

빅데이터 분산처리

강의: 신 인 호 교수



LINUX (리눅스)

Linux(리눅스)



목 차

- 00. 리눅스(Linux) 소개
- 01. 리눅스(Linux) 운영체제
- 02. 리눅스(Linux) 시스템
- 03. 리눅스(Linux) 명령어 입문
- 04. 디렉토리와 파일
- 05. 리눅스 편집기(VI Editor)
- 06. 파일 접근권한 설정
- 07. 관리자권한 실행

분산처리 하둡

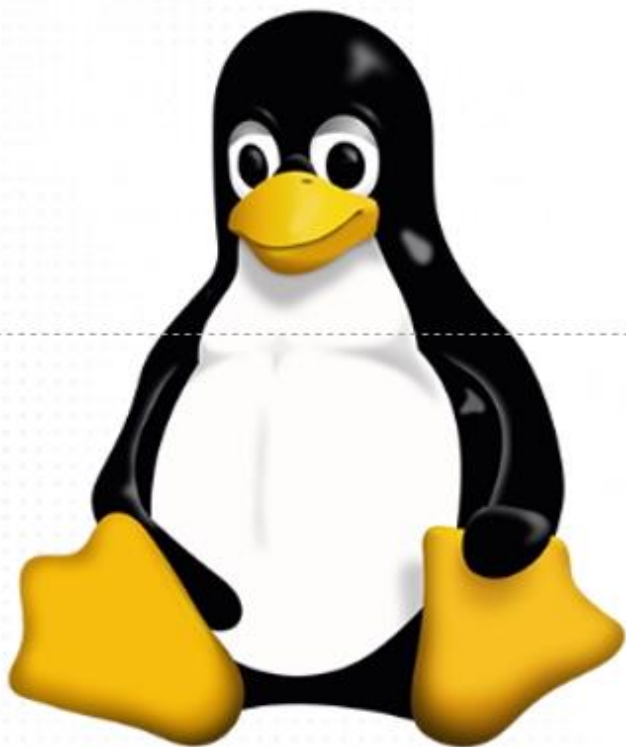


목 차

- 00. 하둡(Hadoop) 소개**
- 01. 하둡 설치 및 운영**
- 02. 하둡 명령어실습**
- 03. 분산처리-WordCount(작성/실습)**
- 05. 빅데이터(수집,적재, 처리,탐색, 분석,응용)**
- 06. 하둡 Echo System 실습**

00.리눅스(Linux) 소개





LINUX (리눅스)

21세기
가장 주목받는
운영체제



✓ 핸드폰(안드로이드)의 운영체제



✓ TV 셋톱 박스나 무선공유기의
운영체제



가전제품의 운영체제

**우리 삶에 상당히
깊숙이 들어와 있는
운영체제**



서버

설치 무료
높은 안정성

서버용 프로그램 대부분이 리눅스를 지원

이식성과 확장성이 뛰어난 Linux



쉬워진 유지보수

뛰어난 범용성

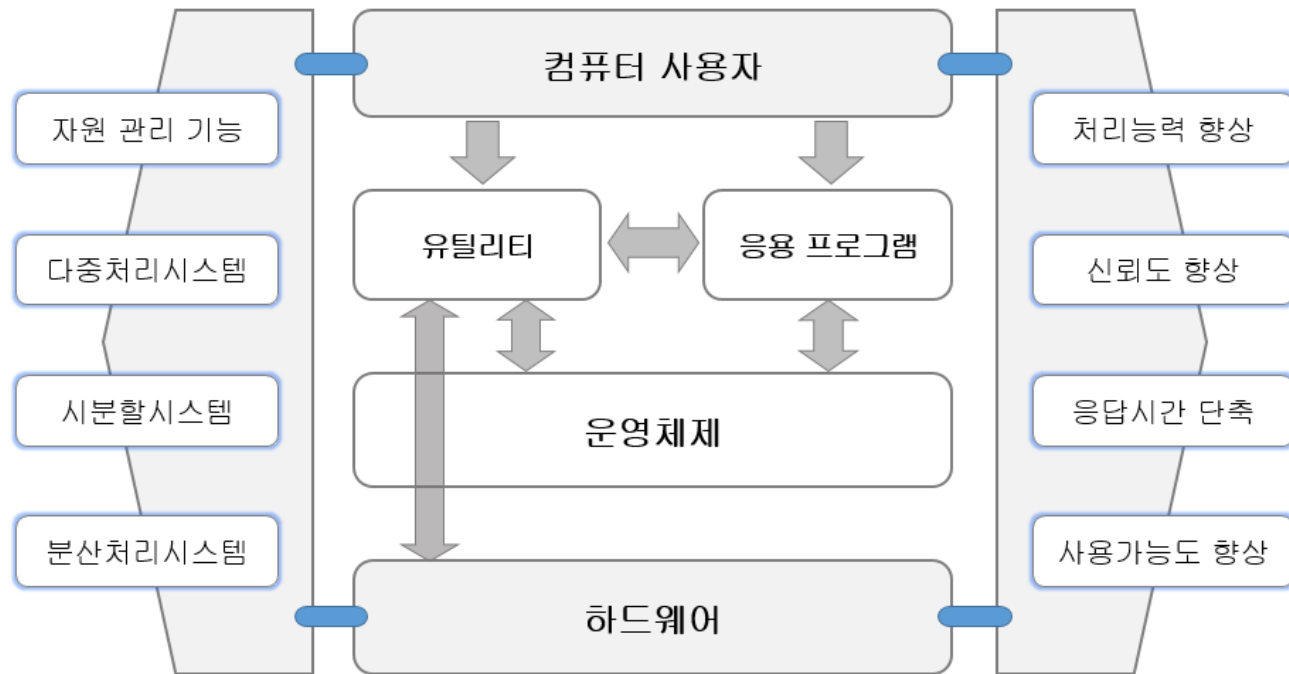


01.리눅스(Linux) 운영체제



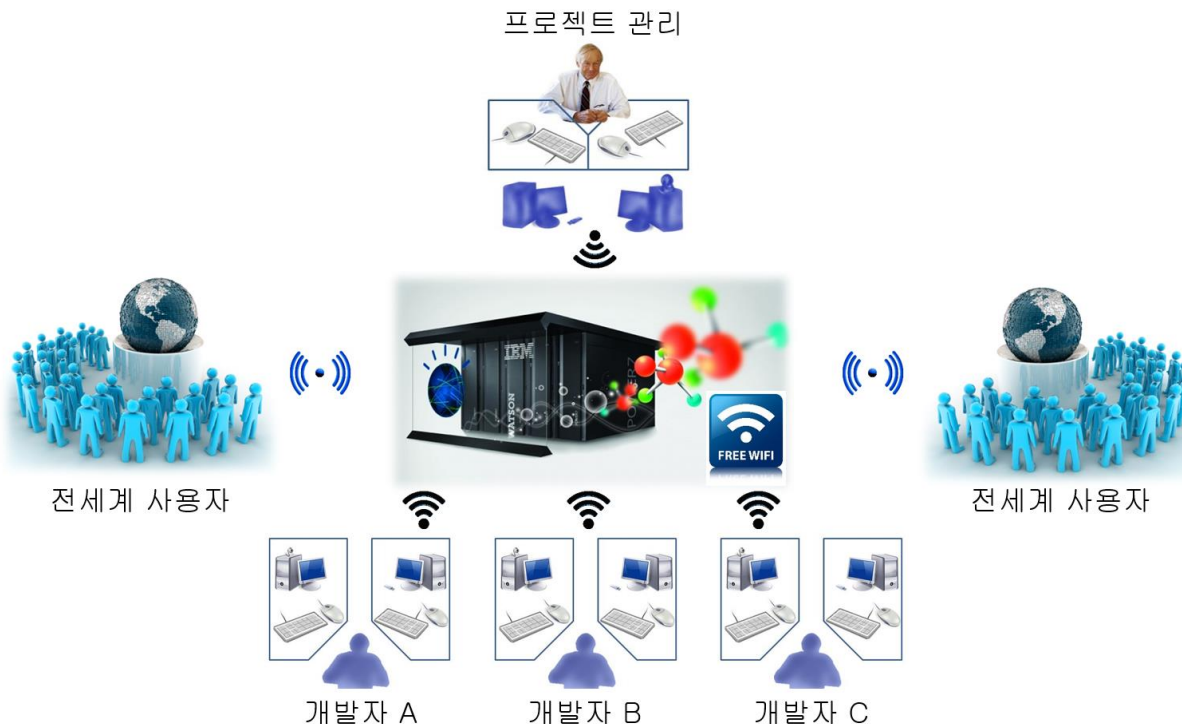
운영체제

- 운영체제는 컴퓨터와 같은 기계장치인 하드웨어와 컴퓨터에 설치되는 모든 소프트웨어를 관리하는 실행관리자
- 사용자가 누구든지 컴퓨터를 효율적으로 사용할 수 있도록 관리
- 운영체제의 역할과 목적



리눅스 운영체제

- 핀란드의 헬싱키 대학에서 운영체제를 공부하던 리누스 토발즈에 의해 개발
- 뉴스그룹의 구성원들과 전 세계 네티즌들의 많은 관심과 지대한 노력으로 구축
- 리눅스 업그레이드와 버그 해결



GNU 프로젝트

- 리누스 토발즈가 리눅스 커널을 개발하기에 앞서 리처드 스톨만에 의해 시작
- 모두가 공유할 수 있는 소프트웨어를 만드는 것을 모티브
- 자유 소프트웨어 재단 FSF(Free Software Foundation)을 설립하여 GNU 프로젝트에서 제작한 S/W 지원
 - GPL 라이선스를 따르도록 규정
 - 자유 소프트웨어의 수정과 공유의 자유 보장
 - S/W 사용, 수정, 복사, 재배포 권리에 대한 사용허가서

■ GPL(General Public License)

- GNU 일반 공중 사용 허가서(GNU General Public License, GNU GPL 또는 GPL)는 자유 소프트웨어 재단에서 만든 자유 소프트웨어 라이선스로, 소프트웨어의 실행, 연구, 공유, 수정의 자유를 최종 사용자에게 보장한다.
- 대표적으로 리눅스 커널이 이용하는 사용 허가이다. GPL은 가장 널리 알려진 강한 카피레프트 사용 허가이며, 이 허가를 가진 프로그램을 사용하여 새로운 프로그램을 만들게 되면 파생된 프로그램 역시 같은 카피레프트를 가져야 한다.

리눅스 시스템의 구성요소

➤ 커널

- 리눅스 토발즈에 의해 생긴 용어
- 명령을 수행 받아 해당 작업을 수행하는 기능
- 리눅스 커널과 셸, 사용자 프로그램의 관계

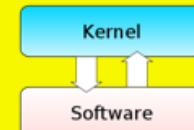


사용자 프로그램 (X윈도우, 웹 서버, DB 서버, 통신 프로그램 등)

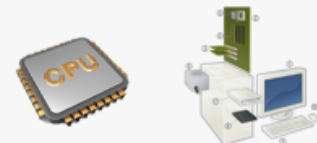
셸 (명령어 해석기; bash, csh 등)



커널 (진정한 의미의 리눅스)



하드웨어 (CPU, RAM, HDD)



리눅스 시스템의 구성요소

➤ 셸

- 사용자와 운영체제 커널 사이의 인터페이스를 감싸는 층

■ Shell의 의미

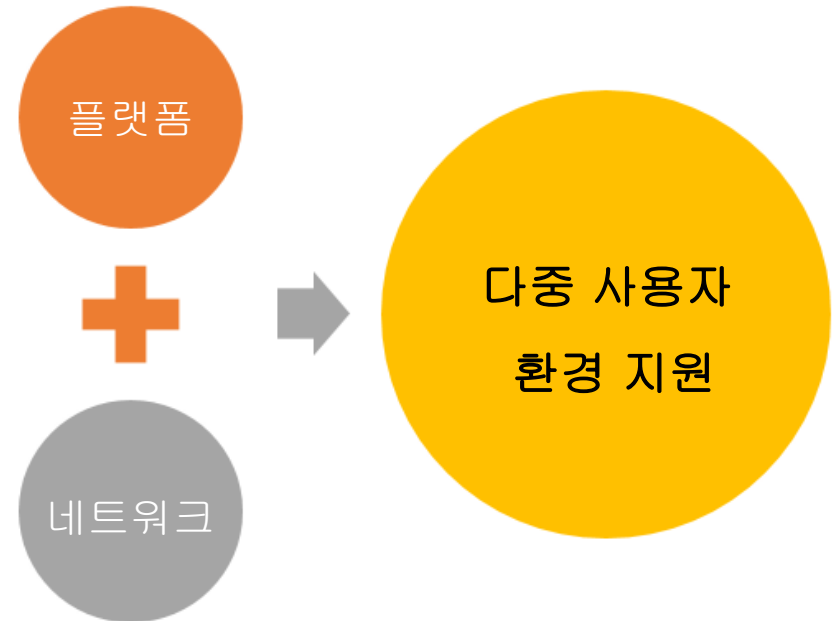
- 리눅스에서 대화형 사용자 인터페이스를 의미
- 사용자가 입력하는 명령어를 이해하고 실행하는 역할
- 운영체제의 바깥 계층에 위치
- 사용자와 커널의 의사소통을 담당
- cshell, bourneshell, bashshell, tcshell, cshell 등의 다양한 종류

■ Shell의 역할

- 입력을 읽고 해당 명령행을 분석
- 특수 문자들을 평가
- 파이프(|), 리다이렉션(>), 백그라운드(&) 프로세스를 설정
- 시그널 처리 (예) **Ctrl** + **C**)

리눅스 시스템의 특징

- 독립된 플랫폼을 갖는 운영체제
- 빠른 업그레이드
- 강력한 네트워크 지원
- 다중 작업과 가상 터미널 환경지원
- 유닉스와 리눅스의 완벽한 호환
- 공개형 오픈 소스의 운영체제
- 다중 사용자 환경 지원
- 저사양 컴퓨터에서도 서버 구축 가능



리눅스의 종류

1. 우분투(Ubuntu)

우분투는 데비안(Debian) 리눅스를 기반의 리눅스입니다.

2. 페도라(Fedora)

페도라는 레드햇(Redhat)의 선행프로젝트 리눅스입니다.

3. 리눅스 민트(Linux Mint)

우분투를 기반으로 제작된 리눅스입니다.

4. 오픈수세(OpenSUSE)

수세리눅스의 개발사인 노벨사의 후원을 받아 제작되었습니다.

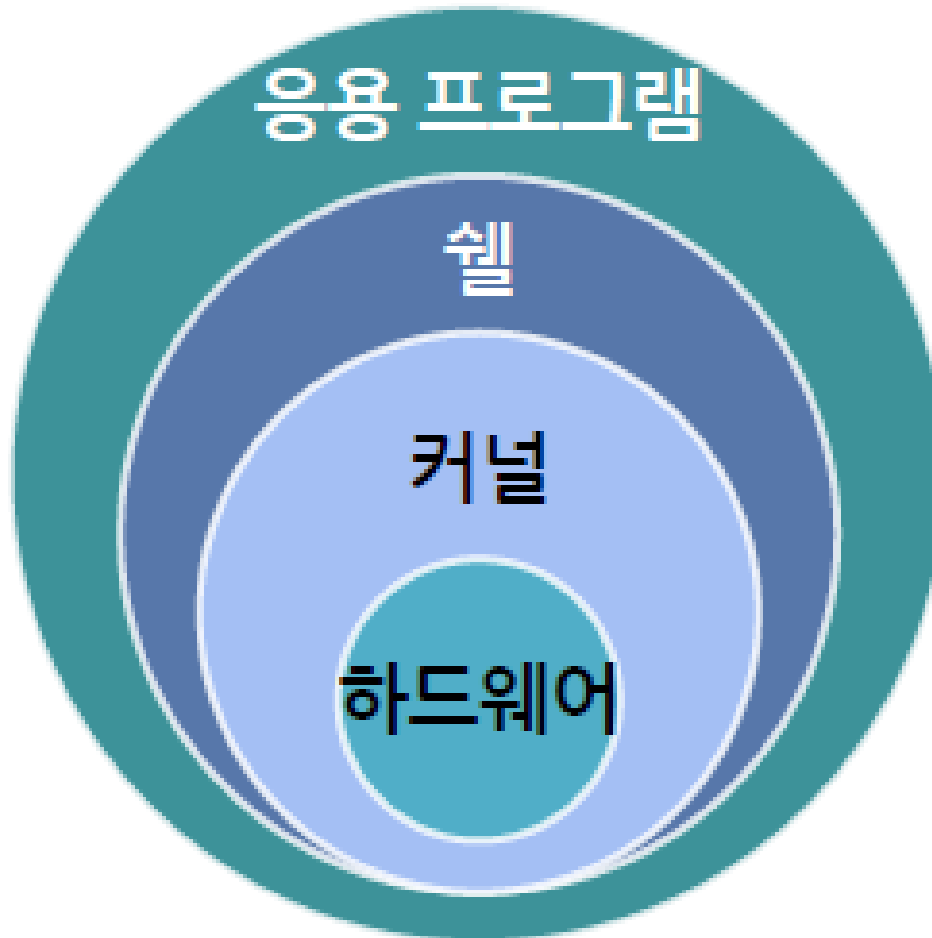
5. 센트OS(CentOS)

레드햇 엔터프라이즈를 그대로 빌드하여 상표권만 제거한 리눅스 입니다.

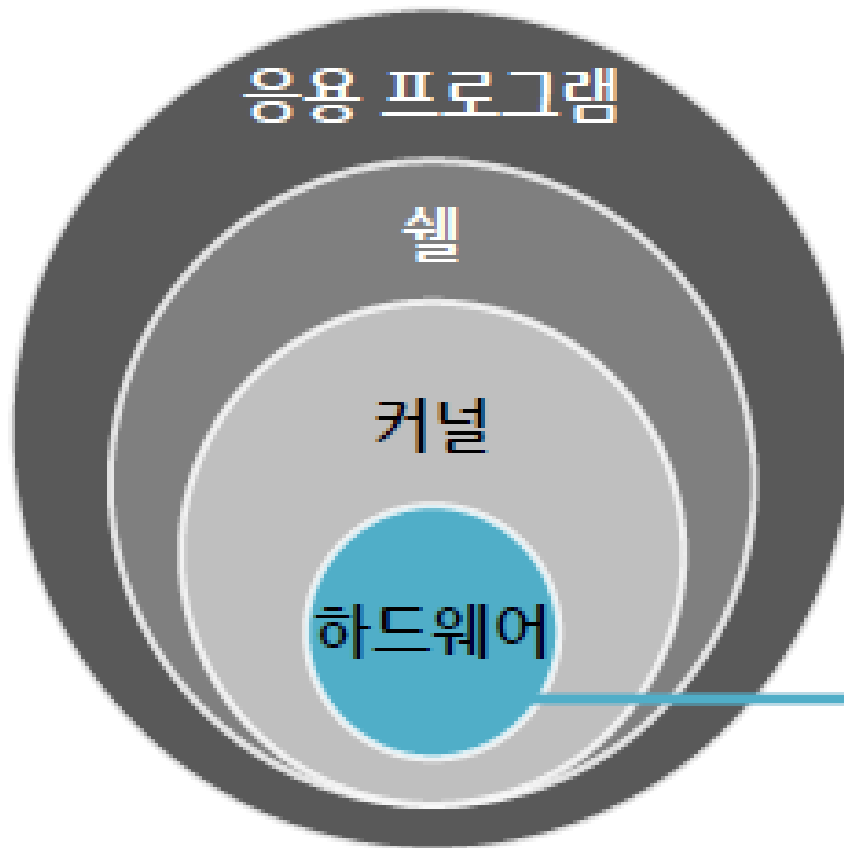
02.리눅스(Linux) 시스템



리눅스 시스템 구조(전체)

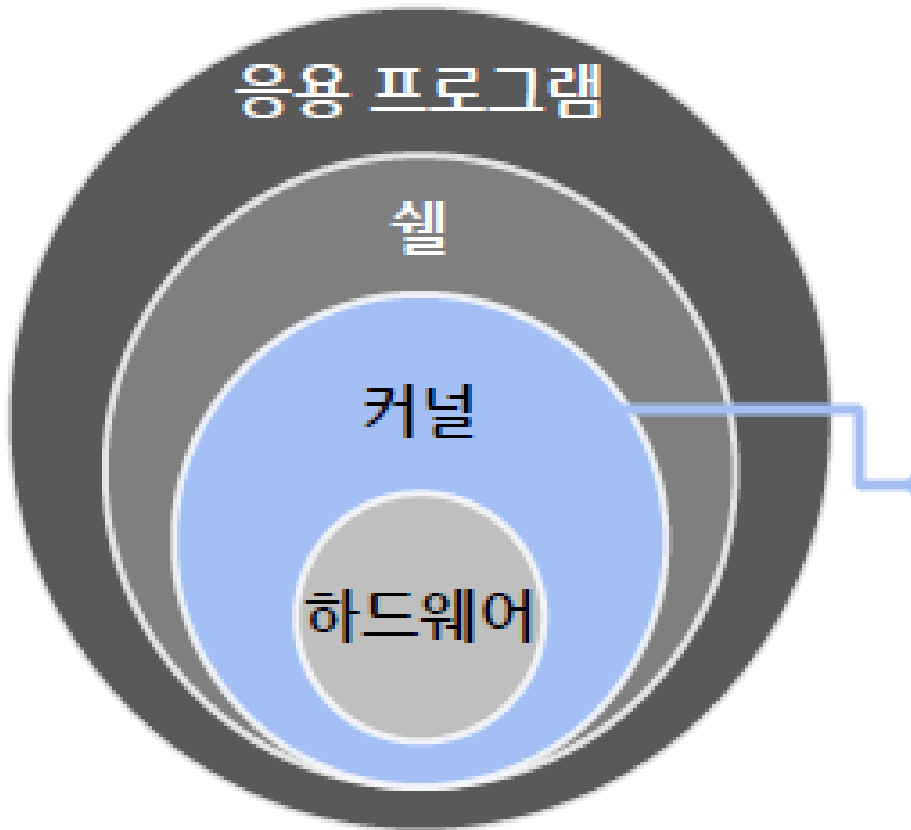


리눅스 시스템 구조(H/W)



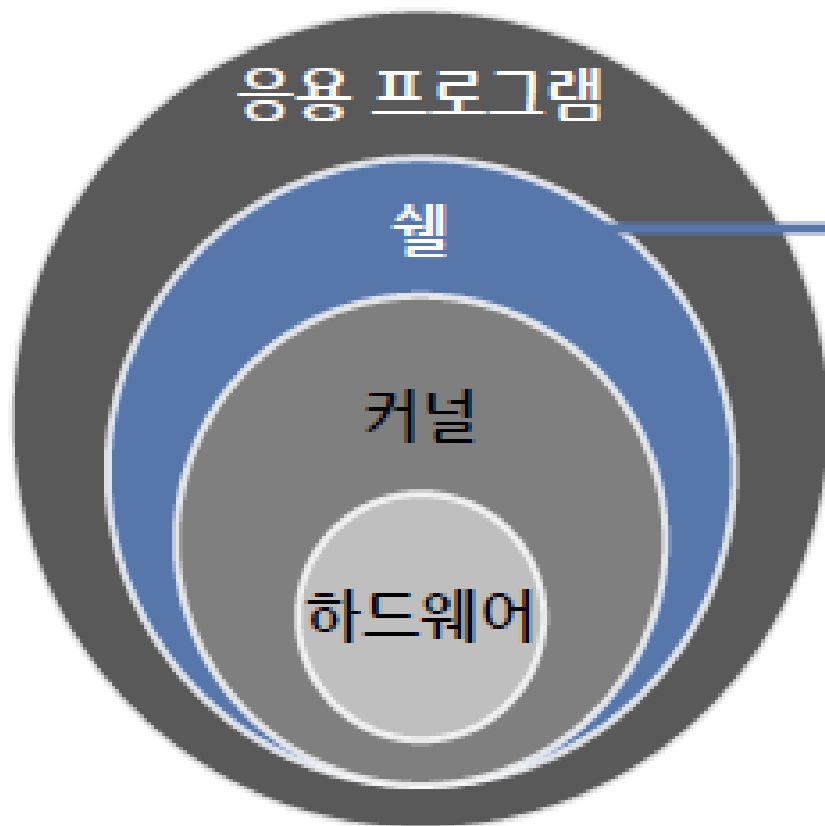
- CPU, 메모리, 하드디스크와 같은 요소들로 구성됨

리눅스 시스템 구조(커널)



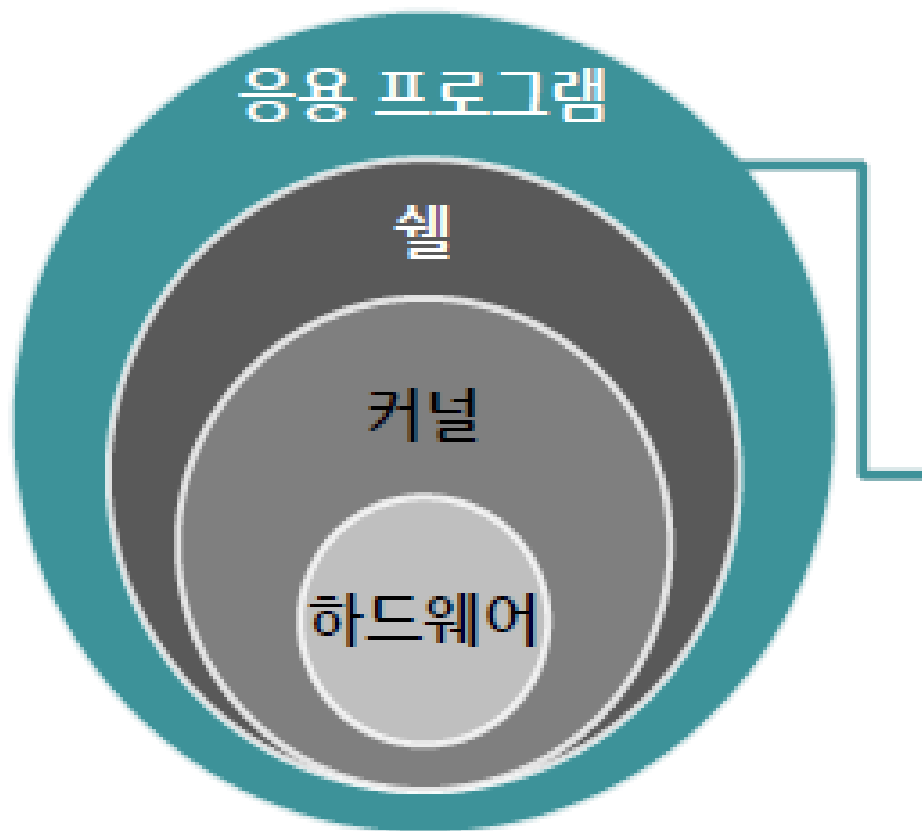
- 실제적인 운영체제
- 하드웨어를 관리하고 제어하는 코드로 구성
- 응용 프로그램들과 통신하며 시스템의 모든 자원을 관리하는 역할

리눅스 시스템 구조(셸)



- 명령어 해석기
- 커널과 응용 프로그램의
가교 역할
- 리눅스는 기본적으로
Bash을 사용함

■ 리눅스 시스템 구조(응용프로그램)



- x 윈도우 시스템, 통신 프로그램, 각종 편집기, 웹서버 등 일반 사용자가 접하는 모든 프로그램을 말함

■ 윈도우 운영체제 VS. 리눅스 운영체제

윈도우 운영체제

- (주) 마이크로 소프트사에서만 만들어서 배포
- 커널 버전 별로 한가지 GUI 버전만 있음
- 정식 커널 버전
 - 윈도우 XP : 5.1
 - 윈도우 Vista : 6.0
 - 윈도우 7 : 6.1
 - 윈도우 10 : 6.4

리눅스 운영체제

- 공개된 소스코드이며 누구나 사용 가능
- 한 가지 커널 버전으로 여러 회사, 단체들이 자신만의 GUI 버전을 만들어서 배포
➔ 이를 'x 윈도우'라고 함
- 여러 회사, 단체들이 만들어 배포하는 GUI가 있는 리눅스를 '리눅스 배포판'이라 함

대표적인 리눅스 배포판

레드햇 계열



CentOS

CentOS

데비안 계열



우분투

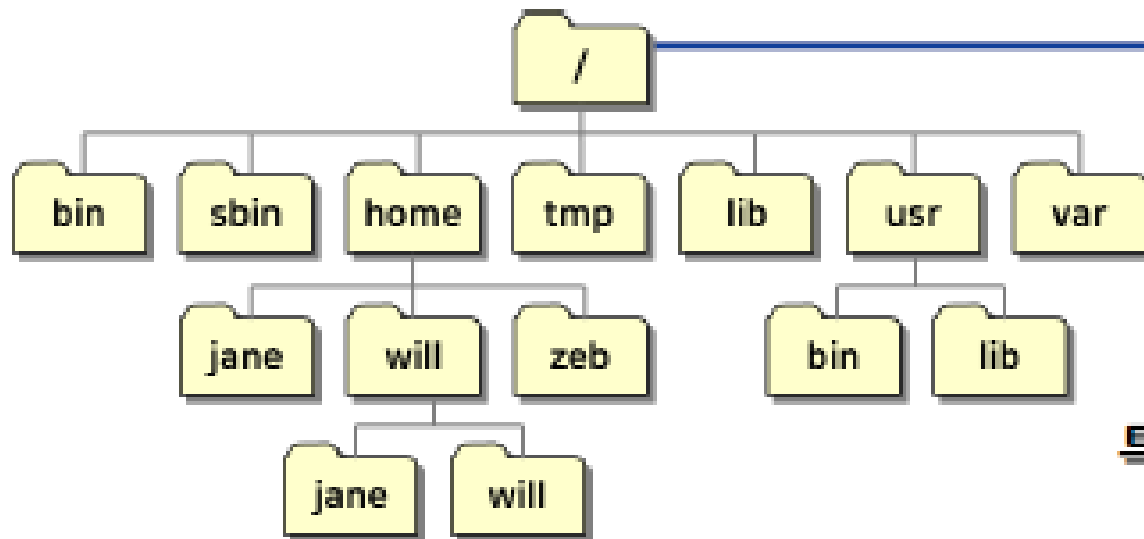
라즈비안

차이점 : 소프트웨어를 설치하는 명령어

명령어 'yum'을 통해
설치 서버에서 응용프로그램
다운로드

명령어 'apt'를 통해
설치 서버에서 응용프로그램
다운로드

파일 시스템구조



루트 디렉터리
(Root directory)

Root 디렉토리 아래로
특정한 디렉토리들로 구성됨

파일 시스템구조

- 디렉토리마다 용도가 정의되어 있으며,
- 디렉토리들의 구성을 통해 리눅스 시스템이 관리됨

디렉토리 이름	설명
/	Root 디렉토리, 최상위 디렉토리
/bin	리눅스 기초 명령어 파일들이 저장된 디렉토리
/boot	커널 및 부팅 관련된 파일들
/dev	시스템 장치파일로, 이른바 디바이스 파일들이 존재함(시리얼, 하드디스크 등) → 실제 파일이 아니고 장치들을 관리하기 위해 사용되는 파일
/etc	시스템 관련 설정 파일들

파일 시스템구조

디렉토리 이름	설명
/home	일반 사용자 홈 디렉토리들
/lib	공유 라이브러리 폴더
/media	CD-ROM, USB 드라이브와 같은 탈부착이 가능한 장치들의 마운트 포인트로 사용
/proc	시스템 정보 파일들
/root	Superuser의 홈 디렉토리
/sbin	시스템 관리용 실행 파일
/tmp	임시 파일 생성용 폴더

리눅스 파티션 권장 크기

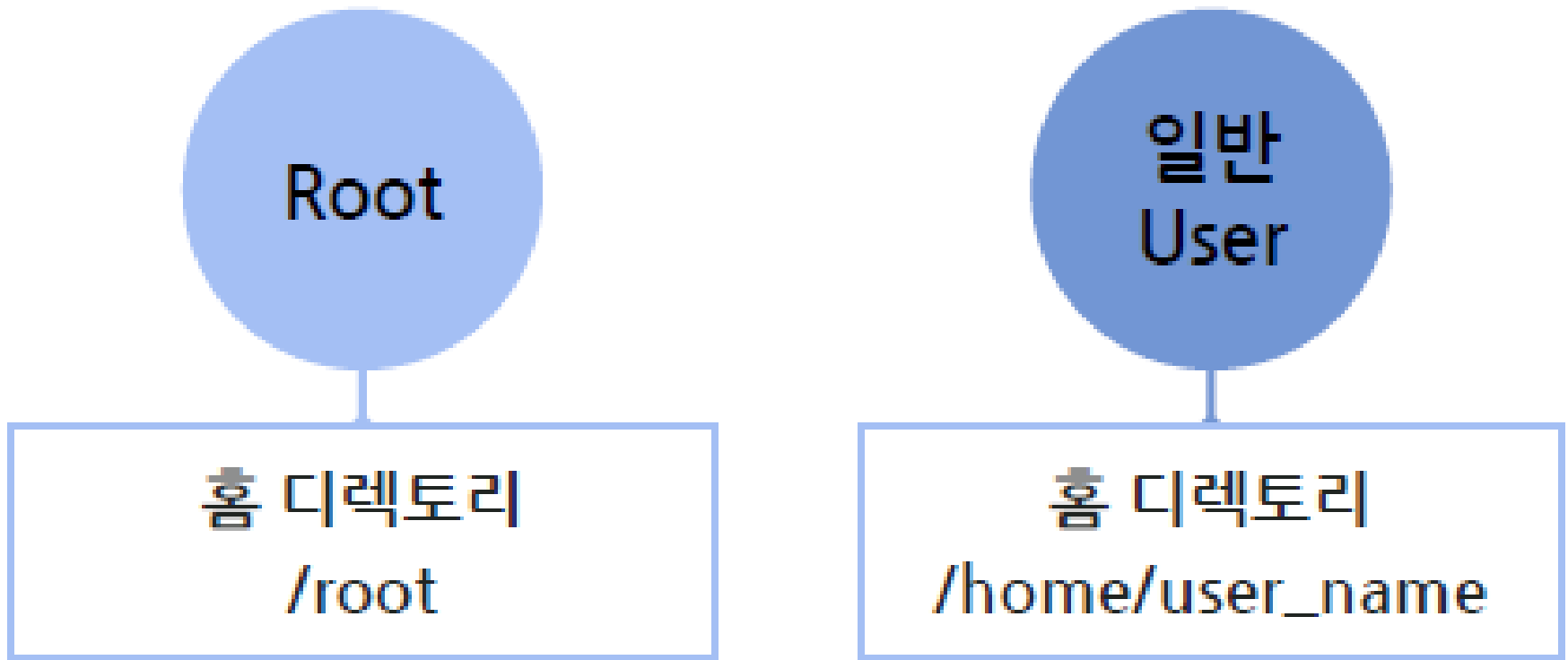
- 리눅스의 용도에 따라 각 파티션의 크기를 정한다.
- 리눅스 전체 용량이 80GB인 경우 권장크기

마운트 포인트	권장 크기	비고
/	10GB	루트 파티션
/bin		기본 명령어가 들어 있음
/sbin		시스템 관리용 명령어가 들어 있음
/etc		시스템의 환경 설정과 관련된 파일이 들어 있음
/boot	4GB	부팅 커널이 저장됨
/media		외부 장치를 마운트하기 위해 제공됨
/usr	설치할 응용 프로그램에 따라 크기 다름 (주로 20GB 내외).	응용 프로그램이 주로 저장됨

파일 시스템구조

/lib		프로그램의 라이브러리가 저장됨
/dev		장치 파일들이 저장됨
/proc		시스템의 프로세서 정보, 프로그램 정보, 하드웨어 정보 등이 들어 있음
/tmp	4GB	임시 파일이 저장됨
/var	10GB	로그, 캐시 파일 등이 저장됨
/root		시스템 관리자인 root의 홈 디렉터리
/home	사용자가 많을 수록 많이 할당 (나머지 용량)	사용자별 공간
/lost+found		파일 시스템 복구를 위한 디렉터리
swap 파티션	RAM의 2배 정도	RAM 부족 시 사용되는 공간

■ 사용자 계정(User account)



리눅스 명령어의 규칙

- 모든 명령어는 영문 소문자
- 모든 계정의 사용자가 사용할 수 있는 명령어
- Root 권한 사용자 만 사용 할 수 있는 명령어

내부 명령어

셸 프로그램에
포함되어 있는 명령어
(예 : cd)

외부 명령어

실행파일이
따로 존재하는 명령어
(예 : ls는 /bin/ls라는 파일로
존재)

리눅스 설치 관련 소프트웨어

1. Securable: 하드웨어의 가상화 설정확인

<https://securable.softonic.kr/download>

2. Oracle VirtualBox : 가상 머신

<https://www.virtualbox.org/wiki/Downloads>

3. Cent OS: Linux OS

<https://www.centos.org/download>

4. Putty : Telnet프로그램

<https://www.putty.org>

5. Filezilla: Ftp 프로그램

<https://filezilla-project.org/download.php>

2025년 리눅스 마스터 시험일정



LINUX (리눅스)

종목	등급	회차	차수	접수일자	시험일자	합격자 발표
리눅스 마스터	1급	2501회	1차	01.20.(월) ~ 02.07.(금)	03.08.(토)	03.28.(금)
			2차	03.31.(월) ~ 04.11.(금)	05.10.(토)	05.30.(금)
		2502회	1차	07.28.(월) ~ 08.08.(금)	09.13.(토)	10.03.(금)
			2차	10.06.(월) ~ 10.17.(금)	11.08.(토)	11.28.(금)
	2급	2501회	1차	01.20.(월) ~ 02.05.(수)	01.21.(화) ~ 02.06.(목)	시험종료 즉시
			2차	01.21.(화) ~ 02.07.(금)	03.08.(토)	03.28.(금)
		2502회	1차	04.28.(월) ~ 05.07.(수)	04.29.(화) ~ 05.08.(목)	시험종료 즉시
			2차	04.29.(화) ~ 05.09.(금)	06.14.(토)	07.04.(금)
		2503회	1차	07.28.(월) ~ 08.06.(수)	07.29.(화) ~ 08.07.(목)	시험종료 즉시
			2차	07.29.(화) ~ 08.08.(금)	09.13.(토)	10.03.(금)
		2504회	1차	10.27.(월) ~ 11.05.(수)	10.28.(화) ~ 11.06.(목)	시험종료 즉시
			2차	10.28.(화) ~ 10.07.(금)	12.13.(토)	'26.01.02.(금)



리눅스 마스터 자격증 안내

03.리눅스(Linux) 명령어 입문



현재 시각과 날짜 출력

현재 시각과 날짜 출력 : date

리눅스 시스템에서 설정되어 있는 현재 시각과 날짜를 출력하기 위해서 사용되는 명령어는 date 입니다. date 명령어의 사용형식은 \$ 프롬프트 다음에 date 명령을 입력한 다음 **[Enter]**를 누르면 됩니다.

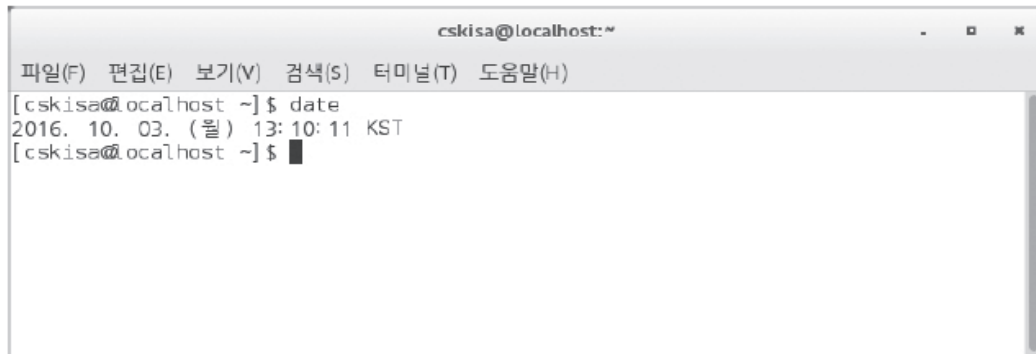
| 예제 2-1 |

현재의 날짜와 시각을 출력합니다.

\$ date

기능 시스템의 현재 시각과 날짜 출력

형식 date **[Enter]**



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ date  
2016. 10. 03. (월) 13:10:11 KST  
[cskisa@localhost ~]$
```

[그림 2-2] date 명령어 실행 결과

화면 지우기 : clear

| 예제 2-5 |

터미널 창에서 수행했던 모든 명령과 실행결과를 지웁니다.

```
$ clear
```

기능 터미널 창에 수행했던 모든 명령과 실행결과를 화면에서 지움

형식 clear

```
cskisa@localhost:~
```

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

				1	2	3		1	2	3	4	5	6	7			1	2	3	4	5	
4	5	6	7	8	9	10		8	9	10	11	12	13	14		6	7	8	9	10	11	12
11	12	13	14	15	16	17		15	16	17	18	19	20	21		13	14	15	16	17	18	19
18	19	20	21	22	23	24		22	23	24	25	26	27	28		20	21	22	23	24	25	26
25	26	27	28	29	30	31		29	30							27	28	29	30	31		

```
[cskisa@localhost ~]$ clear
```

[그림 2-6] clear 명령어 실행

로그인 네임과 ID 확인

| 예제 2-6 |

리눅스 시스템에 접속하여 사용 중인 로그인 네임을 출력합니다.

\$ logname

기능 시스템에 접속하여 사용 중인 로그인 네임 확인

형식 logname



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ logname  
cskisa  
[cskisa@localhost ~]$
```

[그림 2-7] logname 명령어 실행 결과

로그인 네임과 ID 확인

| 예제 2-7 |

현재 리눅스 시스템에 접속한 사용자의 아이디를 출력합니다.

\$ users

기능 시스템에 접속하여 사용 중인 사용자들의 ID 확인

형식 users



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ users  
cskisa cskisa  
[cskisa@localhost ~]$
```

[그림 2-8] users 명령어 실행 결과

로그인 네임과 ID 확인

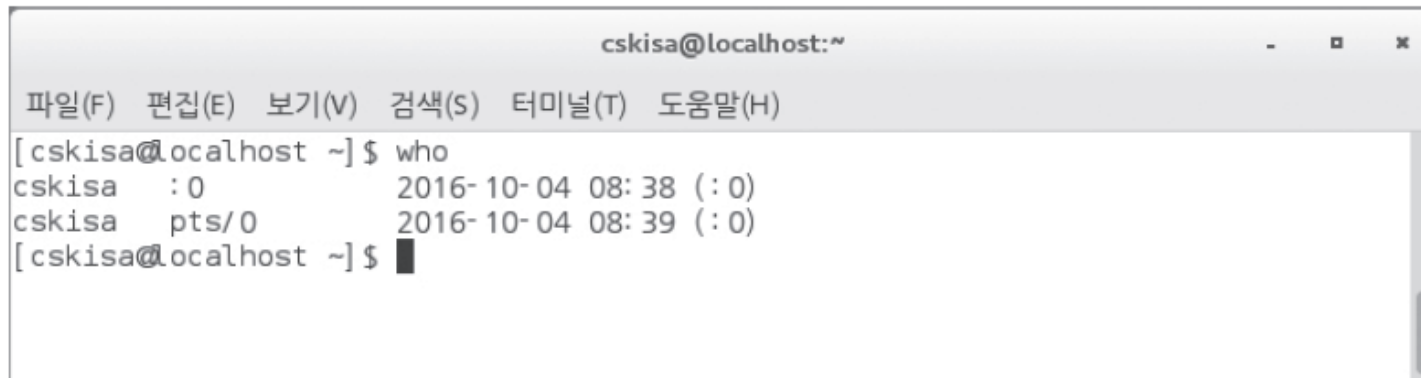
| 예제 2-8 |

리눅스 시스템에 로그인 한 모든 사용자의 계정과 터미널, 로그인 시각 등에 대한 정보를 출력합니다.

\$ who

기능 시스템에 로그인 한 모든 사용자의 계정과 터미널, 로그인 시각 등 확인

형식 who



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ who  
cskisa    :0                2016-10-04 08:38 (:0)  
cskisa    pts/0            2016-10-04 08:39 (:0)  
[cskisa@localhost ~]$
```

[그림 2-9] who 명령어 실행 결과

로그인 네임과 ID 확인

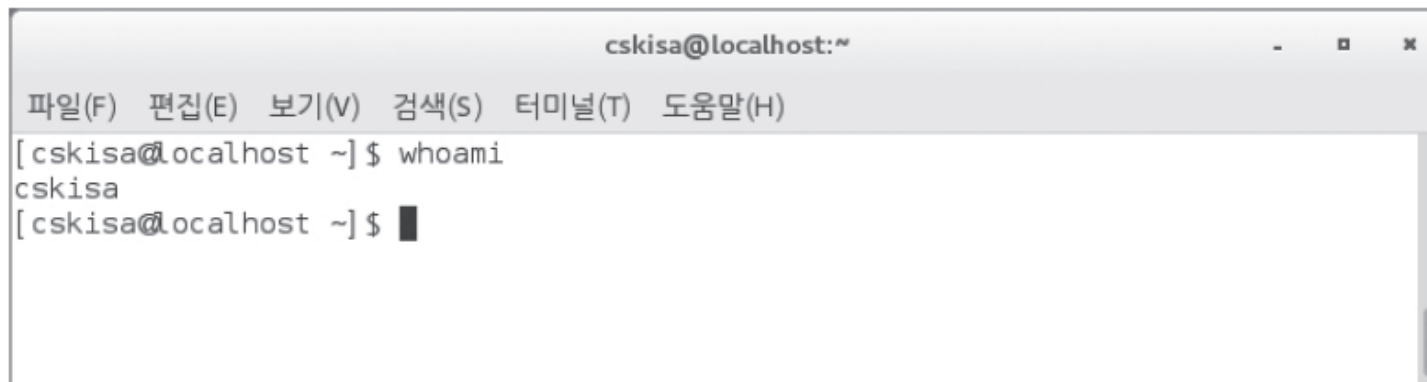
| 예제 2-9 |

현재 리눅스 시스템에 동시에 접속된 사용자를 출력합니다.

```
$ whoami
```

기능 시스템에 동시에 로그인 한 사용자 확인

형식 whoami



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ whoami  
cskisa  
[cskisa@localhost ~]$
```

[그림 2-10] whoami 명령어 실행 결과

로그인 네임과 ID 확인

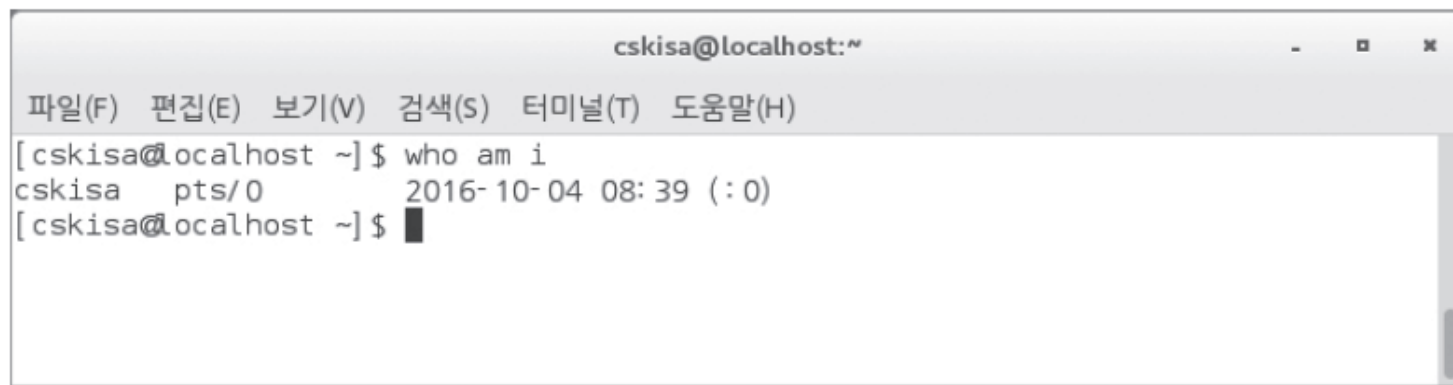
| 예제 2-10 |

현재 리눅스 시스템에 접속한 사용자 계정의 아이디와 pts, 접속한 시각 등 상세한 정보를 출력합니다.

```
$ who am i
```

기능 시스템에 접속한 사용자의 상세한 정보 확인

형식 who am I



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ who am i  
cskisa pts/0 2016-10-04 08:39 (:0)  
[cskisa@localhost ~]$
```

[그림 2-11] who am i 명령어 실행 결과

시스템 정보 출력 : uname

| 예제 2-11 |

현재 사용 중인 시스템에 대한 정보를 출력합니다.

\$ **uname** [옵션]

기능 시스템에 대한 정보 확인

형식 uname [옵션]

옵션 -a : 시스템의 모든 정보 확인

-m : 시스템이 사용 중인 하드웨어 정보 확인

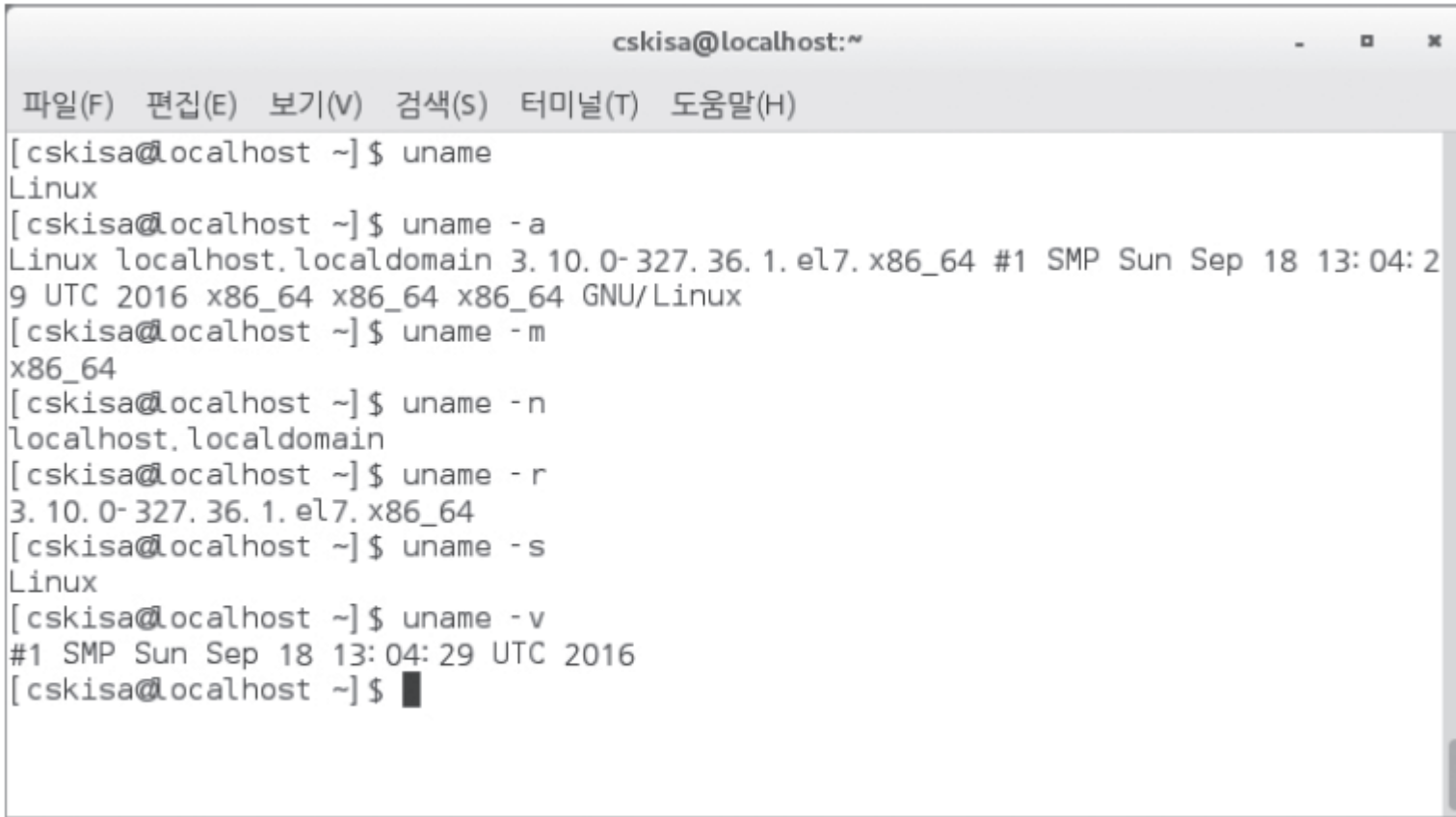
-n : 호스트네임 확인

-r : 운영체제의 릴리즈번호 확인

-s : 운영체제의 이름 확인

-v : 운영체제의 버전출시 일자 확인

시스템 정보 출력 : uname



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ uname  
Linux  
[cskisa@localhost ~]$ uname -a  
Linux localhost.localdomain 3.10.0-327.36.1.el7.x86_64 #1 SMP Sun Sep 18 13:04:29 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux  
[cskisa@localhost ~]$ uname -m  
x86_64  
[cskisa@localhost ~]$ uname -n  
localhost.localdomain  
[cskisa@localhost ~]$ uname -r  
3.10.0-327.36.1.el7.x86_64  
[cskisa@localhost ~]$ uname -s  
Linux  
[cskisa@localhost ~]$ uname -v  
#1 SMP Sun Sep 18 13:04:29 UTC 2016  
[cskisa@localhost ~]$
```

[그림 2-12] uname 명령어 실행 결과

호스트 네임 출력 : hostname

| 예제 2-12 |

현재 사용 중인 리눅스 시스템의 호스트 네임을 출력합니다.

```
$ hostname
```

기능 시스템의 호스트네임 정보 확인

형식 hostname



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ hostname  
localhost.localdomain  
[cskisa@localhost ~]$
```

[그림 2-13] hostname 명령어 실행 결과

환경변수 확인 : env

| 예제 2-14 |

현재 시스템에 설정되어 있는 환경변수에 대한 정보를 출력합니다.

```
$ env
```

기능 환경변수에 대한 정보 확인

형식 env



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ env  
XDG_VTNR=1  
SSH_AGENT_PID=3059  
XDG_SESSION_ID=1  
HOSTNAME=localhost.localdomain  
IMSETTINGS_INTEGRATE_DESKTOP=yes  
GPG_AGENT_INFO=/run/user/1000/keyring/gpg:0:1  
VTE_VERSION=3803  
TERM=xterm-256color  
SHELL=/bin/bash  
XDG_MENU_PREFIX=gnome-  
HISTSIZE=1000  
GJS_DEBUG_OUTPUT=stderr  
WINDOWID=33554439  
QTDIR=/usr/lib64/qt-3.3  
QTINC=/usr/lib64/qt-3.3/include
```

[그림 2-15] env 명령어 실행 결과

문자열 표준 출력 : echo

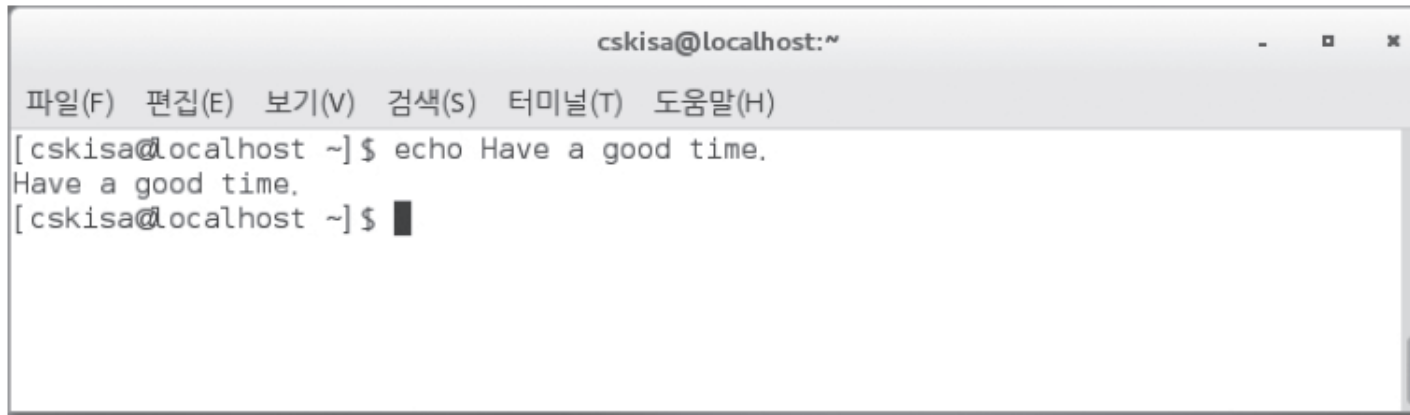
| 예제 2-15 |

지정한 문자열 'Have a good time.'을 표준형식으로 출력합니다.

```
$ echo Have a good time.
```

기능 지정한 문자열 및 환경변수 표준출력

형식 echo 출력할 문자열



A terminal window titled 'cskisa@localhost:~' with a menu bar containing '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal shows the command '[cskisa@localhost ~]\$ echo Have a good time.' being executed, followed by the output 'Have a good time.' and a new prompt line '[cskisa@localhost ~]\$' with a cursor.

[그림 2-16] echo 명령어로 간단한 문자열 출력

옵션 출력 : echo [옵션] 이스케이프

[표 2-1] 이스케이프 문자 종류 및 의미

이스케이프 문자	의미
"\a"	뻑~ 경고음
"\b"	백스페이스 적용
"\c"	마지막에 줄 바꿈 문자 출력 안 함
"\f"	폼 피드 형식으로 출력 (프린터에서 용지변경)
"\n"	줄 바꿈 문자 출력
"\r"	다음 행의 처음으로 커서 이동 (개리지 리턴)
"\t"	수평 탭 정렬
"\v"	수직 탭 정렬
"\w"	역슬래시 (또는 \) 출력
"\nnn"	아스키코드가 8진수(nnn)인 문자 출력

옵션 출력 : echo [옵션] 이스케이프

| 예제 2-16 |

줄 바꿈 옵션을 사용하여 하나의 문장을 두 줄로 출력합니다.

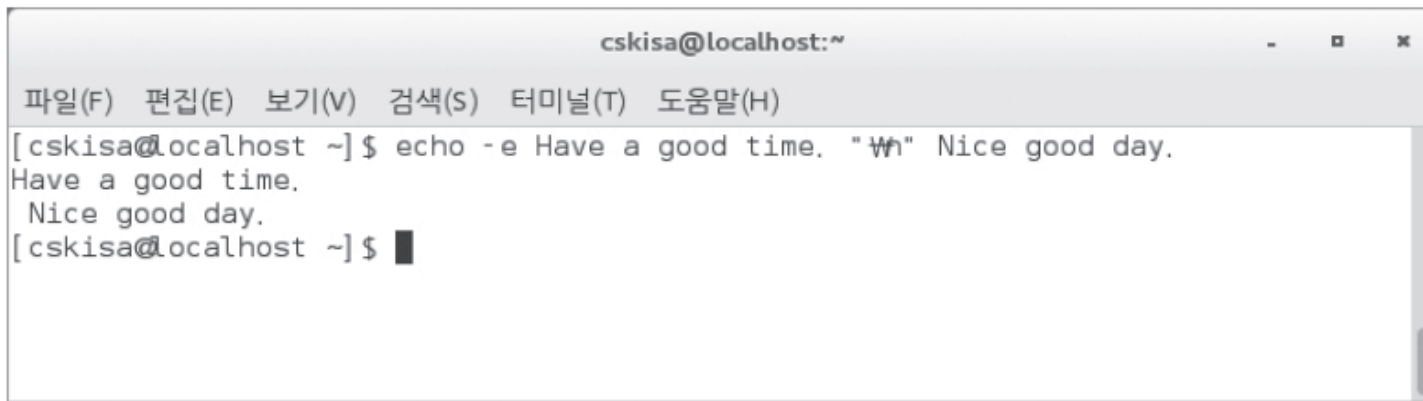
```
$ echo -e Have a good time. "\n" Nice good day.
```

기능 지정한 문자열 및 환경변수 출력

형식 echo [옵션] [문자열] [이스케이프 문자] [문자열] [Enter]

옵션 -n : 문자열을 출력한 다음 줄을 바꾸지 않음

-e : 이스케이프 문자를 사용하기 위한 옵션



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ echo -e Have a good time. "\n" Nice good day.  
Have a good time.  
Nice good day.  
[cskisa@localhost ~]$
```

[그림 2-17] echo 명령어에 옵션과 이스케이프 문자를 사용한 실행 결과

명령어가 존재하는 디렉터리 경로 확인 : which

| 예제 2-17 |

알고 싶은 echo 명령어가 어느 디렉터리에 존재하는지에 대해서 알려줍니다.

```
$ which echo
```

기능 명령어가 어느 디렉터리에 존재하는지 확인

형식 which 명령어



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ which echo  
/usr/bin/echo  
[cskisa@localhost ~]$
```

[그림 2-18] which 명령어 실행 결과

지금까지 사용한 명령어 리스트 확인 : history

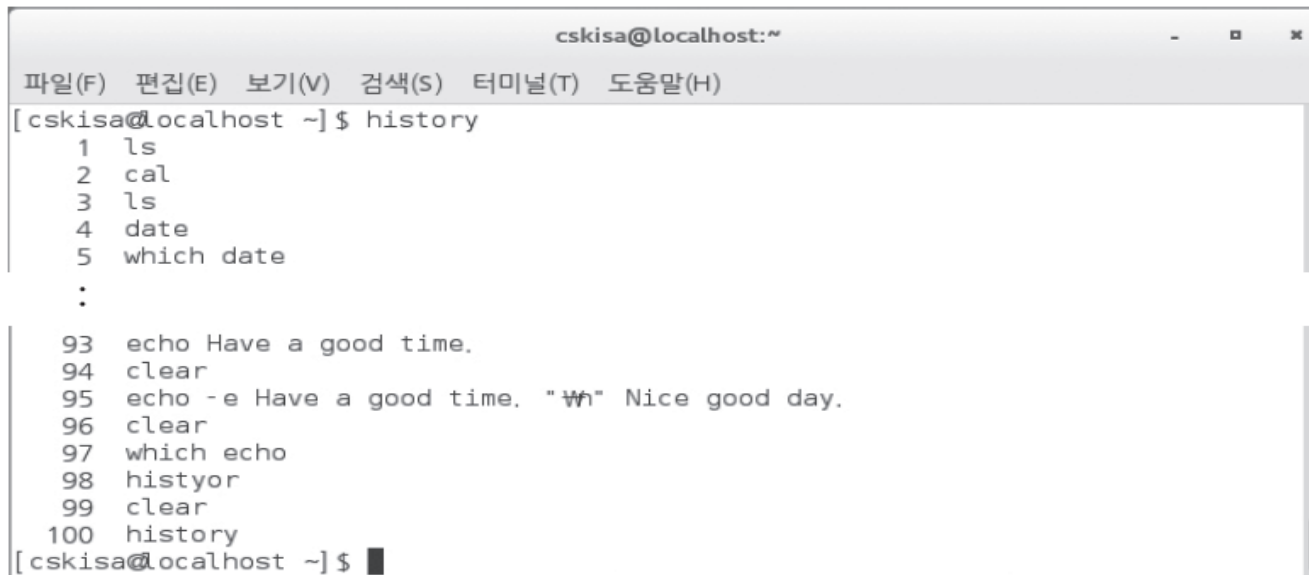
| 예제 2-18 |

리눅스 시스템의 터미널 창에서 지금까지 사용한 모든 명령어 리스트를 출력합니다.

\$ history

기능 : 지금까지 사용했던 모든 명령어들을 보여줌

형식 : history



```
cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ history
 1  ls
 2  cal
 3  ls
 4  date
 5  which date
:
93  echo Have a good time.
94  clear
95  echo -e Have a good time. "w" Nice good day.
96  clear
97  which echo
98  histyor
99  clear
100 history
[cskisa@localhost ~]$
```

[그림 2-19] history 명령어 실행 결과

사용했던 명령을 라인번호로 다시 실행 : ![라인번호]

| 예제 2-19 |

리눅스 시스템에서 사용했던 명령들 중에서 특정 명령을 수행하기 위해 라인번호를 느낌표와 함께 지정하여 수행했던 명령을 다시 실행하도록 합니다.

\$!97

기능 : 사용했던 모든 명령어들 중에서 97번째 수행한 명령어를 다시 사용

형식 : [옵션][라인번호] [Enter]

The screenshot shows a terminal window titled 'cskisa@localhost:~'. The command history is displayed as follows:

```

93 echo Have a good time.
94 clear
95 echo -e Have a good time. "\n" Nice good day.
96 clear
97 which echo
98 histyor
99 clear
100 history
101 history which echo
102 history
103 i97
104 history

```

At the bottom of the terminal, the user enters the command `[cskisa@localhost ~]$ !97`. This command re-executes the command at line 97 of the history, which is `which echo`. The output of this command is `/usr/bin/echo`. The terminal prompt then returns to `[cskisa@localhost ~]$`.

[그림 2-20] history 명령어 후 옵션명령 !97 실행 결과

라인번호를 지정하여 명령 삭제 : history -[옵션] [라인번호]

| 예제 2-20 |

리눅스 시스템의 터미널 창에서 수행했던 명령들 중에서 특정 라인번호를 지정하여 선택적으로 수행했던 명령을 삭제합니다.

```
$ history -d 100
```

기능 지금까지 사용했던 모든 명령어 중에서 100번째 수행한 명령어를 다시 사용

형식 history [옵션] [번호] Enter

옵션 -d : 사용했던 명령어 중에서 라인번호를 지정하여 삭제
-c : 이전에 사용했던 모든 명령어를 삭제

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
 94 clear
 95 echo -e Have a good time. "ㄴ" Nice good day.
 96 clear
 97 which echo
 98 histyor
 99 clear
100 history
101 history which echo
102 history
103 i97
104 history
[cskisa@localhost ~]$ !97
which echo
/usr/bin/echo
[cskisa@localhost ~]$ history -d 100
[cskisa@localhost ~]$
  
```

[그림 2-21] history -d 100 명령어 실행

라인번호를 지정하여 명령 삭제 : history -[옵션] [라인번호]

| 예제 2-20 |

리눅스 시스템의 터미널 창에서 수행했던 명령들 중에서 특정 라인번호를 지정하여 선택적으로 수행했던 명령을 삭제합니다.

```
$ history -d 100
```

기능 지금까지 사용했던 모든 명령어 중에서 100번째 수행한 명령어를 다시 사용

형식 history [옵션] [번호] Enter

옵션 -d : 사용했던 명령어 중에서 라인번호를 지정하여 삭제

-c : 이전에 사용했던 모든 명령어를 삭제

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
 94 clear
 95 echo -e Have a good time. "ㄴ" Nice good day.
 96 clear
 97 which echo
 98 histyor
 99 clear
100 history
101 history which echo
102 history
103 i97
104 history
[cskisa@localhost ~]$ !97
which echo
/usr/bin/echo
[cskisa@localhost ~]$ history -d 100
[cskisa@localhost ~]$

```

[그림 2-21] history -d 100 명령어 실행

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
 92 clear
 93 echo Have a good time.
 94 clear
 95 echo -e Have a good time. "ㄴ" Nice good day.
 96 clear
 97 which echo
 98 histyor
 99 clear
100 history which echo
101 history
102 i97
103 history
104 which echo
105 history -d 100
106 history
[cskisa@localhost ~]$

```

[그림 2-22] history -d 100 명령어 실행 결과

사용했던 모든 명령들을 한꺼번에 삭제 : history -[옵션]

| 예제 2-21 |

리눅스 시스템의 터미널 창에서 수행했던 모든 명령들을 한꺼번에 삭제합니다.

```
$ history -c
```

기능 지금까지 사용했던 명령들을 모두 삭제

형식 history [옵션] [번호]

옵션 -d : 사용했던 명령들 중에서 삭제할 번호에 해당하는 명령 삭제

-c : 이전에 사용했던 명령들을 모두 삭제

```

cskisa@localhost ~$ history -c
92 clear
93 echo Have a good time.
94 clear
95 echo -e Have a good time. "\n" Nice
96 clear
97 which echo
98 histyor
99 clear
100 history which echo
101 history
102 i97
103 history
104 which echo
105 history -d 100
106 history
[cskisa@localhost ~]$ history -c

```

[그림 2-23] history -c 명령어 실행

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
95 echo -e Have a good time. "\n" Nice good day.
96 clear
97 which echo
98 histyor
99 clear
100 history which echo
101 history
102 i97
103 history
104 which echo
105 history -d 100
106 history
[cskisa@localhost ~]$ history -c
[cskisa@localhost ~]$ history
1 history
[cskisa@localhost ~]$

```

[그림 2-24] history -c 명령어 실행 결과

04.디렉토리와 파일

01. 파일의 구성요소
02. 리눅스의 파일 종류
03. 디렉터리와 파일관리
04. 디렉터리 관리 명령어
05. 파일관리 명령어



SECTION
01

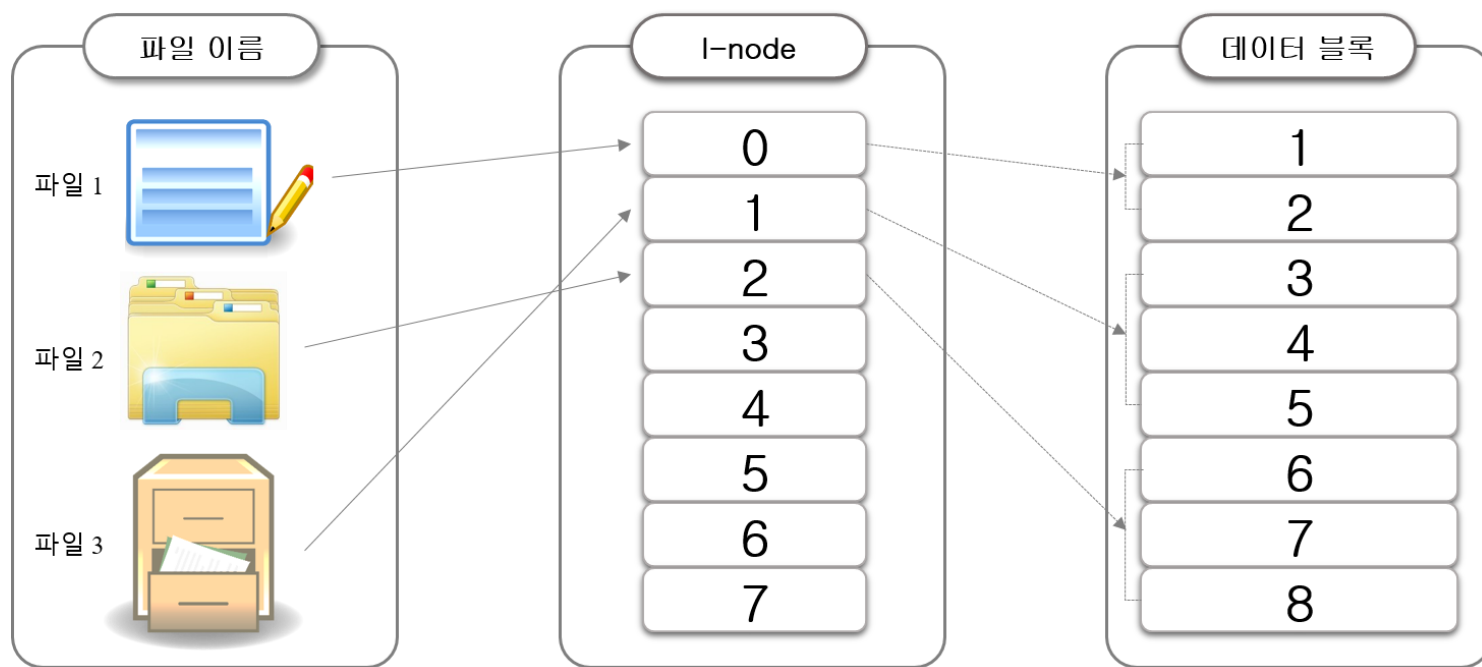
리눅스 운영체제

• 리눅스 운영체제 이해하기



1. 운영체제의 역할과 기능에 대해서 이해합니다.
2. 리눅스 운영체제의 종류와 특징에 대해 살펴봅니다.
3. 리눅스 윈도우환경의 구성요소에 대해서 이해합니다.

- 리눅스 시스템에서는 모든 처리과정을 파일단위로 처리
- 파일의 구성은 계층적인 구조의 특성
- 파일의 세 가지 요소 → 모두 충족해야만 파일로 인정



파일 이름

- 사용자가 파일을 사용목적에 맞도록 사용하고 해당 파일에 정확히 접근할 수 있도록 구별할 수 있는 변별력을 제공
- 변별력은 파일을 조작하기 위한 과정에서도 활용
- 리눅스 시스템에서 관리하는 모든 데이터는 파일이라는 단일 인터페이스를 가지고 있음
- 리눅스 시스템에서는 파일의 속성을 지정하는 특별한 확장명은 제공되지 않음

I-node

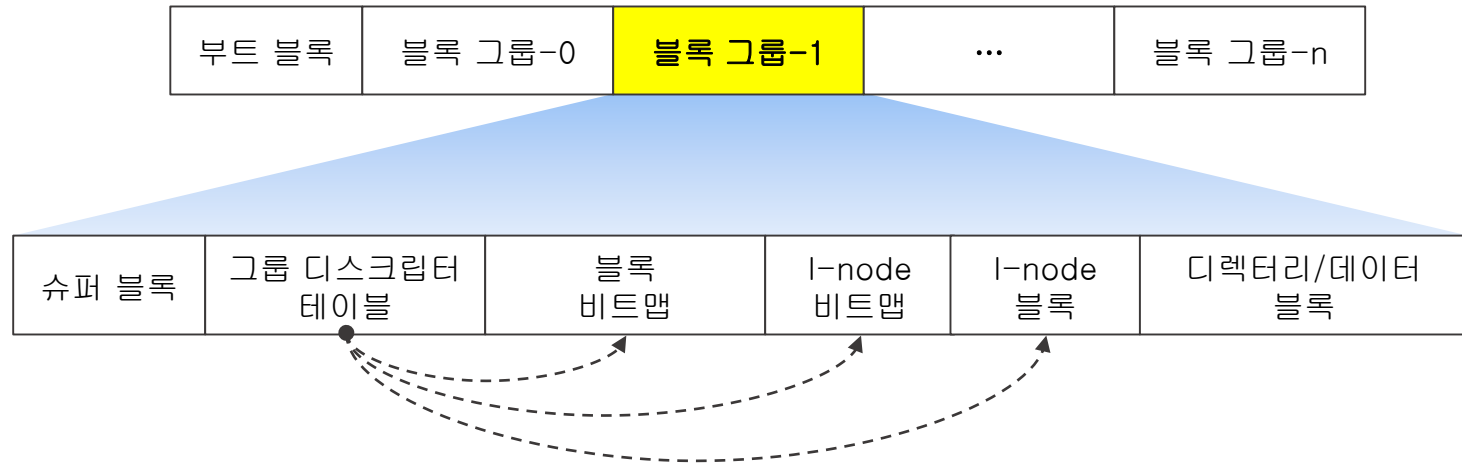
- Index Node라고 불리는 I-node는 파일을 기술하는 디스크 상에서의 데이터 구조를 의미
- 하나의 파일을 생성하면 하나의 I-node가 생성되고 파일을 구별할 수 있도록 I-node 번호가 부여됨
- 부여된 I-node 번호를 찾고자 할 경우에는 파일의 정보를 통해 알 수 있음
- I-node에 저장된 정보

- 파일의 종류
- 파일의 소유권 - 사용자(소유자)와 그룹
- 파일의 액세스 모드
- 파일의 타임스탬프 (파일 갱신일)

데이터 블록

- Data Block은 파일에서 데이터를 저장하는 블록을 의미
- 데이터 블록에는 일반 파일이나 디렉터리 파일의 데이터가 존재
- 디스크 장치에 파일을 저장할 때 실제 데이터는 특별한 구분 없이 디스크에 저장
- 디스크의 어느 한 곳에 차례대로 쌓여 있는 부분을 데이터 블록이라고 함

- Data Block 개념



- 블록 그룹은 블록들의 모임을 의미
- OS 커널이 같은 파일에 속하는 데이터 블록은 같은 블록 그룹에 저장하는 기능을 제공하므로 파일의 단편화를 줄일 수 있음
- 블록 그룹의 개수와 크기는 해당 파티션 블록의 크기에 따라 달라짐

SECTION
02

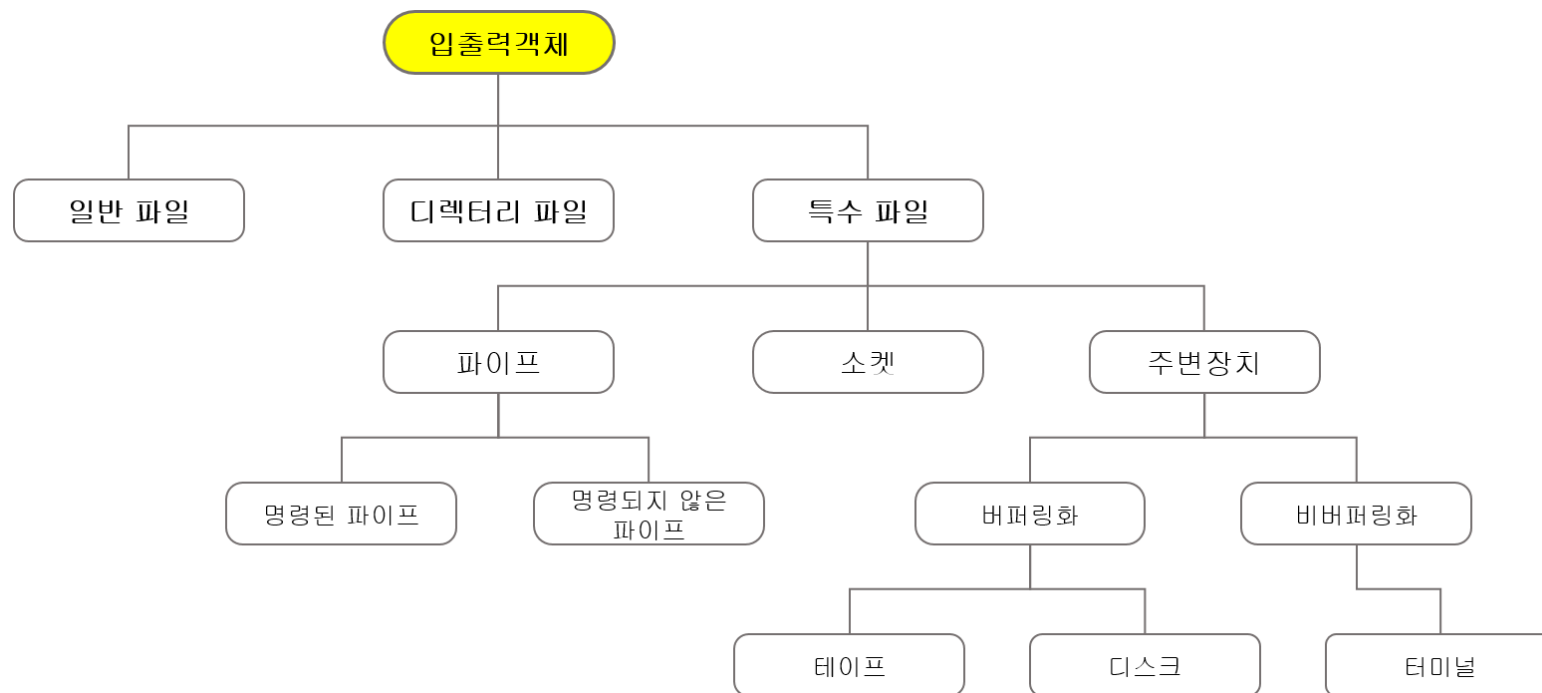
리눅스의 파일 종류

- 리눅스 파일의 종류 이해하기



1. 리눅스 파일의 종류에 대해서 살펴봅니다.
2. 사용목적에 따라 구분되는 파일의 종류에 대해 이해합니다.

- 리눅스 시스템에서 관리하는 모든 데이터는 파일(현재 실행중인 프로세스, 일반 파일, 하드웨어 등)로 인식하여 모든 자원을 파일로 다룸
- 하나의 프로세스는 하나의 작업 디렉터리를 가짐



■ 일반 파일

- 데이터를 관리하는데 주로 사용되는 일상적인 파일을 의미
- 각종 실행 파일이나 텍스트 파일, 이미지 파일, 프로그램 소스 파일 등
- 실행 파일이나 이미지 파일의 경우 데이터가 바이너리 형태로 저장
- 일반 파일을 정규 파일이라고도 하며 정규 파일은 표준 입출력 시스템의 호출을 통해 참조됨

■ 디렉터리 파일

- 리눅스 시스템에서는 디렉터리 역시 파일로 취급되며 디렉터리 파일은 해당 디렉터리에 저장된 파일이나 하위 디렉터리의 정보를 가짐
- 디렉터리는 일반 파일과 마찬가지로 디스크에 저장되어 다른 파일을 조작하고 액세스하는데 필요한 정보를 보유
- 디렉터리라는 파일을 이용해서 전체 파일을 하나의 트리 구조로 관리

■ 링크 파일

- 리눅스와 유닉스에서 자주 사용되며 사용자에게 편리성을 제공 해 주는 파일
- 링크라는 개념을 도입해 여러 개의 이름이 하나의 l-node 에 연결 되도록 수행하여 실제 파일이나 디렉터리 혹은 다른 링크를 가리킴
- 링크를 추가한다는 의미는 단순히 디렉터리 엔트리를 만드는 것
- 파일에는 하드 링크와 심볼릭 링크가 존재하며 링크 파일을 생성 하기 위해서는 ln 명령을 사용

● 하드 링크

- 원본 파일을 복사하여 원본 파일과 동일한 내용의 다른 사본 파일을 만드는 것을 의미
- 원본 파일과 링크 파일 두 개가 서로 다른 파일로 취급되므로 둘 중 어느 하나를 삭제하더라도 나머지 하나는 그대로 남아 있음
- 하드 링크에서는 원본 파일의 내용이 변경될 경우 복사된 링크 파일의 내용 또한 자동으로 변경됨
- 하드 링크는 동일한 i-node를 갖는 파일을 생성하는 것을 의미하며
- 파일의 실제 i-node의 정보를 공유하기 때문에 마치 동일한 파일이 여러 곳에 존재하는 것처럼 보이게 됨

● 심볼릭 링크

- 리눅스 시스템에서 통상적으로 사용되는 링크 파일이 바로 심볼릭 링크 파일이며 다른 표현으로는 소프트 링크라고도 함
- 윈도우 운영체제의 ‘바로 가기’ 기능이나 단축 아이콘과 같은 역할 수행
- 이미 생성된 심볼릭 링크 파일을 삭제하여도 존재된 원본 파일에는 어떠한 영향도 미치지 않음
- 기호화된 링크라고도 하며 절대경로 또는 상대경로 형태로 구성된 다른 파일이나 디렉터리에 대한 참조를 포함하고 있는 특별한 종류의 파일
- 심볼릭 링크는 다른 파일이나 디렉터리에 대한 경로로서 리눅스 운영체제에서 자동으로 해석하고 추적하는 텍스트 문자열을 포함

특수 파일

- 리눅스가 지원하는 파이프, 소켓, 주변장치(디바이스)를 의미
- 디바이스는 윈도우 운영체제에서 하드디스크가 C:, D:, E: 등으로 표시되지만
- 리눅스 시스템에서는 /dev/hda1, ../dev/hda3과 같이 사용됨
- 파일의 영역은 키보드, 모니터, 마우스, 디스크 드라이브 등과 같이 컴퓨터의 모든 자원이 특수 파일로 사용되고 다루어 짐

■ 문서 파일과 이진 파일

- 문서 파일은 키보드로 입력 받은 값만을 가지고 있으며 ASCII 문자로 구성되어 있음
- ASCII 문자는 알파벳 대/소문자, 숫자, 공백, 탭, 제어문자, 문장부호를 포함하여 128개의 코드로 구성되어 매칭
- 이진 파일은 문자가 아닌 데이터 파일이 들어 있으며 기계어인 0과 1의 값으로 구성되어 있음
- 이진 파일은 프로그램을 코딩하여 컴파일 하는 과정에서 생성되는 파일을 의미하며 실행파일이라고도 불림

SECTION
03

디렉터리와 파일 관리

- 디렉터리와 파일 관리 이해하기

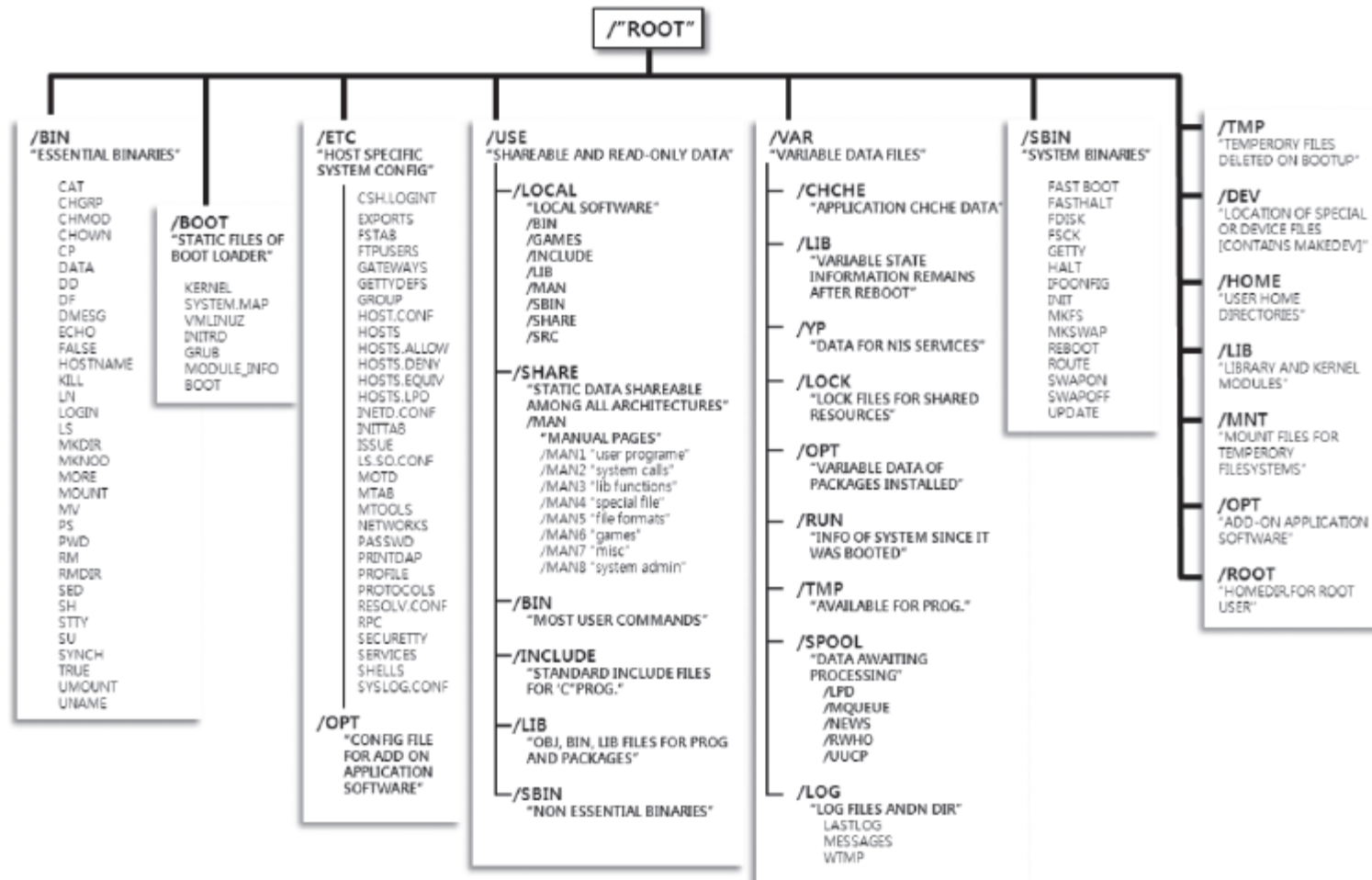


1. 디렉터리의 계층 구조에 대해서 살펴봅니다.
2. 절대 경로와 상대 경로에 대한 의미를 이해합니다.
3. 파일과 디렉터리명 생성규칙에 대해서 살펴봅니다.

■ 디렉터리 계층 구조

- 디렉터리의 구조는 계층적 형태인 트리 구조로 구성되어 있으며
- 사용하고자 하는 목적에 따라 수행되는 명령어의 성격과 내용 및 사용권한 부여 등
- 사용목적에 따라 디렉터리로 구분하여 제공

- 리눅스 시스템의 디렉터리 구조



| 예제 3-1 |

리눅스 시스템의 루트 디렉터리와 심볼릭 링크 파일을 확인하기 위해 `-F` 옵션과 루트를 의미하는 `/`를 선언하여 존재하는 리스트를 확인합니다.

```
$ ls -F /
```

기능 `-F`: 파일의 종류를 실행파일은 `*`, 디렉터리는 `/`, 심볼릭 링크는 `@` 표시

형식 `ls [옵션] /`



```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls -F /
bin@  dev/  home/  lib64@ mnt/  proc/  run/  srv/  tmp/  var/
boot/ etc/  lib@  media/ opt/  root/  sbin@ sys/  usr/
[cskisa@localhost ~]$
  
```

[그림 3-5] 디렉터리 파일과 심볼릭 링크 파일 확인

[표 3-1] 디렉터리와 심볼릭 링크 파일의 주요기능

디렉터리	기능
bin@	기본 명령어가 존재하는 심볼릭 링크 파일
dev/	시스템 디바이스 장치 파일이 존재하는 디렉터리
home/	일반 사용자들이 사용하는 홈 디렉터리
lib64@	64bit 운영체제 호환성과 관련된 파일이 존재하는 심볼릭 링크 파일
mnt/	입/출력장치 등 파일시스템을 임시로 마운트 하는 디렉터리
proc/	프로세스 정보 등 커널 관련정보가 저장되는 디렉터리
run/	실행 중인 서비스와 관련된 파일이 저장되는 디렉터리
srv/	FTP 또는 web 등 시스템에서 제공하는 서비스 데이터가 저장되는 디렉터리
tmp/	시스템 사용 중 발생하는 임시 데이터가 저장되는 디렉터리
var/	시스템 운영 중에 발생하는 데이터와 로그 관련 정보가 저장되는 디렉터리
boot/	부팅에 필요한 커널 파일이 존재하는 디렉터리
etc/	리눅스 시스템 설정을 위한 각종 파일이 존재하는 디렉터리
lib@	공유 라이브러리가 존재하는 심볼릭 링크 파일
media/	USB, CD-ROM과 같은 외부 장치를 마운트(연결)할 때 사용되는 디렉터리
opt/	리눅스 시스템에 존재하지 않는 프로그램이 추가 설치되는 디렉터리
root/	root 계정의 홈 디렉터리로 '/'(루트) 디렉터리와는 성격이 다른 디렉터리
sbin@	시스템 운영관련 파일이 존재하는 심볼릭 링크 파일
sys/	리눅스 커널과 관련된 파일이 존재하는 디렉터리
usr/	기본 실행 파일과 라이브러리 파일, 헤더 파일 등의 파일이 존재하는 디렉터리

- 작업 디렉터리

- 사용자가 리눅스 시스템에 접속하여 현재 사용하고 있는 디렉터리를 의미
- 현재 작업 중인 디렉터리는 점(.)으로 표시되고 현재 작업 중인 디렉터리의 위치를 상세하게 확인하기 위해서는 pwd 명령을 사용

- 홈 디렉터리

- 각 사용자에게 할당되는 디렉터리로 사용자 계정을 처음 만들 때 지정
- 홈 디렉터리의 표시는 틸드(~)를 사용하여 guest_01 계정의 홈 디렉터리는 guest_01~과 같이 표시됨

절대경로와 상대경로

- 리눅스 시스템에서 제공하고 있는 경로는 특정파일이나 디렉터리의 위치를 나타내 줌
- 경로의 구분자는 슬래시(/)를 사용하여 구분되도록 표기
- 리눅스에서 제공되는 디렉터리의 구조는 트리 형태로 구성되어 있기 때문에 경로에 대한 이해 필요
- 경로 표기방법에는 절대경로와 상대경로 표기방법 제공

- 절대경로

- 루트 디렉터리인 /를 기준으로 제공되는 디렉터리를 의미
- 항상 루트(/) 디렉터리 위치부터 시작해야 하며 다음과 같은 특징이 있음

- 반드시 루트를 의미하는 '/'로 시작해야 함
- 루트 디렉터리부터 중간 단계의 디렉터리를 모두 표시해야 함
- 특정 위치를 가리키는 절대 경로명은 항상 동일해야 함

● 상대경로

- 절대경로인 루트(/)를 포함하지 않는 경로로 셸의 작업 디렉터리가 상대경로의 기준이 됨
- 현재 자신이 속해있는 디렉터리는 '.'으로 표현하고 바로 상위 디렉터리는 '..'으로 표현하며 다음과 같은 특징이 있음

- '/' 이외의 문자로 시작해야 함
- 현재 디렉터리의 위치에서 서브 디렉터리로 내려갈 경우 서브 디렉터리명을 추가
- 현재 디렉터리 위치에서 상위 디렉터리로 이동하려면 ..(마침표 두 개)를 추가
- 상대 경로명은 현재 위치한 디렉터리의 위치에 따라 달라질 수 있음

파일과 디렉터리명 생성 규칙

- 리눅스 시스템에서 파일명과 디렉터리명을 생성할 때는 다음과 같은 규칙을 준수해야 함

- '/' 는 파일명이나 디렉터리명에 사용할 수 없음
- 파일과 디렉터리 이름에는 알파벳, 숫자, -, _, .(마침표) 만 사용할 수 있음
- 공백, *, |, ", ', @, #, \$, %, ^, & 등은 사용할 수 없음
- 알파벳 대/소문자는 엄격하게 구별되어 다른 글자로 취급됨
- 파일과 디렉터리명을 선언할 때 점(.)으로 시작하면 숨김 파일로 간주됨

SECTION
04

디렉터리 관리 명령어

- 디렉터리 관리 명령어 종류와 기능 이해하기



1. 현재 디렉터리의 위치를 확인합니다.
2. 디렉터리의 위치 이동과 내용을 확인합니다.
3. 디렉터리의 생성과 삭제방법에 대해서 실습합니다.

현재 디렉터리의 위치 확인

- 현재 작업 중인 디렉터리의 위치를 확인하기 위해서는 print working directory의 약어인 pwd 명령을 사용

| 예제 3-2 |

리눅스 시스템에서의 현재 작업 중인 디렉터리의 위치를 확인합니다.

```
$ pwd
```

기능 현재 디렉터리의 위치를 절대경로로 보여줌

형식 pwd [Enter]



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~] $ pwd  
/home/cskisa  
[cskisa@localhost ~] $
```

[그림 3-6] 현재 디렉터리의 위치 확인

디렉터리의 위치 이동

- 현재 디렉터리 위치에서 다른 디렉터리의 위치로 이동하기 위해서는 change directory의 약어인 cd 명령을 사용

| 예제 3-3 |

리눅스 시스템의 현재 디렉터리에서 지정하는 특정 디렉터리 dev로 이동한 다음 이동한 위치를 확인합니다.

```
$ cd /dev
```

기능 현재 디렉터리의 위치에서 지정한 디렉터리 위치로 이동

형식 cd [디렉터리명]



```
cskisa@localhost:/dev
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ cd /dev
[cskisa@localhost dev]$ pwd
/dev
[cskisa@localhost dev]$
```

[그림 3-7] 절대경로명을 사용하여 디렉터리의 위치 이동

- 상대 경로명으로 디렉터리 이동하기

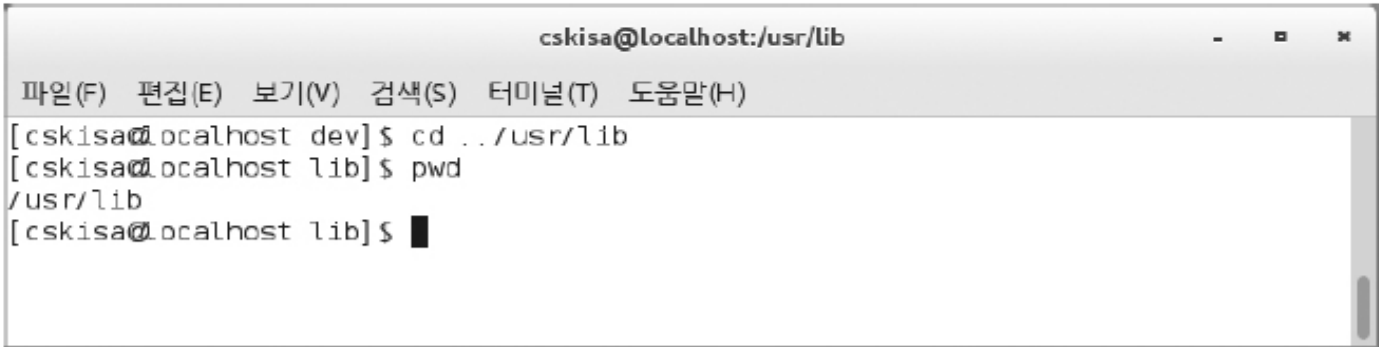
| 예제 3-4 |

터미널 창에서 `cd` 명령으로 `/usr/lib` 디렉터리로 상대 경로를 이용해 디렉터리의 위치를 이동한 다음 `pwd` 명령으로 이동된 현재 디렉터리의 위치에 대한 정보를 출력합니다.

```
$ cd ../usr/lib
```

기능 상대 경로명을 사용하여 현재 디렉터리의 위치에서 지정한 디렉터리 위치로 이동

형식 `cd [상대 경로명/디렉터리명]`



```
cskisa@localhost:/usr/lib
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost dev]$ cd ../usr/lib
[cskisa@localhost lib]$ pwd
/usr/lib
[cskisa@localhost lib]$
```

[그림 3-8] 상대 경로명을 사용하여 디렉터리의 위치 이동

- 홈 디렉터리로 이동하기

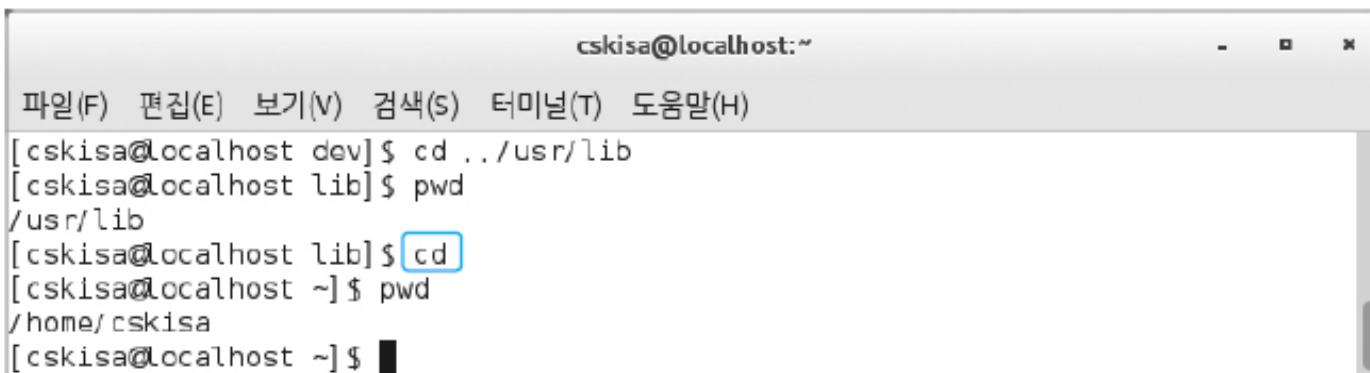
| 예제 3-5 |

현재 작업 중인 디렉터리의 위치에서 홈 디렉터리로 바로 이동합니다.

\$ cd

기능 해당 계정의 홈 디렉터리로 이동

형식 cd



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost dev]$ cd ../usr/lib  
[cskisa@localhost lib]$ pwd  
/usr/lib  
[cskisa@localhost lib]$ cd  
[cskisa@localhost ~]$ pwd  
/home/cskisa  
[cskisa@localhost ~]$
```

[그림 3-9] cd 명령을 사용하여 홈 디렉터리로 이동

- 홈 디렉터리로 이동하기

- `cd ~` : 홈 디렉터리를 나타내는 기호인 '~'를 사용해서 홈 디렉터리로 이동
- `cd` : 이동할 목적지를 지정하고 않고 해당 계정의 홈 디렉터리로 이동
- `cd /home/cskisa` : 절대 경로명을 사용하여 홈 디렉터리로 이동
- `cd ../../home/cskia` : 상대 경로명을 사용하여 홈 디렉터리로 이동

■ 디렉터리 내용 확인

- List의 약어인 ls 명령으로 디렉터리의 내용 확인

\$ ls

기능 디렉터리에 존재하는 파일이나 서브 디렉터리에 대한 정보 확인

형식 ls [옵션] [파일명 또는 디렉터리명]

옵션 -a : 숨겨진 파일까지 포함하여 모든 파일 리스트를 출력

-d : 지정한 디렉터리에 존재하는 파일과 디렉터리의 정보 출력

-n : 호스트네임 확인

-i : 첫 번째 열에 I-node 번호를 출력

-l : 파일의 상세한 정보를 출력

-A : .와 ..를 제외한 모든 파일 리스트를 출력

-F : 파일의 종류를 실행파일은 '*', 디렉터리는 '/', 심볼릭 링크는 '@' 표시

-L : 심볼릭 링크 파일은 원본파일의 정보를 출력

-R : 하위 디렉터리 리스트 출력

- 현재 디렉터리의 내용 확인 : ls

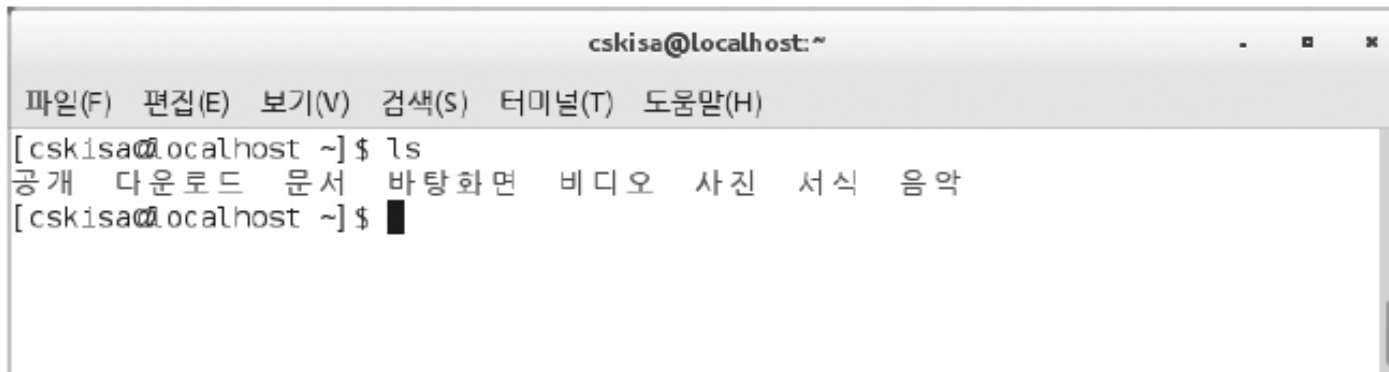
| 예제 3-6 |

현재 디렉터리에 존재하는 디렉터리와 파일의 종류를 확인합니다.

```
$ ls
```

기능 현재 디렉터리의 내용 확인

형식 ls



A terminal window titled 'cskisa@localhost: ~' with standard window controls. The menu bar includes '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The command prompt shows '[cskisa@localhost ~]\$ ls' followed by the output: '공개 다운로드 문서 바탕화면 비디오 사진 서식 음악'. The prompt then returns to '[cskisa@localhost ~]\$' with a cursor.

[그림 3-10] ls 명령으로 디렉터리의 내용 확인

- 숨김 파일 확인 : -a 옵션

| 예제 3-7 |

현재 디렉터리에 존재하는 파일과 디렉터리 목록뿐만 아니라 숨김 파일까지도 모두 출력합니다.

```
$ ls -a
```

기능 현재 디렉터리의 숨김 파일 확인

형식 `ls [옵션]`

```
cskisa@localhost: ~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls -a
.          .bash_history  .bashrc  .esd_auth  .mozilla  문서      사진
..         .bash_logout   .cache   .kde       공개      바탕화면  서식
.ICEauthority .bash_profile .config  .local     다운로드  비디오    음악
[cskisa@localhost ~]$
```

[그림 3-11] 숨김 파일 확인

- 파일의 종류 확인 : -F 옵션

- 파일명 뒤에 /는 디렉터리 파일, @는 심볼릭 링크, *는 실행 파일, 아무런 표시가 없으면 일반 파일을 의미

| 예제 3-8 |

- **Step 01** | 현재 디렉터리에 존재하는 파일의 종류와 기호가 함께 표시되도록 파일의 종류를 확인합니다.

```
$ ls -F
```

기능 파일의 종류가 기호와 함께 표시되어 파일의 종류 확인

형식 `ls [옵션]`

- **Step 02** | 현재 디렉터리에 존재하는 파일의 종류와 숨김 파일까지 포함하여 모든 파일의 종류를 한꺼번에 확인합니다.

```
$ ls -aF
```

기능 숨김 파일까지 함께 모든 파일의 종류를 확인

형식 `ls [옵션][옵션]`

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls -F
공개/  다운로드/  문서/  바탕화면/  비디오/  사진/  서식/  음악/
[cskisa@localhost ~]$ ls -aF
./      .bash_history  .bashrc  .esd_auth  .mozilla/  문서/  사진/
../     .bash_logout  .cache/  .kde/      공개/      바탕화면/  서식/
.ICEauthority  .bash_profile .config/ .local/   다운로드/  비디오/  음악/
[cskisa@localhost ~]$
  
```

[그림 3-12] 숨김 파일을 포함한 모든 파일의 종류 확인

- 지정한 디렉터리의 내용 확인

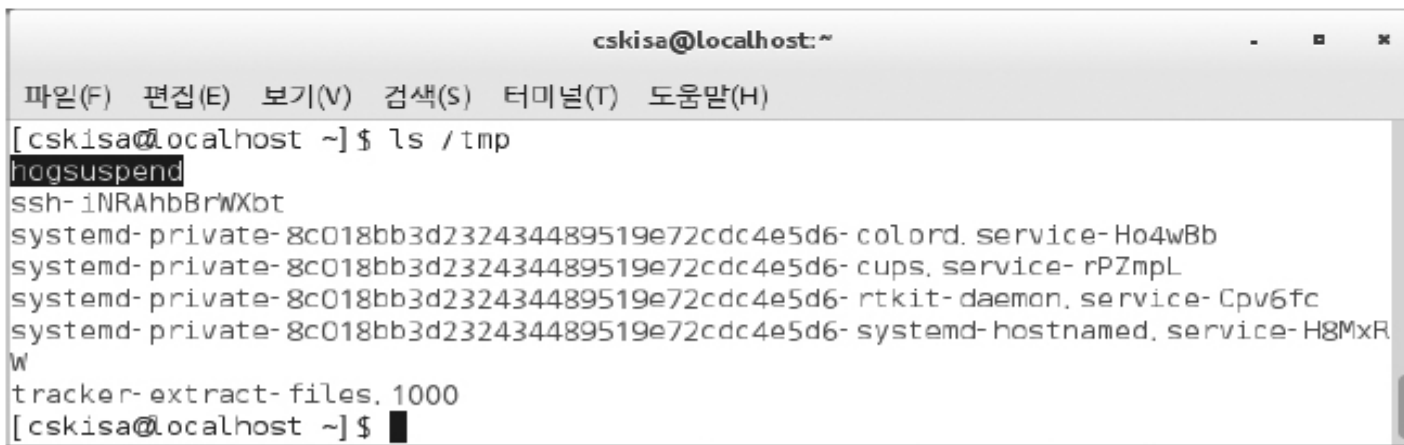
| 예제 3-9 |

tmp 디렉터리에 존재하는 파일과 서브 디렉터리의 종류를 출력합니다.

```
$ ls /tmp
```

기능 다른 위치에 있는 디렉터리의 내용 확인

형식 `ls [디렉터리명]`



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls /tmp  
hogsuspend  
ssh-1NRAhbBrWXot  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-color.service-Ho4wBb  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-cups.service-rPZmpl  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-rtkit-daemon.service-Cpv6fc  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-systemd-hostnamed.service-H8MxRW  
tracker-extract-files.1000  
[cskisa@localhost ~]$
```

[그림 3-13] /tmp 디렉터리에 있는 내용 확인

- 현재 디렉터리에서 특정 디렉터리에 존재하는 목록 확인하기

| 예제 3-10 |

현재 디렉터리의 위치에서 tmp 디렉터리에 존재하는 파일과 서브 디렉터리의 리스트를 출력합니다.

```
$ ls -F /tmp
```

기능 다른 위치에 있는 디렉터리의 내용 확인

형식 `ls [옵션] [디렉터리명]`



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls -F /tmp  
hogsuspend/  
ssh-1NRAhbBrWXbt/  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-color.service-Ho4wBb/  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-cups.service-rPZmpl/  
systemd-private-8c018bb3d232434489519e72cdc4e5d6-rtkit-daemon.service-Cpv6fc/  
tracker-extract-files.1000/  
[cskisa@localhost ~]$
```

[그림 3-14] /tmp 디렉터리에 있는 파일의 종류와 내용 확인

- 디렉터리에 존재하는 파일의 상세정보 출력 : -l 옵션

| 예제 3-11 |

현재 디렉터리에 존재하는 모든 파일의 상세한 정보를 출력합니다.

```
$ ls -l
```

기능 디렉터리에 존재하는 파일의 상세정보 출력

형식 ls [옵션] [Enter]

```

cskisa@localhost: ~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls -l
합계 4
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 공  개
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 다운로드
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 문서
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 바탕화면
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 비디오
drwxr-xr-x. 2 cskisa cskisa 4096 10월  2 08:50 사진
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 서식
drwxr-xr-x. 2 cskisa cskisa  6 10월  1 21:58 음악
[cskisa@localhost ~]$
  
```

[그림 3-15] 디렉터리에 존재하는 파일의 상세정보 출력

- 디렉터리에 존재하는 파일의 상세정보 출력 : -l 옵션

[표 3-2] 파일에 대한 상세정보

구분	의미
d	파일의 유형 : [표 3-3 참조]
rwxr-xr-x	파일 접근권한 (rwx : 사용권한, r-x : 그룹권한, r-x : 타인권한)
2	하드 링크의 개수
cskisa	파일 소유자
cskisa	파일이 속한 그룹
6	파일의 크기 (byte 단위)
10월1 21:58	파일이 마지막으로 수정된 날짜와 시간
공개	파일 이름

- 디렉터리에 존재하는 파일의 상세정보 출력 : -l 옵션

[표 3-3] 파일의 종류

문자	파일 종류
-	일반 정규적인 파일
d	디렉터리 파일
l	심볼릭 링크 파일
b	블록 단위의 Read/Write 블록장치 파일
c	섹터 단위의 Read/Write 문자장치 파일
p	프로세스 간 통신에 사용되는 특수 파일 (파이프 파일)
s	네트워크 통신에 사용되는 특수 파일 (소켓 파일)

- 디렉터리의 자체정보 확인 : -l 옵션

| 예제 3-12 |

루트 디렉터리에 존재하는 모든 파일의 상세한 정보를 출력합니다.

```
$ ls -l /
```

기능 루트 디렉터리에 존재하는 파일의 정보 확인

형식 `ls [옵션] [디렉터리]`

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls -l /
합계 32
lrwxrwxrwx.  1 root root    7 10월  1 19:54 bin -> usr/bin
dr-xr-xr-x.  4 root root 4096 10월  1 23:42 boot
drwxr-xr-x. 19 root root 3100 11월  6 11:59 dev
drwxr-xr-x. 137 root root 8192 11월  6 11:58 etc
drwxr-xr-x.  3 root root   19 10월  1 21:58 home
lrwxrwxrwx.  1 root root    7 10월  1 19:54 lib -> usr/lib
lrwxrwxrwx.  1 root root    9 10월  1 19:54 lib64 -> usr/lib64
drwxr-xr-x.  2 root root    6  8월 12 2015 media
drwxr-xr-x.  2 root root    6  8월 12 2015 mnt
drwxr-xr-x.  3 root root   15 10월  1 20:01 opt
dr-xr-xr-x. 178 root root    0 11월  6 11:58 proc
  
```

[그림 3-16] 루트 디렉터리에 존재하는 파일의 정보 확인

- 디렉터리의 자체정보 확인 : -ld 옵션

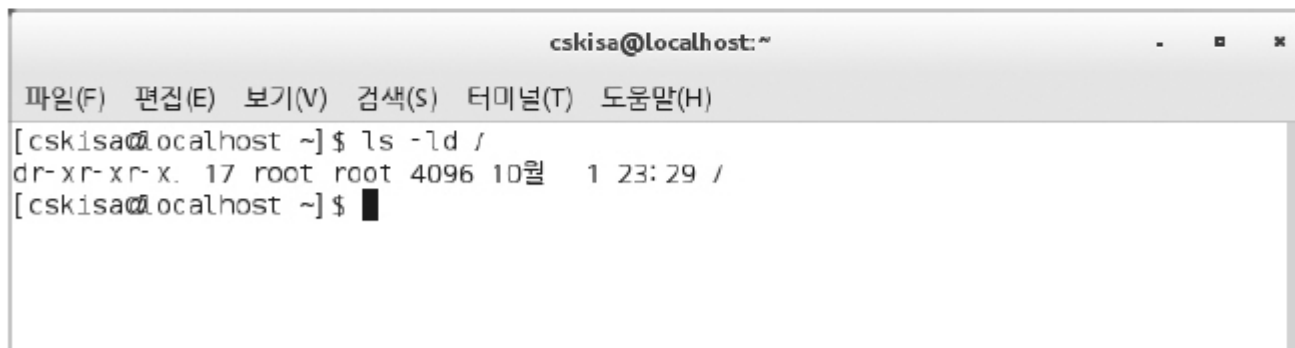
| 예제 3-13 |

루트 디렉터리의 자체정보를 ls 명령과 옵션 -ld를 함께 선언하여 출력합니다.

```
$ ls -ld /
```

기능 루트 디렉터리의 자체정보 확인

형식 ls [옵션][옵션] [디렉터리]



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls -ld /  
dr-xr-xr-x. 17 root root 4096 10월  1 23:29 /  
[cskisa@localhost ~]$
```

[그림 3-17] 루트 디렉터리의 자체정보 확인

- l-node 번호 확인 : -i 옵션

| 예제 3-14 |

현재 디렉터리에 존재하는 파일에 부여된 I-node 번호를 출력합니다.

```
$ ls -i
```

기능 파일에 각각 부여된 I-node 번호 확인

형식 ls [옵션]



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls -i  
72476995 공개 105102042 문서 72476996 비디오 34586232 서식  
3116515 다운로드 105102041 바탕화면 34586234 사진 3116516 음악  
[cskisa@localhost ~]$
```

[그림 3-18] I-node 번호 확인

- 심볼릭 링크 명령 : `dir`, `vdir`

| 예제 3-15 |

`ls` 명령에 대한 심볼릭 링크를 출력하기 위해 `dir` 명령을 사용하고 `ls -l` 명령에 대한 심볼릭 링크를 출력하기 위해 `vdir` 명령을 사용합니다.

```
$ dir
```

기능 `ls` 명령에 대한 심볼릭 링크 명령

형식 `dir`

```
$ vdir
```

기능 `ls -l` 명령에 대한 심볼릭 링크 명령

형식 `vdir`



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ dir  
공개 다운로드 문서 바탕화면 비디오 사진 서식 음악  
[cskisa@localhost ~]$ vdir  
합계 4  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 공개  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 다운로드  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 문서  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 바탕화면  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 비디오  
drwxr-xr-x. 2 cskisa cskisa 4096 10월 2 08:50 사진  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 서식  
drwxr-xr-x. 2 cskisa cskisa 6 10월 1 21:58 음악  
[cskisa@localhost ~]$
```

[그림 3-19] dir과 vdir 심볼릭 링크 명령 수행결과

- 해당파일 존재 유무 : ls [파일명]

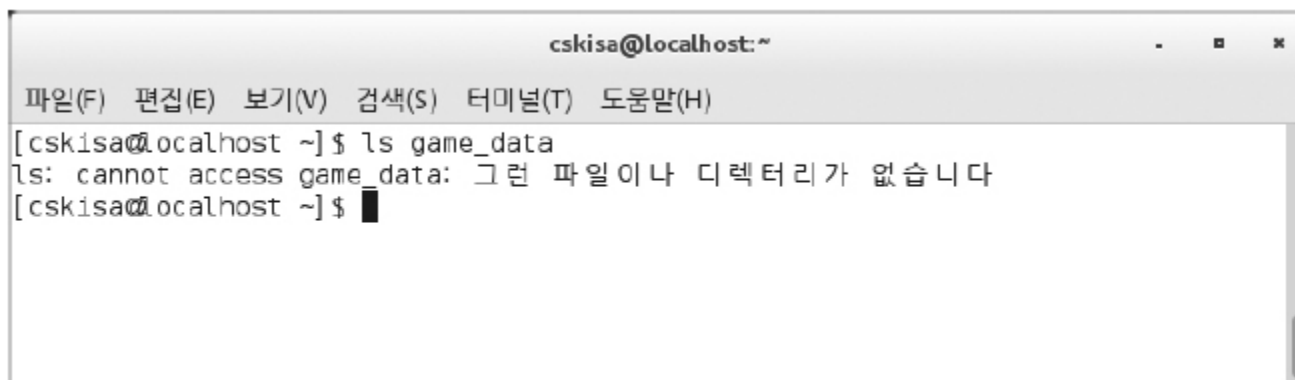
| 예제 3-16 |

리눅스 시스템에 game_data 파일이 존재하는지를 확인합니다.

```
$ ls game_data
```

기능 해당파일이 존재하는지 확인

형식 ls [파일명] [Enter]



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls game_data  
ls: cannot access game_data: 그런 파일이나 디렉터리가 없습니다  
[cskisa@localhost ~]$
```

[그림 3-20] game_data 파일이 존재하는지 확인

■ 디렉터리 생성

- Make directory의 약어인 mkdir 명령으로 디렉터리 생성

```
$ mkdir
```

기능 새로운 디렉터리 생성

형식 mkdir [옵션] 생성할 디렉터리명

옵션 -p : 하위 디렉터를 계층적으로 생성할 때 중간단계의 디렉터리가 없으면 자동으로 중간단계 디렉터를 생성하면서 전체 디렉터를 생성

- 새로운 디렉터리 한 개 생성하기

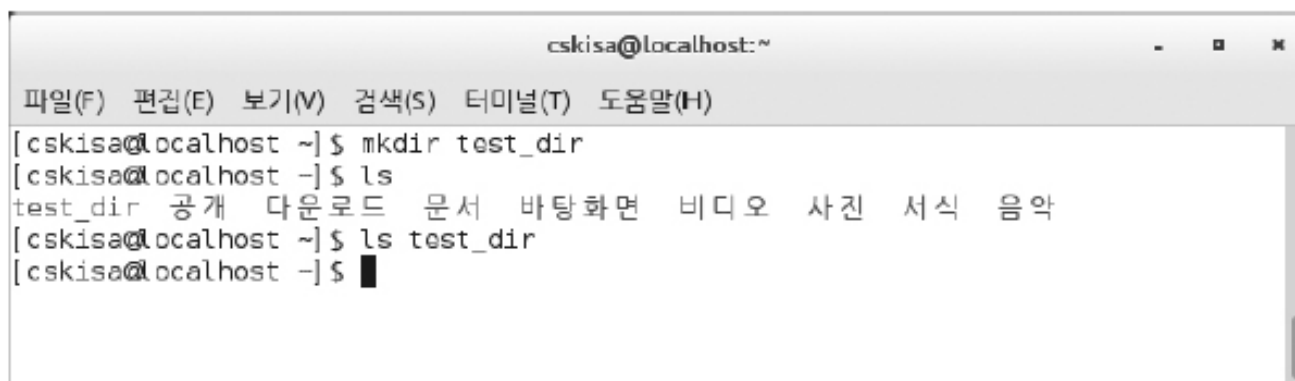
| 예제 3-17 |

현재 디렉터리의 위치에서 test_dir 디렉터를 생성한 다음 test_dir 디렉터리가 제대로 생성되어 존재하는지에 대해 확인합니다.

```
$ mkdir test_dir
```

기능 새로운 test_dir 디렉터리 생성

형식 mkdir 디렉터리명



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ mkdir test_dir  
[cskisa@localhost ~]$ ls  
test_dir  공개  다운로드  문서  바탕화면  비디오  사진  서식  음악  
[cskisa@localhost ~]$ ls test_dir  
[cskisa@localhost ~]$
```

[그림 3-21] test_dir 디렉터리 생성 후 존재여부 확인

- 새로운 디렉터리 한 번에 여러 개 생성하기

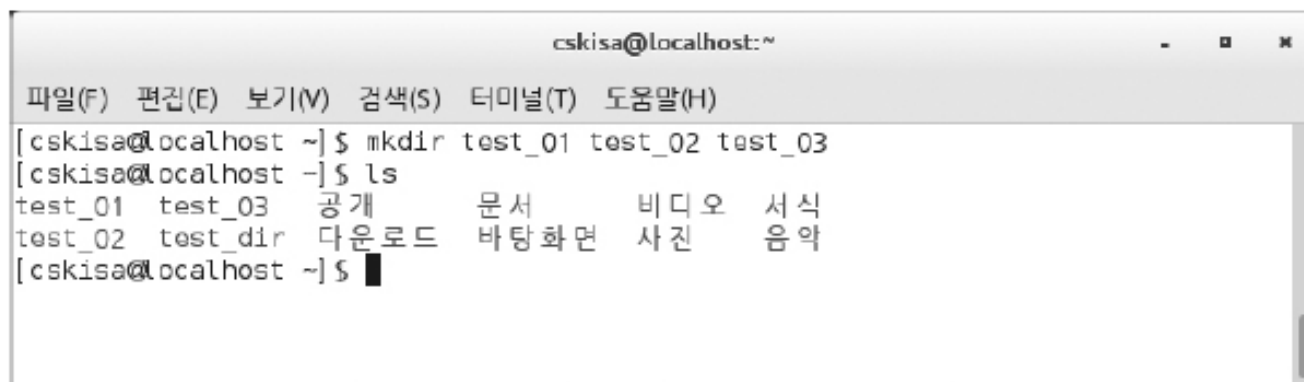
| 예제 3-18 |

현재 디렉터리의 위치에 mkdir 명령으로 test_01, test_02, test_03 디렉터를 새롭게 생성한 다음 ls 명령으로 생성된 디렉터리가 존재하는지에 대해 확인합니다.

```
$ mkdir test_01 test_02 test_03
```

기능 한 번에 세 개의 새로운 디렉터리 test_01, test_02, test_03 디렉터리 생성

형식 mkdir 디렉터리명1 디렉터리명2 디렉터리명3



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ mkdir test_01 test_02 test_03  
[cskisa@localhost ~]$ ls  
test_01 test_03 공개 문서 비디오키 서식  
test_02 test_dir 다운로드 바탕화면 사진 음악  
[cskisa@localhost ~]$
```

[그림 3-22] 한 번에 세 개의 디렉터리 생성 후 존재여부 확인

- 중간 디렉터리 자동으로 생성하기 : -p 옵션

| 예제 3-19 |

중간 디렉터리의 구조를 갖는 tmp/data/guest 디렉터를 생성하기 위해 mkdir 명령과 옵션 -p를 사용하여 새로운 계층형 디렉터를 생성합니다.

```
$ mkdir -p tmp/data/guest
```

기능 계층적 디렉터리 tmp/data/guest 생성

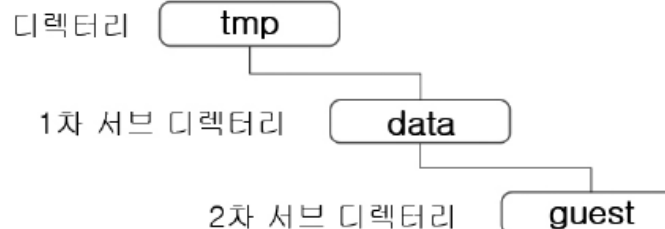
형식 mkdir [옵션] 계층적 디렉터리명

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ mkdir -p tmp/data/guest
[cskisa@localhost ~]$ ls
test_01 test_03 tmp 다운로드 바탕화면 사진 음악
test_02 test_dir 공개 문서 비디오 서식
[cskisa@localhost ~]$ ls -R tmp
tmp:
data
tmp/data:
guest
tmp/data/guest:
[cskisa@localhost ~]$

```

[그림 3-23] 계층적 디렉터리 tmp/data/guest 생성 후 확인



[그림 3-24] 계층적 디렉터리 생성 구조

■ 디렉터리 삭제

- Remove directory의 약어인 rmdir 명령으로 디렉터리 삭제

```
$ rmdir
```

기능 기존에 존재하는 디렉터리 삭제

형식 `rmdir` [옵션] 삭제할 디렉터리명

옵션 `-p`: 삭제하고자 하는 디렉터리를 삭제할 때 부모 디렉터리가 빈 디렉터리로 존재할 경우에는 부모 디렉터리도 자동으로 삭제

- test_dir 디렉터리 삭제

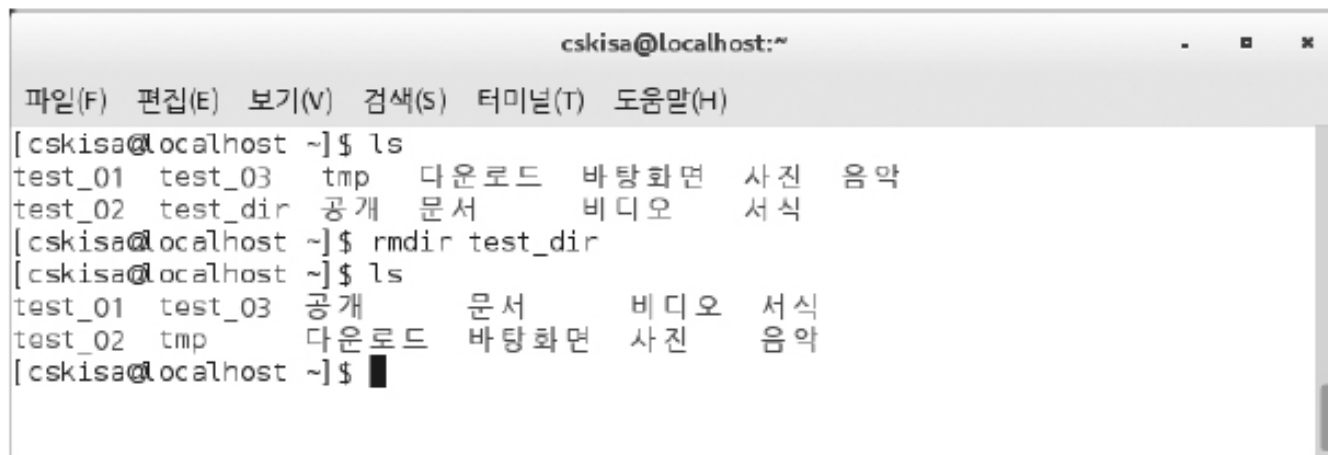
| 예제 3-20 |

현재 디렉터리에 존재하는 test_dir 디렉터를 삭제합니다.

```
$ rmdir test_dir
```

기능 기존에 존재하는 디렉터리 삭제

형식 rmdir 디렉터리명



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls  
test_01 test_03 tmp 다운로드 바탕화면 사진 음악  
test_02 test_dir 공개 문서 비디오 서식  
[cskisa@localhost ~]$ rmdir test_dir  
[cskisa@localhost ~]$ ls  
test_01 test_03 공개 문서 비디오 서식  
test_02 tmp 다운로드 바탕화면 사진 음악  
[cskisa@localhost ~]$
```

[그림 3-25] 디렉터리 존재 확인 후 test_dir 디렉터리 삭제

- 비어있지 않은 디렉터리 삭제하기 : rm 명령과 -r 옵션

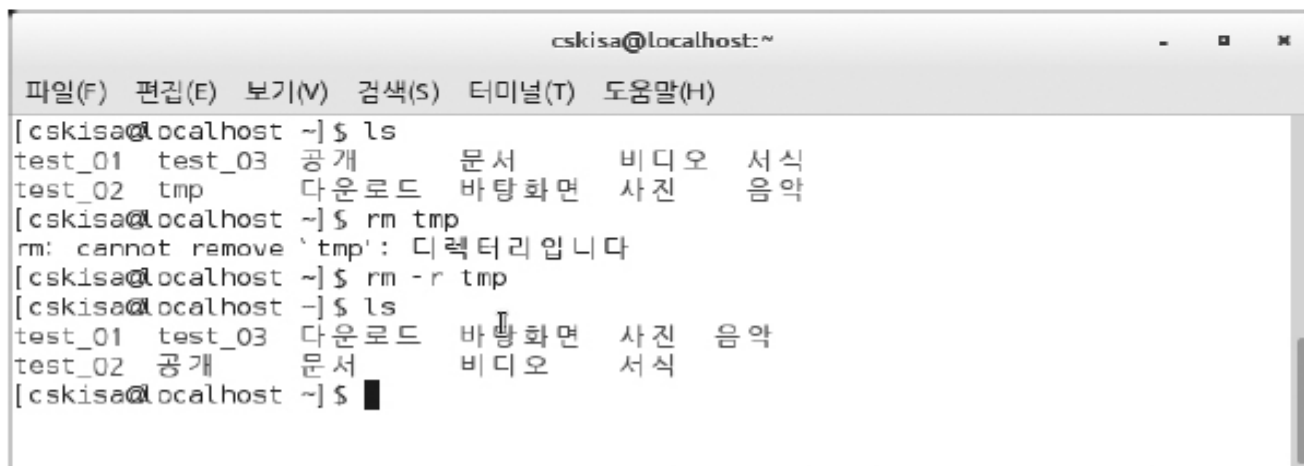
| 예제 3-21 |

이번 예제에서는 비어있지 않은 디렉터리 tmp/data/guest를 한꺼번에 삭제하기 위해 rm명령과 함께 옵션 -r을 사용합니다.

```
$ rm -r tmp
```

기능 비어있지 않은 디렉터리를 한꺼번에 삭제

형식 rm [옵션] 디렉터리명



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ ls  
test_01 test_03 공개 문서 비디오 서식  
test_02 tmp 다운로드 바탕 화면 사진 음악  
[cskisa@localhost ~]$ rm tmp  
rm: cannot remove 'tmp': 디렉터리입니다  
[cskisa@localhost ~]$ rm -r tmp  
[cskisa@localhost ~]$ ls  
test_01 test_03 다운로드 바탕 화면 사진 음악  
test_02 공개 문서 비디오 서식  
[cskisa@localhost ~]$
```

[그림 3-26] 비어있지 않은 tmp 디렉터리 삭제

SECTION
05

파일 관리 명령어

- 파일내용과 파일관리에 유용한 명령어 이해하기



1. 특정 파일의 내용을 확인합니다.
2. 파일의 복사, 이동, 삭제에 대해서 실습합니다..
3. 파일 링크와 파일관리에 유용한 명령에 대해 학습합니다.

파일 생성

- 파일을 생성할 때는 touch 명령을 사용하고 파일의 내용을 검색할 때는 grep 명령을 사용
- 이미 생성된 파일의 위치를 찾기 위해서는 find 명령을 사용하고
- 특정 명령이 어느 디렉터리에 존재하는지를 알기 위해서는 whereis와 which 명령을 사용

- 실습을 위한 디렉터리 생성 : mkdir [디렉터리명]

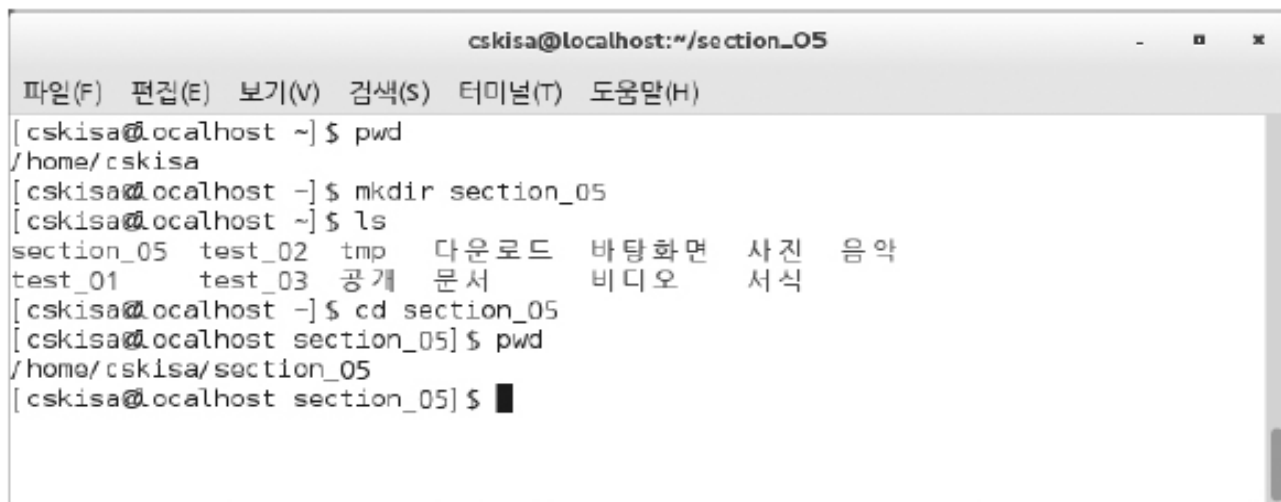
| 예제 3-22 |

앞으로의 실습을 위해 section_05 디렉터를 현재의 위치에 생성합니다.

```
$ mkdir section_05
```

기능 새로운 section_05 디렉터리 생성

형식 mkdir 디렉터리명



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ pwd
/home/cskisa
[cskisa@localhost ~]$ mkdir section_05
[cskisa@localhost ~]$ ls
section_05  test_02  tmp      다운로드  바탕화면  사진      음악
test_01     test_03  공개     문서       비디오    서식
[cskisa@localhost ~]$ cd section_05
[cskisa@localhost section_05]$ pwd
/home/cskisa/section_05
[cskisa@localhost section_05]$
```

[그림 3-27] 새 디렉터리 section_05 생성

- 빈 파일 생성 : touch

```
$ touch
```

형식 touch [-acm] [r ref_file | -t time]

옵션 -a : 최근에 작업한 파일의 접근시간만 변경 적용

-c : 존재하지 않는 파일은 새로 만들지 않음

-m : 최근에 작업한 파일의 수정시간만 변경 적용

-t [[CC]YY]MMDDhhmm[.ss] : 시간을 직접 입력

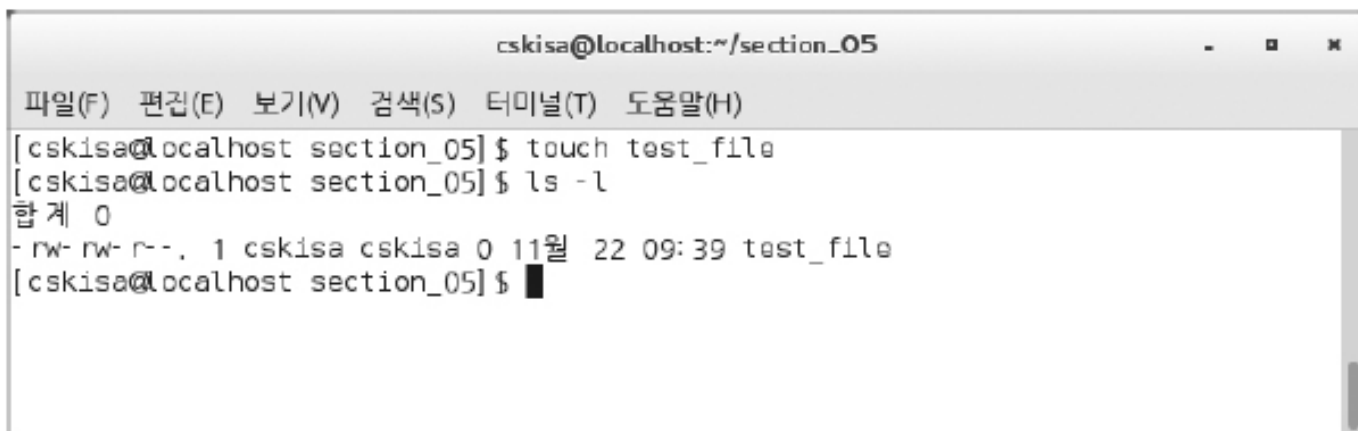
| 예제 3-23 |

새로운 빈 파일을 생성하기 위해 touch 명령으로 비어있는 text_file을 새로 생성합니다.

```
$ touch test_file
```

기능 새로운 test_file 파일 생성

형식 touch 파일명



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ touch test_file
[cskisa@localhost section_05]$ ls -l
합계 0
-rw-rw-r--. 1 cskisa cskisa 0 11월 22 09:39 test_file
[cskisa@localhost section_05]$
```

[그림 3-28] 새로운 파일 test_file 생성

| 예제 3-24 |

리눅스 시스템의 현재 시각을 확인한 다음 이미 존재하는 test_file 파일의 최종 수정시각을 시스템의 현재 시각으로 변경하기 위해 touch 명령을 사용합니다.

```
$ date
```

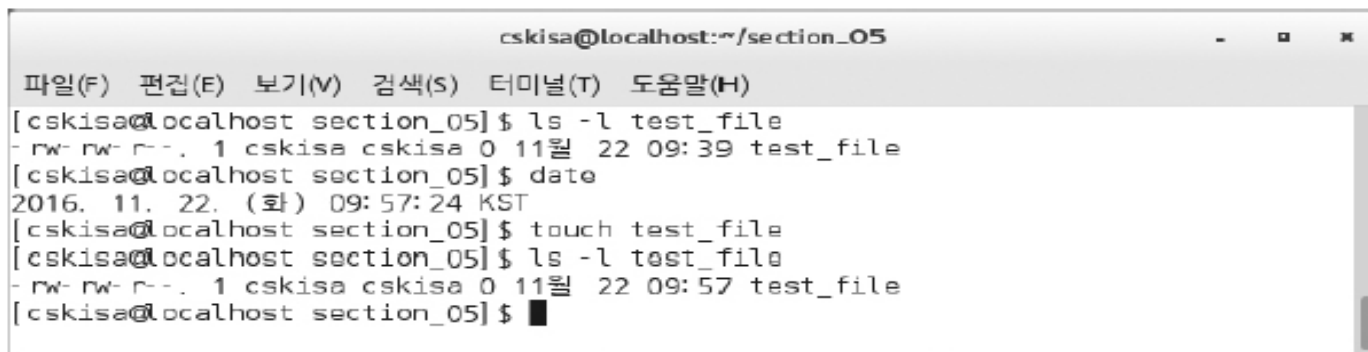
기능 시스템의 현재시각 확인

형식 date [Enter]

```
$ touch test_file
```

기능 이미 존재하는 test_file 파일의 수정시각을 현재시각으로 변경

형식 touch 파일명 [Enter]



```

cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 11월 22 09:39 test_file
[cskisa@localhost section_05]$ date
2016. 11. 22. (화) 09:57:24 KST
[cskisa@localhost section_05]$ touch test_file
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 11월 22 09:57 test_file
[cskisa@localhost section_05]$
  
```

[그림 3-29] test_file 파일의 수정시간을 현재시각으로 변경

● 파일 생성 시각 설정 : -t 옵션

```
$ touch -t [[CC]YY]MMDDhhmm[.ss]
```

기능 파일에 시각을 지정하여 표시

형식 touch [옵션] [시각표시] Enter ↵

옵션 -CC : 연도의 첫 두 자리

-YY : 연도의 마지막 두 자리

-MM : 01~12 월 지정

-DD : 01~31 일 지정

-hh : 00~23 시간 지정

-mm : 00~59 분 지정

-.ss : 00~59 초 지정 (반드시 앞에 점(.)이 삽입되어야 함)

[표 3-4] YY값에 따른 CC의 연도지정 옵션

옵션	연도 지정		
YY	69~99	00~38	39~68
CC	19	20	ERROR

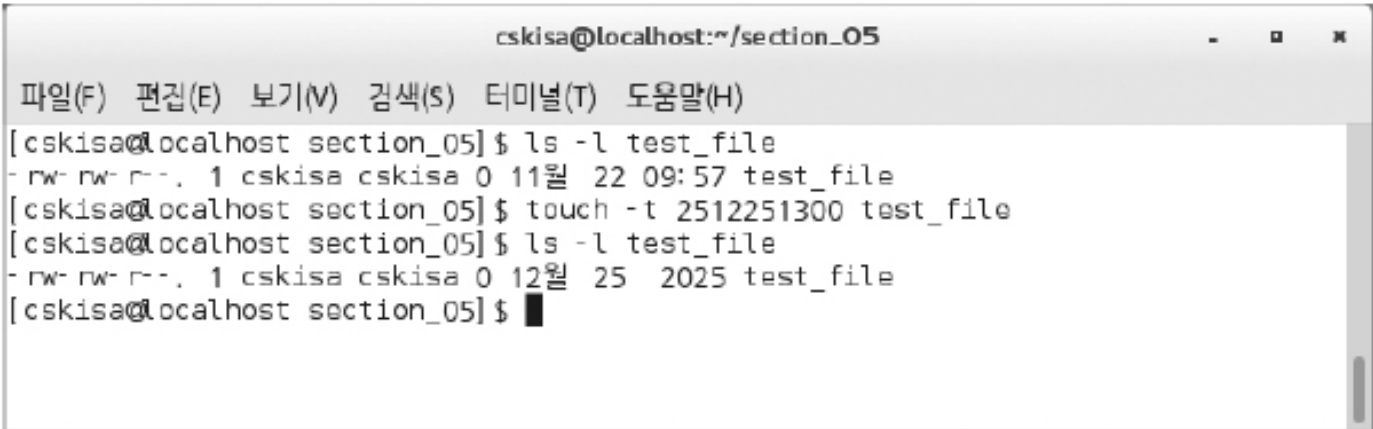
| 예제 3-25 |

현재 디렉터리에 존재하는 test_file의 최종 수정시각을 2025년 12월 25일 13:00분으로 변경합니다.

```
$ touch -t 2512251300 test_file
```

기능 이미 존재하는 test_file 파일의 수정시각을 2025년 12월 25일 13:00분으로 변경

형식 touch [옵션] [시간표시]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 11월 22 09:57 test_file
[cskisa@localhost section_05]$ touch -t 2512251300 test_file
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 12월 25 2025 test_file
[cskisa@localhost section_05]$
```

[그림 3-30] test_file의 시간설정 (2025년 12월 25일 13:00)

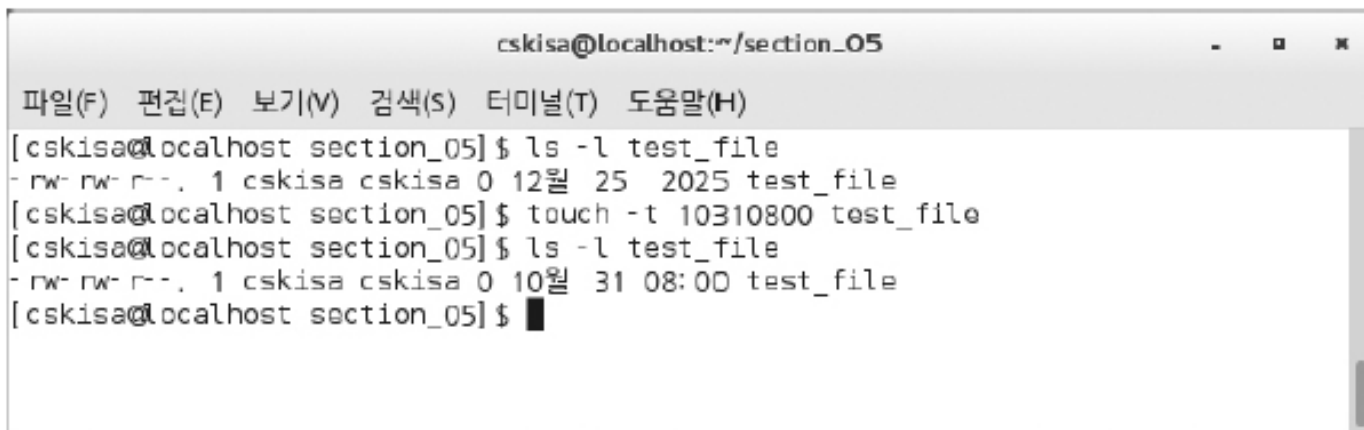
| 예제 3-26 |

test_file의 최종수정 시각을 연도지정을 제외하고 10월 31일 08:00분으로 변경합니다.

```
$ touch -t 10310800 test_file
```

기능 이미 존재하는 test_test 파일의 수정시각을 10월 31일 08:00분으로 변경

형식 touch [옵션] [시간표시]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 12월 25 2025 test_file
[cskisa@localhost section_05]$ touch -t 10310800 test_file
[cskisa@localhost section_05]$ ls -l test_file
-rw-rw-r--. 1 cskisa cskisa 0 10월 31 08:00 test_file
[cskisa@localhost section_05]$
```

[그림 3-31] test_file의 시간설정 (10월 31일 08:00)

파일 복사

- Copy의 약어인 cp 명령을 사용하여 파일이나 디렉터리를 복사

\$ cp

기능 디렉터리 또는 파일을 복사

형식 cp [옵션] [첫 번째 인자] [두 번째 인자]

옵션 -i : 복사할 파일명 또는 디렉터리명이 존재하면 덮어쓸 것인지를 물어보는 방법

-r : 디렉터리를 복사할 때 사용

[표 3-5] cp 명령 옵션의 기능

옵션	기능
-a, -dpR	원본파일의 속성과 링크 정보를 그대로 유지하면서 복사
-b	원본파일의 백업본을 생성
-d	원본파일이 심볼릭 링크된 파일일 경우에는 그 자체까지 복사
-f	복사하고자 하는 파일 이름이 겹칠 경우에는 강제 복사
-i	복사할 파일이름이 겹치면 물어보는 프롬프트 출력
-l	하드 링크 형식으로 복사
-P	원본파일의 경로가 지정되어 있을 경우 그대로 복사
-p	원본파일의 소유주, 그룹 소유주, 퍼미션, 시간 등을 동일하게 복사
-r	디렉터리 안에 있는 모든 하위 디렉터리 파일까지 복사
-s	심볼릭 링크를 생성
-u	복사되는 파일이 원본과 변경 날짜가 같거나 더욱 최신이면 복사하지 않음
-v	복사할 원본파일 이름과 복사되는 파일이름을 보여줌
-x	원본파일과 복사되는 파일의 시스템이 다르면 복사하지 않음
--help	cp 명령 사용에 대한 도움말
--version	cp 명령의 버전 확인

- 원본파일을 파일로 복사할 경우

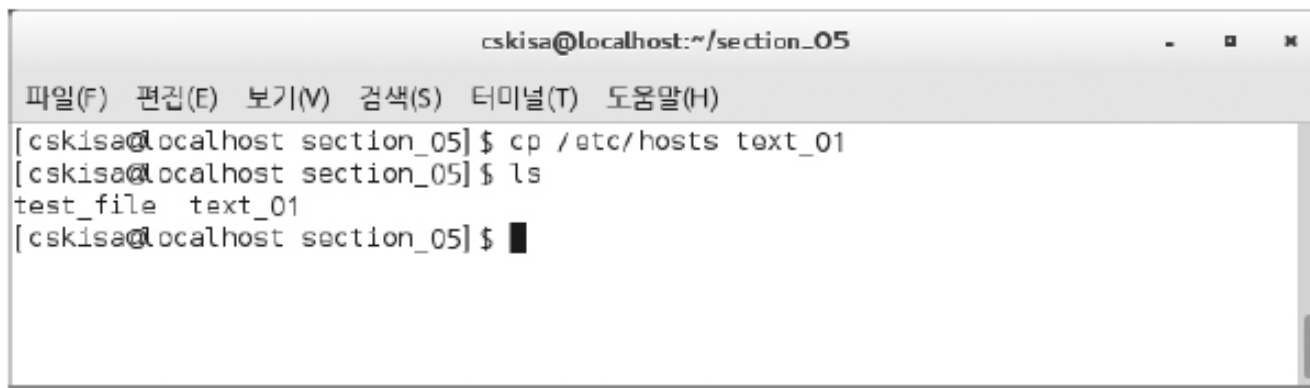
| 예제 3-27 |

etc 디렉터리에 존재하는 기존 파일 hosts를 text_01 파일명으로 복사한 다음 ls 명령으로 복사된 파일이 존재하는지에 대해 확인합니다.

```
$ cp /etc/hosts text_01
```

기능 원본파일 hosts를 text_01 파일로 복사

형식 cp 원본파일이 존재하는 [디렉터리명/파일명] 복사할 [파일명] [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ cp /etc/hosts text_01
[cskisa@localhost section_05]$ ls
test_file  text_01
[cskisa@localhost section_05]$
```

[그림 3-33] etc/hosts 원본파일을 복사한 text_01 파일

- 원본파일을 디렉터리로 복사할 경우

| 예제 3-28 |

현재 디렉터리에 새로운 temp 디렉터를 생성한 다음 현재 디렉터리에 존재하는 파일 text_01을 temp 디렉터리에 복사하고 ls 명령으로 temp 디렉터리에 복사한 파일이 존재하는지에 대해 확인합니다.

```
$ mkdir temp
```

기능 새로운 temp 디렉터리 생성

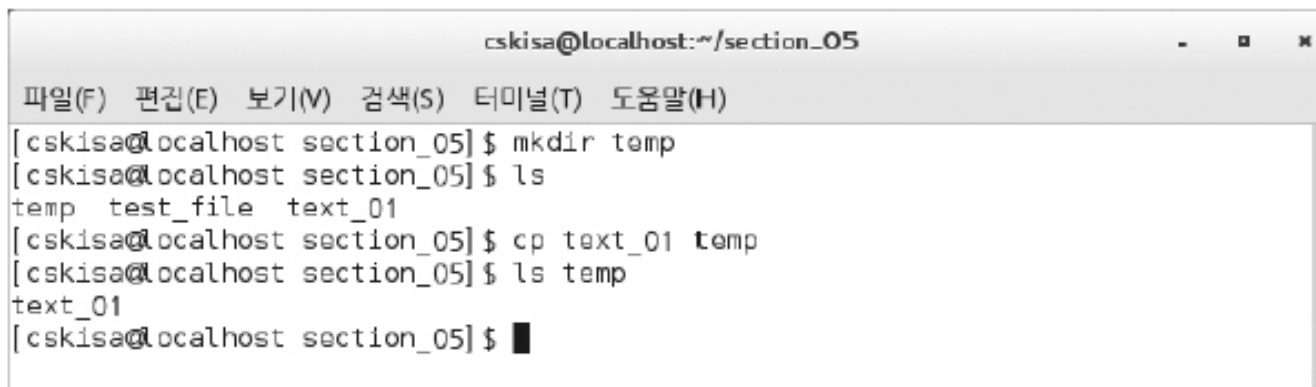
형식 mkdir [디렉터리명]

- 원본파일을 디렉터리로 복사할 경우

```
$ cp text_01 temp
```

기능 text_01 파일을 temp 디렉터리로 복사

형식 cp [복사할 원본파일] [복사파일을 저장할 디렉터리명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ mkdir temp
[cskisa@localhost section_05]$ ls
temp test_file text_01
[cskisa@localhost section_05]$ cp text_01 temp
[cskisa@localhost section_05]$ ls temp
text_01
[cskisa@localhost section_05]$
```

[그림 3-34] temp 디렉터리에 text_01 파일 복사

- 원본파일을 디렉터리로 복사할 경우

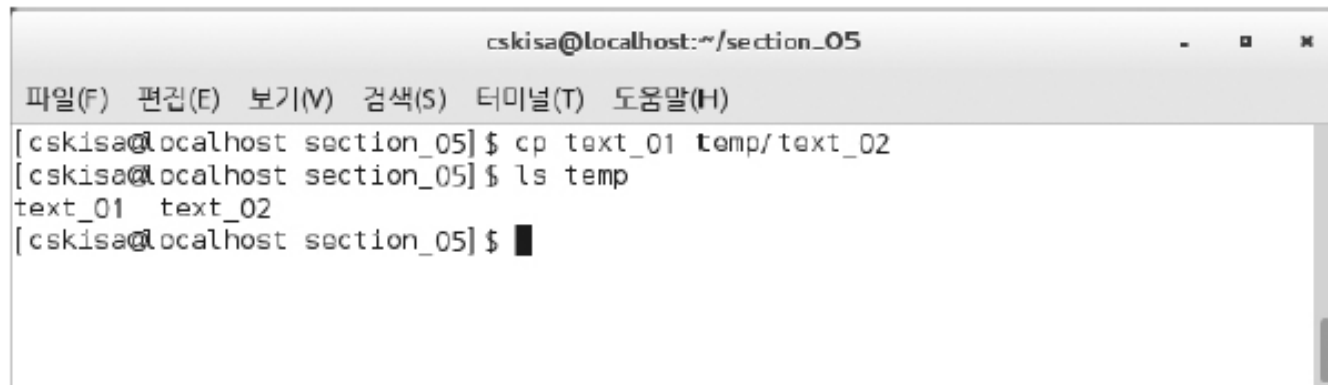
| 예제 3-29 |

현재 디렉터리에 존재하는 text_01 파일을 temp 디렉터리에 text_02 파일명으로 복사한 다음 복사된 text_02 파일이 temp 디렉터리에 존재하는지에 대해 확인합니다.

```
$ cp text_01 temp/text_02
```

기능 원본파일 text_01을 temp 디렉터리에 text_02 파일명으로 복사

형식 cp [원본파일] [디렉터리명/복사될 파일명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05] $ cp text_01 temp/text_02
[cskisa@localhost section_05] $ ls temp
text_01  text_02
[cskisa@localhost section_05] $
```

[그림 3-35] 원본파일 text_01을 temp 디렉터리에 text_02 파일명으로 복사

- 원본파일을 디렉터리로 복사할 경우

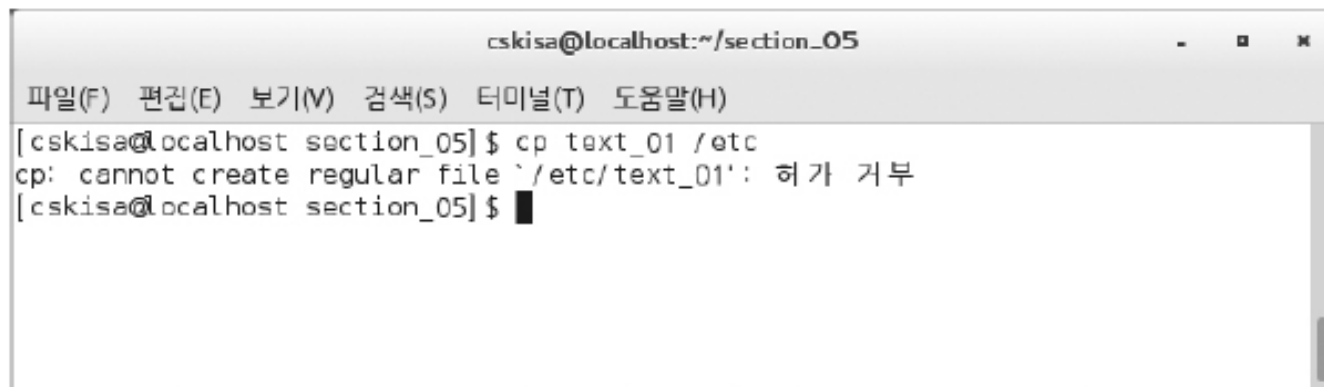
| 예제 3-30 |

쓰기 권한이 없는 etc 디렉터리에 파일을 복사할 경우 ‘허가 거부’ 메시지가 출력됩니다.

```
$ cp text_01 /etc
```

기능 이미 존재하는 text_01 파일을 /etc 디렉터리에 복사

형식 cp [원본 파일명] [복사될 디렉터리명] [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ cp text_01 /etc
cp: cannot create regular file `/etc/text_01': 허가 거부
[cskisa@localhost section_05]$
```

[그림 3-36] 원본파일 text_01을 etc 디렉터리에 복사할 경우 오류 메시지 출력

- 한꺼번에 여러 개의 원본파일을 디렉터리로 복사할 경우

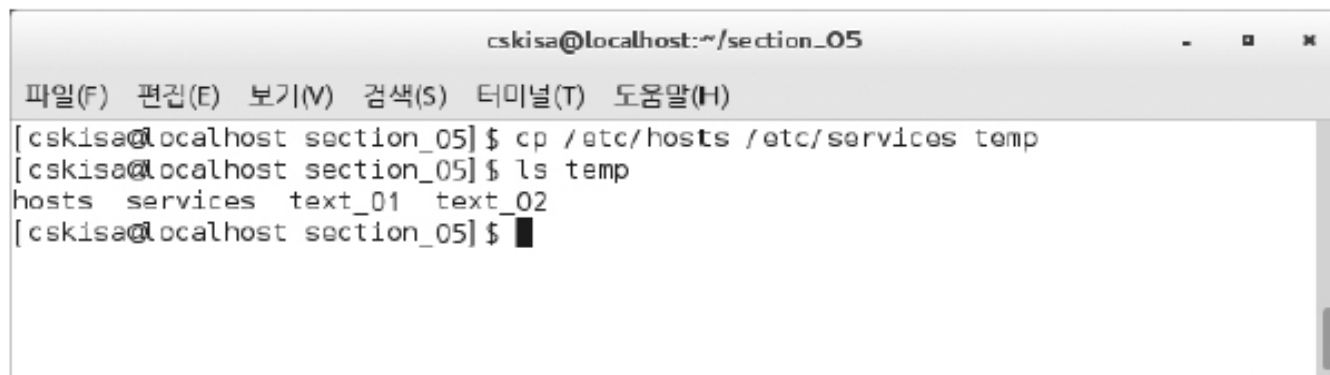
| 예제 3-31 |

etc 디렉터리에 이미 존재하는 hosts 파일과 services 파일 2개를 temp 디렉터리로 한꺼번에 복사한 다음 temp 디렉터리에 파일이 존재하는지를 확인합니다.

```
$ cp /etc/hosts /etc/services temp
```

기능 이미 존재하는 hosts 파일과 services 파일을 한꺼번에 temp 디렉터리에 복사

형식 cp [디렉터리명/파일명] [디렉터리명/파일명] [복사될 디렉터리명] [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05] $ cp /etc/hosts /etc/services temp
[cskisa@localhost section_05] $ ls temp
hosts services text_01 text_02
[cskisa@localhost section_05] $
```

[그림 3-37] 두 개의 파일을 temp 디렉터리에 복사

- 디렉터리를 복사할 경우

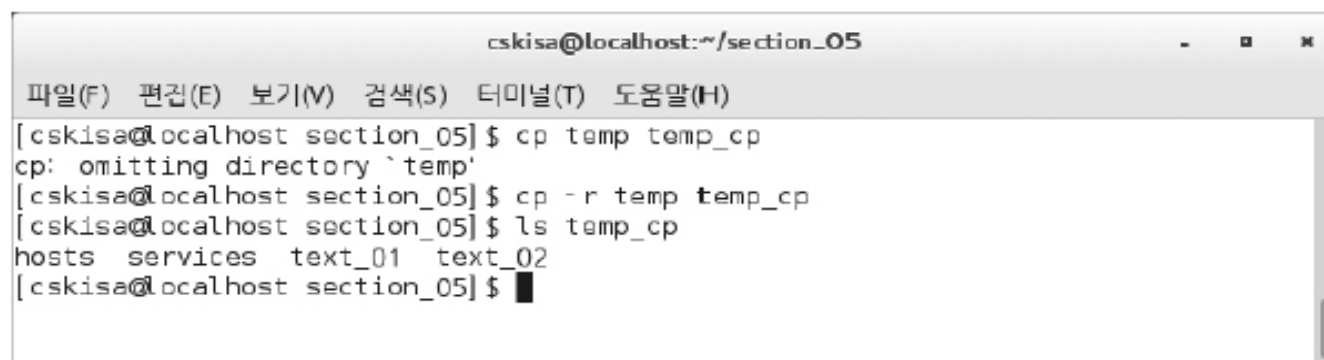
| 예제 3-32 |

현재 디렉터리에 존재하는 temp 디렉터를 temp_cp 디렉터리로 복사합니다. 이 과정에서 복사되는 temp_cp 디렉터리가 존재하지 않을 경우에는 자동으로 생성되도록 옵션 -r과 함께 cp 명령을 사용합니다.

```
$ cp -r temp temp_cp
```

기능 이미 존재하는 temp 디렉터를 temp_cp 디렉터리로 복사

형식 cp [옵션] [원본 디렉터리명] [복사될 디렉터리명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ cp temp temp_cp
cp: omitting directory `temp'
[cskisa@localhost section_05]$ cp -r temp temp_cp
[cskisa@localhost section_05]$ ls temp_cp
hosts  services  text_01  text_02
[cskisa@localhost section_05]$
```

[그림 3-38] temp 디렉터를 temp_cp 디렉터리로 복사

- 파일을 복사할 때 대화식 방법 사용 : -i 옵션

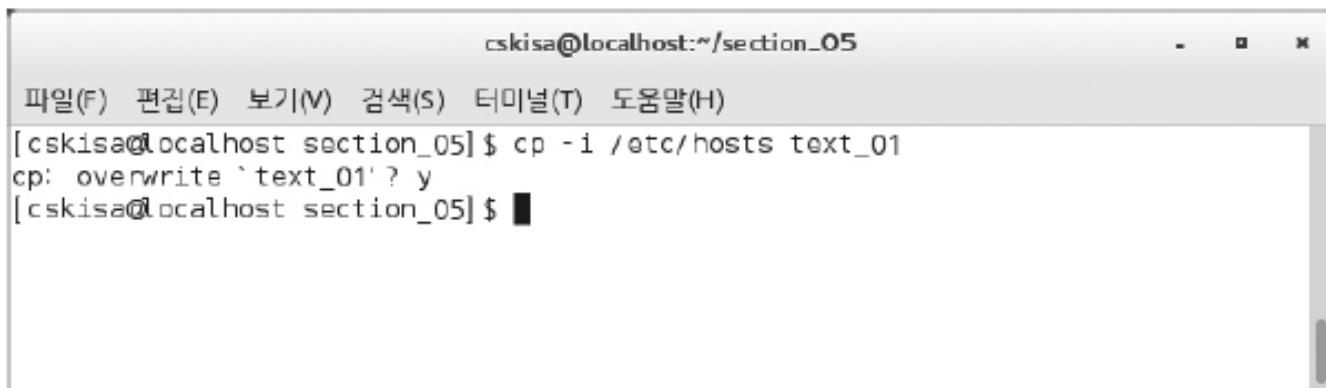
| 예제 3-33 |

파일을 복사할 때 이미 존재하는 동일한 text_01 파일명으로 복사할 경우 덮어씌울 것인지에 대한 옵션이 나타나도록 cp 명령과 함께 옵션 -i를 선언합니다.

```
$ cp -i /etc/hosts text_01
```

기능 복사될 파일명이 이미 존재하는 파일일 경우 덮어씌울 것인지를 물으며 진행

형식 cp [옵션] [디렉터리명/원본파일명] [복사될 파일명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ cp -i /etc/hosts text_01
cp: overwrite `text_01'? y
[cskisa@localhost section_05]$
```

[그림 3-39] 대화식 방법으로 파일 복사

파일 찾기

- 리눅스 시스템에서 찾고자 하는 파일은 find 명령을 사용

\$ find

기능 검색 조건에 맞는 파일을 지정한 위치에서 찾을

형식 find 경로 검색조건 [옵션] [동작] [Enter.↵]

옵션 -name : 파일 이름으로 검색

-type : 파일 타입으로 검색

-user : 로그인 아이디로 사용자가 소유한 모든 파일 검색

-perm : 지정한 접근권한과 일치하는 파일 검색

옵션 -exec 명령 {} \ : 검색된 파일에 명령을 실행

-ok 명령{} \ : 사용자의 확인을 받아서 명령을 실행

-print : 검색된 파일의 절대 경로명을 화면에 출력 (기본 동작)

-ls : 검색결과를 긴 목록형식으로 출력

[표 3-6] find 조건옵션의 기능

옵션	기능
-empty	비어있는 파일 찾기
-group	지정한 그룹이 소유하고 있는 파일 찾기
-name	지정한 파일찾기 (와일드카드 사용 : *, '?', '[]') ⇒ 와일드카드를 사용할 경우 작은따옴표를 선언하여 파일이름을 정확히 표현
-newer	지정한 파일보다 최근에 생성된 파일 찾기
-size	지정한 파일크기 n으로 파일 찾기 (c : byte, k : kbyte, w : word) ⇒ +n : n보다 큰 값, n : 정확한 n 값, n- : n보다 작은 값
-type	찾고자 하는 파일의 타입 지정 ⇒ b : 블록 디바이스 파일, d : 디렉터리, f : 파일, l : 심볼릭 링크
-user	지정한 사용자가 소유하고 있는 파일 찾기

[표 3-7] find 동작옵션의 기능

옵션	기능
-print	찾은 파일 리스트를 보여줌
-prune	현재 디렉터리의 하위 디렉터리로는 내려가지 않음
-exec [명령] {} \;	찾은 파일을 대상으로 주어진 [명령]을 수행 ⇒ \ 앞에는 공백이 들어가야 하고 세미콜론(;)은 붙여야함
-ok [명령] {} \;	exec와 비슷한 기능으로 [명령]을 적용하기 전 물어봄 ⇒ \ 앞에는 공백이 들어가야 하고 세미콜론(;)은 붙여야함
-ls	검색결과 긴 목록형식으로 출력

- user 디렉터리에 존재하는 ls 파일 찾기

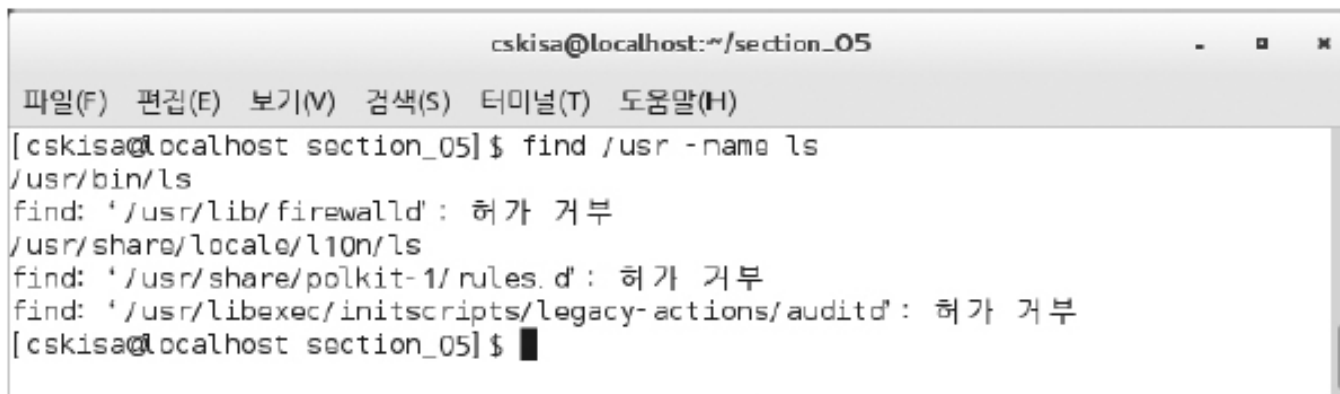
| 예제 3-34 |

usr 디렉터리에 존재하는 ls 파일을 찾기 위해 find 명령과 함께 옵션 -name을 사용합니다.

```
$ find /usr -name ls
```

기능 /usr 디렉터리에 존재하는 ls 파일 찾기

형식 find [디렉터리명] [조건옵션] [파일명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ find /usr -name ls
/usr/bin/ls
find: '/usr/lib/firewalld': 허가 거부
/usr/share/locale/l10n/ls
find: '/usr/share/polkit-1/rules.d': 허가 거부
find: '/usr/libexec/initscripts/legacy-actions/auditd': 허가 거부
[cskisa@localhost section_05]$
```

[그림 3-40] usr 디렉터리에 존재하는 ls 파일 찾기

- 알파벳 h로 시작하는 파일목록 출력

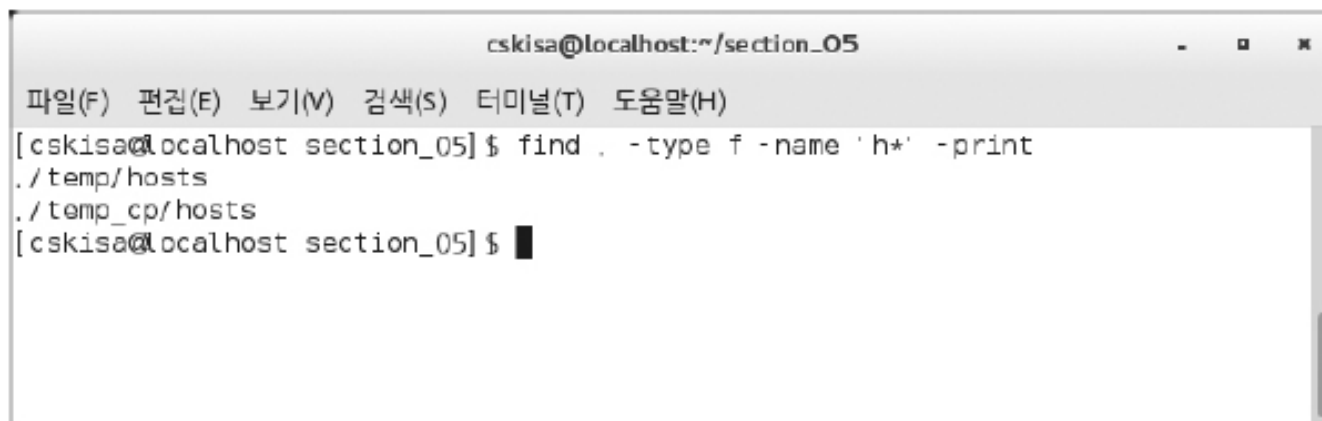
| 예제 3-35 |

현재 디렉터리부터 하위 디렉터리까지의 경로에서 파일이름이 알파벳 h로 시작하는 모든 파일을 찾아서 파일목록을 화면에 출력합니다.

```
$ find . -type f -name 'h*' -print
```

기능 파일타입 중에서 알파벳 h로 시작하는 파일명을 찾아서 출력

형식 find [디렉터리 경로] [조건옵션] [인자] [조건옵션] [인자] [동작옵션] [Enter.]



A terminal window titled 'cskisa@localhost:~/section_05' with standard window controls. The menu bar shows '파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)'. The terminal content shows the command `[cskisa@localhost section_05]$ find . -type f -name 'h*' -print` being executed, resulting in two lines of output: `./temp/hosts` and `./temp_cp/hosts`. The prompt `[cskisa@localhost section_05]$` is visible at the bottom with a cursor.

[그림 3-41] 현재 디렉터리부터 하위 디렉터리까지의 h로 시작하는 파일 찾기

- 현재 디렉터리부터 하위 디렉터리까지 비어있는 파일목록 출력

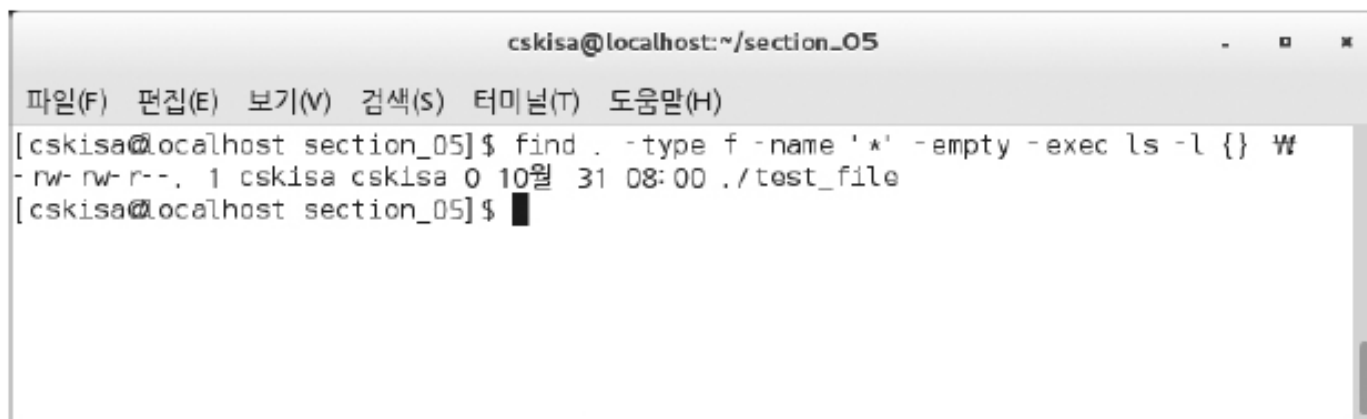
| 예제 3-36 |

현재 디렉터리부터 하위 디렉터리까지의 경로 중에서 비어있는 파일목록을 출력합니다.

```
$ find . -type f -name '*' -empty -exec ls -l {} \;
```

기능 현재 디렉터리부터 하위 디렉터리까지 빈파일 리스트 출력

형식 find [경로] [조건옵션] [인자] [와일드카드] [조건옵션] [동작옵션] W; [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ find . -type f -name '*' -empty -exec ls -l {} W
-rw-rw-r--. 1 cskisa cskisa 0 10월 31 08:00 ./test_file
[cskisa@localhost section_05]$
```

[그림 3-42] 현재 디렉터리부터 하위 디렉터리까지의 빈파일 찾기

● 사용자 홈 디렉터리에 h로 시작하는 파일목록 출력

| 예제 3-37 |

- **Step 01** | 톨드(~)를 사용하여 사용자 홈 디렉터리로 이동합니다.

```
$ cd ~
```

기능 사용자 홈 디렉터리로 이동

형식 cd [이동할 디렉터리명] [Enter]

- **Step 02** | 현재 디렉터리에 존재하는 파일이름 중에서 h로 시작하는 모든 파일목록을 출력합니다.

```
$ find -type f -name 'h*' -print
```

기능 사용자 홈 디렉터리에 존재하는 파일 중에서 h로 시작하는 파일 찾기

형식 find [조건옵션] [인자] [조건옵션] [와일드카드] [동작옵션] [Enter]

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05] $ cd ~
[cskisa@localhost ~] $ find -type f -name 'h*' -print
./local/share/gvfs-metadata/home
./local/share/gvfs-metadata/home-6736b2ce.log
./section_05/temp/hosts
./section_05/temp_cp/hosts
[cskisa@localhost ~] $
  
```

[그림 3-43] 사용자 홈 디렉터리에서 h로 시작하는 파일 찾기

파일 내용 출력

- gedit를 사용하여 간단한 gedit_test.txt 파일을 생성 후 실습



[그림 3-47] 텍스트파일이 저장된 경로명과 파일명

- 파일내용 연속출력 : cat

```
$ cat
```

기능 파일내용을 연속해서 출력

형식 cat [옵션] 출력할 파일명 [Enter.]

옵션 -n : 파일내용을 행 번호를 붙여서 출력

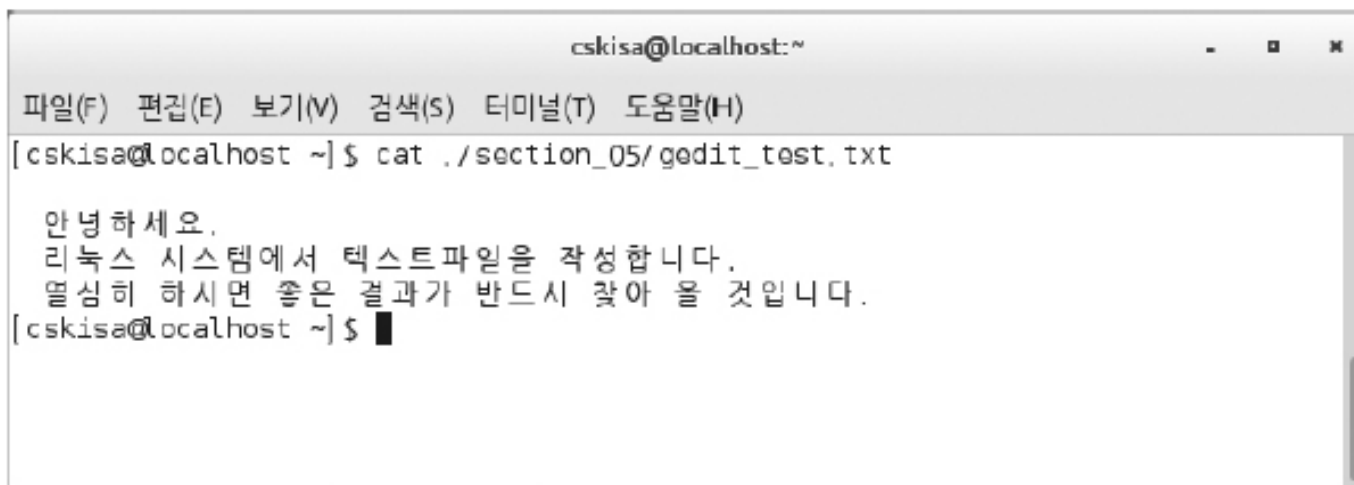
| 예제 3-38 |

section_05 디렉터리에 존재하는 gedit_test.txt 파일의 내용을 연속적으로 출력합니다.

```
$ cat ./section_05/gedit_test.txt
```

기능 section_05 디렉터리에 gedit_test.txt 파일내용 연속출력

형식 cat [디렉터리명/파일명] [Enter↵]



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ cat ./section_05/gedit_test.txt  
  
안녕하세요.  
리눅스 시스템에서 텍스트파일을 작성합니다.  
열심히 하시면 좋은 결과가 반드시 찾아 올 것입니다.  
[cskisa@localhost ~]$
```

[그림 3-48] gedit_test.txt 파일내용 출력

- 파일내용을 행 번호와 같이 출력하기

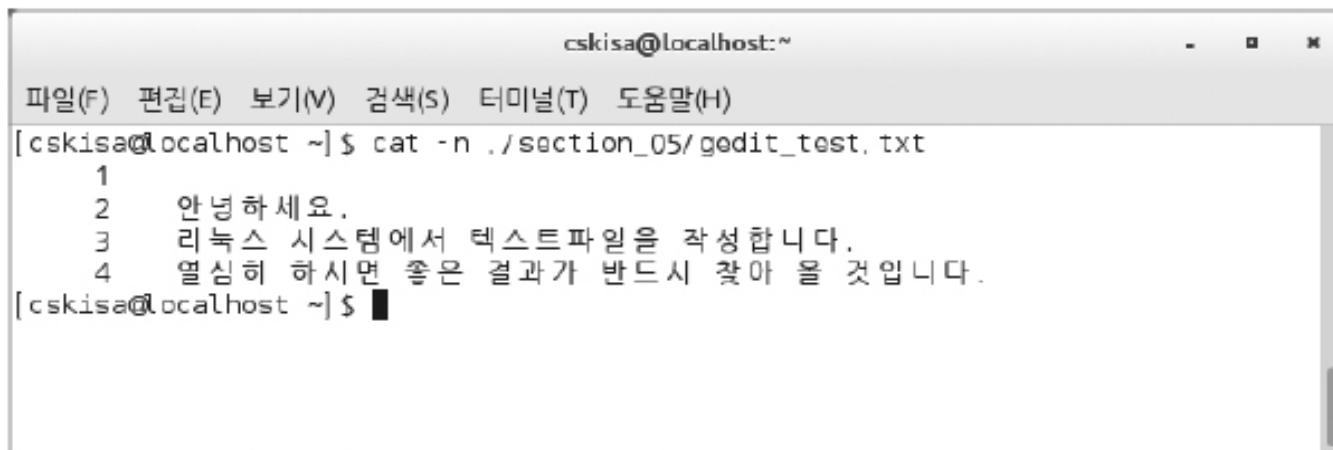
| 예제 3-39 |

section_05 디렉터리에 존재하는 gedit_test.txt 파일의 내용을 행 번호와 함께 연속으로 출력합니다.

```
$ cat -n ./section_05/gedit_test.txt
```

기능 section_05 디렉터리에 gedit_test.txt 파일내용을 행 번호와 함께 연속출력

형식 cat [옵션] [디렉터리명/파일명] [Enter]



```
cskisa@localhost: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ cat -n ./section_05/gedit_test.txt  
1  
2     안녕하세요.  
3     리눅스 시스템에서 텍스트파일을 작성합니다.  
4     열심히 하시면 좋은 결과가 반드시 찾아 올 것입니다.  
[cskisa@localhost ~]$
```

[그림 3-49] gedit_test.txt 파일내용에 행 번호가 나타나도록 출력

- 화면 단위로 파일내용 출력 : more 명령

```
$ more
```

기능 파일내용을 화면 단위로 출력

형식 more [옵션] 출력할 파일명

옵션 +행 번호 : 지정한 행 번호부터 파일내용이 화면단위로 출력

- /etc/services 파일의 내용을 more 명령으로 출력

| 예제 3-40 |

- **Step 01** | ect 디렉터리에 존재하는 services 파일의 내용을 화면단위로 출력합니다.

```
$ more /etc/services
```

기능 etc 디렉터리에 존재하는 services 파일의 내용을 화면단위로 출력

형식 more [디렉터리명/파일명]

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ more /etc/services
# /etc/services:
# $Id: services,v 1.55 2013/04/14 ovasik Exp $
#
# Network services, Internet style
# IANA services version: last updated 2013-04-10
#
# Note that it is presently the policy of IANA to assign a single well-known
# port number for both TCP and UDP; hence, most entries here have two entries
# even if the protocol doesn't support UDP operations.

```

[그림 3-50] more 명령으로 파일내용 출력

- **Step 02** | `[Space Bar]`를 이용하여 etc 디렉터리에 존재하는 services 파일의 내용을 화면단위로 확인합니다.

```

cskisa@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
# /etc/services:
# $Id: services,v 1.55 2013/04/14 ovasik Exp $
#
# Network services, Internet style
# IANA services version: last updated 2013-04-10
#
# Note that it is presently the policy of IANA to assign a single well-known
# port number for both TCP and UDP; hence, most entries here have two entries
# even if the protocol doesn't support UDP operations.
-- More-- (0%

```

[그림 3-51] 스페이스바를 이용하여 화면단위로 내용확인

- **Step 03** | etc 디렉터리에 존재하는 services 파일의 내용확인 과정을 q를 입력하여 종료합니다.

```

cskisa@localhost: ~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
discard      9/udp      sink null
sysstat      11/tcp     users
sysstat      11/udp     users
daytime      13/tcp
daytime      13/udp
qotd         17/tcp     quote
qotd         17/udp     quote
msp          18/tcp
# message send protocol (histori
c)
[cskisa@localhost ~] $
  
```

[그림 3-52] q를 입력하여 more 명령 종료

파일의 특정 문자열 검색

- grep 명령을 사용하여 파일 내에서 특정 문자열을 검색

\$ grep

기능 찾고자 하는 문자열 검색

형식 tail [옵션] [찾을 문자열] [파일명]

옵션 -c : 찾을 문자열이 속한 행의 개수 출력

-H : 파일명과 함께 출력

-i : 대소문자를 구별하지 않고 출력

-l : 해당 패턴을 포함하는 파일이름 출력

-n : 찾으려는 문자열이 속해있는 행의 번호를 함께 출력

-r : 현재 경로부터 하위 경로까지 검색하여 출력

-v : 찾으려는 문자열이 없는 행 출력

-w : 패턴 표현식을 하나의 단어로 취급하여 검색

- Section_05 디렉터리로 이동한 다음 /etc/services 파일을 ex_data 로 복사 후 DHCP가 포함된 행 번호와 내용 출력

| 예제 3-41 |

- Step 01 | cd 명령으로 작업할 section_05 디렉터리로 이동합니다.

```
$ cd section_05
```

기능 section_05 디렉터리로 이동

형식 cd [이동할 디렉터리명]

- Step 02 | cp 명령으로 /etc/services 파일을 ex_data로 복사합니다.

```
$ cp /etc/services ex_data
```

기능 디렉터리에 존재하는 services 원본파일일 ex_data 파일명으로 복사

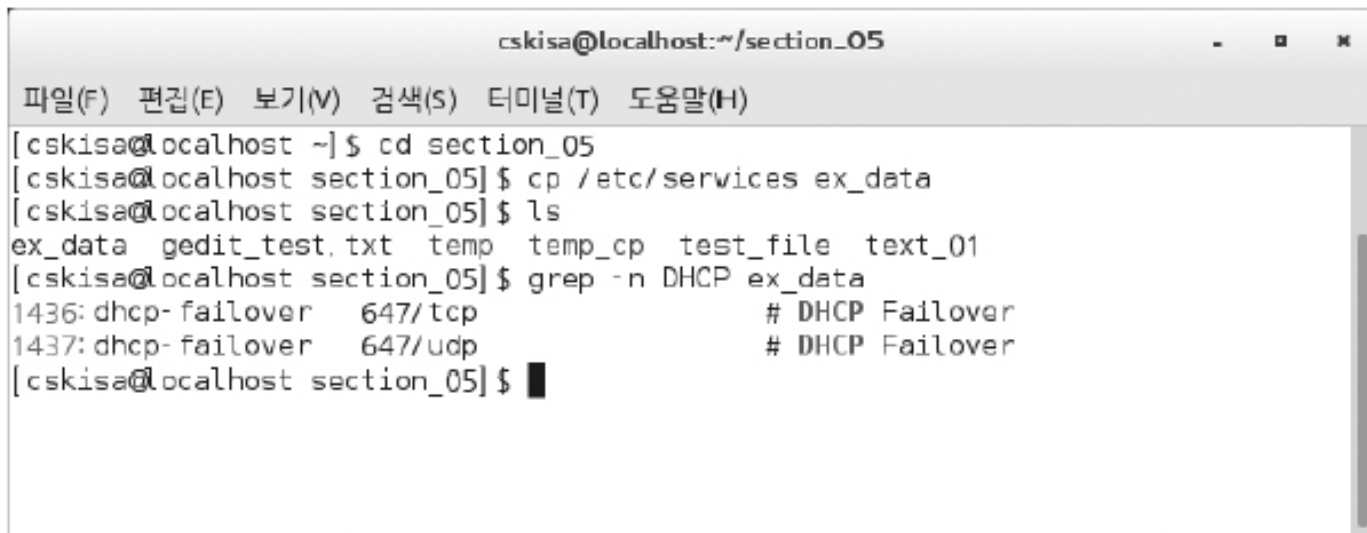
형식 cp [디렉터리명/원본파일명] [복사될 파일명]

- **Step 03** | grep 명령으로 특정 문자열 DHCP를 ex_data 파일에서 찾아서 해당 행 번호와 함께 출력되도록 옵션 -n을 지정합니다.

```
$ grep -n DHCP ex_data
```

기능 ex_data 파일에서 특정 문자열 DHCP가 포함된 부분을 행 번호와 함께 출력

형식 grep [옵션] [특정 문자열] [검색 대상인 파일명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ cd section_05
[cskisa@localhost section_05]$ cp /etc/services ex_data
[cskisa@localhost section_05]$ ls
ex_data  gedit_test.txt  temp  temp_cp  test_file  text_01
[cskisa@localhost section_05]$ grep -n DHCP ex_data
1436: dhcp-failover    647/tcp          # DHCP Failover
1437: dhcp-failover    647/udp          # DHCP Failover
[cskisa@localhost section_05]$
```

[그림 3-53] DHCP가 포함된 행 출력

특정 명령 위치 검색

- 고정된 경로에서 특정 명령 검색 : whereis

```
$ whereis
```

기능 찾고자 하는 명령의 바이너리 파일이나 매뉴얼 파일의 위치 검색

형식 whereis [옵션] 명령

옵션 -b : 바이너리 파일만 검색

-m : 매뉴얼 파일만 검색

-s : 소스 파일만 검색

- whereis 명령으로 cp명령과 mv명령의 존재 디렉터리 확인

| 예제 3-42 |

- Step 01 | 파일복사 cp 명령이 어느 디렉터리에 존재하는지를 확인합니다.

```
$ whereis cp
```

기능 cp 명령이 어느 디렉터리에 위치하는지를 검색

형식 whereis [파일명]

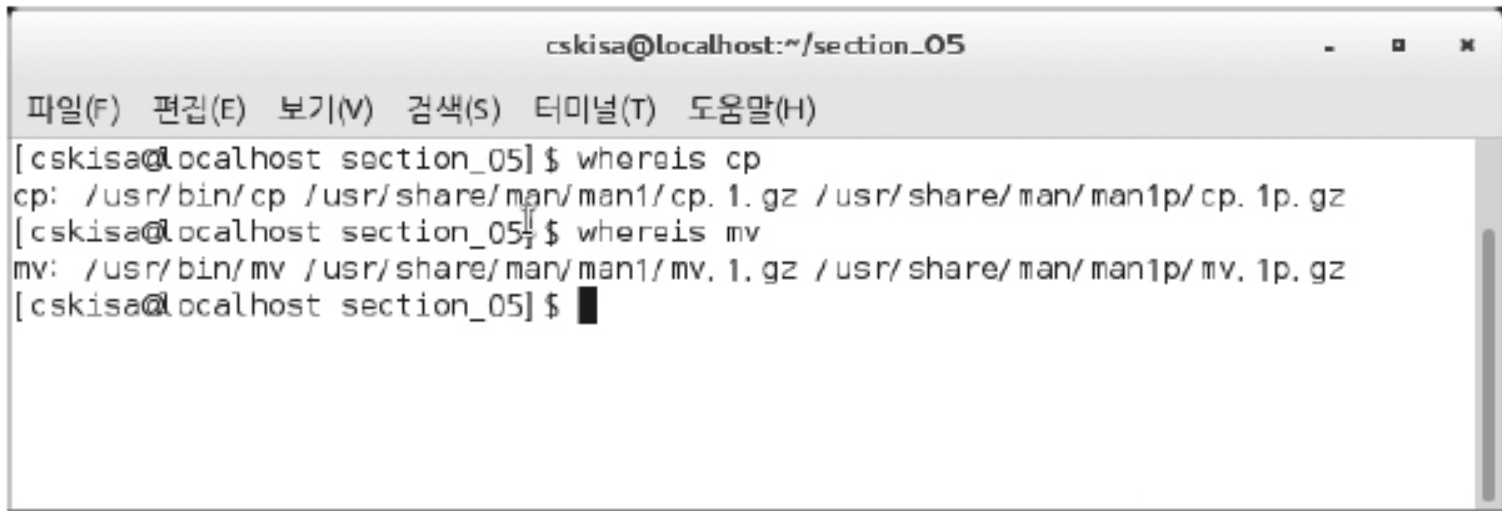
- Step 02 | 디렉터리 이동 mv 명령이 존재하는 디렉터리의 위치를 확인합니다.

```
$ whereis mv
```

기능 cp 명령이 어느 디렉터리에 위치하는지를 검색

형식 whereis [파일명]

- whereis 명령으로 cp명령과 mv명령의 존재 디렉터리 확인



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ whereis cp
cp: /usr/bin/cp /usr/share/man/man1/cp.1.gz /usr/share/man/man1p/cp.1p.gz
[cskisa@localhost section_05]$ whereis mv
mv: /usr/bin/mv /usr/share/man/man1/mv.1.gz /usr/share/man/man1p/mv.1p.gz
[cskisa@localhost section_05]$
```

[그림 3-55] cp 명령과 mv 명령이 존재하는 디렉터리의 위치 검색

- 경로 또는 alias 출력 : which
 - which 명령은 사용자 초기화 파일에 지정된 alias를 찾거나
 - PATH 환경 변수에 지정된 경로를 순서대로 찾아가며 mv명령 파일이 어느 디렉터리에 존재하는 지를 검색

```
$ whereis mv
```

기능 지정된 에일리어스 또는 PATH 환경 변수로 지정된 경로 검색

형식 `which [명령]`

- 경로 또는 alias 출력 : which

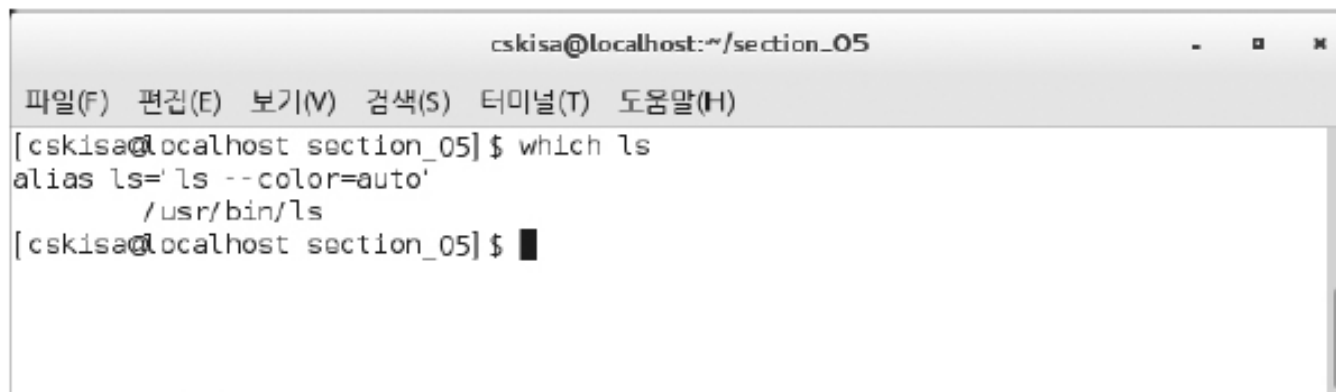
| 예제 3-43 |

which 명령으로 ls 명령에 지정된 에일리어스와 경로를 검색합니다.

```
$ which ls
```

기능 ls 명령에 지정된 에일리어스와 경로 검색

형식 which [명령]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05] $ which ls
alias ls='ls --color=auto'
      /usr/bin/ls
[cskisa@localhost section_05] $
```

[그림 3-56] ls 명령에 지정된 에일리어스와 경로 검색

파일 이동

- move 약어인 mv 명령으로 파일이나 디렉토리를 이동

\$ mv

기능 파일이나 디렉토리를 이동 또는 파일명이나 디렉터리명 변경

형식 mv [원본 파일명] [이동할 파일명]

옵션 -i : 이동할 디렉터리에 동일한 파일명이나 디렉터리가 존재하는지 메시지 출력

- 파일을 파일로 이동

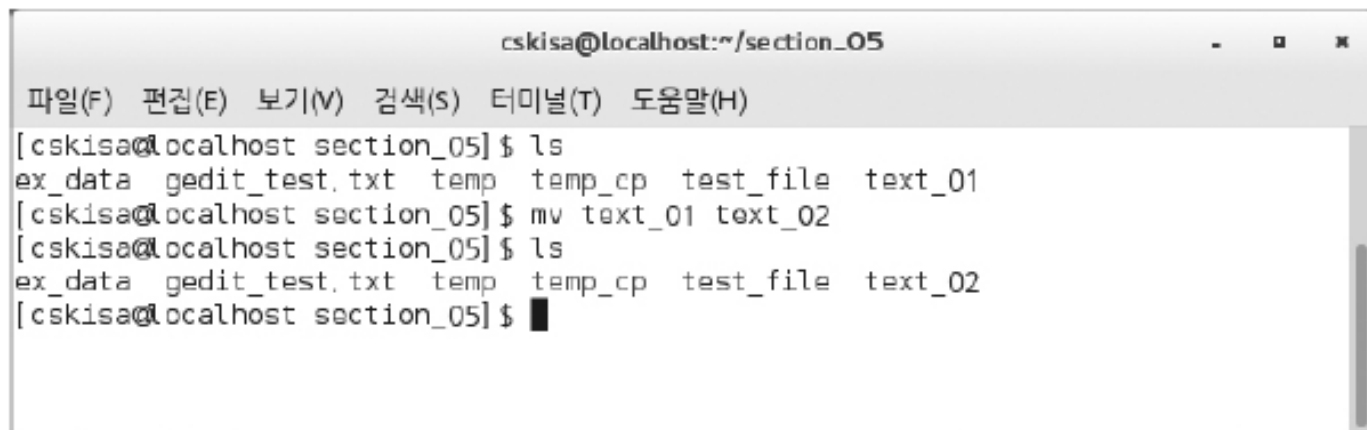
| 예제 3-44 |

현재 디렉터리에 존재하는 text_01 파일을 text_02 파일로 이동합니다.

```
$ mv text_01 text_02
```

기능 기존에 존재하는 text_01 파일을 text_02 파일로 이동

형식 mv [원본 파일명] [이동할 파일명] [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01
[cskisa@localhost section_05]$ mv text_01 text_02
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_02
[cskisa@localhost section_05]$
```

[그림 3-57] mv 명령으로 text_01 파일을 text_02 파일로 이동

- 파일을 다른 디렉터리로 이동

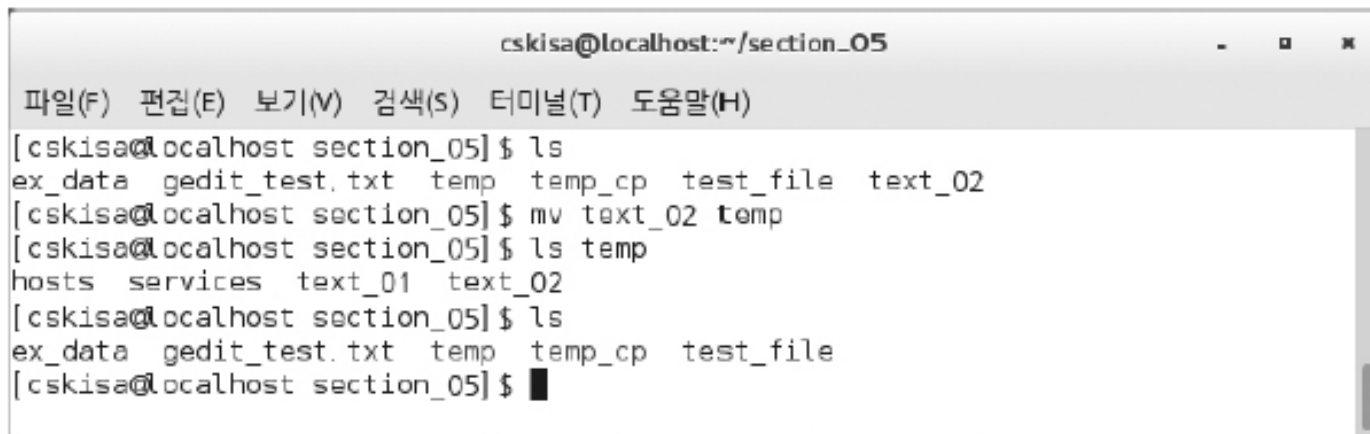
| 예제 3-45 |

현재 디렉터리에 존재하는 text_02 파일을 temp 디렉터리로 이동합니다.

```
$ mv text_02 temp
```

기능 현재 디렉터리에 존재하는 text_02 파일을 temp 디렉터리로 이동

형식 mv [원본 파일명] [이동할 디렉터리명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_02
[cskisa@localhost section_05]$ mv text_02 temp
[cskisa@localhost section_05]$ ls temp
hosts services text_01 text_02
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file
[cskisa@localhost section_05]$ █
```

[그림 3-58] mv 명령으로 text_02 파일을 temp 디렉터리로 이동

- 여러 파일을 다른 디렉터리로 이동

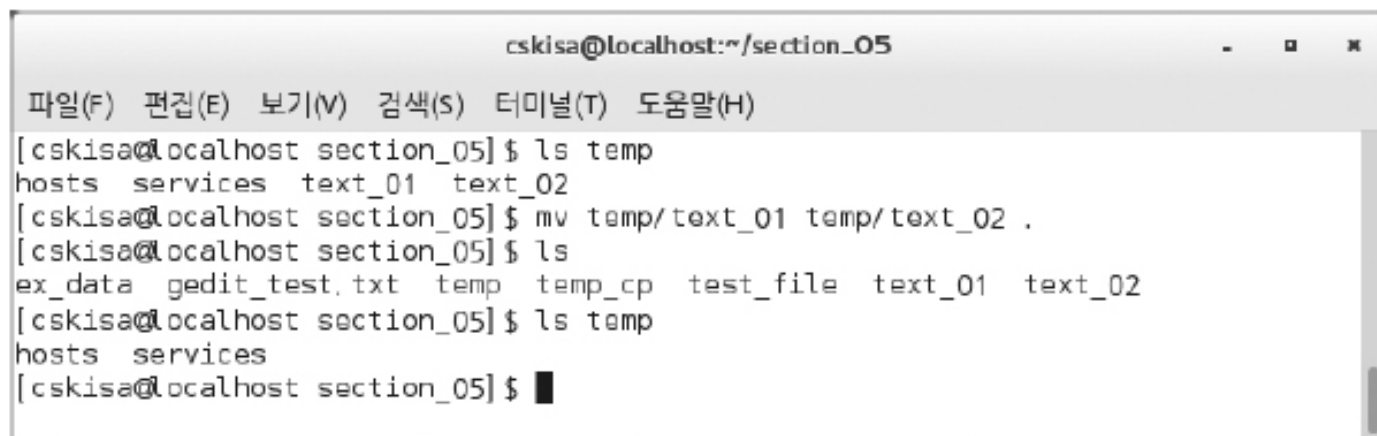
| 예제 3-46 |

temp 디렉터리에 존재하는 text_01 파일과 text_02 파일을 현재 디렉터리로 이동합니다.

```
$ mv temp/text_01 temp/text_02 .
```

기능 temp 디렉터리에 존재하는 text_01, text_02 파일을 현재 디렉터리로 이동

형식 mv [디렉터리명/파일명1] [디렉터리명/파일명2] [이동할 디렉터리명] [Enter.]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05] $ ls temp
hosts services text_01 text_02
[cskisa@localhost section_05] $ mv temp/text_01 temp/text_02 .
[cskisa@localhost section_05] $ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05] $ ls temp
hosts services
[cskisa@localhost section_05] $
```

[그림 3-59] temp 디렉터리에 있는 text_01, text_02 파일을 현재 디렉터리로 이동

- 여러 파일을 다른 디렉터리로 이동

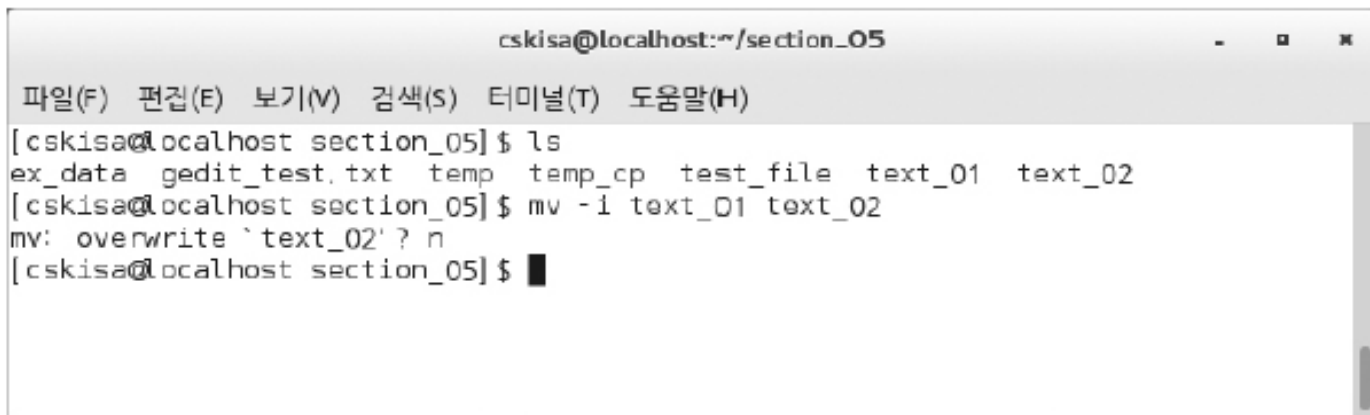
| 예제 3-47 |

현재 디렉터리에서 text_01 파일과 text_02 파일을 이동할 때 동일한 파일명이 존재할 경우 덮어씌울 것인지에 대해 y 또는 n으로 확인하면서 진행합니다.

```
$ mv -i text_01 text_02
```

기능 text_01 파일을 text_02 파일로 이동

형식 mv [옵션] [이동할 원본파일명] [이동 후 파일명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls
ex_data  gedit_test.txt  temp  temp_cp  test_file  text_01  text_02
[cskisa@localhost section_05]$ mv -i text_01 text_02
mv: overwrite `text_02'? n
[cskisa@localhost section_05]$
```

[그림 3-60] 동일한 파일명이 존재할 때 덮어씌울 것인지에 대한 메시지 출력

- 디렉터리를 다른 디렉터리로 이동

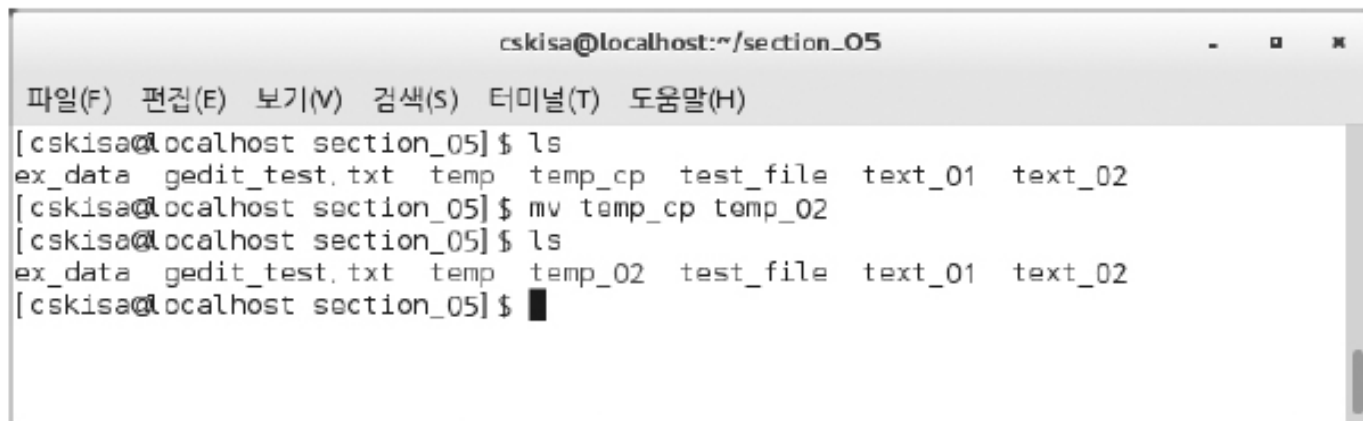
| 예제 3-48 |

현재 디렉터리에 존재하는 temp_cp 디렉터를 temp_02 디렉터리 이름으로 이동합니다.

```
$ mv temp_cp temp_02
```

기능 temp_cp 디렉터를 존재하지 않는 temp_02 디렉터리로 이동

형식 mv [이동할 원본 디렉터리명] [이동 후 디렉터리명]



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05]$ mv temp_cp temp_02
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_02 test_file text_01 text_02
[cskisa@localhost section_05]$
```

[그림 3-61] temp_cp 디렉터를 temp_02 디렉터리로 이동

- 디렉터리를 다른 디렉터리로 이동

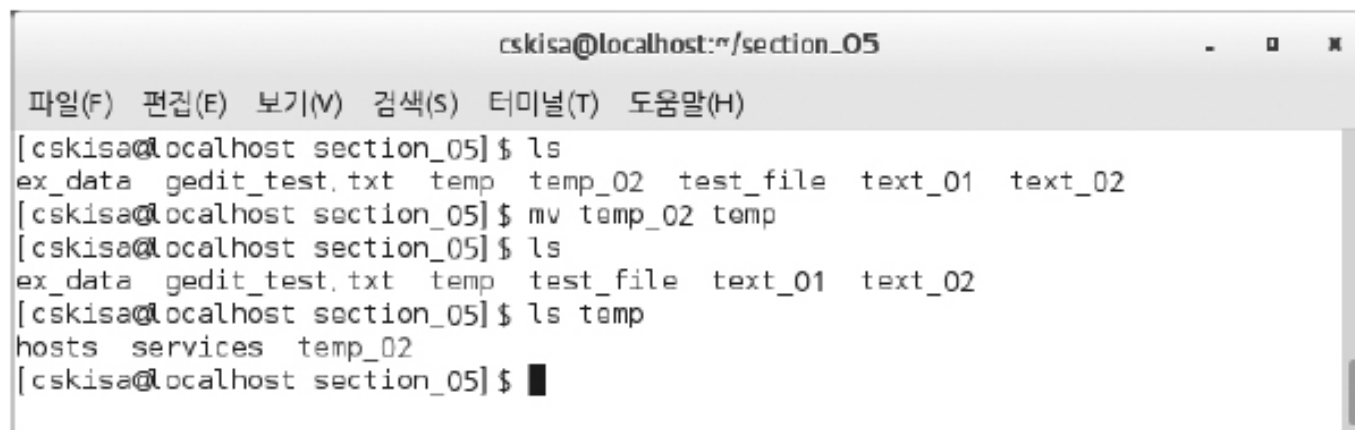
| 예제 3-49 |

현재 디렉터리에 존재하는 temp_02 디렉터를 temp 디렉터리로 이동합니다.

```
$ mv temp_02 temp
```

기능 temp_02 디렉터를 temp 디렉터리로 이동

형식 mv [이동할 원본 디렉터리명] [이동 후 디렉터리명]

A terminal window titled 'cskisa@localhost:~/section_05' with a menu bar containing '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal shows the following commands and output:

```
[cskisa@localhost section_05] $ ls
ex_data  gedit_test.txt  temp  temp_02  test_file  text_01  text_02
[cskisa@localhost section_05] $ mv temp_02 temp
[cskisa@localhost section_05] $ ls
ex_data  gedit_test.txt  temp  test_file  text_01  text_02
[cskisa@localhost section_05] $ ls temp
hosts  services  temp_02
[cskisa@localhost section_05] $
```

[그림 3-62] temp_02 디렉터를 temp 디렉터리로 이동

파일 삭제

- remove 약어인 rm 명령으로 파일을 삭제
- 디렉토리를 삭제할 때는 옵션 -r을 함께 사용

\$ rm

기능 파일 또는 디렉터리 삭제

형식 rm [옵션] [파일명 또는 디렉터리명]

옵션 -i : 삭제할 파일명 또는 디렉터리명이 존재하면 삭제할 것인지를 물어보는 방법

-r : 지정한 디렉토리를 삭제할 때 사용 (특히 비어있지 않은 디렉토리를 삭제)

-ri : 지정한 디렉토리를 삭제할 때 내용을 하나씩 확인하면서 삭제

- 일반 파일 삭제

| 예제 3-50 |

- **Step 01** | 현재 디렉터리에 존재하는 text_file을 text_file_cp 파일명으로 복사합니다.

```
$ cp test_file test_file_cp
```

기능 이미 존재하는 test_file을 test_file_cp 파일명으로 복사

형식 cp [원본 파일명] [복사될 파일명] [Enter]

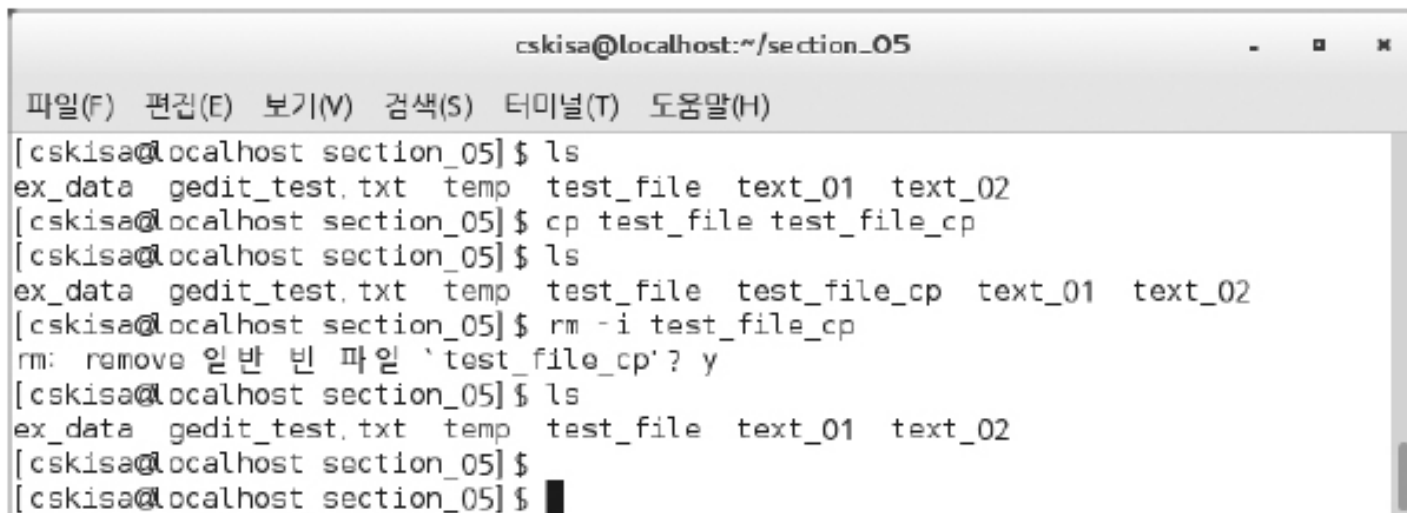
- **Step 02** | 현재 디렉터리에 존재하는 text_file을 삭제한 것인지에 대한 여부를 확인하면서 진행합니다.

```
$ rm -i test_file_cp
```

기능 이미 존재하는 test_file_cp 파일 삭제

형식 rm [옵션] [삭제할 파일명] [Enter]

- **Step 03** | 현재 디렉터리에서 text_file이 삭제되었는지에 대해 확인합니다.

A terminal window titled 'cskisa@localhost:~/section_05' with a menu bar containing '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal shows the following commands and output:

```
[cskisa@localhost section_05]$ ls
ex_data  gedit_test.txt  temp  test_file  text_01  text_02
[cskisa@localhost section_05]$ cp test_file test_file_cp
[cskisa@localhost section_05]$ ls
ex_data  gedit_test.txt  temp  test_file  test_file_cp  text_01  text_02
[cskisa@localhost section_05]$ rm -i test_file_cp
rm: remove 일반 빈 파일 'test_file_cp'? y
[cskisa@localhost section_05]$ ls
ex_data  gedit_test.txt  temp  test_file  text_01  text_02
[cskisa@localhost section_05]$
[cskisa@localhost section_05]$
```

[그림 3-63] 복사된 test_file_cp 파일 삭제

- 디렉터리 삭제

| 예제 3-51 |

- **Step 01** | 현재 디렉터리에 존재하는 temp 디렉터를 temp_cp 디렉터리로 복사합니다.

```
$ cp -r temp temp_cp
```

기능 이미 존재하는 temp 디렉터를 temp_cp 디렉터리명으로 복사

형식 cp [옵션] [원본 디렉터리명] [복사될 디렉터리명]

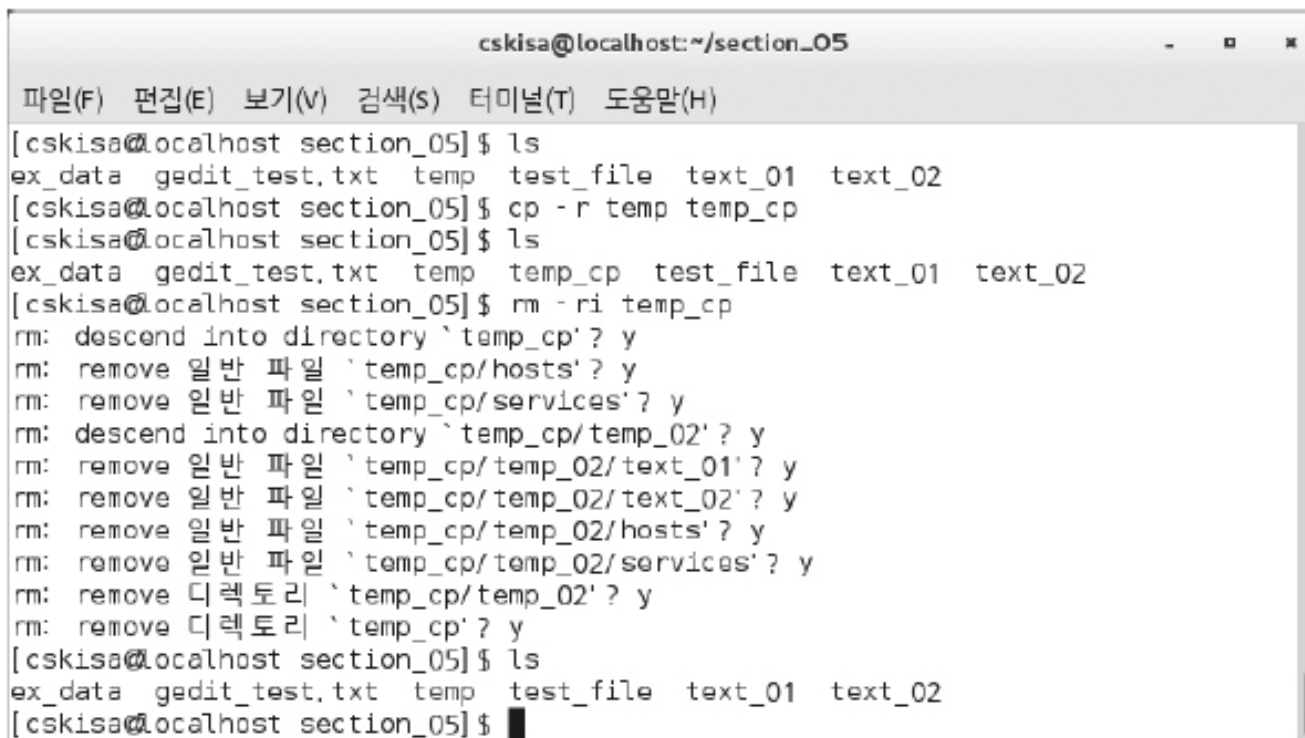
- **Step 02** | 비어있지 않은 temp_cp 디렉터를 삭제하기 위해 rm 명령과 함께 옵션 -ri를 선언하여 삭제여부를 하나씩 확인하면서 진행합니다.

```
$ rm -ri temp_cp
```

기능 이미 존재하는 temp_cp 디렉터를 하나하나 확인하면서 삭제

형식 rm [옵션] [삭제할 디렉터리명]

- **Step 03** | temp_cp 디렉터리에 존재하는 일반 파일과 디렉터리를 정말 삭제할 것인지에 대해 y 또는 n으로 결정합니다.



```
cskisa@localhost: ~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp test_file text_01 text_02
[cskisa@localhost section_05]$ cp -r temp temp_cp
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05]$ rm -ri temp_cp
rm: descend into directory `temp_cp'? y
rm: remove 일반 파일 `temp_cp/hosts'? y
rm: remove 일반 파일 `temp_cp/services'? y
rm: descend into directory `temp_cp/temp_02'? y
rm: remove 일반 파일 `temp_cp/temp_02/text_01'? y
rm: remove 일반 파일 `temp_cp/temp_02/text_02'? y
rm: remove 일반 파일 `temp_cp/temp_02/hosts'? y
rm: remove 일반 파일 `temp_cp/temp_02/services'? y
rm: remove 디렉토리 `temp_cp/temp_02'? y
rm: remove 디렉토리 `temp_cp'? y
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp test_file text_01 text_02
[cskisa@localhost section_05]$
```

[그림 3-64] temp_cp 디렉터리 삭제

- **Step 04** | [예제 3-51]에서 실습한 결과 복사된 temp_cp 디렉터리를 삭제하는 과정에서 하나 씩 삭제 여부를 묻는 과정에서 n을 입력하면 디렉터리가 삭제되는지에 대해 살펴봅니다.

```

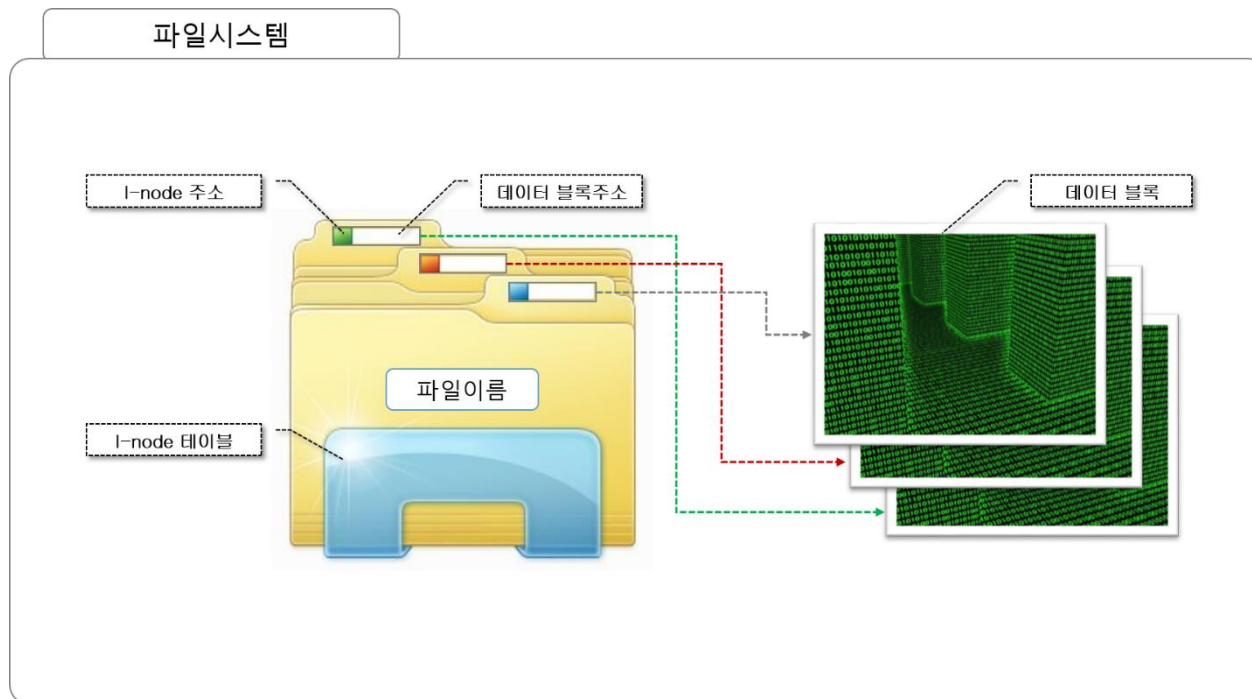
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ cp -r temp temp_cp
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05]$ rm -ri temp_cp
rm: descend into directory `temp_cp'? y
rm: remove 일반 파일 `temp_cp/hosts'? n
rm: remove 일반 파일 `temp_cp/services'? n
rm: descend into directory `temp_cp/temp_02'? n
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05]$ rm -ri temp_cp
rm: descend into directory `temp_cp'? y
rm: remove 일반 파일 `temp_cp/hosts'? n
rm: remove 일반 파일 `temp_cp/services'? n
rm: descend into directory `temp_cp/temp_02'? y
rm: remove 일반 파일 `temp_cp/temp_02/text_01'? y
rm: remove 일반 파일 `temp_cp/temp_02/text_02'? n
rm: remove 일반 파일 `temp_cp/temp_02/hosts'? n
rm: remove 일반 파일 `temp_cp/temp_02/services'? y
rm: remove 디렉토리 `temp_cp/temp_02'? y
rm: cannot remove `temp_cp/temp_02': 디렉터리가 비어있지 않음
[cskisa@localhost section_05]$ ls
ex_data gedit_test.txt temp temp_cp test_file text_01 text_02
[cskisa@localhost section_05]$

```

[그림 3-65] temp_cp 디렉터리가 삭제되지 않음

파일 링크

- 하나의 파일이 복잡한 디렉터리 계층 구조를 가진 다단계로 짧게 줄여 다른 파일명으로 붙여서 간단하게 사용하기 위함
- MS 윈도우의 [바로 가기]와 같은 기능을 수행



- 현재 디렉터리에 존재하는 파일의 I-node 확인

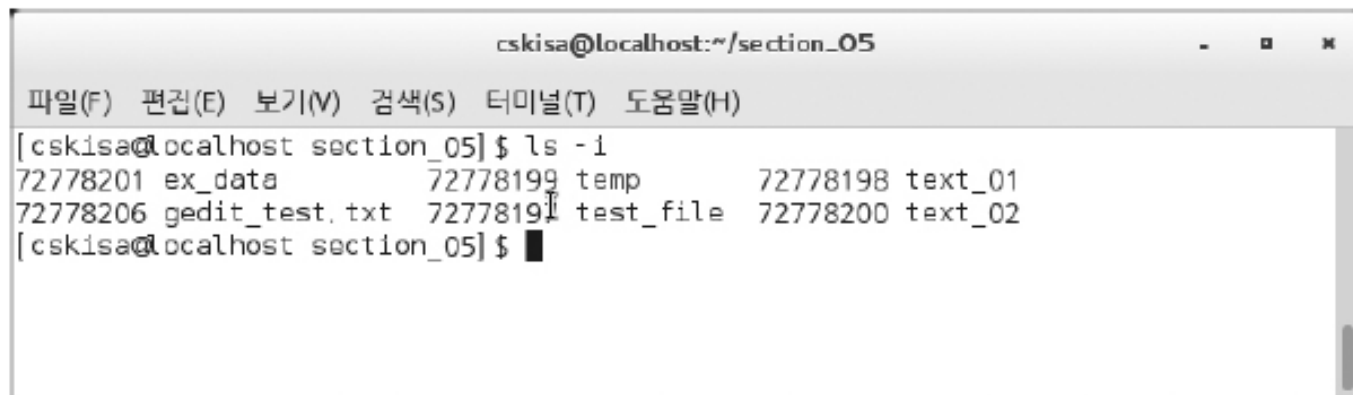
| 예제 3-52 |

현재 디렉터리에 존재하는 파일의 I-node 번호를 확인합니다.

```
$ ls -li
```

기능 파일에 부여된 I-node 번호 확인

형식 `ls [옵션] [Enter]`



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -li
72778201 ex_data          72778199 temp          72778198 text_01
72778206 gedit_test.txt  72778197 test_file    72778200 text_02
[cskisa@localhost section_05]$
```

[그림 3-67] 파일에 부여된 I-node 번호 확인

- 하드 링크 만들기 : ln 명령

- Link의 약어인 ln 명령으로 하드 링크 생성
- 리눅스 시스템에서는 파일에 여러 개의 이름을 명명할 수 있으며 이때 부여하는 파일명을 하드 링크

\$ ln

기능 링크 파일 생성

형식 ln [옵션] 원본파일명 링크파일명

옵션 -b : 링크 파일을 생성할 때 대상 파일이 이미 존재한다면 백업 파일 생성 후 링크 파일 생성

-d : 디렉터리에 대한 하드 링크 파일 생성을 가능하게 함 (단, root 권한으로 수행하더라도 시스템 권한 제한으로 인해 실패 가능성 높음)

-f : 대상 파일이 존재할 경우 대상 파일을 지우고 링크 파일을 생성

-i : 대상 파일이 존재할 경우 대상 파일을 지울 것인지 확인 요청

-s : 심볼릭 링크 파일 생성

-S : 백업 파일을 생성할 경우 접미사를 지정

-t, --target-directory=DIRECTORY : 링크 파일을 생성할 디렉터리 지정

| 예제 3-53 |

현재 디렉터리에 존재하는 ex_data 파일에 대한 하드 링크 파일 ex_data.ln을 생성한 다음 생성된 파일목록을 확인합니다.

```
$ ln ex_data ex_data.ln
```

기능 기존 파일에 대한 하드 링크 파일 생성

형식 ln [원본 파일명] [하드 링크 파일명] [Enter]

```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -l
한계 668
-rw-r--r--. 1 cskisa cskisa 670293 11월 30 22:10 ex_data
-rw-r--r--. 1 cskisa cskisa 159 11월 30 22:07 gedit_test.txt
drwxrwxr-x. 3 cskisa cskisa 47 11월 30 22:15 temp
-rw-rw-r--. 1 cskisa cskisa 0 10월 31 08:00 test_file
-rw-r--r--. 1 cskisa cskisa 158 11월 30 22:13 text_01
-rw-r--r--. 1 cskisa cskisa 158 11월 30 22:03 text_02
[cskisa@localhost section_05]$ ln ex_data ex_data.ln
[cskisa@localhost section_05]$ ls -l
한계 1324
-rw-r--r--. 2 cskisa cskisa 670293 11월 30 22:10 ex_data
-rw-r--r--. 2 cskisa cskisa 670293 11월 30 22:10 ex_data.ln
-rw-r--r--. 1 cskisa cskisa 159 11월 30 22:07 gedit_test.txt
drwxrwxr-x. 3 cskisa cskisa 47 11월 30 22:15 temp
-rw-rw-r--. 1 cskisa cskisa 0 10월 31 08:00 test_file
-rw-r--r--. 1 cskisa cskisa 158 11월 30 22:13 text_01
-rw-r--r--. 1 cskisa cskisa 158 11월 30 22:03 text_02
[cskisa@localhost section_05]$
```

[그림 3-68] ex_data 파일에 대한 ex_data.ln 하드 링크파일 생성

- 심볼릭 링크 만들기 : -s 옵션
 - 하드 링크는 동일한 l-node 번호가 부여되는 반면 심볼릭 링크 파일은 전혀 다른 l-node 번호가 부여됨
 - 즉, 원본 파일과 심볼릭 링크파일은 전혀 다른 파일로 취급된다는 의미임

| 예제 3-54 |

현재 디렉터리에 존재하는 ex_data 파일에 대한 심볼릭 링크파일 ex_data.sl을 생성한 다음 ls 명령과 옵션 -l을 함께 사용하여 ex_data.sl 심볼릭 링크파일에 대한 내용을 상세하게 출력하기 위해 다음 명령을 수행합니다.

```
$ ln -s ex_data ex_data.sl
```

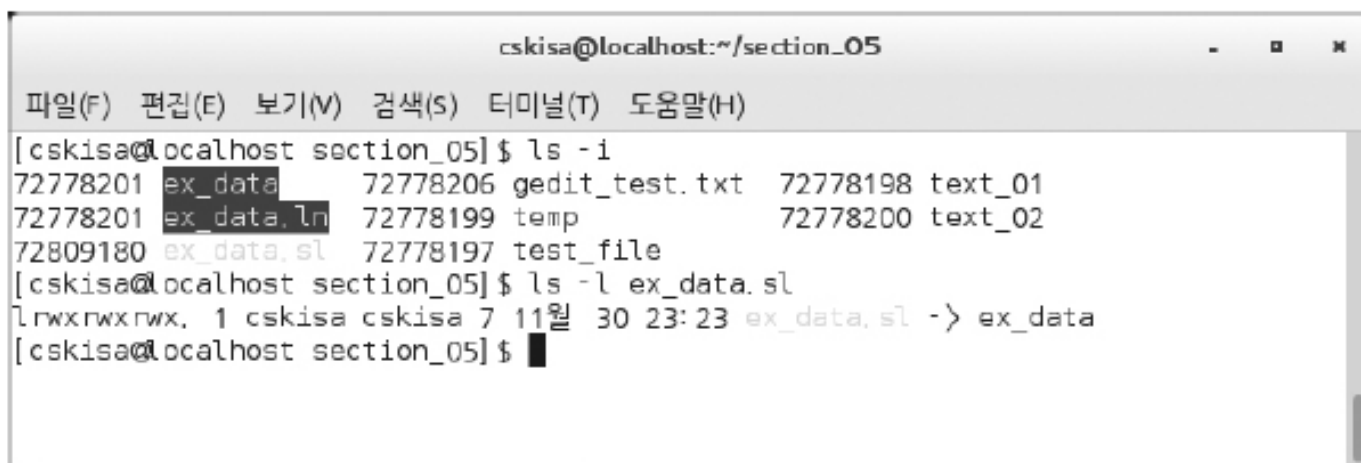
기능 원본파일인 ex_data 파일에 대한 심볼릭 파일 ex_data.sl 생성

형식 ln [옵션] [원본 파일명] [심볼릭 링크 파일명]

```
$ ls -l ex_data.sl
```

기능 심볼릭 파일인 ex_data.sl 파일에 대한 자세한 내용 출력

형식 `ls [옵션] [심볼릭 링크 파일명] [Enter]`



```
cskisa@localhost:~/section_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost section_05]$ ls -l
72778201 ex_data 72778206 gedit_test.txt 72778198 text_01
72778201 ex_data.ln 72778199 temp 72778200 text_02
72809180 ex_data.sl 72778197 test_file
[cskisa@localhost section_05]$ ls -l ex_data.sl
lrwxrwxrwx, 1 cskisa cskisa 7 11월 30 23:23 ex_data.sl -> ex_data
[cskisa@localhost section_05]$
```

[그림 3-69] 심볼릭 링크 파일 생성 후 lnode 번호와 원본파일 확인

05.리눅스 편집기(VI Editor)



- 기존 에디터의 기능을 향상시킨 vim 에디터라고 명명해야 정확한 명칭이지만
- 항상 사용해 오던 vi 에디터의 기능을 향상시킨 에디터로 인식되어 사용되고 있기 때문에 그냥 vi 에디터로 호칭
- 명령 모드, 입력 모드, 라인 모드로 구분되어 모드를 전환하면서 사용하므로 초보자는 어렵다는 선입견을 가질 수 있음

■ 에디터 작업 모드

● 명령 모드

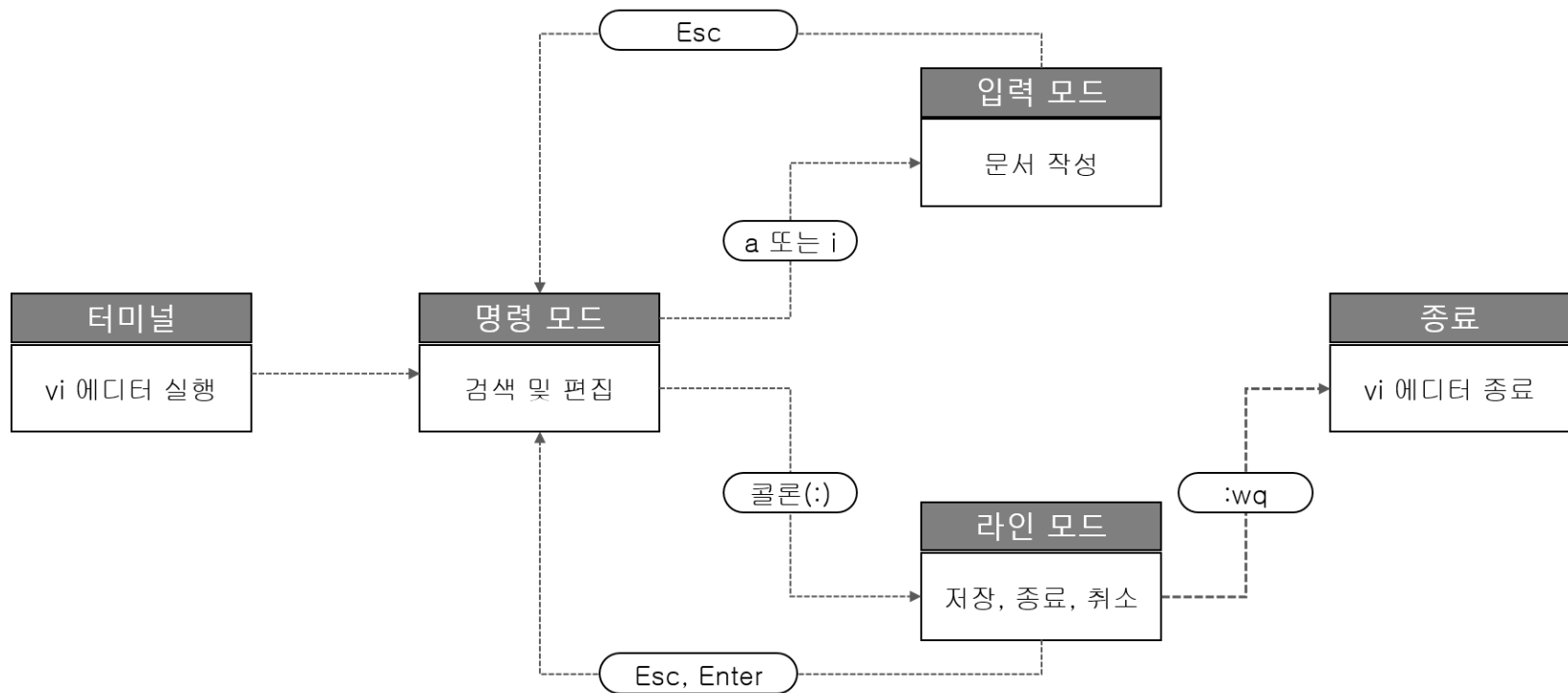
- 한 글자 또는 두 글자 명령을 사용하여 커서를 이동하고 글자를 삭제 또는 교체
- 문자열을 검색하는 등 다양하게 문서를 편집할 수 있는 기능 제공
- 명령이 아닌 키보드 입력은 모두 에러로 간주하며 입력이 잘못되었을 경우에는 경고음

- 입력 모드
 - 입력을 원하는 글자를 입력하고 화면은 입력한 상태 그대로 사용자에게 제공하는 모드
- 라인 모드
 - [Esc]키를 누르고 :(콜론) 프롬프트에서 명령을 입력하여 편집, 검색, 저장 등의 다양한 기능을 제공하는 모드

[표 4-1] vi 에디터에서 저장과 종료할 때 사용되는 명령키

모드	명령키	기능
라인 모드	q	vi 에디터에서 작업한 내용이 없으면 그냥 종료
	q!	작업한 내용을 저장하지 않고 종료
	:w[파일명]	파일명을 지정하면 새 파일로 저장 아니면 그냥 저장
	:wq 또는 :wq!	작업한 내용을 저장하고 vi 에디터를 종료
명령 모드	ZZ (Shift+[Z][Z])	작업한 내용을 저장하고 vi 에디터를 종료

작업 모드 전환



vi 시작과 종료

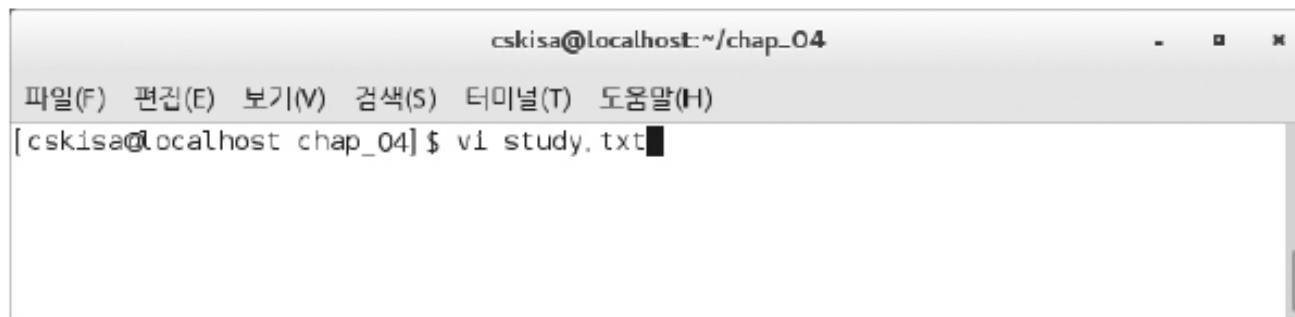
| 예제 4-3 |

- **Step 01** | 여기서는 기존에 저장되어 있는 파일을 불러와 실습하도록 하겠습니다. 불러올 파일은 [예제 4-1]과 [예제 4-2]에서 실습한 study.txt 파일이며 현재 디렉터리의 위치는 chap_04 디렉터리입니다.

```
$ vi study.txt
```

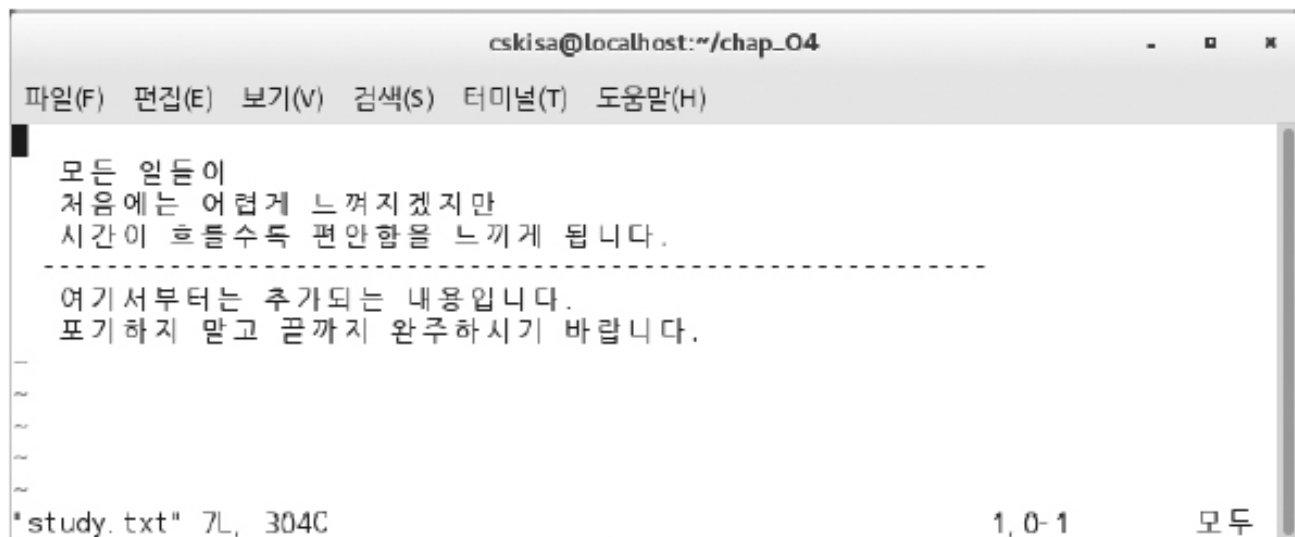
기능 지정된 파일을 읽어오고 파일을 지정하지 않으면 빈 파일이 열림

형식 vi [파일명 지정]



```
cskisa@localhost:~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_04] $ vi study.txt
```

[그림 4-8] chap_04 디렉터리에서 vi 실행



```
cskisa@localhost:~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
모든 일들이
처음에는 어렵게 느껴지겠지만
시간이 흐를수록 편안함을 느끼게 됩니다.
-----
여기서부터는 추가되는 내용입니다.
포기하지 말고 끝까지 완주하시기 바랍니다.
~
~
~
~
~
"study.txt" 7L, 304C                               1, 0-1      모두
```

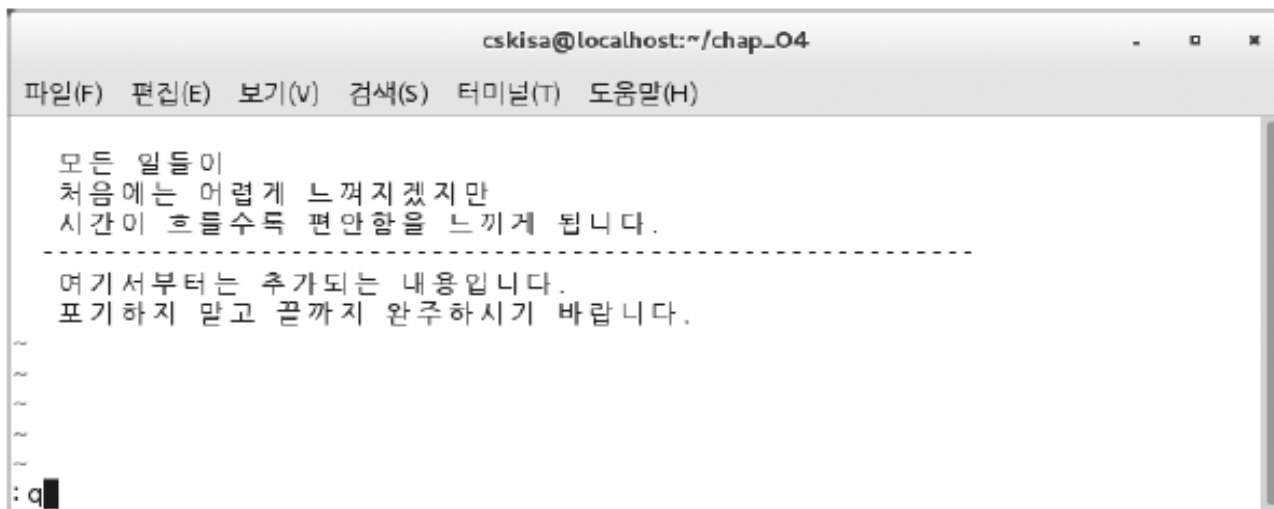
[그림 4-9] vi 에디터 창에서 study.txt 파일 열기

- **Step 02** | vi 명령으로 [그림 4-9]와 같이 study.txt 파일이 열리면 아직 편집과 수정을 하지는 않았지만 일단 저장하고 종료하는 과정에 대해서 먼저 살펴보도록 하겠습니다. study.txt 파일이 열린 vi 에디트 창은 명령 모드로 열린 상태이므로 종료를 위해 **[Esc]**를 누른 다음 콜론(:)을 눌러 라인 모드로 전환하고 명령키 q와 함께 :q를 입력하여 vi 에디터를 종료합니다.

■ :q

기능 명령 모드에서 콜론(:)을 눌러 라인 모드로 전환 (q는 종료 명령키)

형식 :[종료 명령키] **[Enter]**



[그림 4-10] **[Esc]**를 누른 다음 :q 명령키로 vi 에디터 종료

입력 모드로 전환

- chap_04 디렉터리 위치에서 sample_vi.txt 파일 생성

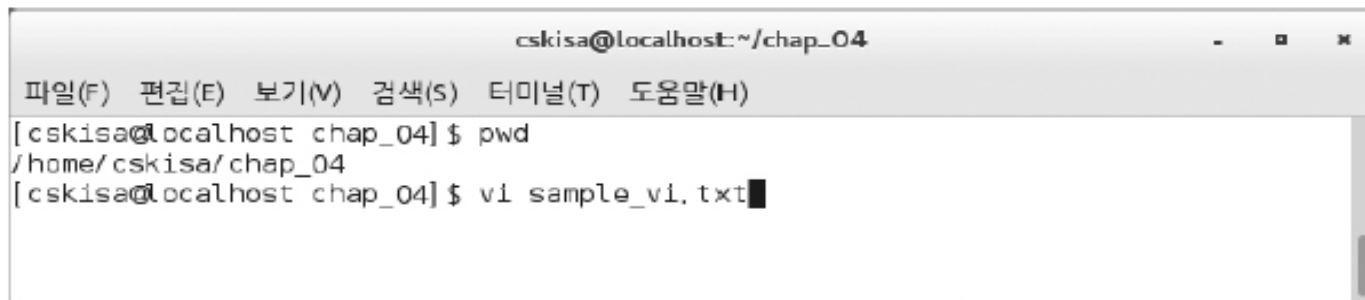
| 예제 4-4 |

- Step 01** | 현재 디렉터리의 위치가 chap_04 디렉터리가 맞는지 pwd 명령으로 확인한 다음 새로운 sample_vi.txt 파일을 vi 명령으로 생성합니다.

```
$ vi sample_vi.txt
```

기능 vi 명령으로 새로운 sample_vi.txt 파일 생성

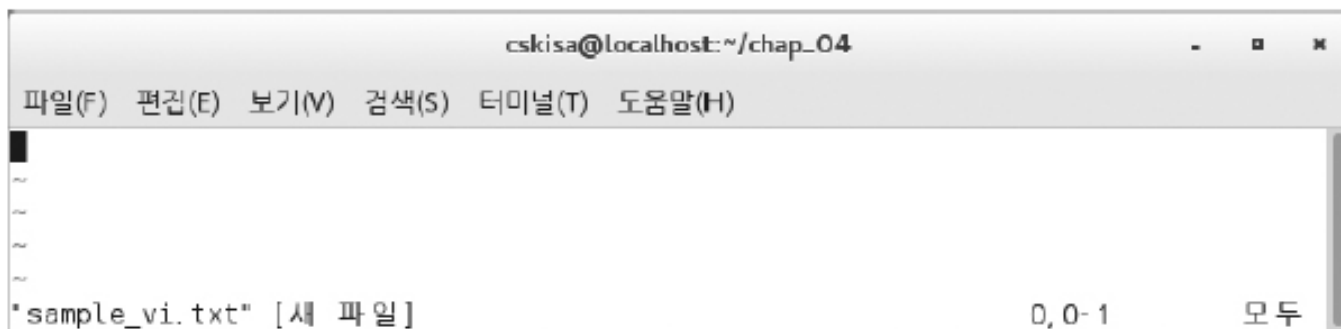
형식 vi [새로 만들 파일명 지정] **[Enter]**



A terminal window titled 'cskisa@localhost: ~/chap_04' with standard window controls. The menu bar shows '파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)'. The command history shows: [cskisa@localhost chap_04] \$ pwd, followed by the output /home/cskisa/chap_04, and then [cskisa@localhost chap_04] \$ vi sample_vi.txt with a cursor at the end of the command line.

[그림 4-11] chap_04 디렉터리에서 vi 명령으로 새 파일 sample_vi.txt 생성

- **Step 02** | vi 에디터 창이 나타나면서 하단에 새 파일 sample_vi.txt 파일에 대한 정보를 보여 줍니다. 파일이 열리면 커서(█)는 맨 처음의 위치에서 깜박거리고 있으며 vi 에디터의 현재 상태는 명령 모드 상태입니다.



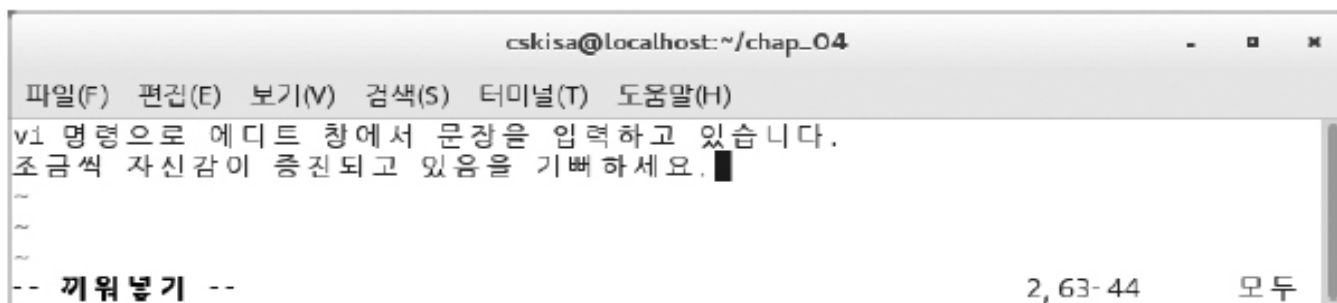
[그림 4-12] vi 명령으로 sample_vi.txt 생성 초기화면

- **Step 03** | vi 명령 실행 후 명령 모드에서 i 명령키를 사용하여 입력 모드로 변환 후 다음 문장을 입력합니다.

vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 자신감이 증진되고 있음을 기뻐하세요.

■ i

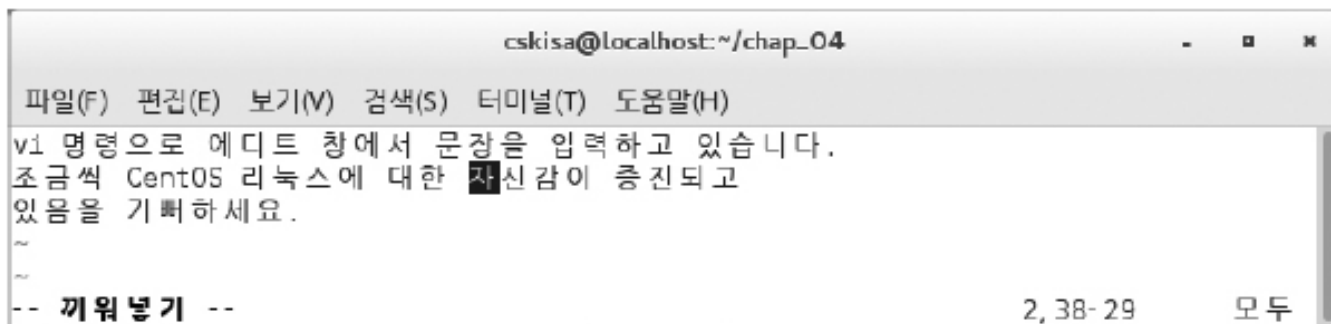
기능 명령 모드에서 명령키 i를 사용하여 입력 모드로 전환
형식 명령키 `[Enter]`



[그림 4-13] 명령키 i를 사용하여 입력 모드로 전환 뒤 문장 입력

- **Step 04** | 키보드의 화살표를 이용하여 밑줄 친 부분을 추가로 입력하고 **[Enter.]**를 쳐서 줄 바꿈을 수행합니다.

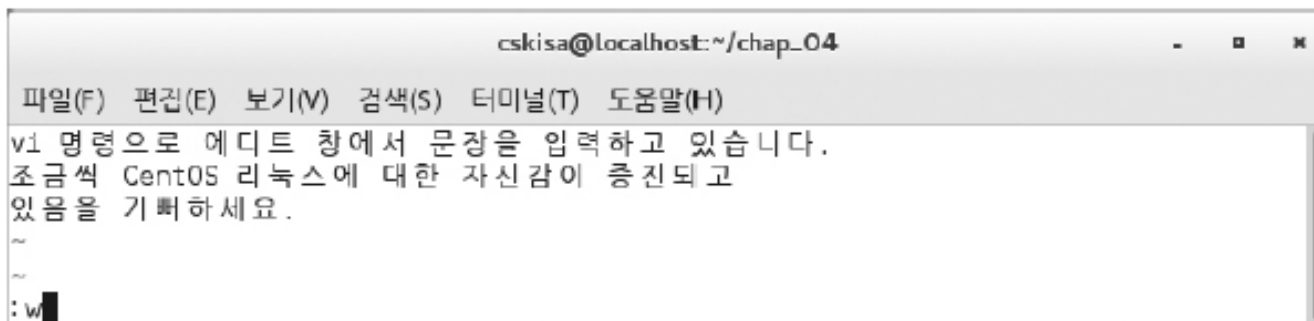
vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 CentOS 리눅스에 대한 자신감이 증진되고
있음을 기뻐하세요.



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 CentOS 리눅스에 대한 자신감이 증진되고
있음을 기뻐하세요.
~
~
-- 끼워 넣기 --                                2, 38-29      모두
```

[그림 4-14] 방향키와 **[Enter.]** 키로 추가내용 입력

- **Step 05** | sample_vi.txt 파일명으로 저장하기 위해 입력 모드에서 [Esc]를 눌러 명령 모드로 전환한 다음 콜론(:)과 함께 저장하는 명령키 w를 :w와 같이 선언하여 작성중인 파일을 저장합니다.

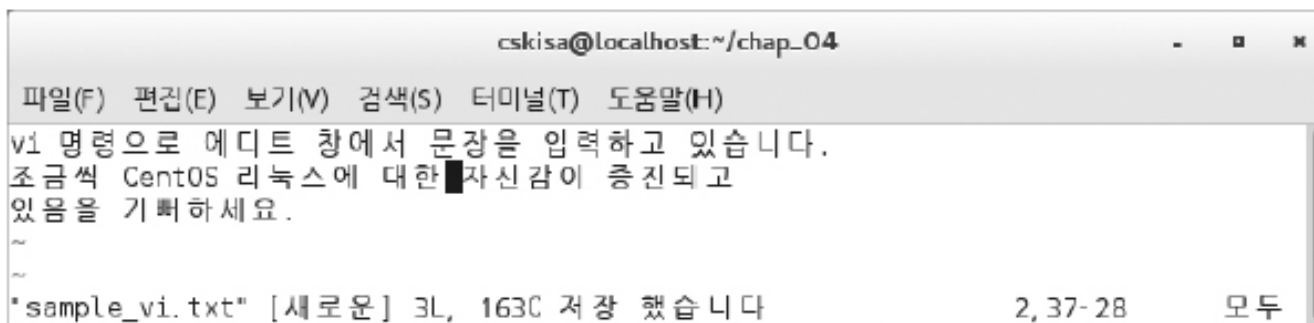


```

cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 CentOS 리눅스에 대한 자신감이 증진되고
있음을 기뻐하세요.
~
~
:w

```

[그림 4-15] [Esc]를 누른 다음 :w 명령키 사용



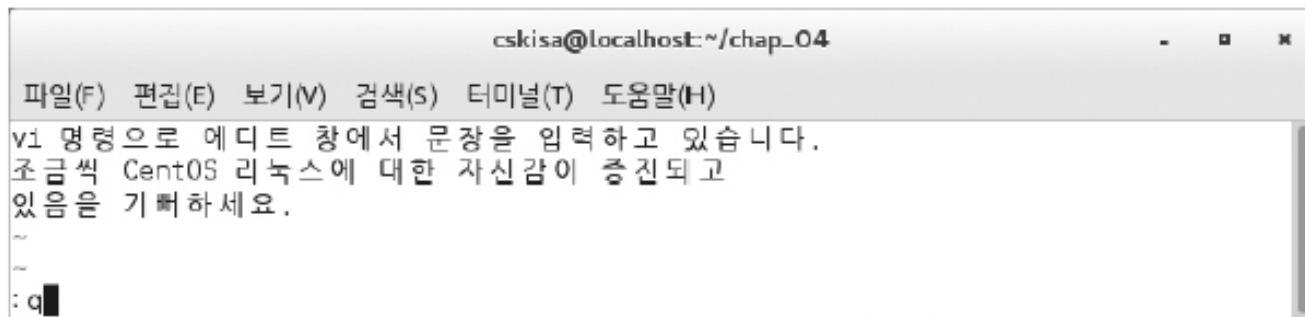
```

cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 CentOS 리눅스에 대한 자신감이 증진되고
있음을 기뻐하세요.
~
~
"sample_vi.txt" [새로운] 3L, 163C 저장 했습니다
2, 37-28      모두

```

[그림 4-16] 작업한 내용을 sample_vi.txt 파일에 저장

- **Step 06** | 저장을 완료하였으면 :q를 눌러 작업 중인 vi 명령을 종료합니다. 이 과정은 이전 단계에서 :wq 명령으로 한꺼번에 저장한 다음 종료해도 됩니다.



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
조금씩 CentOS 리눅스에 대한 자신감이 증진되고
있음을 기뻐하세요.
~
~
: q
```

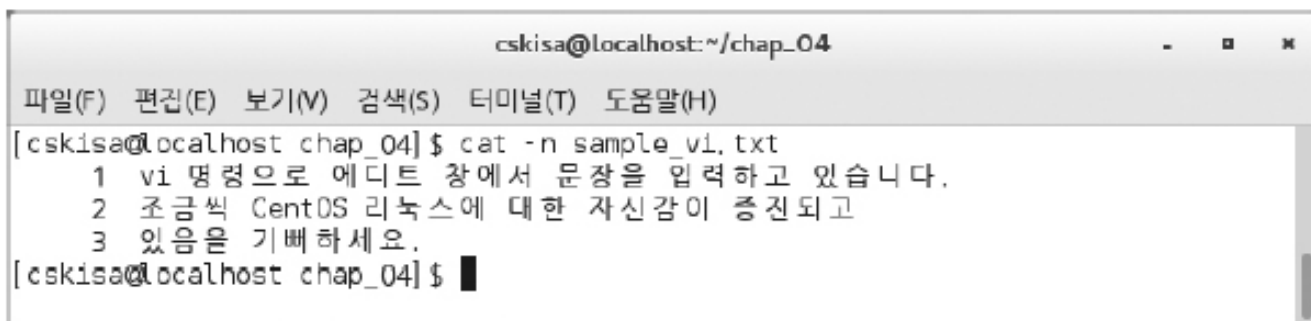
[그림 4-17] :q를 입력하여 vi 에디터 종료

- **Step 07** | 이번에는 저장된 sample_vi.txt 파일의 내용을 행 번호와 함께 출력하도록 하겠습니다.

```
$ cat -n sample_vi.txt
```

기능 현재 디렉터리 있는 sample_vi.txt 파일을 gedit 창으로 열기

형식 cat [옵션] [열고자 하는 파일명]



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_04] $ cat -n sample_vi.txt
  1  vi 명령으로 에디트 창에서 문장을 입력하고 있습니다.
  2  조금씩 CentOS 리눅스에 대한 자신감이 증진되고
  3  있음을 기뻐하세요.
[cskisa@localhost chap_04] $
```

[그림 4-18] 파일내용을 행 번호와 함께 출력

- 입력 모드 전환키

[표 4-2] 명령 모드에서 입력 모드로 전환하는 명령키

명령키	기능	동일한 키
i	현재 커서의 위치부터 입력	[I]
a	현재 커서의 위치 다음 칸부터 입력	[A]
o	현재 커서의 다음 행에 입력	[O]
s	현재 커서 위치에서 한 글자를 지우고 입력	[S]
I	현재 커서가 위치한 행의 맨 앞부터 입력	[Shift] + [I]
A	현재 커서가 위치한 행의 맨 마지막부터 입력	[Shift] + [A]
O	현재 커서가 위치한 이전 행에 입력	[Shift] + [O]
S	현재 커서의 한 행을 지우고 입력	[Shift] + [S]

커서 및 화면 이동

● 커서 이동하기

[표 4-3] 커서 이동을 위해 사용되는 명령키

명령키	기능	동일한 키
^	현재 행의 처음으로 커서 이동	[Home]
\$	현재 행의 마지막으로 커서 이동	[End]
h	커서의 위치를 왼쪽으로 한 칸 이동	[←]
l	커서의 위치를 오른쪽으로 한 칸 이동	[→]
k	커서의 위치를 위쪽으로 한 칸 이동	[↑]
j	커서의 위치를 아래쪽으로 한 칸 이동	[↓]
gg	제일 앞에 있는 첫 행으로 커서 이동	[G] [G]
g	제일 끝에 있는 행으로 커서 이동	[G]
숫자g	해당 숫자의 행으로 커서 이동	:숫자 [G]
:숫자 [Enter-J]	해당 숫자의 행으로 커서 이동	:숫자 [Enter-J]
[Ctrl] + [B]	이전 화면으로 커서 이동	[Page Up]
[Ctrl] + [F]	다음 화면으로 커서 이동	[Page Down]

- 화면 이동하기

[표 4-4] 화면 이동을 위해 사용되는 명령키

명령키	기능	동일한 키
[Ctrl] + u	빈 화면 위로 이동	
[Ctrl] + d	빈 화면 아래로 이동	
[Ctrl] + b	한 화면 위로 이동	Page Up
[Ctrl] + f	한 화면 아래로 이동	Page Down
[Ctrl] + y	화면을 한 행만 위로 이동	
[Ctrl] + e	화면을 한 행만 아래로 이동	

- 특정 행으로 커서 이동하기

[표 4-5] 특정 행으로 커서를 이동하기 위해 사용되는 명령키

명령키	기능
G ((Shift)+g)	커서를 파일의 마지막 행으로 이동
행 번호G	커서를 지정한 행 번호로 이동
행 번호	커서를 지정한 행 번호로 이동
\$	커서를 파일의 마지막 행으로 이동

문자열 검색과 치환

- 특정 문자열 검색

[표 4-6] 특정 문자열을 검색하기 위해 사용되는 명령키

명령키	기능
/문자열 Enter	찾을 문자열을 커서의 위치에서 아래쪽으로 검색
?문자열 Enter	찾을 문자열을 커서의 위치에서 위쪽으로 검색
n	문자열을 찾던 원래 방향으로 다음 문자열 검색
N	문자열을 찾던 반대방향으로 다음 문자열 검색

- Chap_04 디렉터리 위치에서 vi 명령으로 centos_vi.txt 파일 생성 후 주어진 문장을 입력한 다음 문자열 검색과정 실습

< 주어진 문장 >

CentOS 리눅스를 처음 배웁니다.
리눅스에서도 CentOS 리눅스를 열심히 배우면
누구나 리눅스에 자신을 가질 수 있으므로 CentOS 배움에
최선을 다하시기 바랍니다.

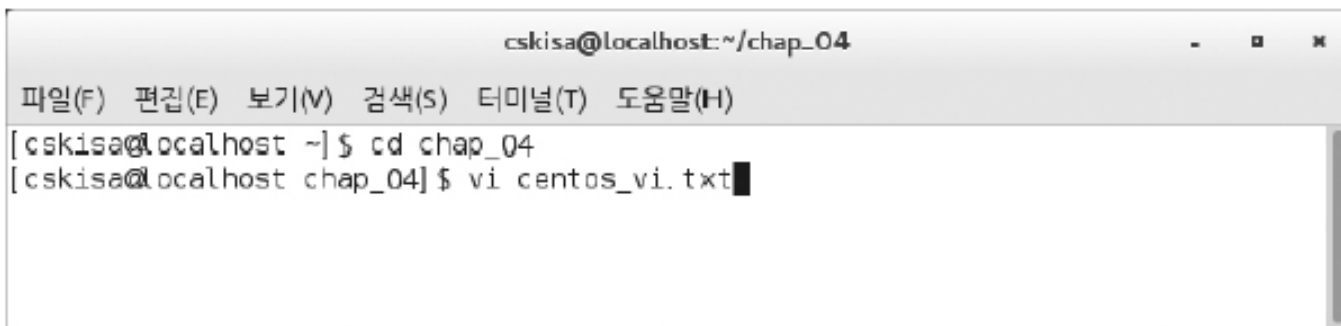
| 예제 4-5 |

- **Step 01** | 터미널 창에서 chap_04 디렉터리 안에 centos_vi.txt 파일을 vi 명령으로 생성합니다.

```
$ vi centos_vi.txt
```

기능 vi 명령으로 새로운 centos_vi.txt 파일 생성

형식 vi [새로 만들 파일명 지정]



The screenshot shows a terminal window titled 'cskisa@localhost: ~/chap_04'. The menu bar includes '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The command history shows the user navigating to the 'chap_04' directory and then running the command 'vi centos_vi.txt' to create a new file.

```
cskisa@localhost: ~/chap_04
[cskisa@localhost ~]$ cd chap_04
[cskisa@localhost chap_04]$ vi centos_vi.txt
```

[그림 4-19] chap_04 디렉터리에서 vi 명령으로 centos_vi.txt 파일생성

- **Step 02** | vi 명령 실행 후 명령 모드에서 i 명령키를 사용하여 입력 모드로 변환하고 나서 다음 문장을 입력합니다.

CentOS 리눅스를 처음 배웁니다.

리눅스에서도 CentOS 리눅스를 열심히 배우면

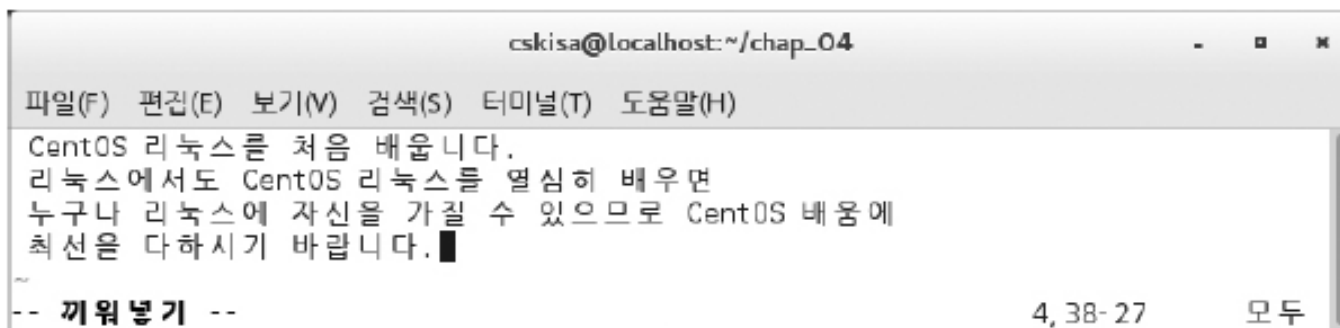
누구나 리눅스에 자신을 가질 수 있으므로 CentOS 배움에

최선을 다하시기 바랍니다.

■ i

기능 명령 모드에서 명령키 i를 사용하여 입력 모드로 전환

형식 명령키



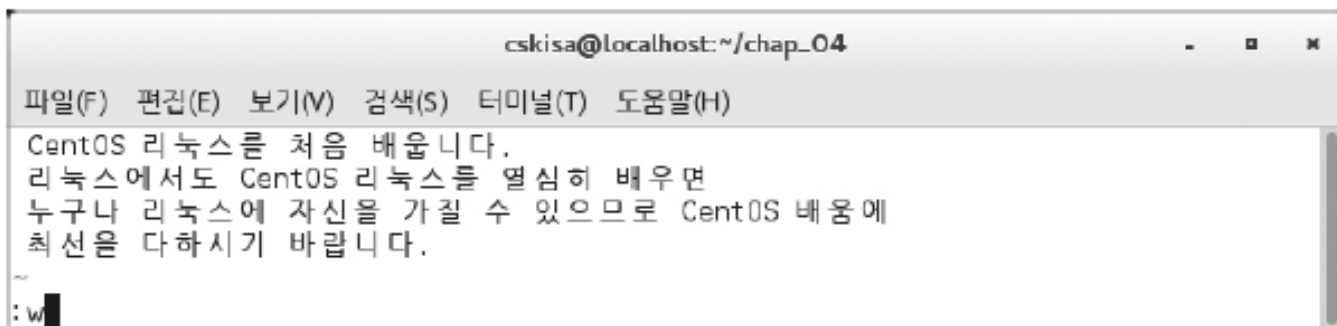
The screenshot shows a terminal window titled 'cskisa@localhost: ~/chap_04'. The window displays the contents of a file being edited in vi editor. The menu bar at the top includes '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The text content of the file is as follows:

```
CentOS 리눅스를 처음 배웁니다.
리눅스에서도 CentOS 리눅스를 열심히 배우면
누구나 리눅스에 자신을 가질 수 있으므로 CentOS 배움에
최선을 다하시기 바랍니다. █
```

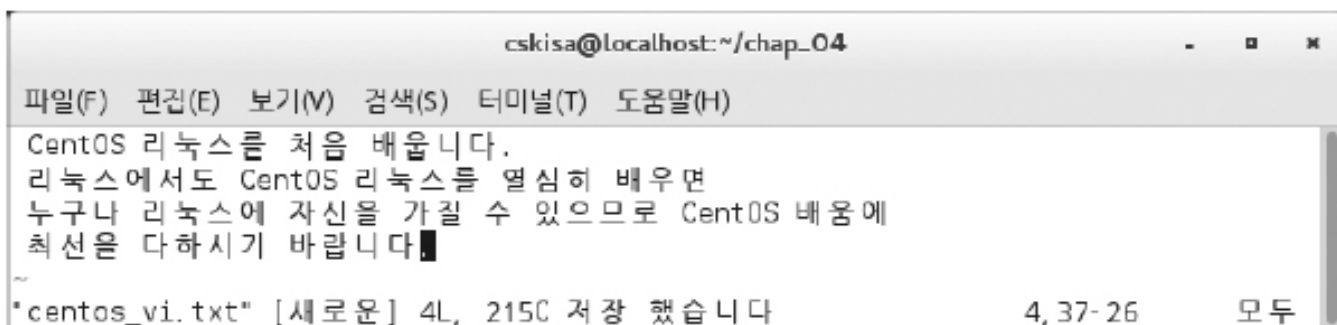
At the bottom of the window, the status line shows '-- 끼워 넣기 --' (insert mode), the cursor position '4, 38-27', and the word '모두' (all).

[그림 4-20] 명령키 i를 사용하여 입력 모드로 전환 뒤 문장 입력

- **Step 03** | centos_vi.txt 파일명으로 저장하기 위해 입력 모드에서 [Esc]를 눌러 명령 모드로 전환한 다음 콜론(:)과 함께 저장하는 명령키 w를 :w와 같이 선언하여 작성중인 파일을 저장합니다.

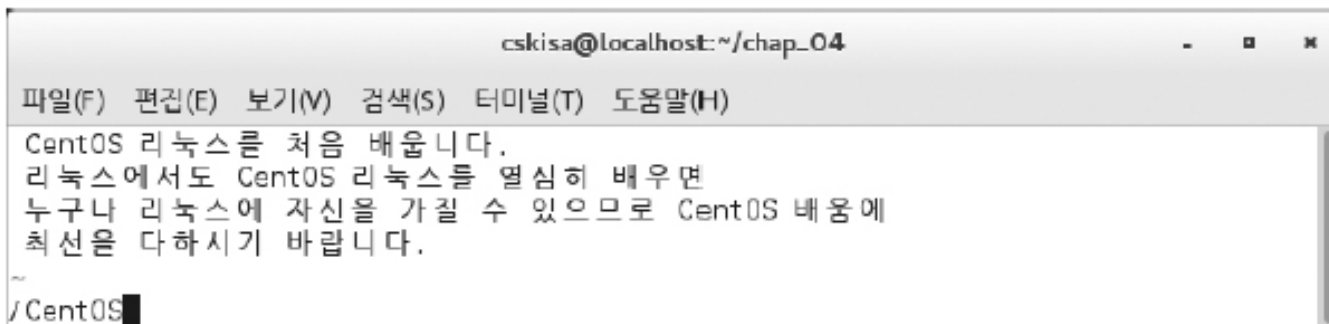


[그림 4-21] [Esc]를 누른 다음 :w 명령키 사용

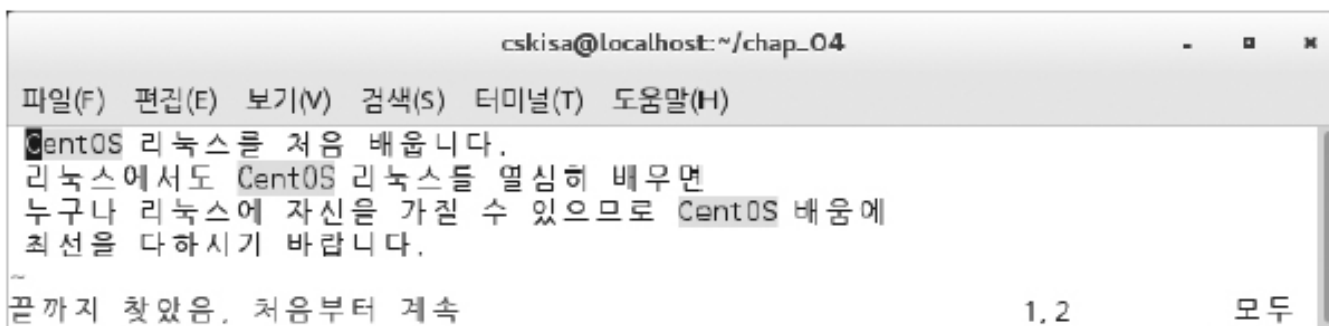


[그림 4-22] 작업한 내용을 centos_vi.txt 파일에 저장

- **Step 04** | 커서의 위치가 마지막 행에 있는 상태에서 문자열을 검색하기 위해 /를 입력하면 커서의 위치가 마지막 행으로 이동하게 됩니다. 검색할 문자열인 'CentOS'를 입력한 다음 **[Enter]**를 누릅니다.

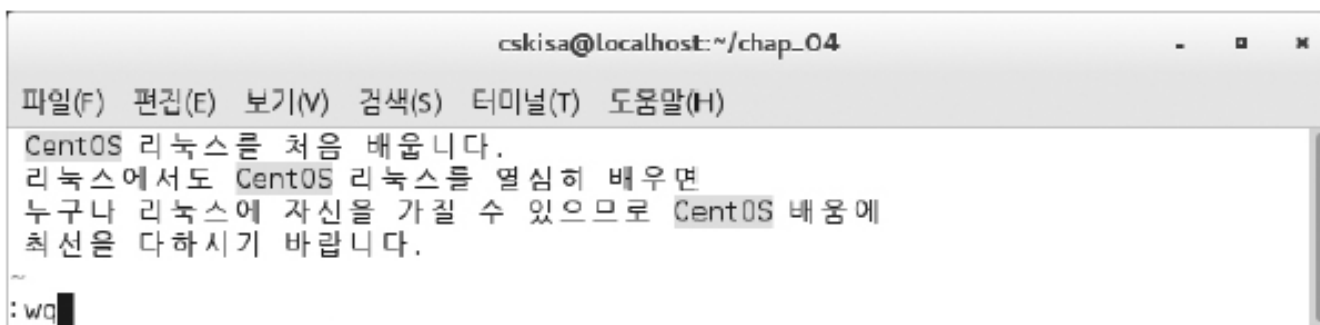


[그림 4-23] /와 문자열 CentOS를 입력 후 **[Enter]**



[그림 4-24] 문자열 CentOS 검색완료

- **Step 05** | 문자열 검색이 완료된 것을 확인하였으면 centos_vi.txt를 저장하기 위해 콜론(:)을 눌러 라인 모드로 전환한 다음 저장과 종료 명령키를 함께 :wq와 같이 선언하여 작성중인 파일을 저장 후 vi를 종료합니다.



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
CentOS 리눅스를 처음 배웁니다.
리눅스에서도 CentOS 리눅스를 열심히 배우면
누구나 리눅스에 자신을 가질 수 있으므로 CentOS 배움에
최선을 다하시기 바랍니다.
~
:wq
```

[그림 4-25] :wq를 입력하여 파일을 저장 후 vi 에디터 종료

- 문자열 치환

[표 4-7] 특정 문자열을 치환하기 위해 사용되는 명령키

명령키	기능
:s/문자열1/문자열2/	커서가 위치한 행에서 첫 번째로 나오는 문자열1을 문자열2로 치환
:%s/문자열1/문자열2/g	파일 전체에서 모든 문자열1을 문자열2로 치환
:<범위>s/문자열1/문자열2/	범위 내 모든 행의 각행에서 첫 번째로 검색되는 문자열1을 문자열 2로 치환
:<범위>s/문자열1/문자열2/g	범위 내 모든 행에서 문자열1을 문자열2로 치환
:<범위>s/문자열1/문자열2/gc	범위 내 모든 행에서 문자열1을 문자열2로 치환할 때 수정할 지에 대한 여부 확인

파일내용 수정 및 삭제

● 파일내용 수정

[표 4-8] 파일내용 수정을 위해 사용되는 명령키

명령키	기능
<code>x</code>	커서가 위치한 내용을 다른 내용으로 수정
<code>cw, #cw</code>	커서 위치부터 현재 단어의 끝까지 수정하고 #에는 수정할 단어 수 지정
<code>s, #s</code>	커서 위치부터 <code>[Esc]</code> 를 누를 때까지 수정하고 #에는 수정할 글자 수 지정
<code>cc</code>	커서가 위치한 행의 내용을 모두 수정
<code>C</code>	커서 위치부터 행의 끝까지 내용을 수정

- 내용 삭제

[표 4-9] 내용 삭제에 사용되는 명령키

명령키	기능
x, #x	커서 위치의 글자를 삭제하고 #은 삭제할 글자 수를 지정
dw, #dw	커서 위치의 단어를 삭제하고 #은 삭제할 단어 수를 지정
dd, #dd	커서 위치의 행을 삭제하고 #은 삭제할 행의 수를 지정
D	커서 위치부터 행의 끝까지 삭제

수행한 명령 취소

- Vi에서는 [Ctrl]+[Z]를 사용하게 되면 화면이 사라지고 프롬프트가 나타나는 결과를 초래

[표 4-10] 이전 명령을 취소할 때 사용되는 명령키

명령키	기능
u	명령을 취소
U	해당 행에서 수행한 모든 명령을 취소
:e!	마지막으로 저장한 내용 이후의 것을 버리고 새로 작업

■ 도움말 사용 : man

- 도움말을 사용하고자 할 때는 manual의 약어인 man 명령을 사용

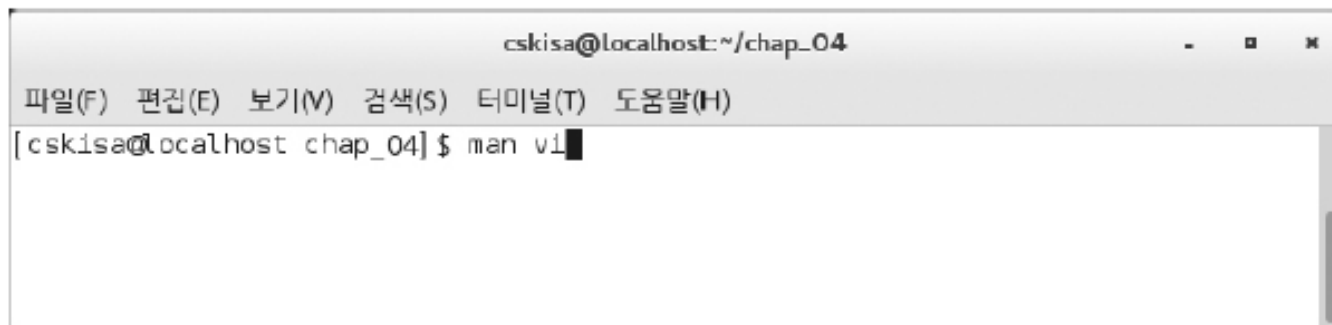
| 예제 4-6 |

vi 명령에 대한 도움말을 출력합니다.

```
$ man vi
```

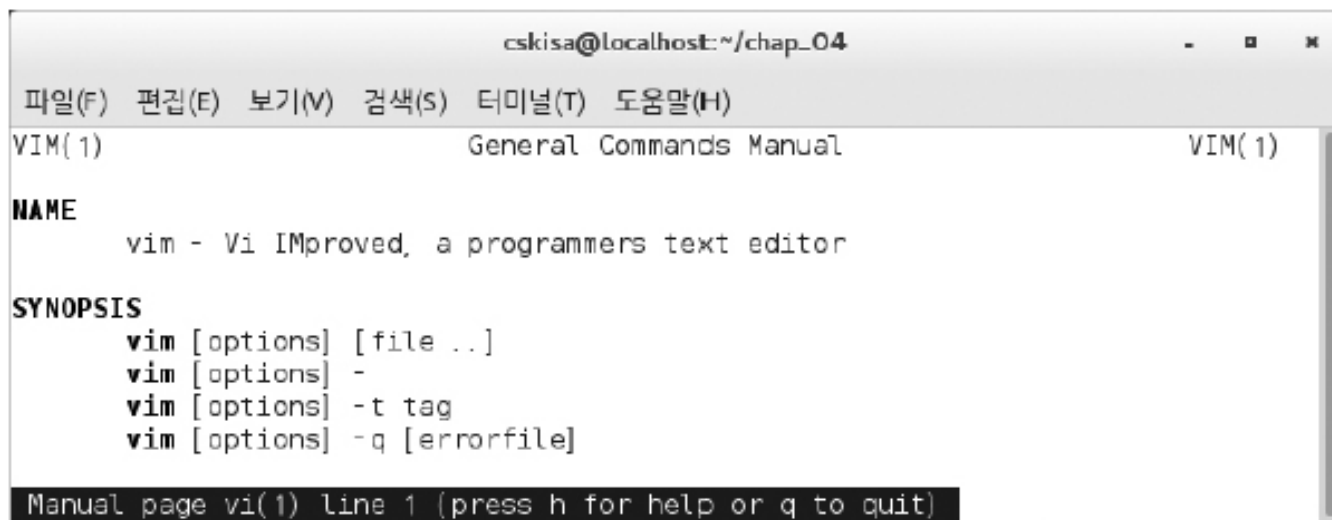
기능 리눅스에 포함된 체계화된 도움말 사용

형식 man [도움말이 필요한 명령]



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_04] $ man vi
```

[그림 4-26] vi 명령에 대한 도움말 요청



```
cskisa@localhost: ~/chap_04
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
VIM(1)                                General Commands Manual          VIM(1)

NAME
    vim - Vi IMproved, a programmers text editor

SYNOPSIS
    vim [options] [file ..]
    vim [options] -
    vim [options] -t tag
    vim [options] -q [errorfile]

Manual page vi(1) line 1 (press h for help or q to quit)
```

[그림 4-27] vi 명령에 대한 도움말 내용

06.파일 접근권한 설정

1. 파일 속성
2. 파일 접근권한
3. 기호를 이용한 접근권한
4. 숫자를 이용한 접근권한
5. 기본 접근권한
6. 특수 접근권한



SECTION
01

파일 속성

- 파일의 속성에 대해 이해하기



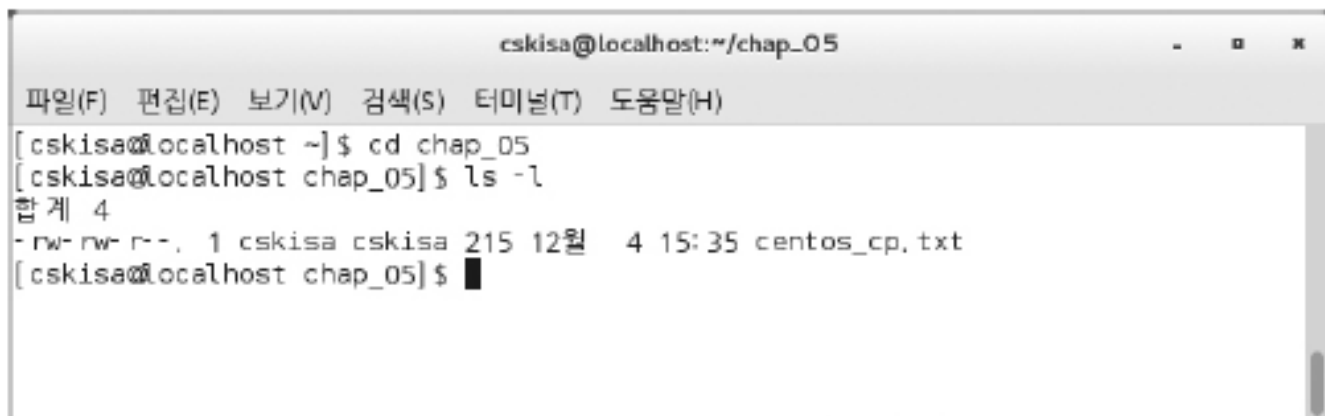
1. 파일의 종류에 대해 살펴봅니다.
2. 파일 소유자와 그룹 등 세부적인 파일 속성을 이해합니다.

- **Step 03** | 사용자 홈 디렉터리에서 chap_05 디렉터리로 이동한 다음 앞에서 복사한 파일 centos_cp.txt에 대한 상세한 정보를 출력하기 위해 `ls -l` 명령을 수행합니다.

```
$ ls -l
```

기능 파일의 정보를 상세하게 출력

형식 `ls [옵션]`



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ cd chap_05
[cskisa@localhost chap_05]$ ls -l
합계 4
-rw-rw-r--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

[그림 5-3] centos_cp.txt 파일의 상세정보 출력

[그림 5-3]에서 보는 바와 같이 centos_cp.txt 파일에 대한 상세한 정보가 출력되었습니다. 파일을 마지막으로 수정한 날짜에는 연도가 생략되어 있는데 이는 파일이 최종적으로 금년에 수정되었음을 의미합니다. 출력된 파일의 정보에 대해 각 항목별로 상세하게 살펴보도록 하겠습니다.

-	rw- rw- r--	.	1	cskisa	cskisa	215	12월 4 15:35	centos_cp.txt
①	②	③	④	⑤	⑥	⑦	⑧	

이와 같은 파일의 상세한 정보에 대해 항목별로 [표 5-1]과 같이 정리하였습니다.

[표 5-1] 파일 속성에 대한 상세 정보

번호	속성 값	의미
①	-	파일의 종류 (-는 일반파일, d는 디렉터리)
②	rw- rw- r--	파일을 읽고(rw-) 쓰고(rw-) 실행(r--)할 수 있는 접근 권한
③	1	하드 링크의 수
④	cskisa	파일 소유자의 로그인 ID
⑤	cskisa	파일 소유자의 그룹 이름
⑥	215	파일의 크기 (byte 단위)
⑦	12월 4 15:35	파일을 최종 수정한 일자와 시각
⑧	centos_cp.txt	파일명

① 파일의 종류

- 파일 속성의 첫 번째 항목은 파일의 종류를 표시
 - - : 일반 파일
 - d : 디렉터리
- 파일의 종류를 확인하기 위한 명령

\$ file

기능 지정한 파일의 종류를 알려줌

형식 file [디렉터리명 또는 파일명]

- 파일의 종류를 파악하기 전에 현재 디렉터리의 위치 확인
 - 현재 디렉터리의 위치 : chap_05
 - 상위 디렉터리로 이동 : ../

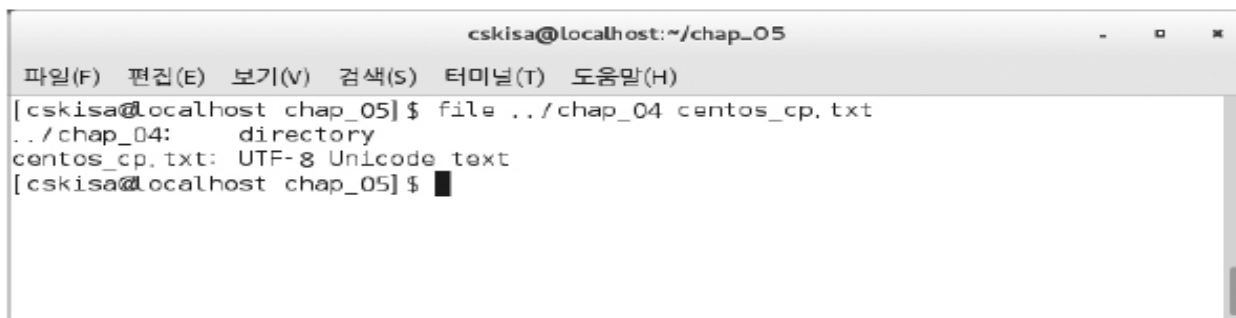
| 예제 5-2 |

현재 디렉터리의 위치에서 chap_04 디렉터리에 존재하는 centos_cp.txt 파일의 종류를 출력합니다.

```
$ file ../chap_04 centos_cp.txt
```

기능 chap_04와 centos_cp.txt 파일의 종류 출력

형식 file [디렉터리명] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05] $ file ../chap_04 centos_cp.txt
.. /chap_04:      directory
centos_cp.txt:   UTF-8 Unicode text
[cskisa@localhost chap_05] $
```

[그림 5-4] chap_04와 centos_cp.txt 파일의 종류 출력

② 파일의 접근권한 표시

- 파일의 속성에서 두 번째 항목인 `rw- rw- r--`의 의미하며 파일에 접근하여 읽고 쓰는 권한을 부여하기 위함
 - `rw-` : 소유자 접근권한
 - `rw-` : 그룹 접근권한
 - `r--` : 기타 사용자 접근권한

③ 하드 링크의 수

- 하드 링크는 하나의 파일에 대해 여러 개의 파일명을 부여할 수 있는 기능
- 3장에서 이미 살펴보았으므로 자세한 사항은 3장을 참조

④ 파일 소유자의 로그인 ID

- 리눅스 시스템에서 취급되는 모든 파일에는 각각의 소유자가 지정되어 있음
- 시스템과 관련된 파일은 대부분 루트계정이 소유자이고 일반 파일은 해당 파일을 생성한 사용자가 파일의 소유자가 됨
- Centos_cp.txt 파일의 경우 사용자 계정 cskisa 아이디로 접속된 상태에서 생성한 파일이므로 파일 소유자의 로그인 ID는 cskisa로 출력됨

⑤ 파일 소유자의 그룹이름

- 시스템 관리자가 사용자를 등록할 때 파일 소유자의 그룹이 결정
- 임의로 소유자가 속할 그룹을 바꿀 수는 없으며 변경하고자 할 경우에는 반드시 시스템 관리자에게 그룹 변경을 요청해야 함
- 그룹 변경에 대해 정의는 /etc/group 디렉터리에서 관리

- 현재 접속 중인 사용자가 속한 그룹 조회

\$ groups

기능 현재 접속 중인 사용자가 속한 그룹을 알려줌

형식 groups [사용자명]

- Groups 명령을 사용할 때 ‘사용자명’을 인자로 지정하면 그 사용자가 속한 그룹을 알려주지만
- 인자를 지정하지 않으면 현재 접속 중인 사용자가 속한 그룹을 알려줌

- 현재 접속 중인 사용자가 속한 그룹 조회

| 예제 5-3 |

현재 접속 중인 사용자가 속한 그룹과 root 계정이 속한 그룹에 대한 정보를 출력합니다.

\$ groups

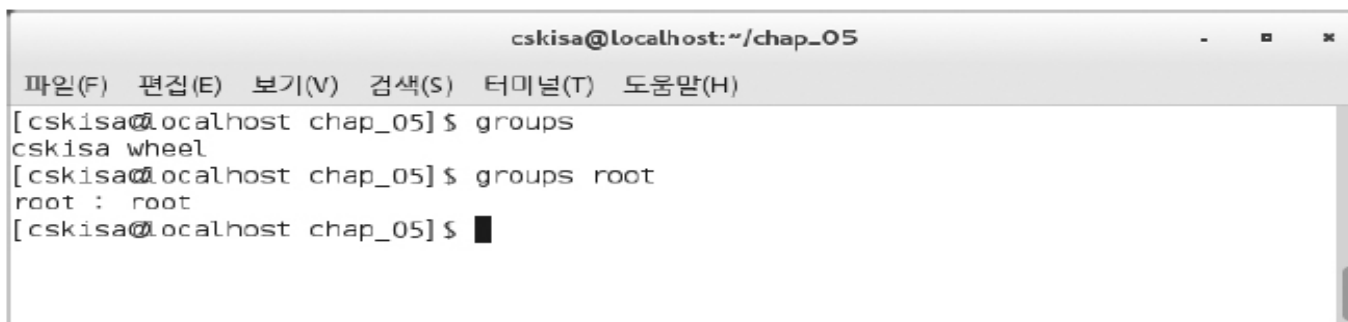
기능 현재 접속 중인 사용자가 속한 그룹을 알려줌

형식 groups [사용자명]

\$ groups root

기능 현재 접속 중인 사용자가 속한 그룹을 알려줌

형식 groups [사용자명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ groups
cskisa wheel
[cskisa@localhost chap_05]$ groups root
root : root
[cskisa@localhost chap_05]$
```

[그림 5-5] 사용자와 root가 속한 그룹 확인

⑥ 파일의 크기

- 파일의 크기는 byte 단위로 알려줌
- [예제 5-1]의 출력결과에서 centos_cp.txt 파일의 크기가 215 byte로 출력

⑦ 파일이 수정된 마지막 일자와 시각

- 파일이 마지막으로 수정된 일자와 시각 표시
- 연도가 표시되지 않는 경우는 금년을 의미

⑧ 파일명

- 실질적으로 사용되고 있는 파일명
- 파일의 종류에 따라 디렉터리 또는 파일의 이름으로 출력



파일 접근권한

- 파일 접근권한에 대해 이해하기



- 1 파일 접근권한의 종류에 대해서 살펴봅니다.
- 2 파일 접근권한 표기방법에 대해 이해합니다.
- 3 파일 접근권한 변경방법을 예제를 통해 실습합니다.

접근권한 종류

- 리눅스 시스템에서 사용되는 파일에 대한 접근권한은 읽기, 쓰기, 실행권한 등 3가지 권한으로 제한

[표 5-2] 파일과 디렉터리 접근권한

접근권한	파일	디렉터리
읽기(r)	해당 파일을 읽거나 복사 가능	ls 명령으로 디렉터리의 목록을 확인 (옵션은 실행권한이 있을 경우만 허용)
쓰기(w)	파일을 수정, 이동, 삭제 가능 (디렉터리에 쓰기권한 있어야함)	해당 파일을 삭제하거나 생성할 수 있음
실행(x)	셸 스크립트 또는 실행파일에 대한 실행 가능	cd 명령 사용 가능하며 파일을 디렉터리 로 이동 또는 복사 가능

접근권한 표기방법

- 리눅스 시스템에 접속한 사용자 카테고리별로 누가 어떤 파일에 대해 읽기, 쓰기, 실행권한을 할 수 있는지에 대해 문자로 표현
- [예제 5-1]에서 실행한 결과 centos_cp.txt 파일의 접근권한 출력

```
- rw- rw- r-- . 1 cskisa cskisa 215 12월 4 15:35 centos_cp.txt
```

기타 사용자 권한

파일의 크기

쓰기 권한

파일 소유자의 로그인 ID

읽기 권한

일반파일

접근권한 표기방법

- 다양한 파일 접근권한의 조합

[표 5-3] 다양한 파일 접근권한 조합

접근권한	의미
<code>rwX r-X r-X</code>	소유자 권한(읽기, 쓰기, 실행), 그룹과 기타 사용자 권한(읽기, 실행)
<code>r-X r-X r-X</code>	소유자, 그룹, 기타 사용자(읽기, 실행) 권한만 부여
<code>rw- --- ---</code>	소유자 (읽기, 쓰기) 권한만 부여 그룹과 기타 사용자는 권한 없음
<code>rw- rw- rw-</code>	소유자, 그룹, 기타 사용자 모두(읽기, 쓰기) 권한만 부여
<code>rwX rwX rwX</code>	소유자, 그룹, 기타 사용자 모두(읽기, 쓰기, 실행) 권한 부여
<code>rwX --- ---</code>	소유자 (읽기, 쓰기, 실행) 권한만 부여
<code>r-- --- ---</code>	소유자 (읽기) 권한만 부여

■ 접근권한 변경 명령 : chmod

- 접근권한 변경은 파일 소유자와 시스템 관리자만 변경 가능하며 파일 접근권한 변경 명령은 change mode의 약어인 chmod 명령을 사용

```
$ chmod
```

기능 디렉터리 또는 파일의 접근권한 변경

형식 chmod [옵션]

옵션 -R : 하위 디렉터리까지 접근권한을 모두 변경

SECTION
03

기호를 이용한 접근권한

- 파일 접근권한 변경을 위해 심볼릭 모드 사용 이해하기



1. 파일 접근권한을 변경하는 심볼릭 모드에 대해 이해합니다.
2. 심볼릭 모드에서 사용되는 문자와 기호를 살펴봅니다.
3. 심볼릭 모드로 파일 접근권한을 변경하는 방법에 대해 실습합니다.

심볼릭 모드

- 기호를 이용한 접근권한 변경방법인 심볼릭 모드는 사용자 카테고리 문자, 연산자 기호, 접근권한 문자를 사용하여 파일 접근권한 변경



- 심볼릭 모드에서 사용되는 문자와 기호

[표 5-4] 심볼릭 모드에서 사용되는 문자와 기호의 종류

구분	문자/기호	의미
사용자 카테고리 문자	u	파일 소유자
	g	파일 소유자가 속한 그룹
	o	파일 소유자와 그룹 이외의 기타 사용자
	a	파일을 사용하려는 전체 사용자
연산자 기호	+	파일 접근권한 부여
	-	파일 접근권한 제거
	=	파일 접근권한 설정
접근권한 문자	r	파일 읽기권한
	w	파일 쓰기권한
	x	파일 실행권한

- 심볼릭 모드에서 다양한 파일 접근권한

[표 5-5] 심볼릭 모드에서 다양한 파일 접근권한 조합

권한 표기	의미
u+w	파일 소유자에게 쓰기(w) 권한부여
u-w	파일 소유자에게 쓰기(w) 권한제거
u=rwx	파일 소유자에게 쓰기(w), 읽기(r), 실행(x) 권한설정
u+x, go+w	소유자에게 실행(x) 권한부여와 그룹 및 기타 사용자에게 쓰기(w) 권한부여
g+w	파일 그룹에게 쓰기(w) 권한부여
g+wx	파일 그룹에게 쓰기(w)와 실행(x) 권한부여
go+w	그룹과 기타 사용자에게 쓰기(w) 권한부여
+wx	파일을 사용하려는 모든 사용자에게 쓰기(w)와 실행(x) 권한부여
a+rwx	모든 사용자에게 쓰기(w), 읽기(r), 실행(x) 권한부여
o-r	파일 기타 사용자에게 읽기(r) 권한제거

심볼릭 모드로 접근권한 변경

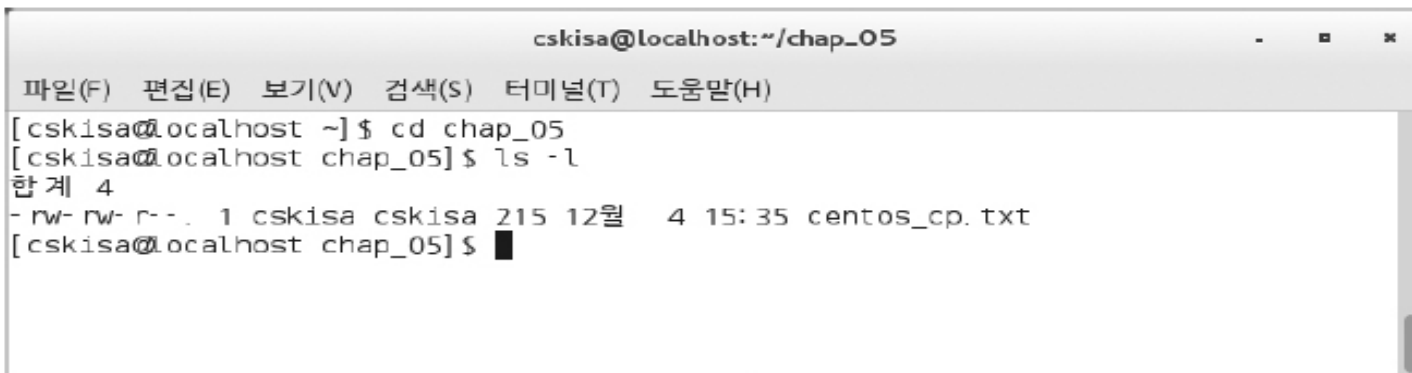
| 예제 5-4 |

- **Step 01** | 터미널 창에서 현재 디렉터리의 위치를 확인하여 chap_05 디렉터리 안으로 이동한 다음 디렉터리에 존재하는 centos_cp.txt 파일에 대해 ls -l 명령으로 파일 접근권한이 현재 어떻게 설정되어 있는지에 대한 상세정보를 출력합니다.

```
$ ls -l
```

기능 파일의 정보를 상세하게 출력

형식 ls [옵션]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ cd chap_05
[cskisa@localhost chap_05]$ ls -l
한계 4
-rw-rw-r--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

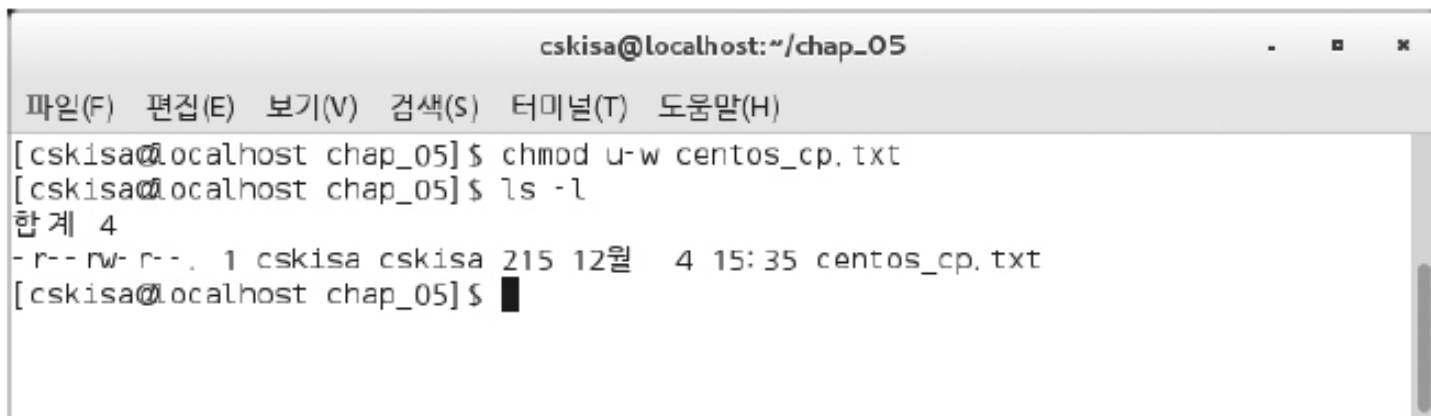
[그림 5-8] centos_cp.txt 파일의 상세정보 출력

- **Step 02** | centos_cp.txt 파일의 현재 접근권한은 rw- rw- r--으로 설정되어 있습니다. 여기에서 접근권한 첫 번째 항목인 소유자 권한 중 쓰기 권한을 제거하기 위해 문자와 연산자 기호를 u-w와 같이 선언하여 접근권한을 변경합니다.

```
$ chmod u-w centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [심볼릭 모드] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05] $ chmod u-w centos_cp.txt
[cskisa@localhost chap_05] $ ls -l
합계 4
-r--rw- r--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05] $
```

[그림 5-9] centos_cp.txt 파일 접근권한 변경

- **Step 03** | centos_cp.txt 파일에 설정되어 있던 접근권한은 `rw- rw- r--`에서 소유자 쓰기 권한제거 옵션 `u-w`를 통해 `r-- rw- r--`으로 접근권한이 변경 설정되었음을 확인합니다.

<접근권한 변경 전>

```
-  rw- rw- r-- . 1  cskisa  cskisa  215  12월  4  15:35  centos_cp.txt
  ▲ 당초
```

<접근권한 변경 후>

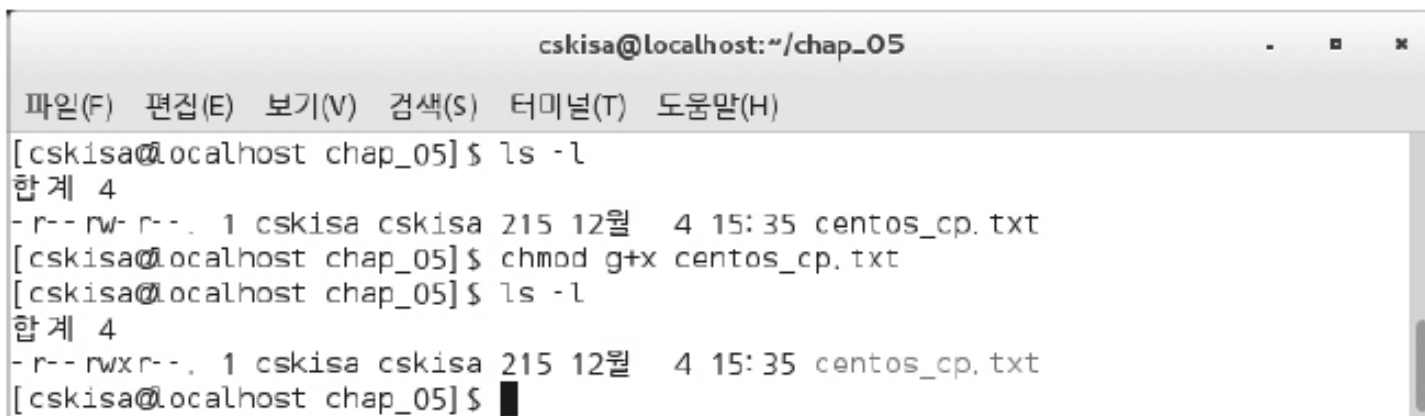
```
-  r-- rw- r-- . 1  cskisa  cskisa  215  12월  4  15:35  centos_cp.txt
  ▲ 변경
```

- **Step 04** | centos_cp.txt 파일의 그룹에 심볼릭 모드 g+x를 지정하여 그룹에 대한 실행권한을 부여합니다.

```
$ chmod g+x centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [심볼릭 모드] [파일명]



```
cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rw-r--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod g+x centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rwxr--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

[그림 5-10] 그룹에 실행권한 부여

- **Step 05** | 이 단계에서는 심볼릭 모드 o+x를 지정하여 기타 사용자에게 파일에 대한 실행권한을 부여합니다.

```
$ chmod o+x centos_cp.txt
```

기능 파일의 접근권한 변경

형식 `chmod [심볼릭 모드] [파일명]`

```

cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rwxr--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod o+x centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$

```

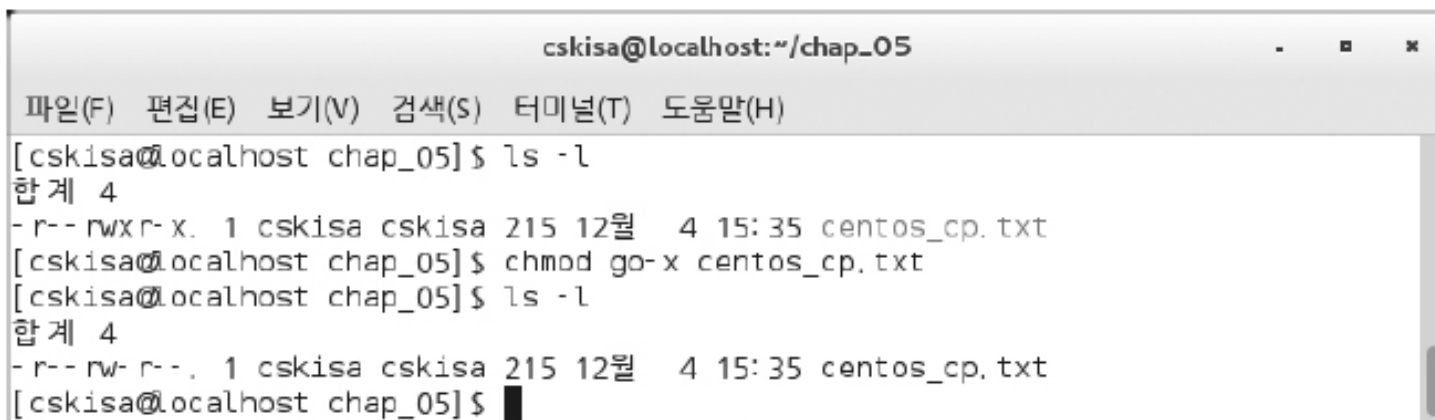
[그림 5-11] 기타 사용자에게 실행권한 부여

- **Step 06** | 다음은 centos_cp.txt 파일에 대한 그룹과 기타 사용자의 파일 실행권한을 심볼릭 모드 go-x를 지정하여 제거합니다.

```
$ chmod go-x centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [심볼릭 모드] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod go-x centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--rw- r-. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

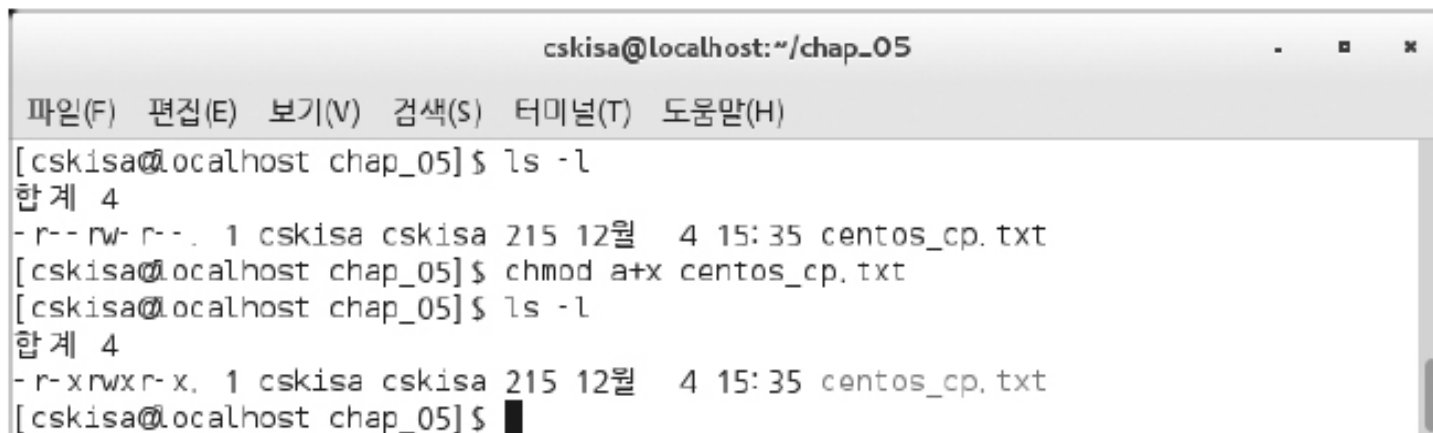
[그림 5-12] 그룹과 기타 사용자의 실행권한 제거

- **Step 07** | 이 단계에서는 심볼릭 모드 a+x를 지정하여 centos_cp.txt 파일을 모든 사용자에게 실행권한을 부여합니다.

```
$ chmod a+x centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [심볼릭 모드] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r--r--r--. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod a+x centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xr-xr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

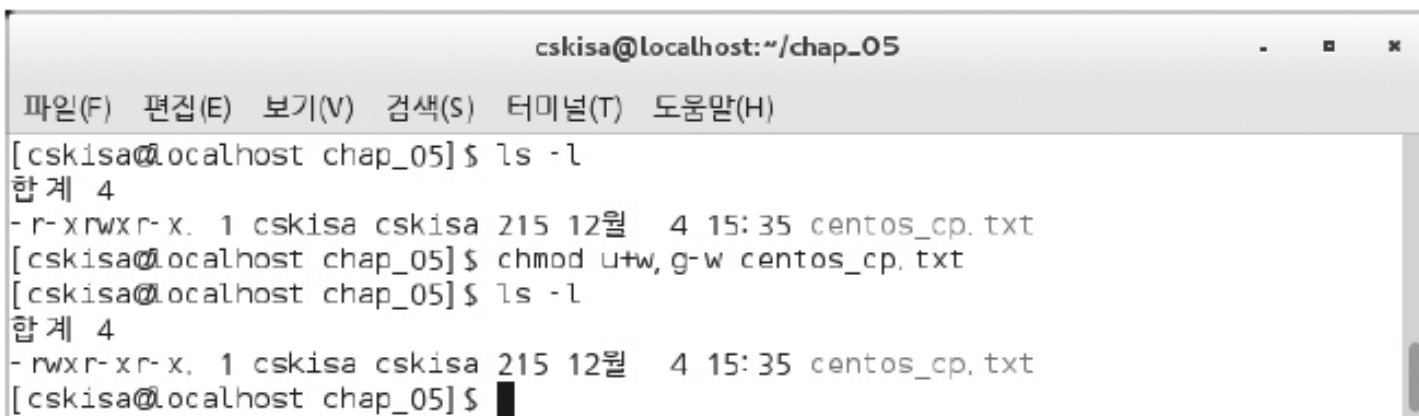
[그림 5-13] 모두에게 실행권한 부여

- **Step 08** | 마지막 단계로 centos_cp.txt 파일 소유자에게 u+w를 지정하여 파일에 대한 쓰기권한을 부여하고 그룹의 쓰기권한은 g-w를 지정하여 파일에 대한 쓰기권한을 각각 제거합니다.

```
$ chmod u+w,g-w centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [심볼릭 모드],[심볼릭 모드] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xrwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod u+w,g-w centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-rwxr-xr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

[그림 5-14] 소유자 쓰기권한 부여와 그룹의 쓰기권한 제거

SECTION
04

숫자를 이용한 접근권한

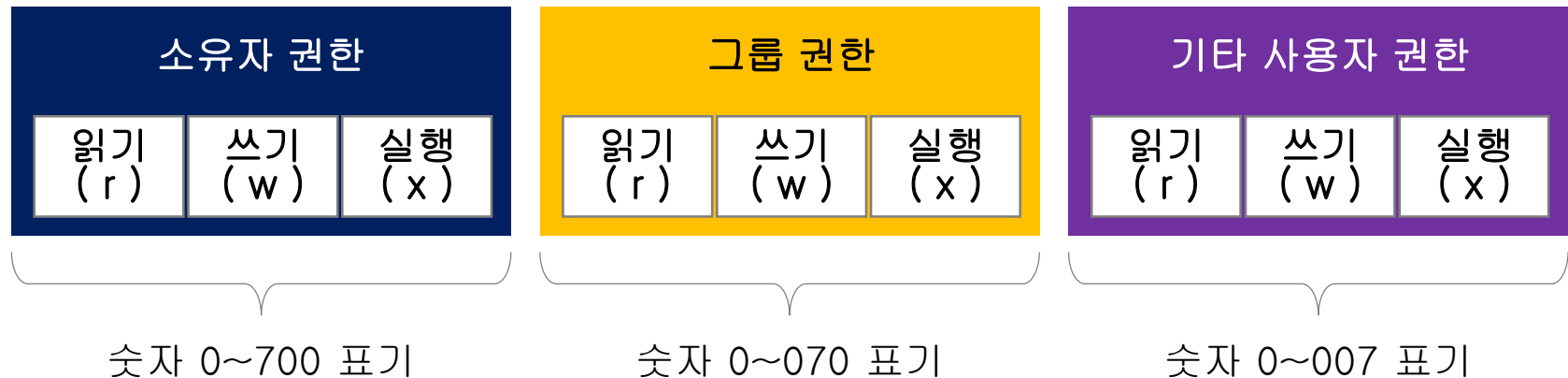
- 파일 접근권한 변경을 위한 숫자모드 사용 이해하기



1. 파일 접근권한을 변경하는 숫자 모드에 대해 살펴봅니다.
2. 숫자 모드에서 숫자로 환산하는 방법에 대해 이해합니다.
3. 숫자 모드로 파일 접근권한을 변경하는 방법에 대해 실습합니다.

숫자 모드

- 접근권한을 조정할 때 문자의 조합은 다소 많아지므로 복잡해지는 단점을 해결하기 좋은 방법인 숫자 모드 활용



- centos_cp.txt 파일의 접근권한을 숫자모드로 표현

```
-  rwx r-x r-x . 1  cskisa  cskisa  215  12월  4  15:35  centos_cp.txt
```



- 파일 접근권한과 숫자의 대응

[표 5-6] 파일 접근권한과 숫자의 대응관계

권한 표기	2진수	8진수	의미	접근권한 예시
rwX	111	7 (4+2+1)	읽기, 쓰기, 실행	rwX rwX rwX → 777
rw-	110	6 (4+2+0)	읽기, 쓰기	rwX r-X r-X → 755
r-w	101	5 (4+0+1)	읽기, 실행	rw- rw- rw- → 666
r--	100	4 (4+0+0)	읽기	r-X r-X r-X → 555
-wX	011	3 (0+2+1)	쓰기, 실행	rw- r-- r-- → 644
-w-	010	2 (0+2+0)	쓰기	rwX --- --- → 700
--X	001	1 (0+0+1)	실행	rw- r-- --- → 640
---	000	0 (0+0+0)	권한 없음	r-- --- --- → 400

■ 숫자 모드로 접근권한 변경

- 숫자의 각 위치가 사용자 카테고리를 나타내고 있으므로 사용자 카테고리를 별도로 지정할 필요 없음(기호 모드와 차이점)



● 숫자 모드로 접근권한 변경

| 예제 5-5 |

- **Step 01** | 터미널 창에서 chap_05 디렉터리 안으로 이동한 다음 디렉터리에 존재하는 centos_cp.txt 파일에 대해 ls -l 명령으로 파일 접근권한이 설정되어 있는 현재 상태를 확인합니다.

```
$ ls -l
```

기능 파일의 정보를 상세하게 출력

형식 ls [옵션] [Enter]

```

cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ ls
chap_03 chap_05 공개 문서 비디오 서식
chap_04 chap_ex 다운로드 바탕화면 사진 음악
[cskisa@localhost ~]$ cd chap_05
[cskisa@localhost chap_05]$ ls
centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-rwxr-xr-x. 1 cskisa cskisa 215 12월 4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
  
```

755(rwx r-x r-x)
7(400+200+100), 5(40+00+10), 5(4+0+1)

[그림 5-18] centos_cp.txt 파일의 접근권한 현재 상태 확인

- **Step 02** | centos_cp.txt 파일에 부여된 접근권한 rwx r-x r-x에서 소유자의 접근권한에서 쓰기권한을 제거하기 위해 숫자 모드 555(r-x r-x r-x)로 접근권한을 변경하면 다음과 같은 결과가 출력됩니다.

```
$ chmod 555 centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [숫자 모드] [파일명]

```

cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-rwxr-xr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod 555 centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xr-xr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
  
```

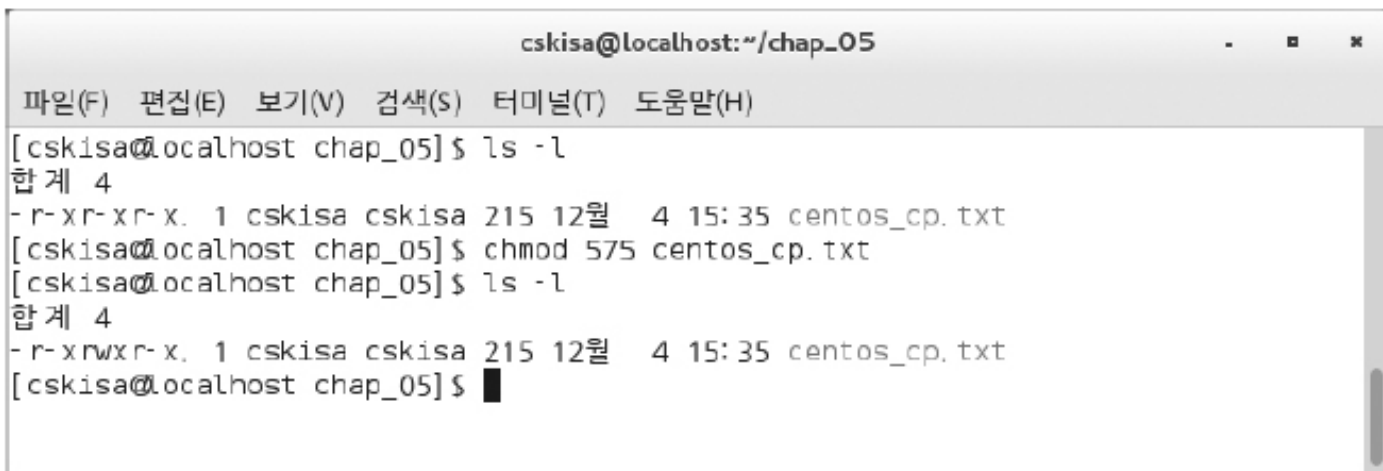
[그림 5-19] centos_cp.txt 파일의 접근권한 555로 변경

- **Step 03** | 이제 centos_cp.txt 파일에 변경된 접근권한 r-x r-x r-x에서 그룹의 접근권한에 쓰기권한을 부여하기 위해 숫자 모드 575(r-x rwx r-x)로 접근권한을 변경해 봅니다.

```
$ chmod 575 centos_cp.txt
```

기능 파일의 접근권한 변경

형식 chmod [숫자 모드] [파일명]



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xr-xr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$ chmod 575 centos_cp.txt
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xrwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
[cskisa@localhost chap_05]$
```

[그림 5-20] centos_cp.txt 파일의 접근권한 575로 변경

SECTION
05

기본 접근권한

- 파일 생성 시 부여되는 기본 접근권한 이해하기



1. 파일 생성 시 부여되는 접근권한에 대해 살펴봅니다.
2. 파일에 부여하지 않을 권한을 지정할 때의 마스크 값에 대해 이해합니다.
3. 마스크 값을 적용하여 기본 접근권한을 변경하는 방법에 대해 실습합니다.

- work 디렉터리와 sample_f.txt 일반파일을 생성한 다음 디렉터리와 일반 파일에 설정된 기본 접근권한 확인

| 예제 5-6 |

- **Step 01** | 현재 디렉터리에 빈파일 sample_f.txt를 생성합니다.

```
$ touch sample_f.txt
```

기능 빈파일 생성

형식 touch [일반 파일명] [Enter]

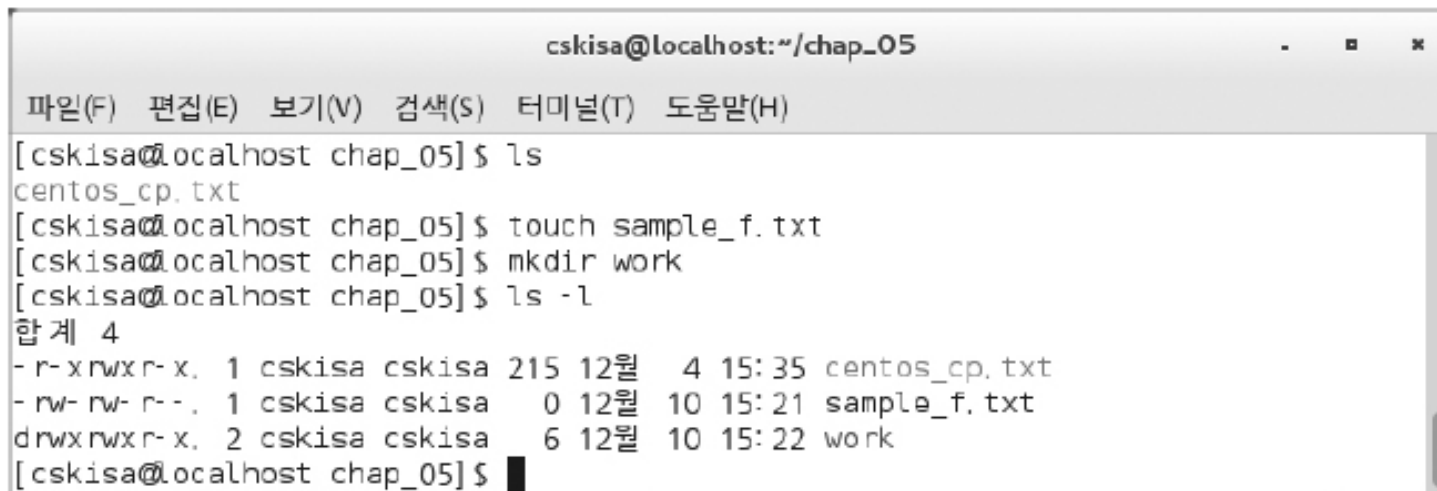
- **Step 02** | 현재 디렉터리에 새로운 work 디렉터를 생성합니다.

```
$ mkdir work
```

기능 새 디렉터리 생성

형식 mkdir [디렉터리명] [Enter]

- **Step 03** | 새로 생성한 sample_f.txt 빈파일과 work 디렉터리가 존재하는지를 확인합니다.



```
cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls
centos_cp.txt
[cskisa@localhost chap_05]$ touch sample_f.txt
[cskisa@localhost chap_05]$ mkdir work
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xrwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
-rw-rw-r--. 1 cskisa cskisa   0 12월 10 15:21 sample_f.txt
drwxrwxr-x. 2 cskisa cskisa   6 12월 10 15:22 work
[cskisa@localhost chap_05]$
```

[그림 5-21] 빈 파일과 새 디렉터리 생성

[예제 5-6]에서 실습한 바와 같이 일반파일 sample_f.txt는 rw- rw- r--와 같이 권한이 설정되어 있으며 work 디렉터리는 rwx rwx r-x와 같이 기본 접근권한이 자동으로 설정되어 있는 것을 확인하였습니다. 리눅스 시스템 환경에 현재 설정된 기본 접근권한을 확인하기 위해서는 umask 명령을 사용하며 사용형식은 다음과 같습니다.

```
$ umask
```

기능 리눅스 시스템에 현재 설정되어 있는 기본 접근권한을 출력하거나 변경

형식 umask [옵션] [마스크 값]

옵션 -s : 마스크 값을 rwx와 같이 문자로 출력

- 현재 리눅스 시스템 환경에 설정된 기본 접근권한 조회

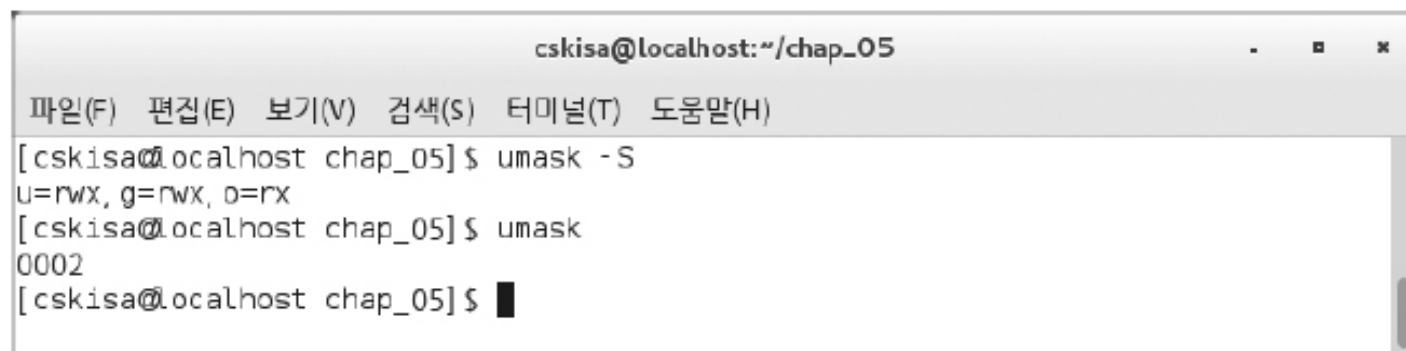
| 예제 5-7 |

현재 사용 중인 리눅스 시스템에 설정되어 있는 파일의 기본 접근권한에 대한 정보를 출력합니다.

```
$ umask -S
```

기능 리눅스 시스템에 현재 설정되어 있는 기본 접근권한을 출력하거나 변경

형식 `umask [옵션] [마스크 값]`



```
cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05] $ umask -S
u=rwx, g=rwx, o=rwx
[cskisa@localhost chap_05] $ umask
0002
[cskisa@localhost chap_05] $
```

[그림 5-22] 리눅스 시스템에 현재 설정된 파일의 기본 접근권한

[예제 5-7]에서 실습한 결과에서 볼 수 있듯이 현재 있는 위치가 chap_05 디렉터리이므로 현재 디렉터리에 설정되어 있는 일반 접근권한을 살펴보면 소유자 권한과 그룹 권한은 읽기, 쓰기, 실행이 모두 부여되어 있고 기타 사용자의 경우에는 읽기와 실행 권한만 부여되어 있음을 확인할 수 있습니다.

umask 명령과 옵션 -S를 함께 실행할 경우에는 현재 리눅스 시스템에 설정된 디렉터리의 기본 접근권한이 u=rwx, g=rwx, o=rx와 같이 문자로 출력되었고 옵션 없이 umask 명령만 사용할 경우에는 숫자 0002가 출력되었습니다. 출력된 0002 숫자는 마스크 값을 의미합니다.

■ 마스크 값 적용

- 마스크 값은 파일이나 디렉터리를 생성할 때 부여하지 않을 권한에 대해서 지정하는 값을 의미
- 인자 없이 umask 명령만 사용하게 되면 현재 설정된 기본 마스크 값을 확인할 수 있음
- umask란 파일이 생성될 때 사용할 파일의 권한에 대해 mask를 인코딩하는 시스템 변수를 의미
- 이러한 권한 값을 퍼미션이라고도 하며 퍼미션은 보통 4자리(레드햇 7.2이하 버전은 3자리)로 표현

- 포미션의 4자리 숫자는 각각의 고유권한을 나타내며 맨 앞자리는 특수 퍼미션 <SetUid, SetGid>을 의미하고
- 다음 자리는 소유자 권한, 그 다음은 그룹 권한, 맨 마지막에는 기타 사용자 권한을 부여
- umask bit 형식

권한 구분	SetUid, SetGid	소유자 권한	그룹 권한	기타 사용자 권한
umask bit	0	0	0	2

프로그램 파일과 프로세스들에게
접근하는 것을 허용하는 기능을 수행

- 리눅스 시스템에서 마스크 값의 적용은 파일과 디렉터리에 따라 각각 다르게 적용됨
- 포미션의 4자리 중에서 특수 퍼미션 <SetUid, SetGid>을 의미하는 맨 앞의 0을 제외하고 002에 대한 의미에 대해서만 살펴보면

[표 5-7] 마스크 값 테이블

마스크 값	0	1	2	3	4	5	6	7
파일의 기본권한	6	6	4	4	2	2	0	0
디렉터리의 기본권한	7	7	6	4	3	2	1	0

- 좀 더 쉽게 이해 할 수 있도록 파일과 디렉터리에 대한 권한설정과 해당 모드의 코드 값을 아래 표와 같이 정리

[표 5-8] 파일과 디렉터리에 대한 기본권한과 해당모드 값

마스크 값		000	001	002	022
파일	권한 값	666	666	664	644
	기본 권한	-rw- rw- rw-	-rw- rw- rw-	-rw- rw- r--	-rw- r-- r--
디렉터리	권한 값	777	776	775	755
	기본 권한	drwx rwx rwx	drwx rwx rw-	drwx rwx r-x	drwx r-x r-x

■ 기본 접근권한

- 리눅스 시스템에서 기본 값을 변경하면 기본 접근권한도 변경되도록 설정되어 있으며
- 기본 접근권한을 출력하거나 변경할 때 사용되는 umask 명령과 함께 마스크 값을 적용하면 기본 접근권한을 변경할 수 있음

- 파일과 디렉터리에 대한 기본 접근권한 변경

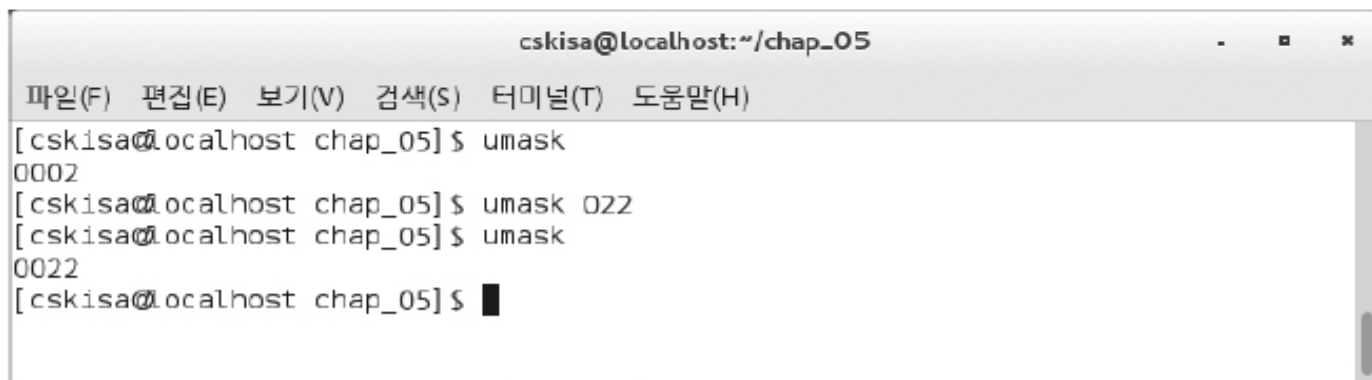
| 예제 5-8 |

- **Step 01** | 현재 설정된 리눅스 시스템의 기본 접근권한을 출력한 다음 기본 접근권한을 마스크 값 022를 적용하여 기본 접근권한을 변경하고 변경된 마스크 값을 확인합니다.

```
$ umask 022
```

기능 리눅스 시스템에 현재 설정되어 있는 기본 접근권한을 출력하거나 변경

형식 `umask [마스크 값]`



A terminal window titled 'cskisa@localhost: ~/chap_05' with a menu bar (파일(F), 편집(E), 보기(V), 검색(S), 터미널(T), 도움말(H)). The terminal shows the following commands and output:

```
[cskisa@localhost chap_05] $ umask
0002
[cskisa@localhost chap_05] $ umask 022
[cskisa@localhost chap_05] $ umask
0022
[cskisa@localhost chap_05] $
```

[그림 5-24] 마스크 값 022로 기본 접근권한

- **Step 02** | 파일 sample_022.txt와 work_022 디렉터리를 touch 명령과 mkdir 명령으로 생성한 다음 ls -l 명령을 수행하여 기본 접근권한을 출력합니다.

```
$ touch sample_022.txt
```

기능 빈파일 생성

형식 touch [일반 파일명]

```
$ mkdir work_022
```

기능 새 디렉터리 생성

형식 mkdir [디렉터리명]

```
cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ touch sample_022.txt
[cskisa@localhost chap_05]$ mkdir work_022
[cskisa@localhost chap_05]$ ls -l
합계 4
-r-xrwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
-rw-r--r--. 1 cskisa cskisa   0 12월 10 17:50 sample_022.txt
-rw-rw-r--. 1 cskisa cskisa   0 12월 10 15:21 sample_f.txt
drwxrwxr-x. 2 cskisa cskisa   6 12월 10 15:22 work
drwxr-xr-x. 2 cskisa cskisa   6 12월 10 17:50 work_022
[cskisa@localhost chap_05]$
```

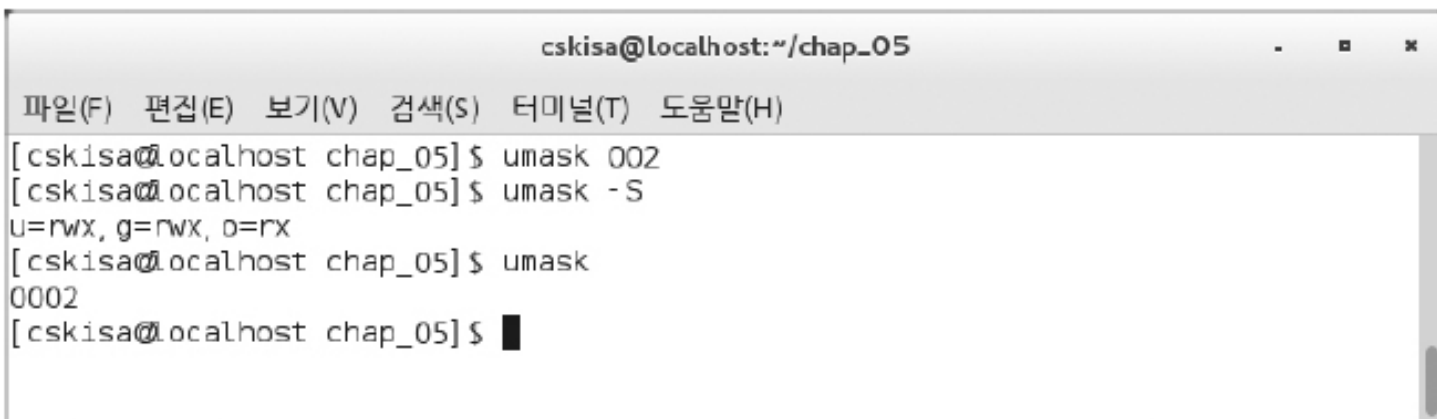
[그림 5-25] sample_022 파일과 work_022 디렉터리 생성

- **Step 03** | 기본 접근권한을 022로 변경 설정한 마스크 값을 원래 설정되어 있던 기본 마스크 값 002로 다시 변경한 다음 umask 명령으로 변경된 기본 접근권한을 확인합니다.

```
$ umask 002
```

기능 리눅스 시스템에 현재 설정되어 있는 기본 접근권한을 출력하거나 변경

형식 `umask [마스크 값]`



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ umask 002
[cskisa@localhost chap_05]$ umask -S
u=rwx,g=rwx,o=rx
[cskisa@localhost chap_05]$ umask
0002
[cskisa@localhost chap_05]$
```

[그림 5-26] 마스크 값 002로 원래 기본 접근권한으로 변경

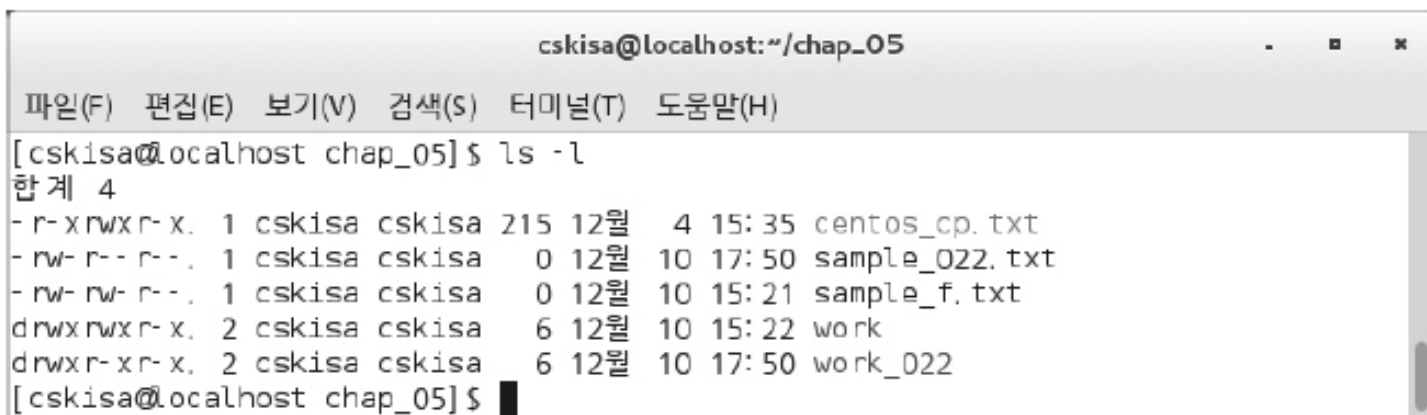
① 파일에 대한 기본 접근권한 : sample_f.txt 파일과 sample_022.txt 파일

umask 022 명령을 수행하기 전에 생성했던 sample_f.txt 파일은 -rw- rw- r--과 같이 설정되어 있는 반면 umask 022 명령을 수행한 다음 생성한 sample_022.txt 파일은 -rw- r-- r--과 같이 다르게 출력된 것을 확인할 수 있습니다. 즉 그룹권한이 rw-과 r--으로 기본 접근권한이 각각 다르게 설정되어 있음을 알 수 있습니다.

② 디렉터리에 대한 기본 접근권한 : work 디렉터리와 work_022 디렉터리

umask 022 명령을 수행하기 전에 생성했던 work 디렉터리는 drwx rwx r-x와 같이 설정되어 있는 반면 umask 022 명령을 수행한 다음 생성한 work_022 디렉터리는 drwx r-x r-x와 같이 다르게 출력된 것을 확인할 수 있습니다. 즉 그룹권한에 대한 설정이 rwx와 r-x로 기본 접근권한이 각각 다르게 설정되어 있음을 알 수 있습니다.

- **Step 04** | 기본 마스크 값을 변경하기 전과 변경한 후에 생성된 파일과 디렉터리에 대한 기본 접근권한에 어떠한 변화가 있는지를 `ls -l` 명령으로 확인합니다.



```

cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05] $ ls -l
한계 4
-r-xrwxr-x. 1 cskisa cskisa 215 12월  4 15:35 centos_cp.txt
-rw-r--r--. 1 cskisa cskisa  0 12월 10 17:50 sample_022.txt
-rw-rw-r--. 1 cskisa cskisa  0 12월 10 15:21 sample_f.txt
drwxrwxr-x. 2 cskisa cskisa  6 12월 10 15:22 work
drwxr-xr-x. 2 cskisa cskisa  6 12월 10 17:50 work_022
[cskisa@localhost chap_05] $

```

[그림 5-27] 마스크 값을 변경해도 기존 생성한 파일과 디렉터리에는 변화 없음

[예제 5-8]을 실습한 결과 `umask 022` 명령으로 생성했던 파일과 디렉터리에 부여된 기본 접근권한은 마스크 값을 `umask 002` 명령으로 다시 변경해도 기존에 생성한 파일과 디렉터리에는 생성 당시의 기본 접근권한이 적용되어 있기 때문에 접근권한에 대한 변화는 없습니다.



특수 접근권한

• 파일 생성 시 부여되는 특수 접근권한 이해하기



1. 특수 접근권한의 종류에 대해 살펴봅니다.
2. 특수 접근권한 설정 값에 대해 이해합니다.
3. 특수 접근권한 설정 값을 적용하는 방법에 대해 실습합니다.

- 특수 접근권한은 SetUid, SetGid, Sticky Bit 세 가지가 있음

[표 5-9] 특수 접근권한 설정 값

접근권한	SetUid	SetGid	Sticky Bit
권한설정 값	4000	2000	1000

[표 5-10] 특수 접근권한과 권한 모드

접근권한			소유자 권한			그룹 권한			기타 사용자 권한		
SetUid	SetGid	Sticky Bit	r	w	x	r	w	x	r	w	x
4000	2000	1000	400	200	100	40	20	10	4	2	1

SetUid

- 특정 파일에 SetUid가 설정되어 있다면 다른 사용자들이 그 파일을 실행하였을 경우와
- 실행되는 동안에는 실행시킨 사용자의 권한(아이디의 권한)이 아닌 파일 소유자 권한으로 실행하게 됨
- SetUid를 사용하여 파일의 소유자 권한을 실행할 수 있도록 설정하려면 접근권한에서 맨 앞자리에 4를 설정해야 함

- 특수 접근권한 SetUid 적용

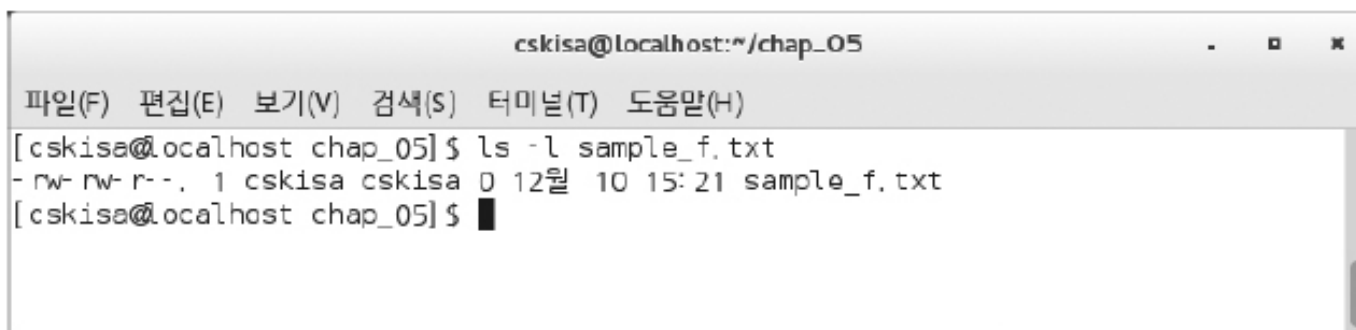
| 예제 5-9 |

- **Step 01** | chap_05 디렉터리에 존재하는 sample_f.txt 파일의 권한을 확인합니다.

```
$ ls -l sample_f.txt
```

기능 파일의 정보를 상세하게 출력

형식 `ls [옵션] [파일명] [Enter]`



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rw-rw-r--. 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$
```

[그림 5-28] sample_f.txt 파일의 상세정보 출력

- **Step 02** | sample_f.txt 파일의 권한이 664(rw-rw-r--)로 설정되어 있는 권한 값을 확인한 다음 권한 값을 4700(-rws----)으로 변경하여 적용합니다.

```
$ chmod 4700 sample_f.txt
```

기능 파일의 접근권한 변경

형식 chmod [SetUid + 숫자 모드] [파일명]

```

cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rw-rw-r--. 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$ chmod 4700 sample_f.txt
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rws-----. 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$
  
```

[그림 5-29] SetUid를 적용하여 sample_f.txt 파일에 특수 접근권한 설정

[예제 5-9]를 통해 sample_f.txt 파일에 특수 접근권한 SetUid를 적용한 결과 접근권한은 rws --- ---으로 변경되었습니다. 여기에서 주의 깊게 살펴봐야 할 부분은 소유자 권한이 rws로 변경되었다는 부분입니다. 이 부분에 대해 좀 더 자세하게 살펴보면 일반 접근권한으로 설정된 sample_f.txt 파일의 숫자 모드는 644(rw- rw- r--)로 설정되어 있었지만 chmod 명령으로 접근권한 값을 4700으로 변경하여 설정하였더니 소유자 권한이 rwx가 아닌 rws로 설정되었음을 확인하였습니다.

SetGid

- 특수 권한이 설정된 파일을 실행할 경우 해당 파일이 실행되는 동안에는 파일 소유 그룹의 권한으로 실행되도록 적용해 주는 기능
- SetGid는 접근권한의 맨 앞자리에 2를 설정해야 하며 이와 같이 설정하면 그룹 권한에서 소유 그룹의 권한 값에
- 실행의 값인 x 대신 s 기호가 표기되고 그룹에 실행 권한이 없을 경우에는 대문자 S가 표기됨
- 특수 접근권한 설정에 있어서 절대모드 표현방법으로 적용할 경우 일반 권한 값에 8진수 2000을 설정하여 해당 파일의 그룹권한 변경

- 특수 접근권한 SetGid 적용

| 예제 5-10 |

- **Step 01** | 현재 디렉터리에 존재하는 sample_f.txt 파일의 상세한 정보를 출력합니다.

```
$ ls -l sample_f.txt
```

기능 파일의 상세한 내용 출력

형식 ls [옵션] [파일명]

- **Step 02** | sample_f.txt 파일에 대한 그룹 소유자 권한을 2700으로 변경합니다.

```
$ chmod 2700 sample_f.txt
```

기능 파일의 접근권한 변경

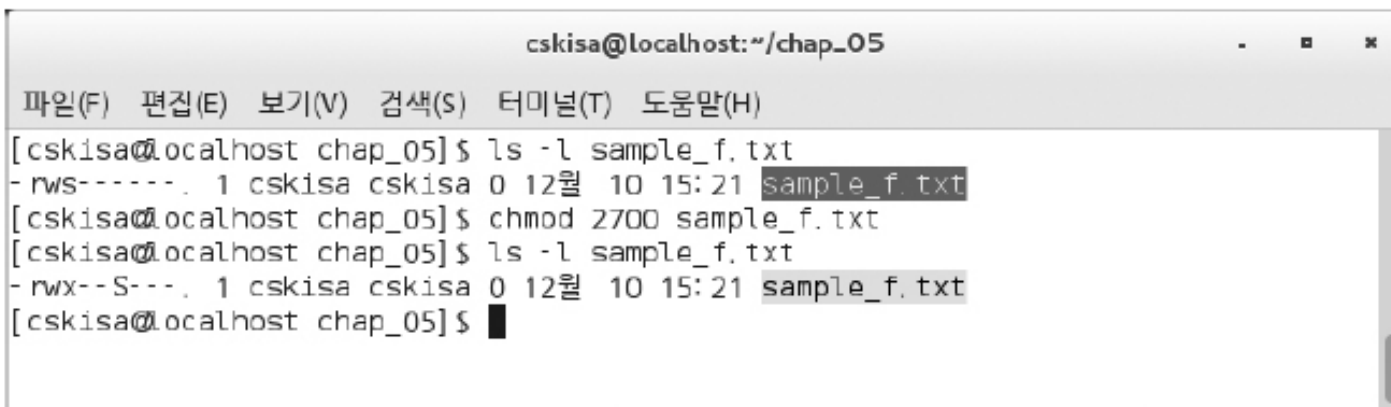
형식 chmod [SetGid + 숫자 모드] [파일명]

- **Step 03** | 특수 접근권한을 변경 설정한 sample_f.txt 파일의 상세한 정보를 출력합니다.

```
$ ls -l sample_f.txt
```

기능 파일의 상세한 내용 출력

형식 `ls [옵션] [파일명]`



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rws----- 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$ chmod 2700 sample_f.txt
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rwx--S--- 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$
```

[그림 5-30] SetGid를 적용하여 sample_f.txt 파일에 특수 접근권한 설정

[예제 5-10]에서 실습한 결과에서 볼 수 있듯이 sample_f.txt 파일에 SetGid의 설정 값을 2700으로 지정하여 그룹 소유자의 특수 접근권한을 변경하였습니다. 실습 파일인 sample_f.txt 파일은 특수 접근권한을 설정하기 전에 설정된 권한은 rws ---- ----에서 rwx --S ----으로 변경된 것을 확인하였습니다. 그리고 SetGid 설정 후 그룹권한에 표기된 권한묶음이 대문자 S가 표기된 것은 그룹에 대한 실행권한이 없기 때문입니다. 소유자에 대한 권한묶음도 rws에서 rwx로 변경되어 적용되었습니다.

■ Sticky Bit

- 일반 파일이 아닌 디렉터리에 대해 특수권한을 설정하는 기능으로 아무런 제약 없이 누구나 디렉터리에 파일을 생성할 수 있음
- 이 디렉터리에 생성되는 파일은 파일을 생성한 소유자의 파일로 귀속되며 다른 사용자는 이 파일에 대해 어떠한 이유라도 삭제할 수 없음
- Sticky Bit는 접근권한의 맨 앞자리에 8진수 1000을 설정해야 하며
- Sticky Bit가 설정되면 기타 사용자의 실행권한에 t가 표시되고 실행 권한이 없는 파일의 경우에는 대문자 T가 표기됨

- 특수 접근권한 Sticky Bit 적용

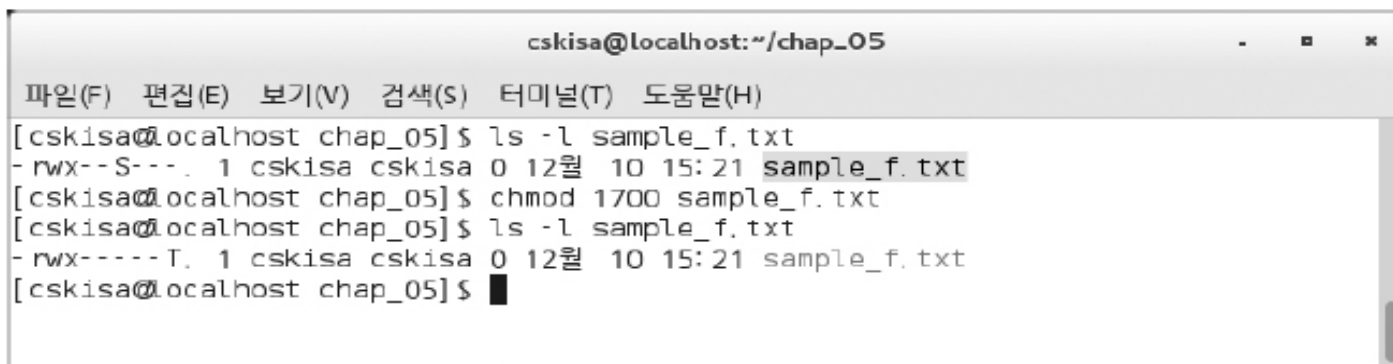
| 예제 5-11 |

- **Step 01** | chap_05 디렉터리에 존재하는 sample_f.txt 파일의 권한설정 값을 ls -l 명령으로 확인한 다음 Sticky Bit의 설정 값을 1700으로 설정한 다음 특수 권한이 설정된 이 파일에 대한 권한묵음을 확인합니다.

```
$ chmod 1700 sample_f.txt
```

기능 파일의 접근권한 변경

형식 chmod [Sticky + 숫자 모드] [파일명] [Enter]



```
cskisa@localhost: ~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rwx--S---. 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$ chmod 1700 sample_f.txt
[cskisa@localhost chap_05]$ ls -l sample_f.txt
-rwx-----T. 1 cskisa cskisa 0 12월 10 15:21 sample_f.txt
[cskisa@localhost chap_05]$
```

[그림 5-31] Sticky Bit를 적용하여 sample_f.txt 파일에 특수 접근권한 설정

- **Step 02** | chap_05 디렉터리에 존재하는 work 디렉터리의 권한설정 값을 ls -ld 명령으로 확인합니다.

```
$ ls -ld work
```

기능 디렉터리의 정보를 상세하게 출력

형식 ls [옵션] [디렉터리명]

- **Step 03** | Sticky Bit의 설정 값을 1755로 설정한 다음 work 디렉터리에 설정된 특수권한을 확인합니다.

```
$ chmod 1755 work
```

기능 디렉터리의 접근권한 변경

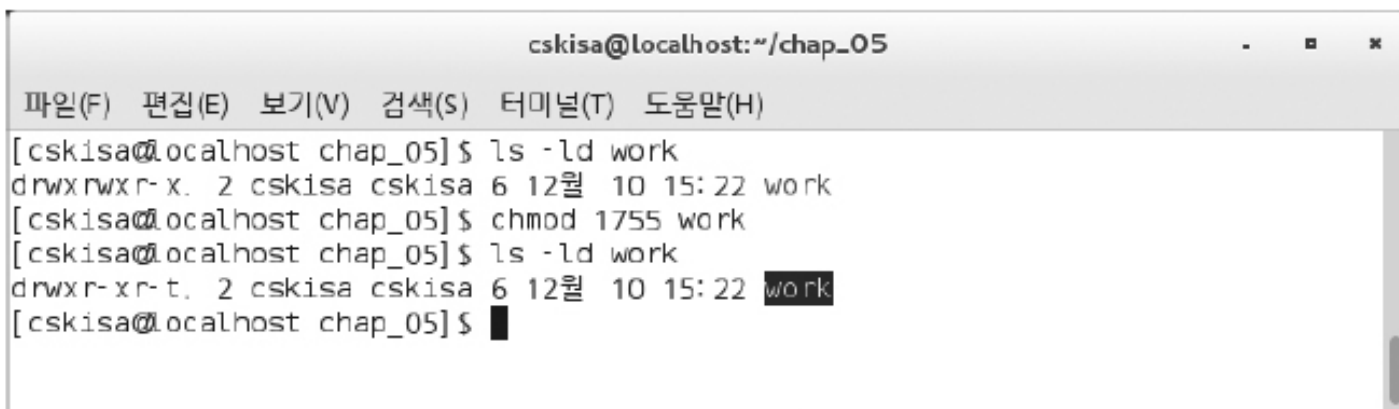
형식 chmod [Sticky Bit + 숫자 모드] [디렉터리명]

- **Step 04** | Sticky Bit의 설정 값을 변경 적용한 work 디렉터리의 권한설정 값을 `ls -ld` 명령으로 확인합니다.

```
$ ls -ld work
```

기능 디렉터리의 정보를 상세하게 출력

형식 `ls [옵션] [디렉터리명]`



```
cskisa@localhost:~/chap_05
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost chap_05]$ ls -ld work
drwxrwxr-x. 2 cskisa cskisa 6 12월 10 15:22 work
[cskisa@localhost chap_05]$ chmod 1755 work
[cskisa@localhost chap_05]$ ls -ld work
drwxr-xr-t. 2 cskisa cskisa 6 12월 10 15:22 work
[cskisa@localhost chap_05]$
```

[그림 5-32] Sticky Bit를 적용하여 work 디렉터리에 특수 접근권한 설정

[예제 5-11]의 Step 01에서 실습한 sample_f.txt 파일에 Sticky Bit 설정 값을 1700으로 설정하였더니 파일에 대한 특수 접근권한이 `rwX --- --T`로 표기되었습니다. 이와 같이 표기된 이유는 기타 사용자에게 실행권한이 없는 파일이기 때문에 대문자 T가 표기된 것입니다.

그리고 Step 02에서는 실습한 work 디렉터리에 Sticky Bit 설정 값을 1755로 설정하였더니 디렉터리에 대한 특수 접근권한이 `rwX rwX r-X`에서 `rwX r-X r-t`로 변경된 것을 확인하였습니다. 좀 더 자세히 살펴보면 기타 사용자의 권한목록이 `r-X`에서 `r-t`와 같이 표기된 것은 work 디렉터리를 다른 사용자가 실행할 수 있는 권한이 부여된 디렉터리임을 의미합니다.

07.관리자 권한 실행

1. 관리자 계정 접속
2. 사용자 계정 관리
3. 그룹관리
4. 소유자와 소유그룹 변경
5. 리눅스 시스템 종료





관리자 계정 접속

• 리눅스 시스템 관리자 계정 이해하기



1. 리눅스 시스템 관리자 계정에 대해 이해합니다.
2. 터미널 창에서 프롬프트를 변경하는 실습을 진행합니다.
3. 관리자 계정과 사용자 계정에 대한 정보를 확인합니다.

- 관리자 계정은 사용자 계정생성과 권한 부여 등의 리눅스 시스템의 전반적인 관리를 수행
- 관리자 계정으로 접속하려면 먼저 터미널 창에 나타난 '\$' 프롬프트를 '#' 프롬프트로 변경한 다음 관리자인 root 계정으로 로그인

■ 프롬프트 변경 : su

- 터미널 창에 나타난 \$ 프롬프트는 일반 사용자 계정으로 접속된 상태를 의미
- 다른 계정으로 교체할 때는 substitute의 약어인 su 명령을 사용
- 일반 사용자 계정에서 root 또는 다른 사용자 계정으로 변경할 경우에는 암호를 물어보지만
- root에서 다른 사용자 계정으로 변경할 경우에는 모든 권한을 가지고 있기 때문에 암호를 묻지 않음

- su 명령어 사용형식

\$ su

기능 다른 계정으로 변경

형식 su [옵션] 계정이름

옵션 -c : 셸을 실행하지 않고 주어진 명령만 수행

-s : 지정된 셸로 접속

-, -1 : 지정한 사용자의 환경변수를 적용하여 접속

- 터미널 창에서 프롬프트 변경

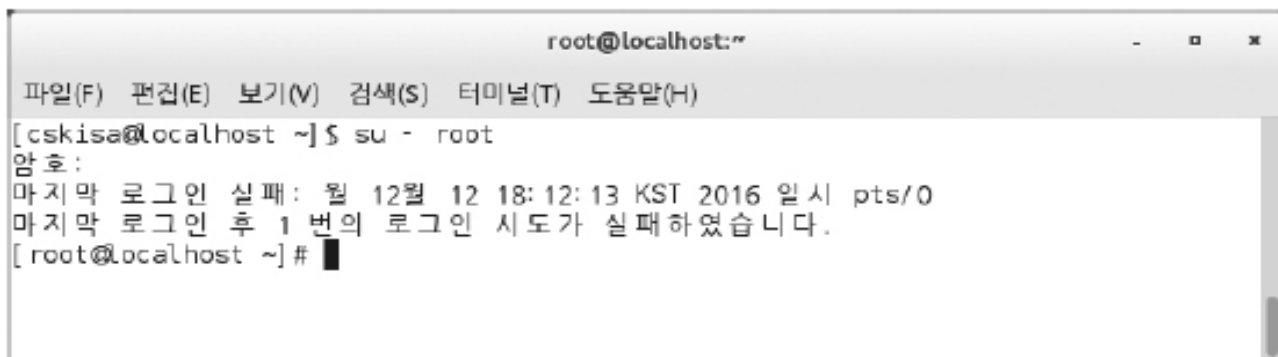
| 예제 6-1 |

- **Step 01** | 터미널 창에서 su 명령과 옵션 -을 함께 선언하여 su - root와 같이 명령을 수행하도록 하고 암호는 리눅스 시스템을 설치할 때 지정했던 spacezone을 입력합니다. 암호는 입력할 때 보안상의 이유로 입력한 암호가 화면에는 나타나지 않습니다.

```
$ su - root
```

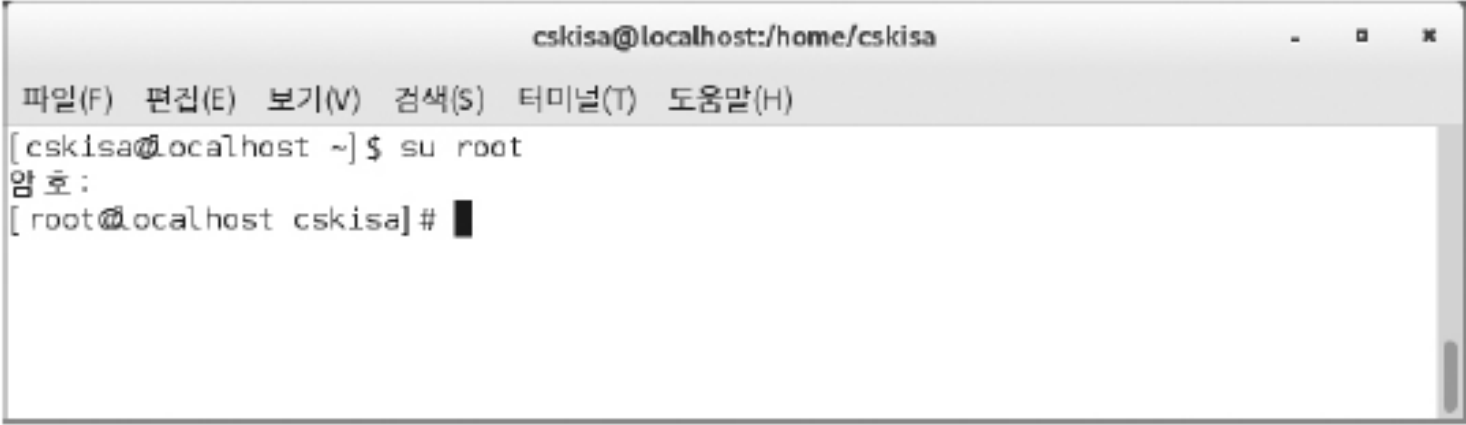
기능 root 계정으로 변경

형식 su [옵션] 계정이름



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ su - root  
암호:  
마지막 로그인 실패: 월 12월 12 18:12:13 KST 2016 일시 pts/0  
마지막 로그인 후 1 번의 로그인 시도가 실패하였습니다.  
[root@localhost ~]#
```

[그림 6-1] 일반 사용자 계정에서 관리자 계정인 root로 변경



```
cskisa@localhost:/home/cskisa
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[cskisa@localhost ~]$ su root
암호:
[root@localhost cskisa]#
```

[그림 6-2] root의 셸 환경변수를 사용하지 않고 root로 변경

- **Step 02** | 터미널 창에서 프롬프트가 #으로 나타난 상태는 관리자 계정인 root로 접속되었다는 것을 의미합니다. 그렇다면 root 계정에서 일반 사용자 계정인 cskisa로 변경하고자 한다면 다음과 같이 명령을 수행하면 됩니다.

```
# su - cskisa
```

기능 root 계정으로 변경

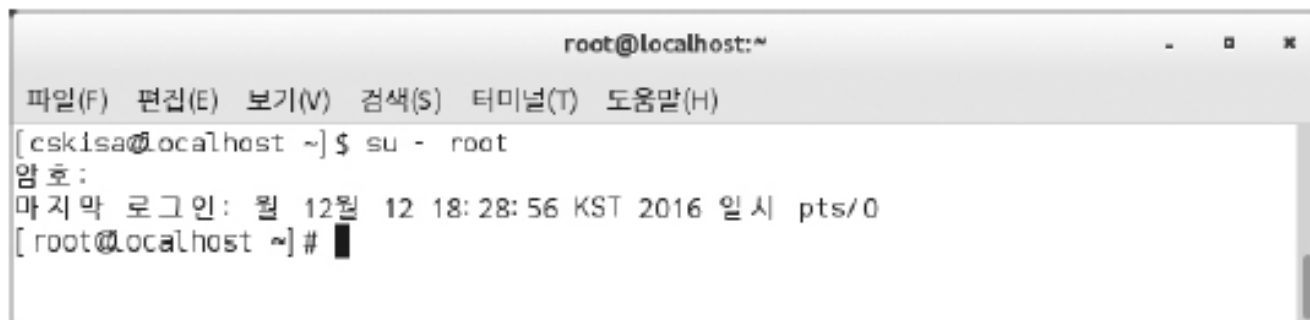
형식 su [옵션] 계정이름



```
cskisa@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ su - root  
암호 :  
마지막 로그인 실패: 월 12월 12 18:12:13 KST 2016 일시 pts/0  
마지막 로그인 후 1 번의 로그인 시도가 실패하였습니다.  
[root@localhost ~]# su - cskisa  
마지막 로그인: 월 12월 12 18:11:30 KST 2016 일시 :0  
[cskisa@localhost ~]$
```

[그림 6-3] root 계정에서 사용자 계정인 cskisa로 변경

- **Step 03** | 다음 실습을 위해 터미널 창에서 root 계정으로 변경합니다.



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[cskisa@localhost ~]$ su - root  
암호:  
마지막 로그인: 월 12월 12 18:28:56 KST 2016 일시 pts/0  
[root@localhost ~]#
```

[그림 6-4] root의 셸 환경변수를 적용하여 root 계정으로 변경

■ 계정정보 확인

- 계정에 대한 일반정보는 /etc/passwd 파일에 저장되어 있으며
- 유닉스를 사용했던 초기시절에 비해 해킹에 대한 위험이 최근에는 급증함에 따라
- 사용자 계정 암호를 /etc/shadow 파일에도 별도로 저장하여 관리

- 계정정보 확인

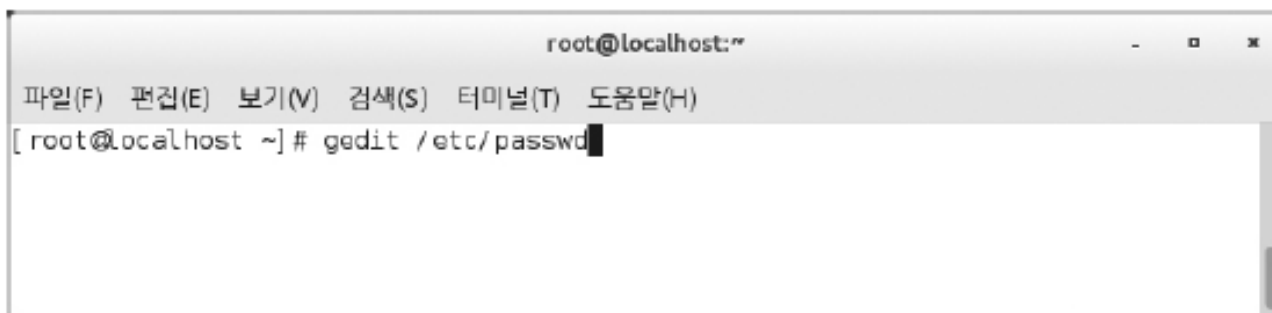
| 예제 6-2 |

- **Step 01** | 터미널 창의 프롬프트가 #으로 표기되어 있는지를 확인한 다음 gedit 또는 vi 에디터를 사용하여 /etc/passwd 파일을 엽니다.

```
# gedit /ect/passwd
```

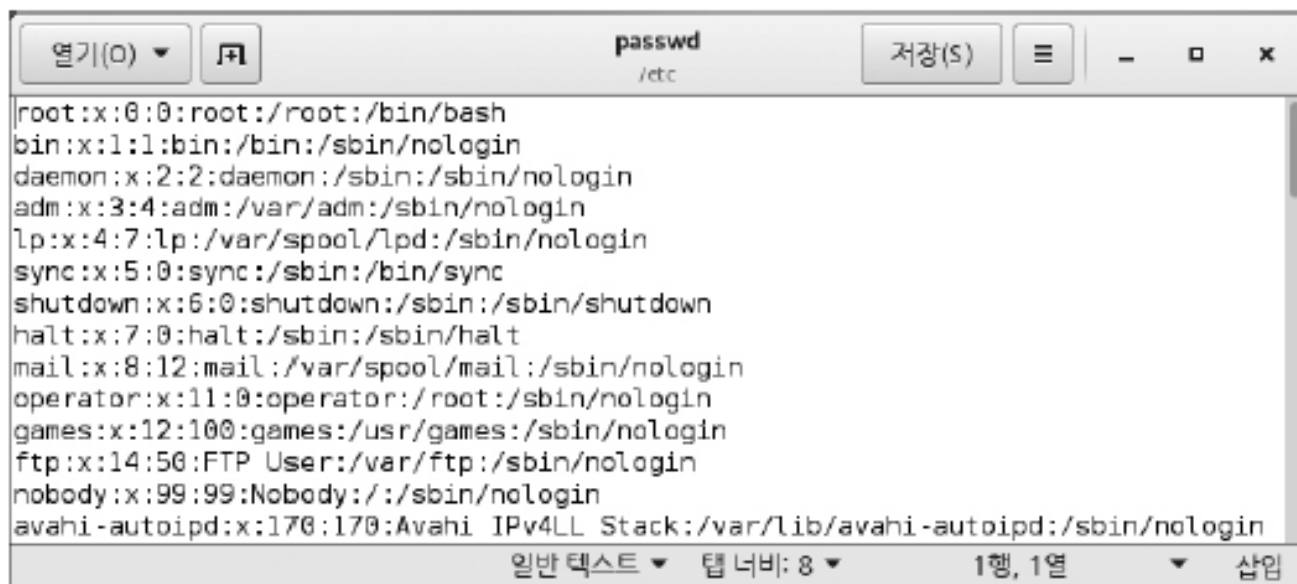
기능 터미널 창에서 지에디트를 실행하여 /etc/passwd 파일 열기

형식 gedit /경로명/파일이름



[그림 6-5] gedit 명령으로 /etc/passwd 파일 열기

- **Step 02** | gedit 창이 활성화되면서 /etc/passwd 파일에 저장되어 있는 root 계정의 일반정보가 자세히 나타나면 맨 위에 있는 첫 줄의 내용을 확인합니다.



```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
```

[그림 6-6] root 계정에 대한 일반정보가 저장된 /etc/passwd 파일내용

- **Step 03** | gedit 창이 활성화되면서 /etc/passwd 파일의 제일 하단에는 사용자 계정인 cskisa에 대해 구성된 정보를 확인할 수 있습니다.

```

pulse:x:171:171:PulseAudio System Daemon:/var/run/pulse:/sbin/nologin
hsqldb:x:96:96::/var/lib/hsqldb:/sbin/nologin
tomcat:x:91:91:Apache Tomcat:/usr/share/tomcat:/bin/nologin
pkuser:x:17:17:Certificate System:/usr/share/pki:/sbin/nologin
gdm:x:42:42::/var/lib/gdm:/sbin/nologin
gnome-initial-setup:x:986:981::/run/gnome-initial-setup:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
postfix:x:89:89:/var/spool/postfix:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin
dovecotnull:x:985:980:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/nologin
tcpdump:x:72:72::/sbin/nologin
cskisa:x:1000:1000:cskisa:/home/cskisa:/bin/bash
  
```

[그림 6-7] cskisa 계정에 대한 일반정보가 저장된 /etc/passwd 파일내용

/etc/passwd 파일에는 root 관리자 계정에 대한 정보와 cskisa 사용자 계정에 대한 정보가 저장되어 있습니다. 두 개의 계정 중에서 cskisa 계정에 대한 /etc/passwd 파일 구성내용을 각 기본 항목별로 자세히 살펴보도록 하겠습니다.

<u>cskisa</u>	<u>:</u>	<u>x</u>	<u>:</u>	<u>1000</u>	<u>:</u>	<u>1000</u>	<u>:</u>	<u>cskisa</u>	<u>:</u>	<u>/home/cskisa</u>	<u>:</u>	<u>/bin/bash</u>
①		②		③		④		⑤		⑥		⑦

① 로그인 ID

로그인 ID는 사용자 계정의 이름을 의미합니다. 로그인 ID는 최대 32자까지 지정할 수 있지만 너무 긴 ID는 효율성이 떨어지기 때문에 다른 운영체제와의 연동을 위해 적당한 길이(8자 정도)로 지정하는 것을 권장합니다. 로그인 ID에는 콜론(:)과 Enter를 제외한다면 어떤 문자라도 상관 없이 사용할 수 있습니다. 그렇지만 동일한 ID를 사용할 수는 없습니다.

② 암호저장 항목

x는 유닉스 초기시절 사용자 암호를 저장하기 위한 항목입니다. 이 항목은 이전 프로그램과의 호환성을 위해 기본 항목을 그대로 유지하고 있으며 x로 표시합니다.

③ UID

리눅스 시스템이 사용자를 구분하기 위해 사용되는 사용자 ID 번호입니다. 일반사용자는 1000번부터 할당되며 사용자에 대한 시스템 관리업무를 위해 내부적으로 사용자를 위해 예약되어 있기 때문에 임의로 변경하지 않는 것이 좋습니다. 사용자를 구별함에 있어 로그인 ID가 다르더라도 UID가 같으면 리눅스 시스템은 동일한 사용자로 판단합니다. 따라서 시스템 관리자는 UID가 중복되지 않도록 관리에 주의를 기울여야 합니다.

④ GID

사용자가 속해 있는 그룹의 ID를 나타냅니다. 리눅스 시스템에서 사용되는 모든 사용자는 무조건 하나 이상의 그룹에 속해 있어야 합니다. GID는 사용자를 등록할 때 정해지며 GID를 지정하지 않으면 자동으로 로그인 ID가 그룹으로 등록됩니다. 등록된 GID는 `/etc/group` 파일에서 확인할 수 있습니다.

⑤ 설명

사용자에 대한 일반적인 정보(이름, 부서, 연락처 등)가 기록되는 항목입니다. 이 항목은 유닉스에서 유래된 기원을 바탕으로 사용자 개인정보^{GECOS}라고도 불립니다.

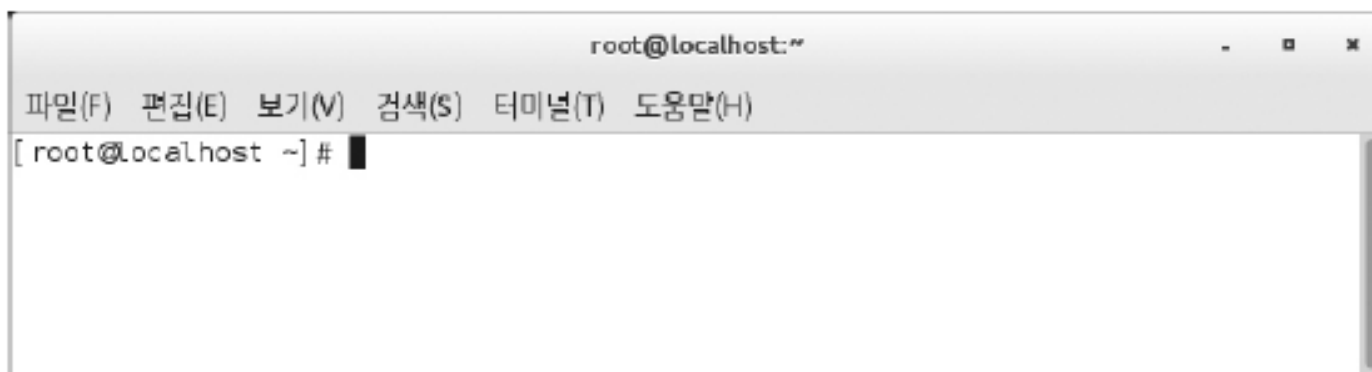
⑥ 홈 디렉터리

이 항목에서는 사용자 계정에 할당된 홈 디렉터리를 절대경로의 형태로 저장됩니다. 홈 디렉터리는 사용자 계정을 생성하면서 자동으로 부여되며 홈 디렉터리 아래에는 서브 디렉터리와 파일 등을 생성하거나 저장할 수 있습니다.

⑦ 로그인 셸

사용자의 로그인 기본 셸을 지정해 줍니다. 현재 CentOS 리눅스에서는 배시 셸을 기본 셸로 적용하여 사용자에게 로그인 셸을 지정합니다.

- **Step 04** | root 계정으로 접속된 상태에서 터미널 창을 열어보면 프롬프트가 '#'으로 나타납니다. 그리고 터미널 창의 상단에 보면 root@localhost:~와 같이 root 계정으로 접속된 상태임을 알 수 있습니다.



[그림 6-11] root 계정으로 접속된 상태에서의 # 프롬프트



사용자 계정 관리

• 사용자 계정 발급과 관리방법 이해하기



1. 사용자 계정을 생성하는 과정에 대해서 살펴봅니다.
2. 사용자 계정정보 수정방법에 대해 살펴봅니다.
3. 사용자 계정과 삭제에 대해서 실습합니다.

■ 사용자 계정생성

- 사용자 계정은 여러 명의 사용자가 리눅스 시스템에 접속하여 사용할 때 사용자를 구별하는 역할을 수행
- 사용자 계정은 사용자가 시스템에 접근할 수 있는 유일한 방법이며
- 시스템 관리자 입장에서 사용자 접근권한을 통제할 수 있는 중요한 역할을 수행
- 그렇기 때문에 사용자 계정 관리는 리눅스 시스템 관리자의 중요한 업무 중 하나에 속함

- 사용자 계정 생성 명령 : adduser 보다 useradd 명령을 사용

\$ useradd

기능 사용자 계정을 새로 생성

형식 useradd [옵션] [생성할 계정이름]

옵션 -c : 사용자 이름 등 추가적인 설명 지정

-d : 사용자 계정의 홈 디렉터리 지정

-D : 기본설정 값을 출력하거나 설정

-e : 유효기간 설정 (YYYY-MM-DD)

-f : 사용자 계정의 유효일자 (-f-175 : 앞으로 175일간 사용가능)

-g : 사용자 계정의 로그인 기본그룹 GID 지정

-G : 2차 그룹의 GID 지정

-o : 사용자 계정의 UID 중복 허용

-p : 사용자 계정의 비밀번호

-s : 사용자 계정의 기본 셸 지정

-u : 사용자 계정의 UID 지정

- 터미널 창에서 프롬프트가 #으로 표기되어 있는지를 확인한 다음 옵션 없이 user_001 사용자 계정 생성

| 예제 6-3 |

- **Step 01** | home 디렉터리에 존재하는 사용자 계정 리스트를 출력합니다.

```
# ls /home
```

기능 home 디렉터리에 존재하는 사용자 계정 리스트 출력

형식 ls [/디렉터리명] [Enter.]

- **Step 02** | 사용자 계정 user_001 계정을 새로 생성한 다음 추가된 계정이 존재하는지를 확인합니다.

```
# useradd user_001
```

기능 user_001 사용자 계정을 새로 생성

형식 useradd [새로 생성할 계정이름]

A terminal window titled 'root@localhost: ~' with a menu bar containing '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal shows the following commands and output:

```
[root@localhost ~]# ls /home
cskisa
[root@localhost ~]# useradd user_001
[root@localhost ~]# ls /home
cskisa user_001
[root@localhost ~]#
```

[그림 6-12] 새로운 사용자 계정 user_001 생성

- 생성된 새 계정에 암호 지정 : passwd 명령 사용형식

passwd

기능 사용자 계정의 암호 수정

형식 passwd [옵션] 계정이름

옵션 -l : 지정한 계정의 암호 잠금

-u : 암호의 잠금 해제

-d : 지정한 계정의 암호 삭제

- 사용자 계정 user_001 계정의 암호를 계정이름과 동일하게 설정

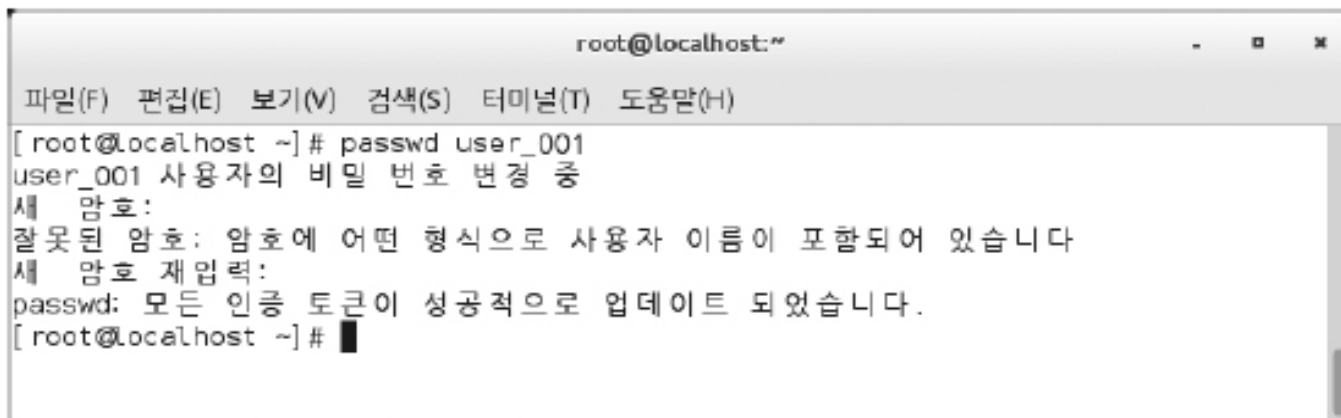
| 예제 6-4 |

사용자 계정인 user_001 계정의 암호를 계정이름과 동일하게 user_001로 설정합니다.

```
# passwd user_001
```

기능 user_001 계정에 암호 설정

형식 passwd [사용자 계정이름]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # passwd user_001  
user_001 사용자의 비밀 번호 변경 중  
새 암호:  
잘못된 암호: 암호에 어떤 형식으로 사용자 이름이 포함되어 있습니다  
새 암호 재입력:  
passwd: 모든 인증 토큰이 성공적으로 업데이트 되었습니다.  
[root@localhost ~] #
```

[그림 6-13] user_001 사용자 계정에 암호 설정

- user_001 계정정보 확인 : /etc/passwd

| 예제 6-5 |

사용자 계정인 user_001 계정에 대한 자세한 정보를 gedit 창에서 확인합니다.

```
# gedit /etc/passwd
```

기능 계정에 대한 정보확인

형식 gedit [/경로명/파일명] [Enter]

```
hsqldb:x:96:96::/var/lib/hsqldb:/sbin/nologin
tomcat:x:91:91:Apache Tomcat:/usr/share/tomcat:/bin/nologin
pkuser:x:17:17:Certificate System:/usr/share/pki:/sbin/nologin
gdm:x:42:42::/var/lib/gdm:/sbin/nologin
gnome-initial-setup:x:986:981:./run/gnome-initial-setup:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
postfix:x:89:89:./var/spool/postfix:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin
dovecotnull:x:985:988:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/nologin
tcpdump:x:72:72:./:/sbin/nologin
cskisa:x:1000:1000:cskisa:/home/cskisa:/bin/bash
user_001:x:1001:1001:./:/home/user_001:/bin/bash
```

[그림 6-14] user_001 계정에 대한 정보 출력

- 암호화된 user_001 계정정보 확인 : /etc/shadow

| 예제 6-6 |

리눅스 시스템을 보호하기 위해 별도로 암호가 저장되어 있는 /etc/shadow 파일에 대한 정보를 확인합니다.

```
# gedit /etc/shadow
```

기능 계정의 암호화된 정보 확인

형식 gedit [/경로명/파일명]



[그림 6-15] 계정에 대한 암호화된 정보 출력

- 홈 디렉터리 지정하여 계정 생성

| 예제 6-7 |

- **Step 01** | 새로 생성할 사용자 계정 user_002의 홈 디렉터를 work_06으로 지정하면 별도의 디렉터리 생성과정 없이 자동으로 홈 디렉터리가 work_06으로 지정됩니다. 암호는 사용자 계정이름과 동일하게 user_002로 입력합니다.

```
# useradd -d /work_06 user_002
```

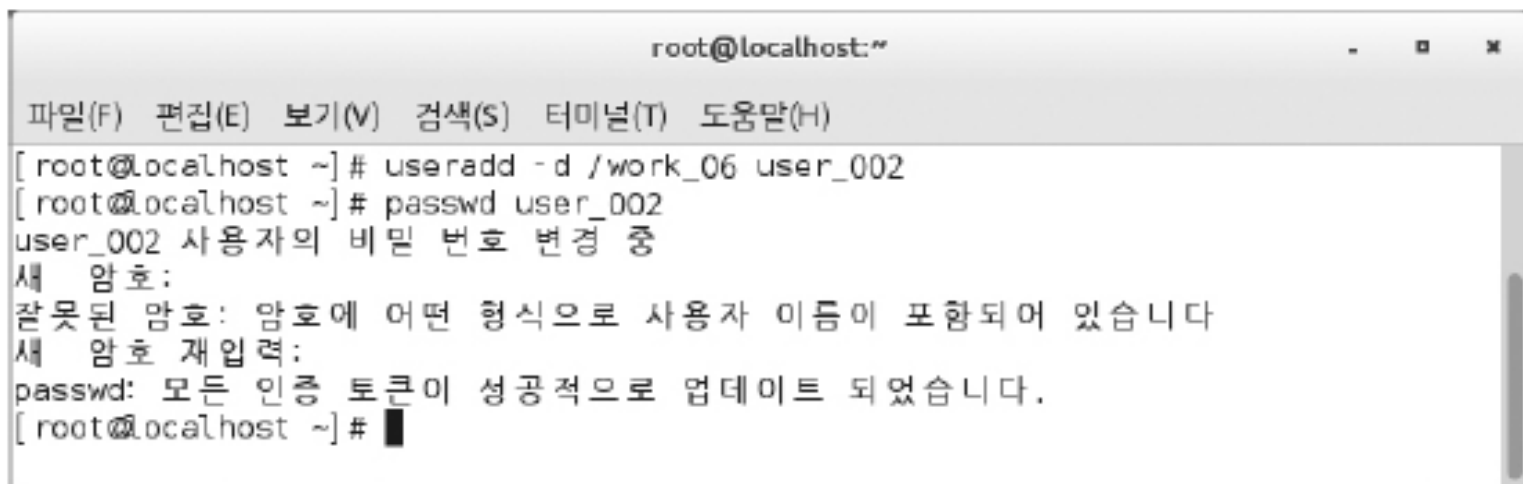
기능 새 계정을 생성하면서 홈 디렉터를 work_02으로 지정

형식 useradd [옵션] [/홈 디렉터리 경로명] [계정이름] [Enter]

```
# passwd user_002
```

기능 user_002 계정에 암호 설정

형식 passwd [계정이름] [Enter]

A terminal window titled 'root@localhost:~' with standard window controls. The menu bar includes '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal content shows the execution of 'useradd -d /work_06 user_002' and 'passwd user_002'. It prompts for a password, receives an error for including the username, and then successfully updates the password token.

```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # useradd -d /work_06 user_002  
[root@localhost ~] # passwd user_002  
user_002 사용자의 비밀번호 변경 중  
새 암호:  
잘못된 암호: 암호에 어떤 형식으로 사용자 이름이 포함되어 있습니다  
새 암호 재입력:  
passwd: 모든 인증 토큰이 성공적으로 업데이트 되었습니다.  
[root@localhost ~] #
```

[그림 6-16] 홈 디렉터리를 지정하여 user_002 계정 생성

- **Step 02** | gedit 창에서 /etc/passwd 파일을 열어 새로 생성한 user_002 계정에 지정된 홈 디렉터리가 work_06으로 적용되었는지 정보를 확인합니다.

```
# gedit /etc/passwd
```

기능 계정에 대한 정보확인

형식 gedit [/경로명/파일명] [Enter]

```
tomcat:x:91:91:Apache Tomcat:/usr/share/tomcat:/bin/nologin
pkiuser:x:17:17:Certificate System:/usr/share/pki:/sbin/nologin
gdm:x:42:42:/:/var/lib/gdm:/sbin/nologin
gnome-initial-setup:x:986:981:/:/run/gnome-initial-setup:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
postfix:x:89:89:/:/var/spool/postfix:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin
dovenull:x:985:980:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/
nologin
tcpdump:x:72:72:/:/sbin/nologin
cskisa:x:1000:1000:cskisa:/home/cskisa:/bin/bash
user_001:x:1001:1001:/:/home/user_001:/bin/bash
user_002:x:1002:1002:/:/work_06:/bin/bash
```

[그림 6-17] user_002 계정의 홈 디렉터리 확인

- 리눅스 파일내용 출력 : tail 명령

\$ tail

기능 파일내용을 마지막 라인부터 10번째 라인까지만 내용출력

형식 tail [옵션] [생성할 계정이름]

옵션 -f : 파일의 마지막 10라인을 실시간으로 계속해서 내용출력

-F : 파일 변동 시 다시 명령을 실행할 필요 없이 실시간으로 내용출력

-n : 지정한 n라인만큼 내용출력

-n +n : 마지막 라인이 아니라 첫 번째 라인부터 시작해 n번째 라인이후 출력

--byte=n : 지정한 n바이트 만큼의 내용을 출력

- user_002 계정의 정보를 tail 명령으로 출력

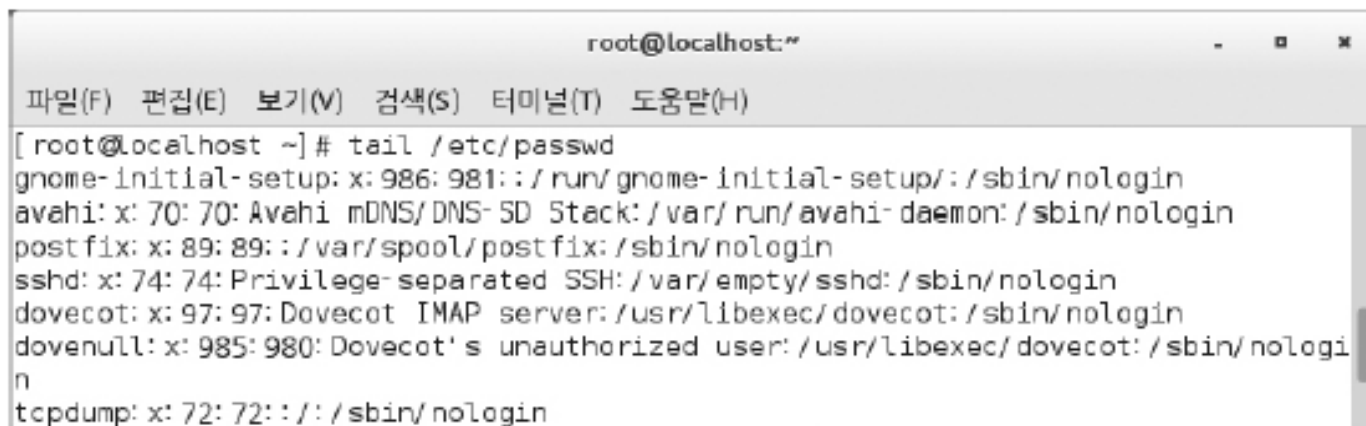
| 예제 6-8 |

- **Step 01** | tail 명령과 함께 /etc/passwd 파일을 지정하여 파일내용을 출력합니다.

```
# tail /etc/passwd
```

기능 파일내용을 마지막 라인부터 10번째 라인까지만 내용출력

형식 tail [옵션] [파일명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# tail /etc/passwd  
gnome-initial-setup:x:986:981:/:run/gnome-initial-setup:/sbin/nologin  
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin  
postfix:x:89:89:/:var/spool/postfix:/sbin/nologin  
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin  
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin  
dovecot:x:985:980:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/nologin  
tcpdump:x:72:72:/:/sbin/nologin
```

(a) tail 명령 실행



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
sshd: x: 74: 74: Privilege-separated SSH: /var/empty/sshd: /sbin/nologin  
dovecot: x: 97: 97: Dovecot IMAP server: /usr/libexec/dovecot: /sbin/nologin  
dovnull: x: 985: 980: Dovecot's unauthorized user: /usr/libexec/dovecot: /sbin/nologin  
tcpdump: x: 72: 72: : / : /sbin/nologin  
cskisa: x: 1000: 1000: cskisa: /home/cskisa: /bin/bash  
user_001: x: 1001: 1001: : /home/user_001: /bin/bash  
user_002: x: 1002: 1002: : /work_06: /bin/bash  
[ root@localhost ~] #
```

(b) tail 명령으로 /etc/passwd 파일내용 출력

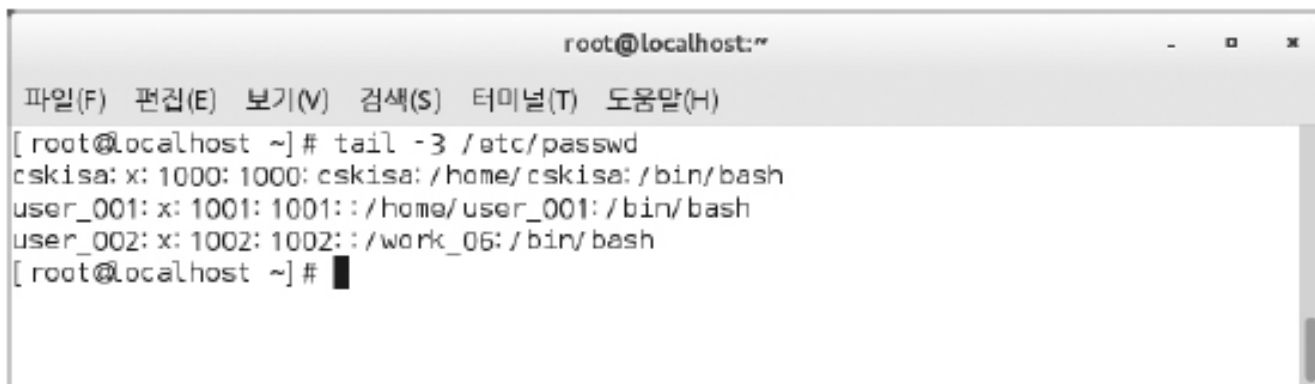
[그림 6-19] tail 명령 실행 및 /etc/passwd 파일내용 출력

- **Step 02** | tail 명령은 기본적으로 마지막 라인부터 10번째 라인까지의 파일내용을 출력하기 때문에 user_002 계정에 대한 정보가 10라인 출력되었습니다. 이번에는 옵션 -n을 사용하여 필요한 라인만큼만 /etc/passwd 파일의 내용을 출력하도록 하겠습니다. 여기서는 마지막 라인부터 3개 라인만 출력되도록 수행합니다.

```
# tail -3 /etc/passwd
```

기능 파일내용을 마지막 라인부터 3번째 라인까지만 내용출력

형식 tail [옵션] [파일명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # tail -3 /etc/passwd  
cskisa:x:1000:1000:cskisa:/home/cskisa:/bin/bash  
user_001:x:1001:1001::/home/user_001:/bin/bash  
user_002:x:1002:1002::/work_06:/bin/bash  
[root@localhost ~] #
```

[그림 6-19] /etc/passwd 파일내용을 마지막 3라인만 출력

■ 사용자 계정정보 수정

- 이미 생성된 사용자 계정에 대한 정보 수정 : usermod 명령 사용

```
$ usermod
```

기능 사용자 계정정보 수정

형식 usermod [옵션] [사용자 계정이름]

옵션 -c : 사용자 이름 등 추가적인 설명 수정

-d : 사용자 계정의 홈 디렉터리 수정

-e : 사용자 계정 만료일(유효기간) 수정 (YYYY-MM-DD)

-f : 사용자 계정 비활성화 일자 수정

-g : 사용자 계정의 로그인 기본그룹 GID 수정

-G : 2차 그룹의 GID 수정

-l : 사용자 계정 이름 변경

-o : 사용자 계정의 UID 중복 허용

-s : 사용자 계정의 기본 셸 수정

-u : 사용자 계정의 UID 수정

- user_002 계정의 UID 변경 : -u 옵션

| 예제 6-9 |

- **Step 01** | 사용자 계정 user_002에 대한 UID를 2001로 변경합니다.

```
# usermod -u 2001 user_002
```

기능 user_002 계정의 UID를 2001로 변경

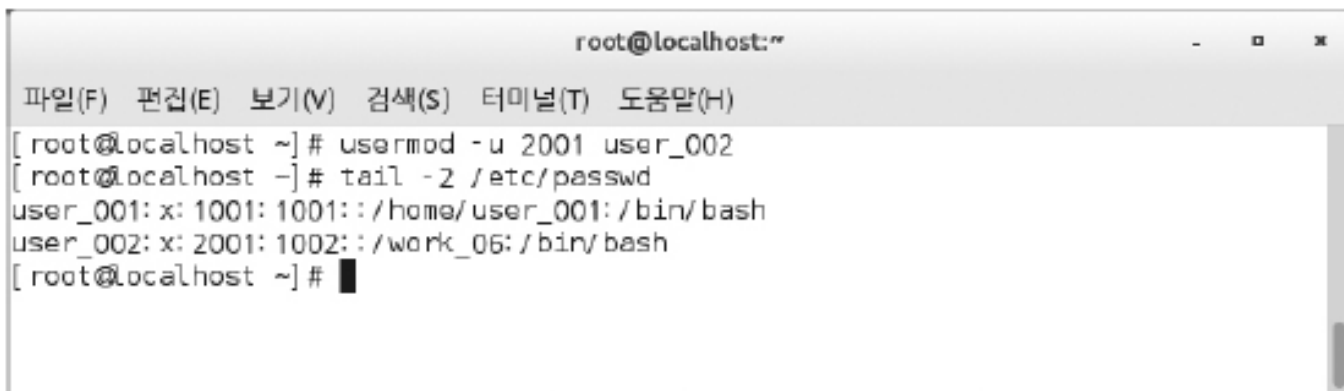
형식 usermod [옵션] [사용자 계정이름]

- **Step 02** | etc 디렉터리에 존재하는 passwd 파일의 내용 중에서 마지막 라인부터 2번째 라인까지만 내용을 출력합니다.

```
# tail -2 /etc/passwd
```

기능 파일내용을 마지막 라인부터 2번째 라인까지만 내용출력

형식 tail [옵션] [파일명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # usermod -u 2001 user_002  
[root@localhost ~] # tail -2 /etc/passwd  
user_001: x: 1001: 1001: : /home/user_001: /bin/bash  
user_002: x: 2001: 1002: : /work_06: /bin/bash  
[root@localhost ~] #
```

[그림 6-20] user_002 계정의 UID를 2001로 변경

- 홈 디렉터리와 로그인 ID 한꺼번에 변경 : -l, -u 옵션 사용

| 예제 6-10 |

- **Step 01** | 사용자 계정 user_002에 대한 로그인 ID를 guest_001로 지정하고 홈 디렉터를 workspace_06 디렉터리로 지정합니다.

```
# usermod -d /home/workspace_06 -l guest_001 user_002
```

기능 user_002 계정의 홈 디렉터리와 로그인 ID 변경

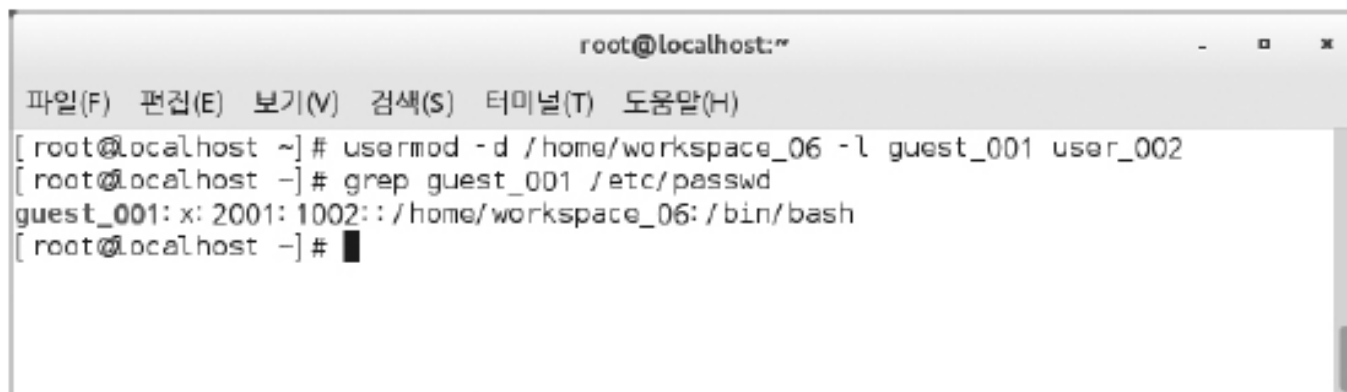
형식 usermod [옵션] [홈 디렉터리] [옵션] [새 로그인 ID] [기존 로그인 ID]

- **Step 02** | 사용자 로그인 ID guest_001에 대한 문자열의 정보를 출력합니다.

```
# grep guest_001
```

기능 지정된 guest_001 문자열을 찾아서 정보를 출력

형식 grep [옵션] [찾을 문자열] [Enter]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# usermod -d /home/workspace_06 -l guest_001 user_002  
[root@localhost ~]# grep guest_001 /etc/passwd  
guest_001:x:2001:1002::/home/workspace_06:/bin/bash  
[root@localhost ~]#
```

[그림 6-21] user_002 사용자 계정의 홈 디렉터리와 로그인 ID 변경

■ 사용자 암호 기한설정 및 변경 : chage 명령 사용

- 사용자 계정의 암호는 주기적으로 변경해 주는 것을 권장

chage

기능 시스템 보안을 위해 사용자 계정에 따른 암호 기한설정 및 변경

형식 chage [옵션] [사용자 계정이름] [Enter]

옵션 -d : 암호를 마지막으로 변경한 일자 → 마지막 날짜

-E : 계정사용 종료일자 (YYYY-MM-DD) → 만료일자

-I : 암호를 유효기간 종료 이후에 계정 비활성화 될 일 수 → 잠금 날짜

-m : 암호를 변경할 수 있는 최소 일 수 (0 이면 아무 때나 변경) → 최소 날짜

-M : 마지막 변경 이후의 암호 유효 일 수 → 최대 날짜

-w : 암호 만료기간 몇 일전에 안내 메시지 출력 → 경고 날짜

-l : 현재 사용자의 암호 만기에 대한 기본정보 출력 (l의 소문자)

-1 : 암호 만료일을 지정하지 않거나 설정 값 해제 (숫자 1일)

- user_001 계정에 설정된 암호에 대한 정보 출력과 기간설정

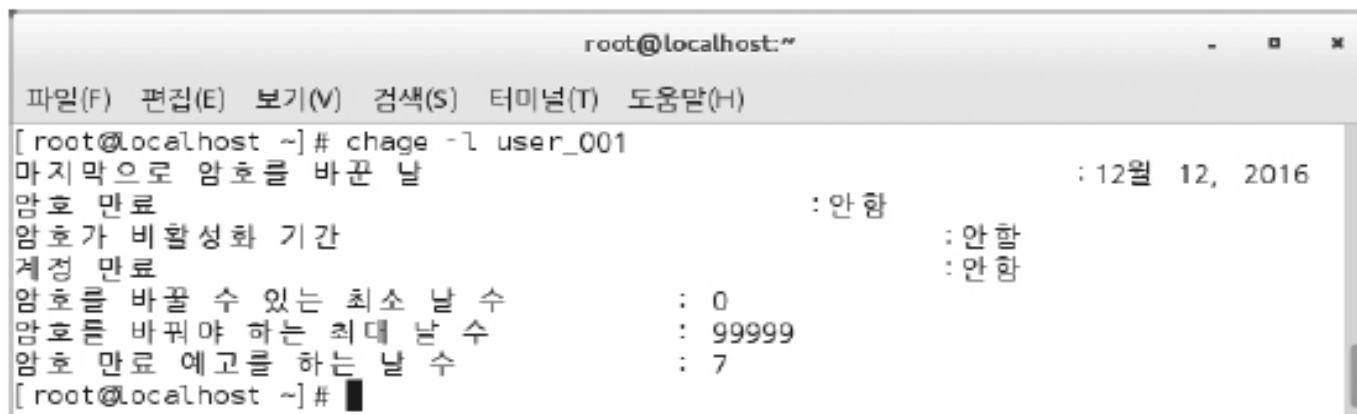
| 예제 6-11 |

- **Step 01** | 사용자 계정 user_001에 설정되어 있는 암호정보를 출력합니다.

```
# chage -l user_001
```

기능 user_001 계정에 설정되어 있는 기본정보 출력

형식 chage [옵션] [사용자 계정이름] [Enter]



```

root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# chage -l user_001
마지막으로 암호를 바꾼 날                : 12월 12, 2016
암호 만료                                : 안 함
암호가 비활성화 기간                    : 안 함
계정 만료                                : 안 함
암호를 바꿀 수 있는 최소 날 수          : 0
암호를 바꿔야 하는 최대 날 수          : 99999
암호 만료 예고를 하는 날 수             : 7
[root@localhost ~]#
  
```

[그림 6-22] 사용자 계정 user_001에 설정된 암호정보 출력

- **Step 02** | 사용자 계정 user_001에 대한 계정사용 만료 일자를 2017년 01월 31일까지로 설정하고 변경된 정보를 확인합니다(연월일 구분은 - 또는 /를 사용).

```
# chage -E 2017/01/31 user_001
```

기능 user_001 계정의 만료 일자를 2017년 01월 31일로 설정

형식 chage [옵션] [사용자 계정이름] [Enter]

```
root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# chage -E 2017/01/31 user_001
[root@localhost ~]# chage -l user_001
마지막으로 암호를 바꾼 날                : 12월 12, 2016
암호 만료                                : 안 함
암호가 비활성화 기간                    : 안 함
계정 만료                                : 1월 31, 2017
암호를 바꿀 수 있는 최소 날 수          : 0
암호를 바꿔야 하는 최대 날 수          : 99999
암호 만료 예고를 하는 날 수            : 7
[root@localhost ~]#
```

[그림 6-23] 사용자 계정 user_001의 만료일자 설정

- **Step 03** | 사용자 계정 user_001에 대해 설정된 암호를 최대 20일까지 사용할 수 있도록 설정한 다음 변경된 내용을 확인합니다.

```
# chage -M 20 user_001
```

기능 user_001 계정의 암호를 바꿔야 하는 최대 일 수를 20일로 설정

형식 chage [옵션] [사용자 계정이름] [Enter]

```
# chage -l user_001
```

기능 user_001 계정에 설정되어 있는 기본정보 출력

형식 chage [옵션] [사용자 계정이름] [Enter]

```

root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# chage -M 20 user_001
[root@localhost ~]# chage -l user_001
마지막으로 암호를 바꾼 날           : 1월 15, 2017
암호 만료                           : 2월 04, 2017
암호가 비활성화 기간                 : 2월 09, 2017
계정 만료                           : 1월 31, 2017
암호를 바꿀 수 있는 최소 날 수       : 0
암호를 바꿔야 하는 최대 날 수       : 20
암호 만료 예고를 하는 날 수         : 7
[root@localhost ~]#
  
```

[그림 6-24] 사용자 계정 user_001의 암호를 바꿔야 하는 최대 유효일수 설정

■ 사용자 계정 삭제 : userdel 명령 사용

- 사용자 계정을 삭제할 때는 관련된 홈 디렉터리와 파일을 모두 삭제

```
$ userdel
```

기능 사용자 계정 삭제

형식 userdel [옵션] 계정이름

옵션 -r : 홈 디렉터리 삭제

-f : 사용 중인 계정이름도 강제 삭제

-rf : 홈 디렉터리이외에 존재하는 파일 삭제

- 사용자 계정 삭제 수행방법

- ① /etc/passwd 파일에서 사용자 계정 삭제 : # userdel -r 계정이름
- ② /etc/group 파일에서 사용자 그룹 삭제 : # groupdel 계정이름
- ③ /home 디렉터리에 있는 사용자의 홈 디렉터리 삭제 : # rm -rf /home/계정이름
- ④ /var/spool/mail에 있는 사용자 메일 삭제 : # rm /var/spool/mail/계정이름
- ⑤ 데이터베이스 연동에서 삭제하려는 계정이름과 암호 삭제
- ⑥ 기타 응용 프로그램에 존재하는 사용자 계정의 정보 삭제
- ⑦ 리눅스 시스템을 재시작

- UID를 이용하여 사용자 계정관련 모든 파일 삭제방법

```
# find -user UID -exec rm -r { } \;
```

기능 파일내용 중에서 UID를 찾아서 관련된 파일을 모두 삭제

형식 `find [옵션] UID [옵션] [명령] {} \;`

- user_0001 사용자 계정 삭제

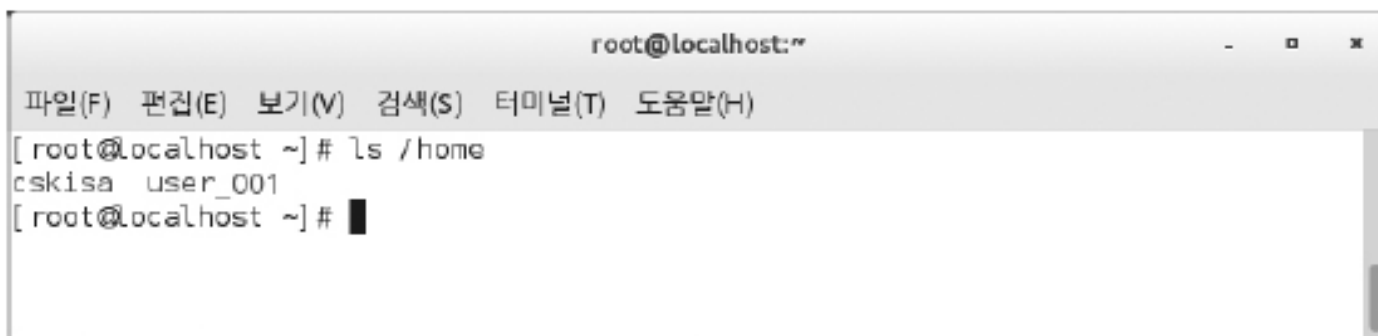
| 예제 6-12 |

- **Step 01** | 홈 디렉터리에 존재하는 사용자 계정이름에 대해 먼저 확인합니다.

```
# ls /home
```

기능 홈 디렉터리에 존재하는 사용자 계정이름 확인

형식 ls [옵션] [디렉터리명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# ls /home  
cskisa user_001  
[root@localhost ~]#
```

[그림 6-25] 홈 디렉터리에 존재하는 사용자 계정 목록

- **Step 02** | 삭제하고자 하는 user_001 계정에 대한 UID 정보 등을 확인합니다.

```
# grep user_001 /etc/passwd
```

기능 /etc/passwd 파일에서 특정 문자열 user_001을 찾을

형식 grep [찾을 문자열] [대상 파일명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# grep user_001 /etc/passwd  
user_001:x:1001:1001::/home/user_001:/bin/bash  
[root@localhost ~]#
```

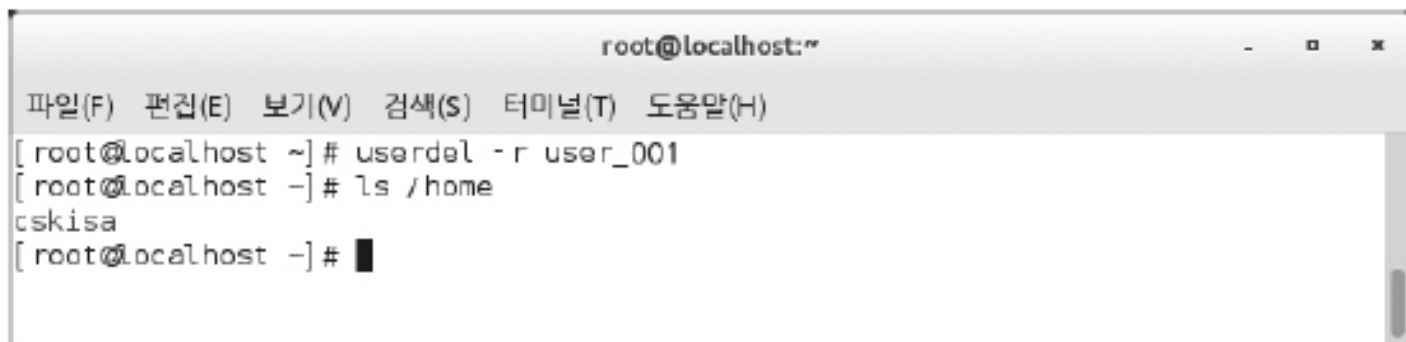
[그림 6-26] 문자열 검색을 통한 user_001 계정의 기본정보 출력

- **Step 03** | 사용자 계정 user_001을 userdel 명령과 옵션 -r을 함께 사용하여 계정을 삭제한 다음 ls /home 명령으로 user_001 계정이 삭제되었는지를 확인합니다.

```
# userdel -r user_001
```

기능 사용자 계정인 user_001 계정을 삭제

형식 userdel [옵션] [계정이름]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# userdel -r user_001  
[root@localhost ~]# ls /home  
cskisa  
[root@localhost ~]#
```

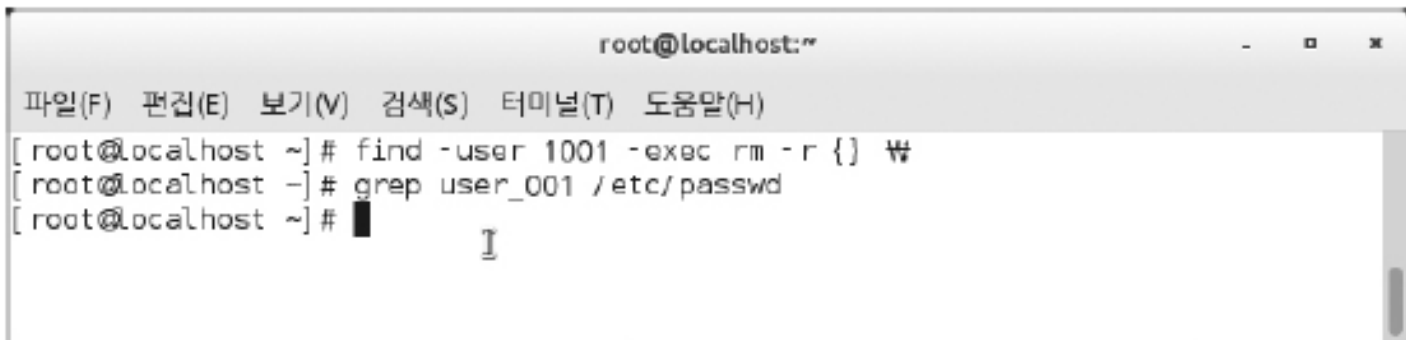
[그림 6-27] user_001 계정 삭제 후 홈 디렉터리 확인

- **Step 04** | 삭제한 사용자 계정 user_001의 UID는 [그림 6-26]에서 확인한 바와 같이 1001이므로 find 명령과 함께 옵션을 사용하여 user_001 계정과 관련된 모든 파일을 한꺼번에 삭제한 다음 grep 명령으로 파일이 남아있는지를 확인합니다.

```
# find -user 1001 -exec rm -r { } \;
```

기능 파일내용 중에서 UID를 찾아서 관련된 파일을 모두 삭제

형식 find [옵션] UID [옵션] [명령] {} ₩:



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# find -user 1001 -exec rm -r {} ₩  
[root@localhost ~]# grep user_001 /etc/passwd  
[root@localhost ~]#
```

[그림 6-28] 삭제된 user_001 계정관련 모든 파일 한꺼번에 삭제 후 확인



그룹 관리

• 그룹 생성과 그룹 정보 이해하기



1. 그룹을 생성하는 방법에 대해 실습합니다.
2. 그룹의 정보를 수정하는 방법에 대해 살펴봅니다.
3. 그룹 암호와 그룹 삭제 방법에 대해 실습합니다.

■ 새 그룹 생성

- 새 그룹은 /etc/group 파일에 저장되며 사용형식은 다음과 같음

```
$ groupadd
```

기능 새 그룹 생성

형식 `groupadd [옵션] 그룹이름`

옵션 `-g gid`: 그룹의 GID를 직접 지정

`-o`: GID의 중복사용 허용

- 옵션 없이 새 그룹 생성

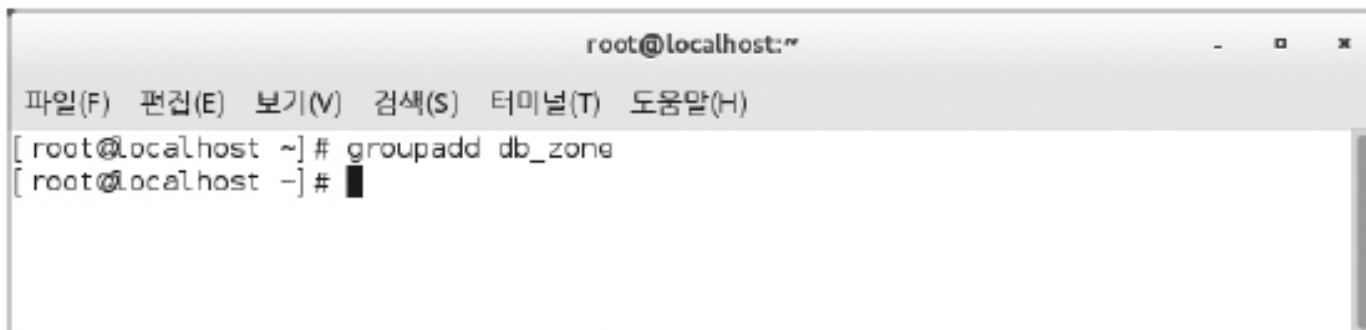
| 예제 6-13 |

- **Step 01** | 새 그룹 db_zone을 옵션 없이 groupadd 명령으로 생성합니다.

```
# groupadd db_zone
```

기능 새 그룹 db_zone을 생성

형식 groupadd [옵션] 그룹이름 [Enter.]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # groupadd db_zone  
[root@localhost ~] #
```

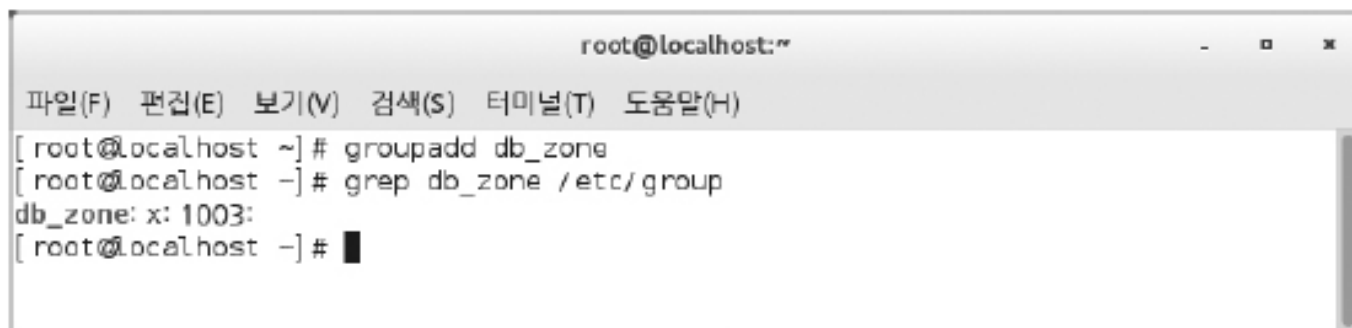
[그림 6-29] 새 그룹 db_zone 생성

- **Step 02** | 새로 생성된 db_zone 그룹에 리눅스 시스템에서 자동으로 부여된 GID를 확인합니다.

```
# grep db_zone /etc/group
```

기능 /etc/group 파일에서 특정 문자열 db_zone을 찾을

형식 grep [찾을 문자열] [대상 파일명]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # groupadd db_zone  
[root@localhost ~] # grep db_zone /etc/group  
db_zone: x: 1003:  
[root@localhost ~] #
```

[그림 6-30] db_zone 그룹에 자동 설정된 GID 확인

- GID를 지정하여 새 그룹 생성

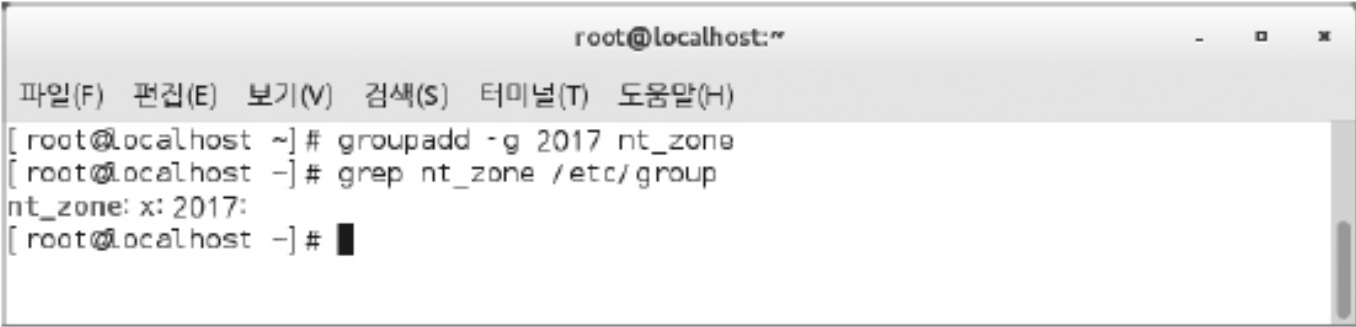
| 예제 6-14 |

GID를 2017로 설정하여 새 그룹 nt_zone을 생성한 다음 GID를 확인합니다.

```
# groupadd -g 2017 nt_zone
```

기능 GID를 2017로 설정하여 새 그룹 nt_zone을 생성

형식 groupadd [옵션] GID 그룹이름



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # groupadd -g 2017 nt_zone  
[root@localhost ~] # grep nt_zone /etc/group  
nt_zone:x:2017:  
[root@localhost ~] #
```

[그림 6-31] GID를 2017로 설정하여 새 그룹 nt_zone을 생성

- 기존 GID를 중복 지정하여 새 그룹 생성

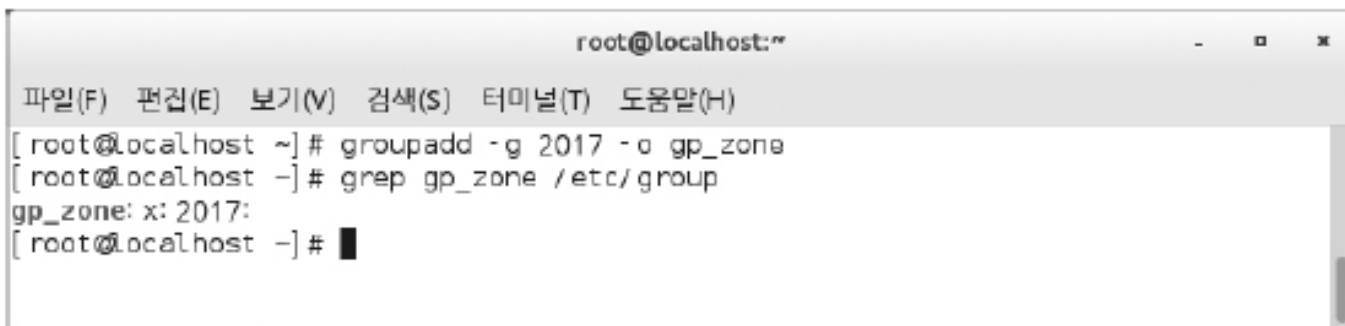
| 예제 6-15 |

기존 GID 2017을 중복 지정하여 새 그룹 gp_zone을 생성한 다음 GID를 확인합니다.

```
# groupadd -g 2017 -o gp_zone
```

기능 기존 GID 2017을 중복 지정하여 새 그룹 gp_zone을 생성

형식 groupadd [옵션] GID [옵션] 그룹이름



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # groupadd -g 2017 -o gp_zone  
[root@localhost ~] # grep gp_zone /etc/group  
gp_zone:x:2017:  
[root@localhost ~] #
```

[그림 6-32] 기존 GID 2017을 중복 지정하여 새 그룹 gp_zone을 생성

■ 그룹 암호 설정 및 삭제

- gpasswd 명령을 사용하여 그룹 암호 설정 및 삭제

```
$ gpasswd
```

기능 그룹 암호 관리 (/etc/group과 /etc/gshadow 파일을 관리)

형식 gpasswd [옵션] 그룹이름 [Enter]

옵션 -a 계정이름 : 계정을 그룹에 추가

-d 계정이름 : 계정을 그룹에서 삭제

-r : 그룹 암호를 삭제

- 기존 그룹에 새 멤버 추가 : -a 옵션 사용

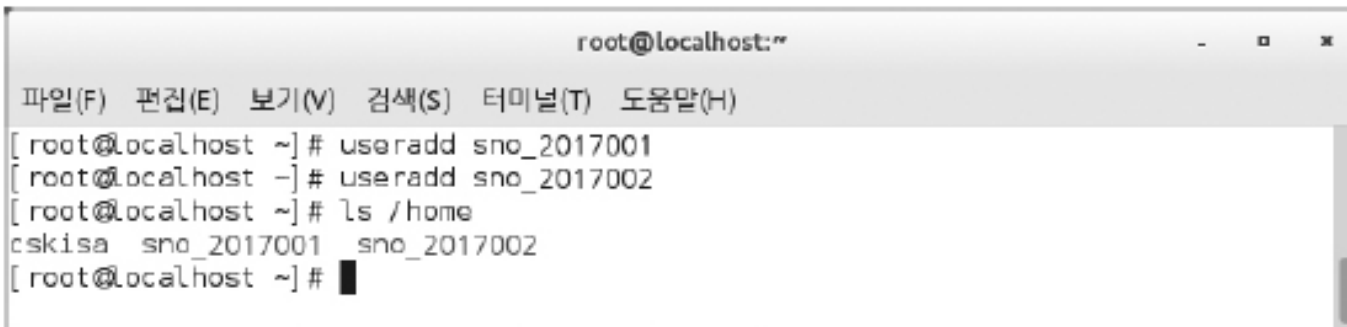
| 예제 6-16 |

- **Step 01** | 사용자 계정 sno_2017001, sno_2017002를 새로 생성한 다음 확인합니다.

```
# useradd sno_2017001
```

기능 새 사용자 계정 sno_2017001 생성

형식 useradd [옵션] 계정이름



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # useradd sno_2017001  
[root@localhost ~] # useradd sno_2017002  
[root@localhost ~] # ls /home  
cskisa  sno_2017001  sno_2017002  
[root@localhost ~] #
```

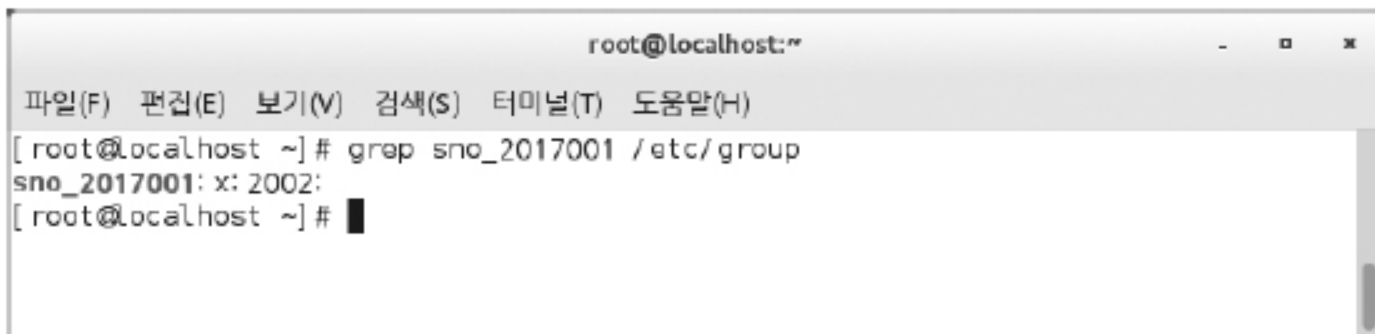
[그림 6-33] 새로운 계정 2개 생성 후 확인

- **Step 02** | 멤버를 추가할 그룹을 미리 확인해 보면 /etc/group 파일에는 멤버가 하나도 없음을 알 수 있습니다.

```
# grep sno_2017001 /etc/group
```

기능 /etc/group 파일에서 문자열 sno_2017001을 찾아서 출력

형식 grep [옵션] 파일명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# grep sno_2017001 /etc/group  
sno_2017001: x: 2002:  
[root@localhost ~]#
```

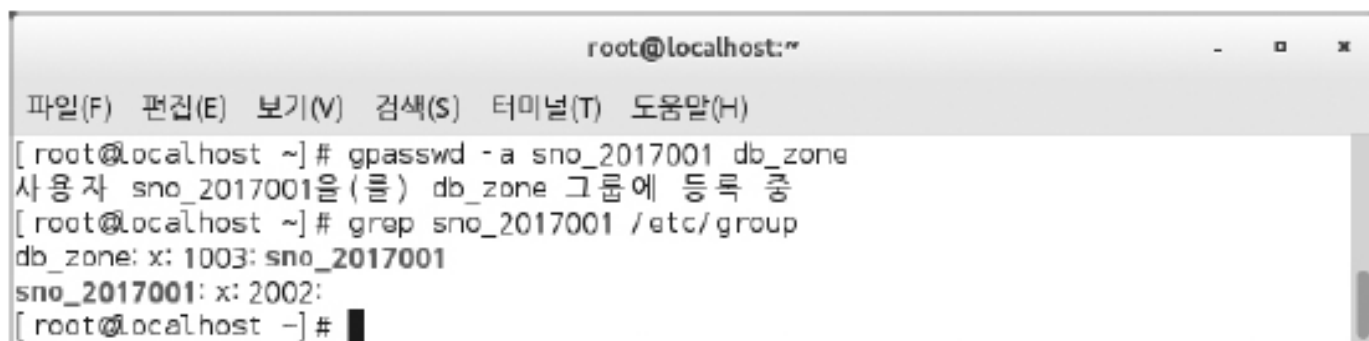
[그림 6-34] 그룹 2017에 존재하는 멤버 확인

- **Step 03** | db_zone 그룹에 사용자 계정 sno_2017001을 새 멤버로 추가한 다음 멤버가 정상적으로 추가되어 있는지를 확인합니다.

```
# gpasswd -a sno_2017001 db_zone
```

기능 사용자 계정 sno_2017001을 db_zone 그룹의 멤버로 추가

형식 gpasswd [옵션] 계정이름 그룹명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# gpasswd -a sno_2017001 db_zone  
사용자 sno_2017001을 (를) db_zone 그룹에 등록 중  
[root@localhost ~]# grep sno_2017001 /etc/group  
db_zone:x:1003:sno_2017001  
sno_2017001:x:2002:  
[root@localhost ~]#
```

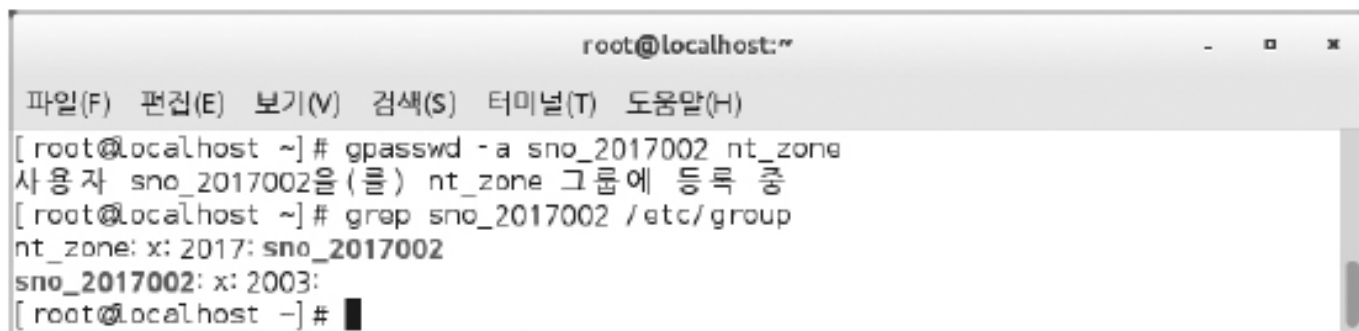
[그림 6-35] db_zone 그룹에 사용자 계정 sno_2017001 멤버 추가 후 존재확인

- **Step 04** | nt_zone 그룹에 사용자 계정 sno_2017002를 새 멤버로 추가한 다음 멤버가 정상적으로 추가되어 있는지를 확인합니다.

```
# gpasswd -a sno_2017002 nt_zone
```

기능 사용자 계정 sno_2017002를 nt_zone 그룹의 멤버로 추가

형식 gpasswd [옵션] 계정이름 그룹명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# gpasswd -a sno_2017002 nt_zone  
사용자 sno_2017002을(를) nt_zone 그룹에 등록 중  
[root@localhost ~]# grep sno_2017002 /etc/group  
nt_zone:x:2017:sno_2017002  
sno_2017002:x:2003:  
[root@localhost ~]#
```

[그림 6-36] nt_zone 그룹에 사용자 계정 sno_2017002 멤버 추가 후 존재확인

- 기존 그룹에 있는 특정 멤버 삭제 : -d 옵션 사용

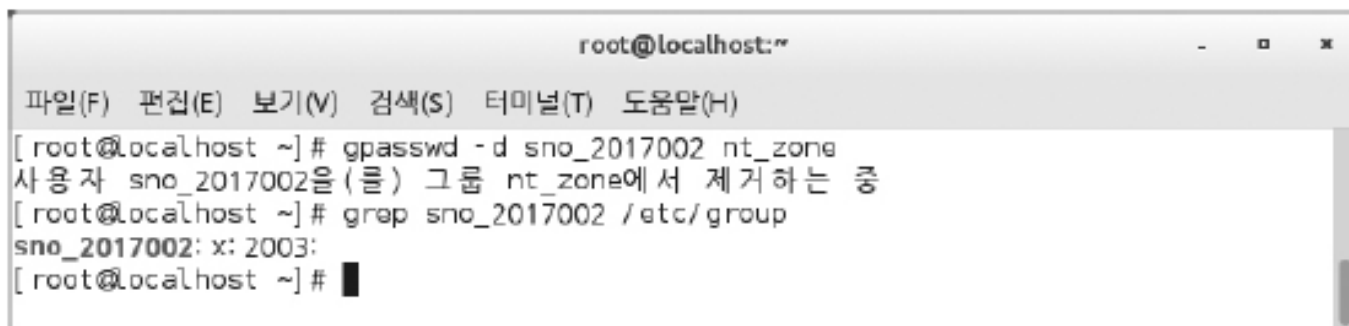
| 예제 6-17 |

nt_zone 그룹에 존재하는 사용자 계정 sno_2017002 멤버를 삭제한 다음 삭제된 멤버가 존재하는지에 대해 확인합니다.

```
# gpasswd -d sno_2017002 nt_zone
```

기능 nt_zone 그룹에 있는 sno_2017002 멤버 삭제

형식 gpasswd [옵션] 계정이름 그룹명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# gpasswd -d sno_2017002 nt_zone  
사용자 sno_2017002을(를) 그룹 nt_zone에서 제거하는 중  
[root@localhost ~]# grep sno_2017002 /etc/group  
sno_2017002: x: 2003:  
[root@localhost ~]#
```

[그림 6-37] nt_zone 그룹에서 sno_2017002 멤버 삭제 후 확인

- 기존 그룹에 암호 설정 : gpasswd

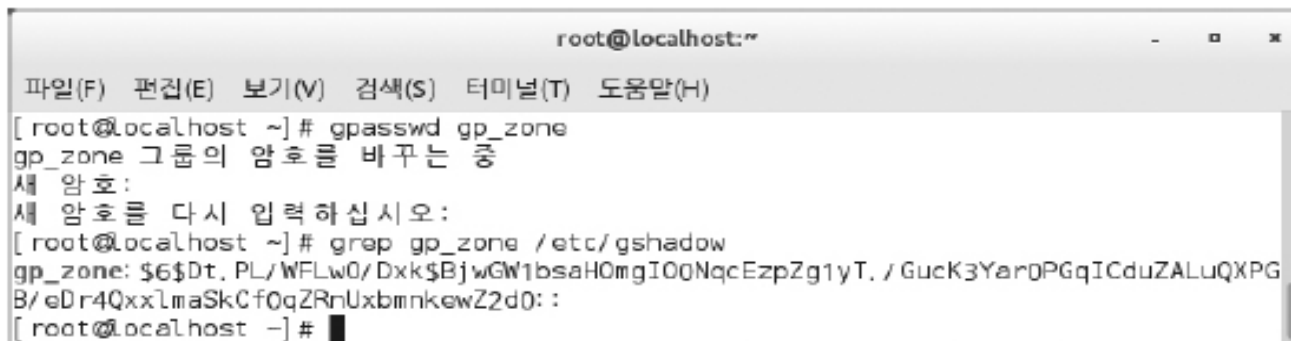
| 예제 6-18 |

- **Step 01** | 앞에서 생성한 gp_zone 그룹에 암호를 그룹이름과 동일한 gp_zone을 설정한 다음 설정된 암호가 저장된 /etc/gshadow 파일의 내용을 출력해서 그룹 암호가 설정되었음을 확인합니다.

```
# gpasswd gp_zone
```

기능 gp_zone 그룹에 암호 설정

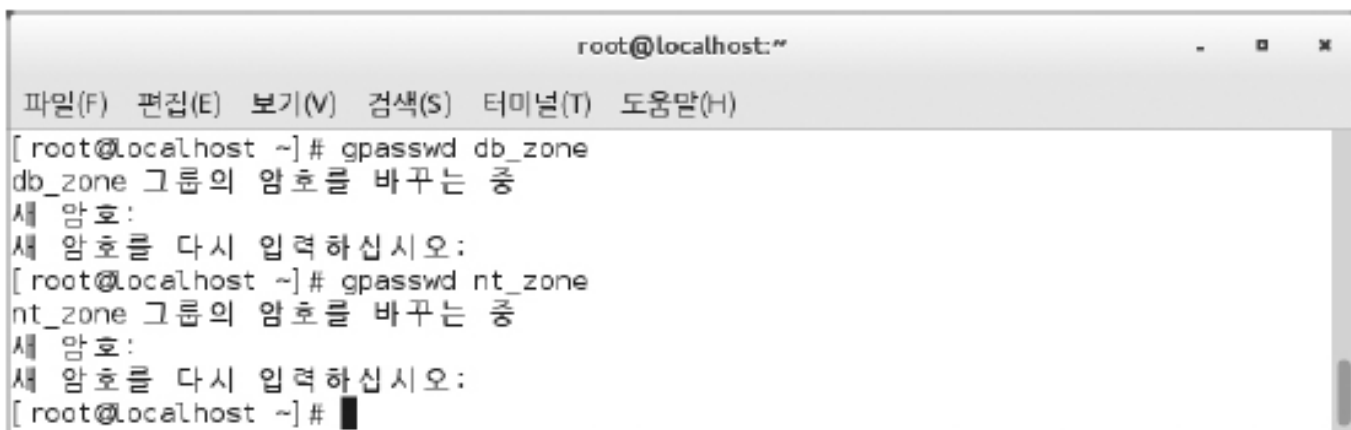
형식 gpasswd 그룹명 [Enter]



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # gpasswd gp_zone  
gp_zone 그룹의 암호를 바꾸는 중  
새 암호:  
새 암호를 다시 입력하십시오:  
[root@localhost ~] # grep gp_zone /etc/gshadow  
gp_zone: $6$Dt.PL/WFLw0/Dxk$BjwGW1bsaH0mgIO0NqcEzpZg1yT./GucK3Yar0PGqICduZALuQXPG  
B/eDr4QxxLmaSkCf0qZRnUxbmnkewZ2d0:  
[root@localhost ~] #
```

[그림 6-38] gp_zone 그룹 암호 설정 후 확인

- **Step 02** | 같은 방법으로 db_zone 그룹과 nt_zone 그룹에 대한 암호를 그룹이름과 동일하게 설정한 다음 /etc/gshadow 파일의 내용을 확인합니다.



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~] # gpasswd db_zone  
db_zone 그룹의 암호를 바꾸는 중  
새 암호:  
새 암호를 다시 입력하십시오:  
[root@localhost ~] # gpasswd nt_zone  
nt_zone 그룹의 암호를 바꾸는 중  
새 암호:  
새 암호를 다시 입력하십시오:  
[root@localhost ~] #
```

[그림 6-39] db_zone그룹과 nt_zone그룹 암호 설정

- 기존 그룹에 설정되어 있는 그룹 암호 삭제 : -r 옵션 사용

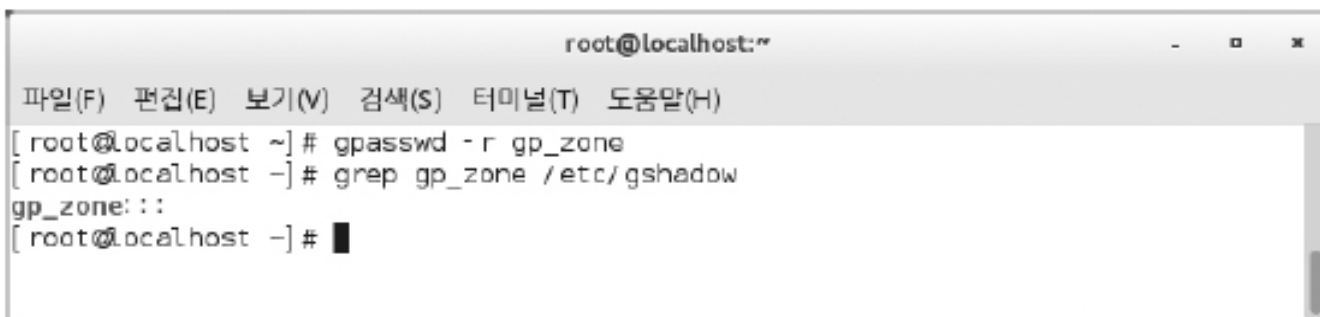
| 예제 6-19 |

gp_zone 그룹에 설정되어 있는 암호 gp_zone을 삭제한 다음 /etc/gshadow 파일의 내용을 출력해서 그룹 암호가 삭제되었음을 확인합니다.

```
# gpasswd -r gp_zone
```

기능 gp_zone 그룹에 설정되어 있는 암호 삭제

형식 gpasswd [옵션] 그룹명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# gpasswd -r gp_zone  
[root@localhost ~]# grep gp_zone /etc/gshadow  
gp_zone:::  
[root@localhost ~]#
```

[그림 6-40] gp_zone 그룹 암호 삭제 후 확인

소속 그룹 관리

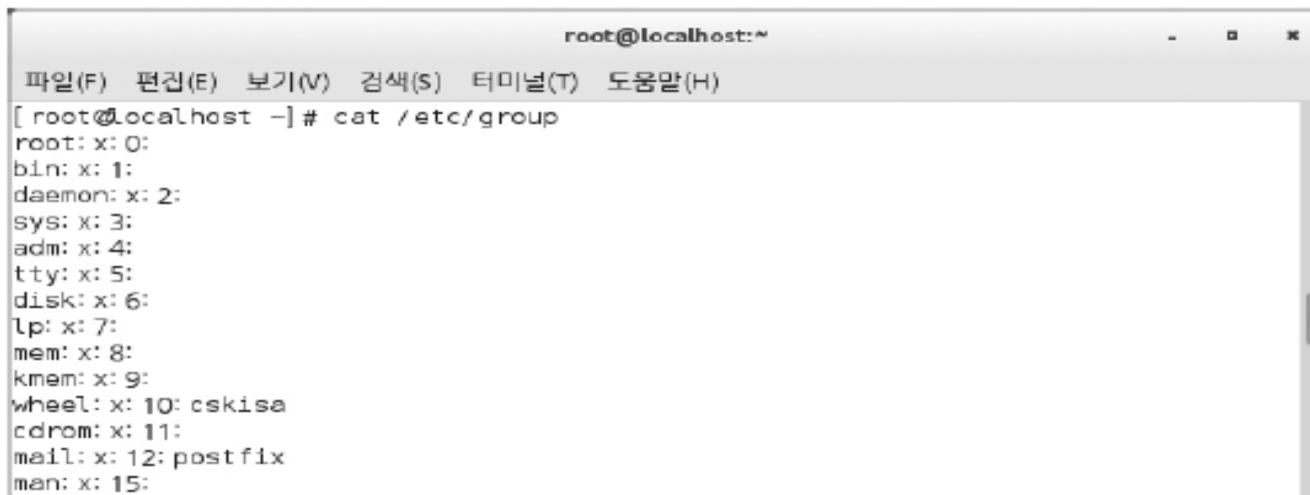
- 기존에 등록되어 있는 그룹 목록 출력 : cat 명령 사용

| 예제 6-20 |

```
# cat /etc/group
```

기능 리눅스 시스템에 등록되어 있는 그룹 목록 출력

형식 cat [옵션] 파일명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[ root@localhost ~] # cat /etc/group  
root:x:0:  
bin:x:1:  
daemon:x:2:  
sys:x:3:  
adm:x:4:  
tty:x:5:  
disk:x:6:  
lp:x:7:  
mem:x:8:  
kmem:x:9:  
wheel:x:10:cskisa  
cdrom:x:11:  
mail:x:12:postfix  
man:x:15:
```

(a) 리눅스 시스템에 등록되어 있는 그룹 목록-1



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
sshd: x: 74:  
dovecot: x: 97:  
dovecot: x: 980:  
stapusr: x: 156:  
stapusr: x: 157:  
stapdev: x: 158:  
tcpdump: x: 72:  
cskisa: x: 1000:  
user_002: x: 1002:  
db_zone: x: 1003: sno_2017001  
nt_zone: x: 2017:  
gp_zone: x: 2017:  
sno_2017001: x: 2002:  
sno_2017002: x: 2003:  
[root@localhost ~] #
```

(b) 리눅스 시스템에 등록되어 있는 그룹 목록-2

[그림 6-41] 리눅스 시스템에 등록되어 있는 그룹 목록 출력

- 사용자 계정으로 접속모드 변경 : su - 계정이름

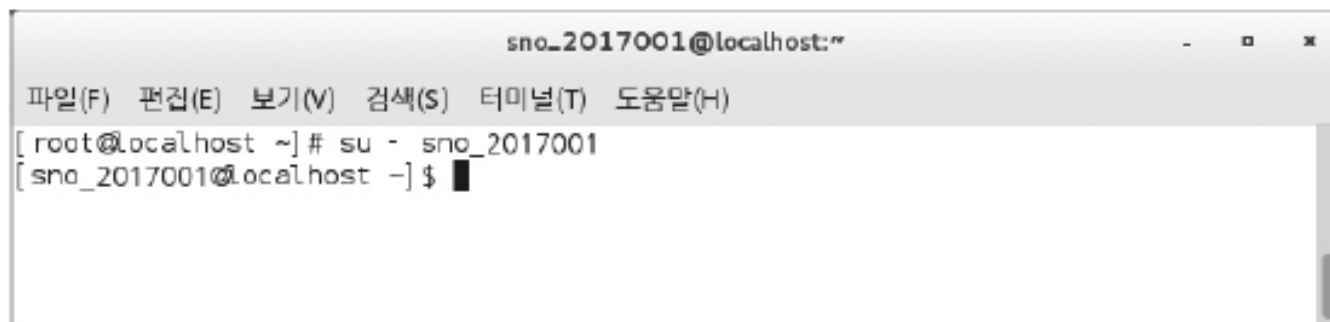
| 예제 6-21 |

현재 소속된 그룹을 변경하기 위해서는 먼저 관리자 계정이 아닌 사용자 계정으로 접속한 다음에 진행해야 합니다. su 명령으로 sno_2017001 사용자 계정으로 접속모드를 변경합니다.

```
# su - sno_2017001
```

기능 sno_2017001 사용자 계정으로 변경

형식 su [옵션] 계정이름



[그림 6-42] 접속모드를 사용자 계정으로 변경

- 소속 그룹정보 출력 : id 명령 사용

\$ id

기능 현재 사용자의 실제 ID와 유효 사용자 ID, 그룹 ID를 출력

형식 id [옵션] [계정이름]

옵션 -g : 사용자의 그룹 ID 출력

-G : 추가 그룹의 ID 출력

-u : 사용자의 UID 출력

-n : u, g, G 옵션과 함께 사용하며 해당하는 ID의 이름 출력

-r : u, g, G 옵션과 함께 사용하며 해당하는 실제 ID를 출력

- 현재 사용자 계정의 그룹 정보 출력

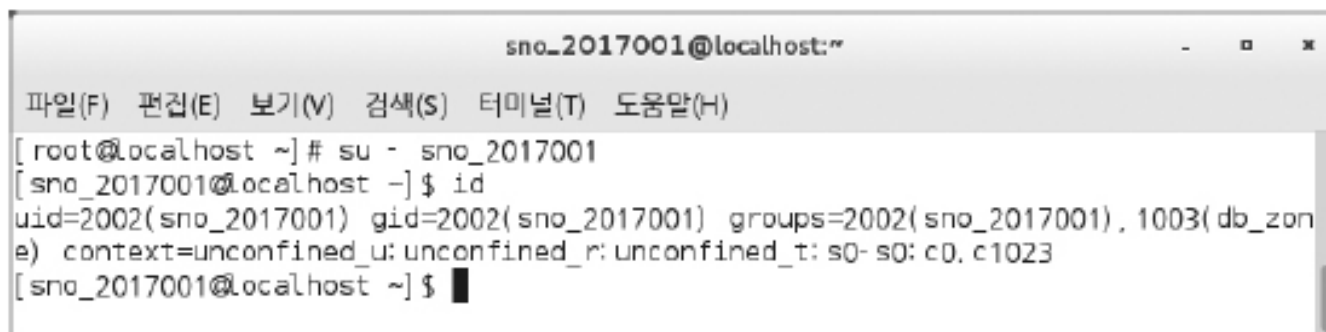
| 예제 6-22 |

현재 사용자 계정의 그룹에 대한 정보를 출력합니다.

```
$ id
```

기능 현재 사용자의 실제 ID와 유효 사용자 ID, 그룹 ID를 출력

형식 id [옵션] [계정이름]



```
sno_2017001@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# su - sno_2017001  
[sno_2017001@localhost ~]$ id  
uid=2002(sno_2017001) gid=2002(sno_2017001) groups=2002(sno_2017001), 1003(db_zon  
e) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0,c1023  
[sno_2017001@localhost ~]$
```

[그림 6-43] sno_2017001 계정의 그룹 정보 출력

- 소속 그룹 변경 : newgrp 명령 사용

```
$ newgrp 그룹명
```

기능 소속 그룹을 지정한 그룹으로 변경

형식 newgrp 그룹명

- sno_2017001 사용자 계정이 속한 그룹 변경 후 내용 출력

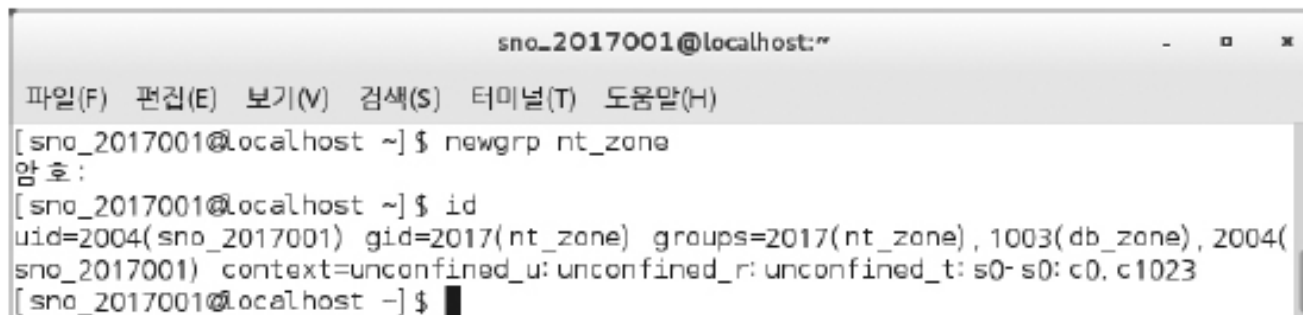
| 예제 6-23 |

sno_2017001 사용자 계정으로 접속된 상태에서 이 계정이 속한 그룹을 nt_zone으로 변경한 다음 id 명령으로 변경된 내용을 확인합니다..

```
$ newgrp nt_zone
```

기능 소속 그룹을 지정한 nt_zone 그룹으로 변경

형식 newgrp 그룹명



```
sno_2017001@localhost:~$ newgrp nt_zone
[sno_2017001@localhost ~]$ id
uid=2004(sno_2017001) gid=2017(nt_zone) groups=2017(nt_zone), 1003(db_zone), 2004(sno_2017001) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0,c1023
[sno_2017001@localhost ~]$
```

[그림 6-44] sno_2017001 계정의 소속 그룹변경과 정보 출력



소유자와 소유그룹 변경

- 파일 및 디렉터리의 소유자와 소유그룹 변경하기



1. 파일 및 디렉터리 소유자와 소유그룹에 대해 살펴봅니다.
2. 파일 및 디렉터리 소유자와 소유그룹 변경방법에 대해 실습합니다.

파일과 디렉터리의 소유권과 그룹 소유권 변경 : chown 명령 사용

- chown 명령어 사용형식

chown

기능 파일이나 디렉터리의 소유권 및 그룹 소유권을 변경

형식 `chown [옵션] [계정이름] [파일명 또는 디렉터리명] [Enter]`

옵션 `-R` : 하위 디렉터리를 모두 포함하여 소유권과 그룹 소유권을 변경

- chown 명령으로 파일과 디렉터리의 소유권 및 그룹 소유권 변경

| 예제 6-24 |

- **Step 01** | 먼저 root 계정으로 로그인 된 상태에서 사용자 계정인 sno_2017002로 접속을 변경한 다음 work 디렉터리와 빈파일 juso.txt를 생성합니다.

```
# ls /home
```

기능 /home 디렉터리에 있는 사용자 계정 목록 출력

형식 ls 디렉터리명

```
# su - sno_2017002
```

기능 사용자 계정 sno_2017002로 접속 변경

형식 su - 계정이름

```
$ mkdir work
```

기능 새로운 work 디렉터리를 생성

형식 mkdir 디렉터리명

```
$ touch juso.txt
```

기능 새로운 빈파일 juso.txt를 생성

형식 touch 파일명

```
sno_2017002@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# ls /home
cskisa sno_2017001 sno_2017002
[root@localhost ~]# su - sno_2017002
[sno_2017002@localhost ~]$ ls
[sno_2017002@localhost ~]$ mkdir work
[sno_2017002@localhost ~]$ touch juso.txt
[sno_2017002@localhost ~]$ ls -l
합계 0
-rw-rw-r--. 1 sno_2017002 sno_2017002 0  1월  6 08:30 juso.txt
drwxrwxr-x. 2 sno_2017002 sno_2017002 6  1월  6 08:30 work
[sno_2017002@localhost ~]$
```

[그림 6-45] 사용자 계정으로 변경 후 work 디렉터리와 juso.txt 파일 생성

- **Step 02** | chown 명령은 일반 사용자 계정에서는 사용할 수 없기 때문에 사용자 계정에서 work 디렉터리와 juso.txt 파일을 생성한 다음에는 다시 root 계정으로 접속해야 합니다.

```
$ su - root
```

기능 root 계정으로 접속 변경

형식 su - 계정이름

- **Step 03** | root 계정으로 접속된 상태에서 sno_2017002 계정에 존재하는 파일의 목록과 소유자 권한을 확인한 다음 chown 명령으로 work 디렉터리의 소유자를 cskisa 계정으로 변경하고 소유자가 변경된 사항을 확인합니다.

```
# ls -l /home/sno_2017002
```

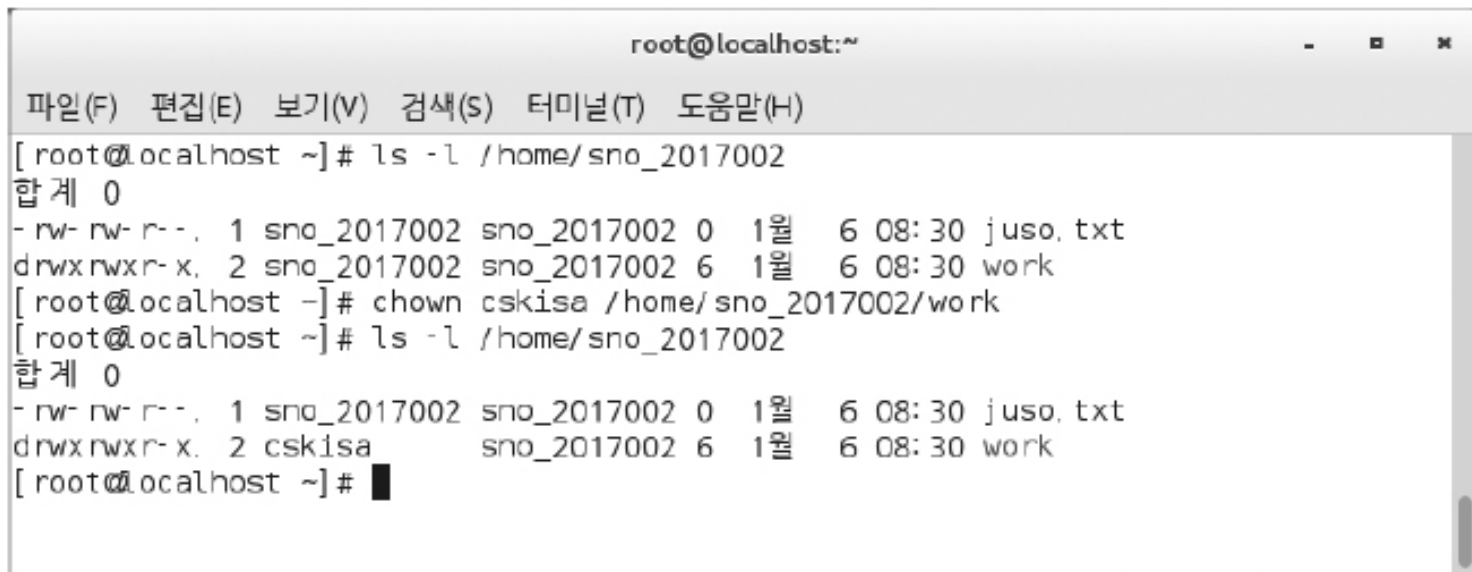
기능 사용자 계정 sno_2017002에서 생성한 디렉터리와 파일 존재 확인

형식 ls [옵션] 디렉터리명/계정이름

```
# chown cskisa /home/sno_2017002/work
```

기능 sno_2017002 계정에서 생성한 work 디렉터리의 소유자를 cskisa로 변경

형식 chown 계정이름 디렉터리명



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# ls -l /home/sno_2017002  
합계 0  
-rw-rw-r--. 1 sno_2017002 sno_2017002 0 1월 6 08:30 juso.txt  
drwxrwxr-x. 2 sno_2017002 sno_2017002 6 1월 6 08:30 work  
[root@localhost ~]# chown cskisa /home/sno_2017002/work  
[root@localhost ~]# ls -l /home/sno_2017002  
합계 0  
-rw-rw-r--. 1 sno_2017002 sno_2017002 0 1월 6 08:30 juso.txt  
drwxrwxr-x. 2 cskisa      sno_2017002 6 1월 6 08:30 work  
[root@localhost ~]#
```

[그림 6-46] work 디렉터리에 대한 소유자를 cskisa 계정으로 변경

- sno_2017002 계정에서 생성한 juso.txt 파일의 사용자 계정과 그룹을 동시에 지정하여 변경

| 예제 6-25 |

항상 chown 명령을 사용하기 전에는 반드시 root 계정으로 로그인 된 상태인지를 먼저 확인합니다. 그리고 사용자 계정 sno_2017002에서 생성한 juso.txt 파일의 소유자와 그룹을 모두 cskisa로 변경한 다음 변경된 사항을 확인합니다.

```
# ls -l /home/sno_2017002
```

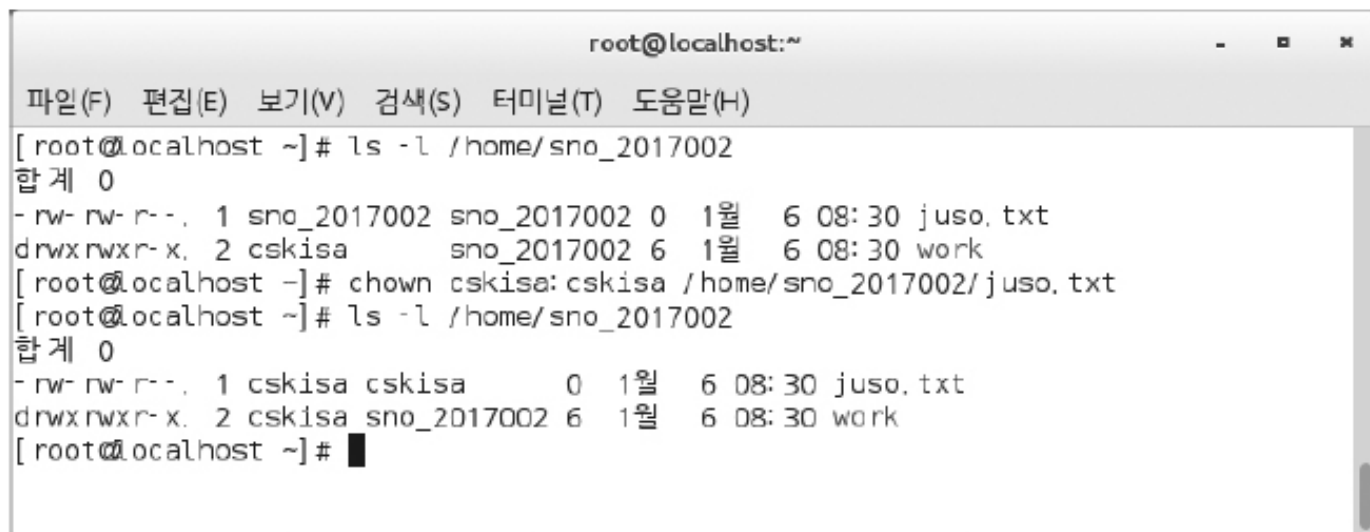
기능 사용자 계정 sno_2017002에서 생성한 디렉터리와 파일 존재 확인

형식 ls [옵션] 디렉터리명/계정이름

```
# chown cskisa:cskisa /home/sno_2017002/juso.txt
```

기능 sno_2017002 계정에서 생성한 juso.txt 파일의 소유자와 그룹을 cskisa로 변경

형식 chown 계정이름:그룹이름 파일명



```

root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# ls -l /home/sno_2017002
합계 0
-rw-rw-r--. 1 sno_2017002 sno_2017002 0 1월 6 08:30 juso.txt
drwxrwxr-x. 2 cskisa      sno_2017002 6 1월 6 08:30 work
[root@localhost ~]# chown cskisa:cskisa /home/sno_2017002/juso.txt
[root@localhost ~]# ls -l /home/sno_2017002
합계 0
-rw-rw-r--. 1 cskisa cskisa      0 1월 6 08:30 juso.txt
drwxrwxr-x. 2 cskisa sno_2017002 6 1월 6 08:30 work
[root@localhost ~]#
  
```

[그림 6-47] juso.txt 파일에 대한 소유자와 그룹을 cskisa로 변경

■ 파일과 디렉터리의 소유그룹 변경 : chgrp 명령 사용

- chgrp 명령어 사용형식

chgrp

기능 파일이나 디렉터리의 그룹 소유권을 변경

형식 chgrp [옵션] [계정이름] [파일명 또는 디렉터리명]

옵션 -R : 하위 디렉터리를 모두 포함하여 그룹 소유권을 변경

- sno_2017001 계정에 새로운 디렉터리와 파일 생성 후 소유그룹 변경

| 예제 6-26 |

- **Step 01** | 먼저 root 계정으로 로그인 된 상태에서 사용자 계정인 sno_2017001로 접속을 변경한 다음 계층적 디렉터리 workspace/test/ex_dir 디렉터리와 scale.txt 빈 파일을 생성합니다.

```
# su - sno_2017001
```

기능 사용자 계정 sno_2017001로 접속 변경

형식 su - 계정이름

```
$ mkdir -p workspace/test/ex_dir
```

기능 새로운 workspace/test/ex_dir 계층적 디렉터리를 생성

형식 mkdir [옵션] 계층적 디렉터리명

```
$ touch scale.txt
```

기능 새로운 빈파일 scale.txt를 생성

형식 touch 파일명

```
$ ls -lR
```

기능 현재 디렉터리에 존재하는 하위 디렉터리 리스트를 출력

형식 ls [옵션] 디렉터리명/계정이름

```
sno_2017001@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# su - sno_2017001  
마지막 로그인: 일 1월 1 16:17:43 KST 2017 일시 pts/0  
[sno_2017001@localhost ~]$ mkdir -p workspace/test/ex_dir  
[sno_2017001@localhost ~]$ touch scale.txt  
[sno_2017001@localhost ~]$ ls -lR  
..  
합계 0  
-rw-rw-r--. 1 sno_2017001 sno_2017001 0 1월 6 09:37 scale.txt  
drwxrwxr-x. 3 sno_2017001 sno_2017001 17 1월 6 09:37 workspace  
  
./workspace:  
합계 0  
drwxrwxr-x. 3 sno_2017001 sno_2017001 19 1월 6 09:37 test  
  
./workspace/test:  
합계 0  
drwxrwxr-x. 2 sno_2017001 sno_2017001 6 1월 6 09:37 ex_dir  
  
./workspace/test/ex_dir:  
합계 0  
[sno_2017001@localhost ~]$
```

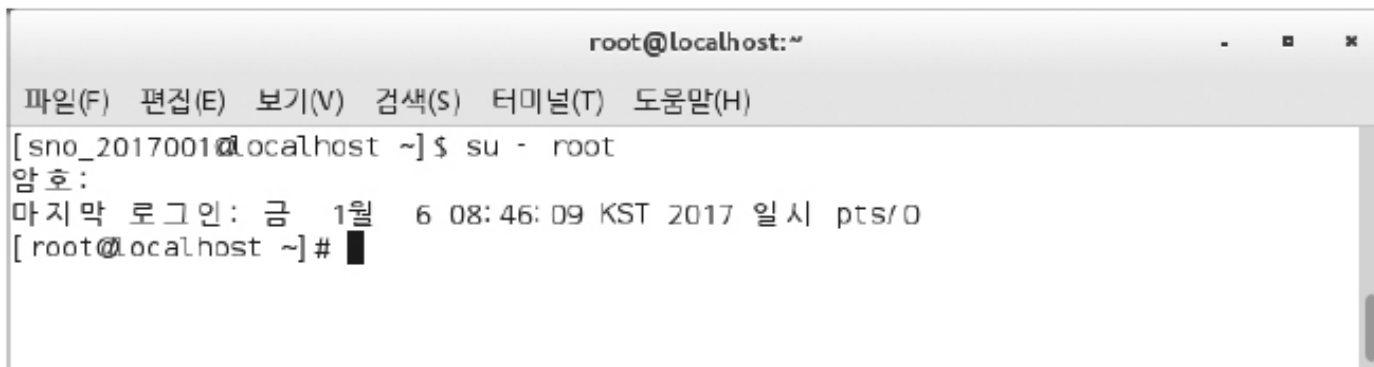
[그림 6-48] sno_2017001 계정에서 계층적 디렉터리와 빈파일 생성

- **Step 02** | chgrp 명령 또한 일반 사용자 계정에서는 사용할 수 없기 때문에 사용자 계정에서 다시 root 계정으로 접속해야 합니다.

```
$ su - root
```

기능 root 계정으로 접속 변경

형식 su - 계정이름



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[sno_2017001@localhost ~]$ su - root  
암호 :  
마지막 로그인: 금 1월 6 08:46:09 KST 2017 일시 pts/0  
[root@localhost ~]#
```

[그림 6-49] 사용자 계정에서 root 계정으로 전환

- **Step 03** | root 계정으로 접속된 상태에서 sno_2017001 계정에 존재하는 파일의 목록과 소유자 권한을 확인한 다음 chgrp 명령과 함께 옵션 -R을 지정하여 소유자와 소유그룹을 cskisa 계정으로 변경하고 변경된 사항을 확인합니다.

```
# ls -l /home/sno_2017001
```

기능 사용자 계정 sno_2017001에서 생성한 디렉터리와 파일 존재 확인

형식 ls [옵션] 디렉터리명/계정이름 [Enter]

```
# chgrp -R cskisa /home/sno_2017001/workspace
```

기능 sno_2017002 계정에서 생성한 workspace 디렉터리의 소유자를 cskisa로 변경

형식 chown 계정이름 디렉터리명 [Enter]

```
$ ls -lR
```

기능 현재 디렉터리에 존재하는 하위 디렉터리 리스트를 출력

형식 `ls [옵션] 디렉터리명/계정이름`

```

root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# ls -l /home/sno_2017001
합계 0
-rw-rw-r--. 1 sno_2017001 sno_2017001 0 1월 6 09:37 scale.txt
drwxrwxr-x. 3 sno_2017001 sno_2017001 17 1월 6 09:37 workspace
[root@localhost ~]# chgrp -R cskisa /home/sno_2017001/workspace
[root@localhost ~]# ls -lR /home/sno_2017001
/home/sno_2017001:
합계 0
-rw-rw-r--. 1 sno_2017001 sno_2017001 0 1월 6 09:37 scale.txt
drwxrwxr-x. 3 sno_2017001 cskisa 17 1월 6 09:37 workspace

/home/sno_2017001/workspace:
합계 0
drwxrwxr-x. 3 sno_2017001 cskisa 19 1월 6 09:37 test

/home/sno_2017001/workspace/test:
합계 0
drwxrwxr-x. 2 sno_2017001 cskisa 6 1월 6 09:37 ex_dir

/home/sno_2017001/workspace/test/ex_dir:
합계 0
[root@localhost ~]#

```

[그림 6-50] 계층적 디렉터리 workspace에 대한 소유그룹을 cskisa 계정으로 변경

- sno_2017001 계정에 생성한 scale.txt 파일의 소유그룹을 cskisa 계정으로 변경한 다음 변경된 사항 출력

| 예제 6-27 |

chgrp 명령 또한 사용하기 전에는 반드시 root 계정으로 로그인 된 상태인지를 먼저 확인합니다. 그리고 사용자 계정 sno_2017001에서 생성한 scale.txt 파일의 소유자와 그룹을 모두 cskisa로 변경한 다음 변경된 사항을 확인합니다.

```
# ls -l /home/sno_2017001
```

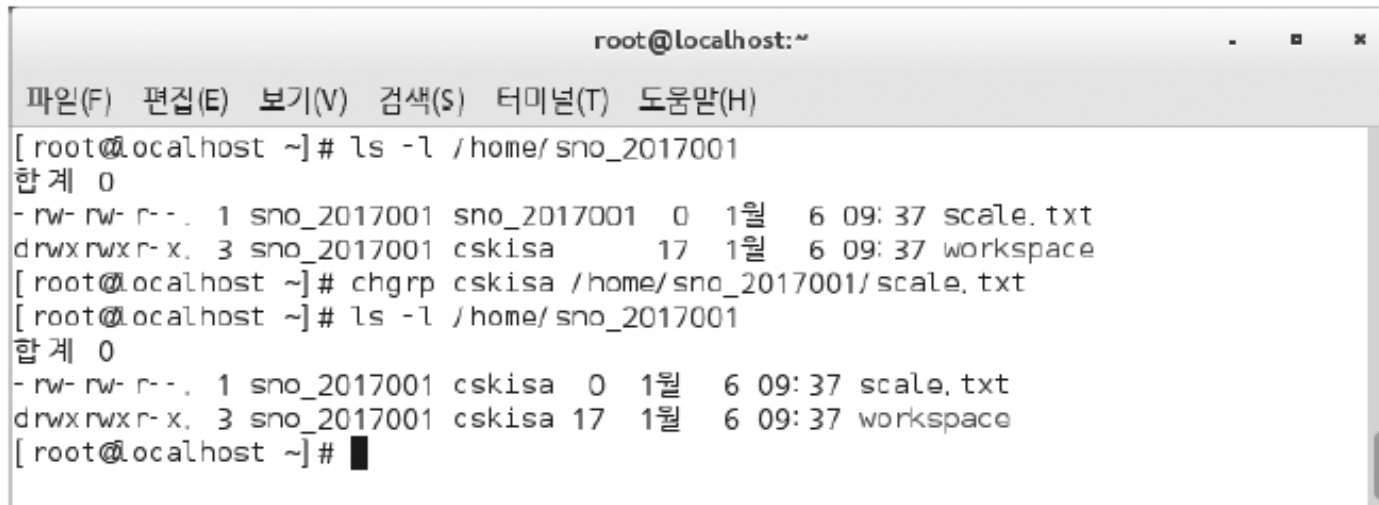
기능 사용자 계정 sno_2017001에서 생성한 디렉터리와 파일 존재 확인

형식 ls [옵션] 디렉터리명/계정이름


```
# chgrp cskisa /home/sno_2017001/scale.txt
```

기능 sno_2017001 계정에서 생성한 scale.txt 파일의 소유그룹을 cskisa로 변경

형식 chgrp 그룹이름 파일명



```

root@localhost:~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[root@localhost ~]# ls -l /home/sno_2017001
합계 0
-rw-rw-r--. 1 sno_2017001 sno_2017001 0 1월 6 09:37 scale.txt
drwxrwxr-x. 3 sno_2017001 cskisa 17 1월 6 09:37 workspace
[root@localhost ~]# chgrp cskisa /home/sno_2017001/scale.txt
[root@localhost ~]# ls -l /home/sno_2017001
합계 0
-rw-rw-r--. 1 sno_2017001 cskisa 0 1월 6 09:37 scale.txt
drwxrwxr-x. 3 sno_2017001 cskisa 17 1월 6 09:37 workspace
[root@localhost ~]#

```

[그림 6-51] scale.txt 파일에 대한 소유그룹을 cskisa 계정으로 변경

SECTION
05

리눅스 시스템 종료

• 리눅스 시스템을 종료하는 방법 이해하기



1. 현재 사용 중인 리눅스 시스템을 종료하는 방법의 종류를 살펴봅니다.
2. 시스템을 종료하는 명령어와 옵션에 대해 이해합니다.
3. 리눅스 시스템을 재부팅하는 방법을 살펴봅니다.

리눅스 시스템 종료 : shutdown 명령 사용

- shutdown 명령어 사용형식

shutdown

기능 리눅스 시스템 종료

형식 shutdown [옵션] [시간] [메시지]

옵션 -c : 이전에 수행했던 shutdown 명령 취소

-f : 빠른 재시작 (fsck 생략 가능)

-h : 종료하며 halt 상태로 이동

-k : 실제 종료는 아니고 사용자들에게 메시지만 전달

-r : 종료 후 재시작

시간 : 종료할 시간 (hh:mm, +m, now)

메시지 : 접속된 모든 사용자에게 보낼 메시지

- 이외에도 poweroff, halt, reboot 명령으로도 리눅스 시스템 종료

시스템 즉시 종료

리눅스 시스템을 즉시 종료하려면 shutdown 명령과 함께 -h 옵션을 사용하여 현재 시각을 의미하는 now를 지정하면 됩니다.

```
# shutdown -h now
```

종료 메시지 보낸 후 종료하기

리눅스 시스템은 여러 사용자가 동시에 접속하여 사용하는 시스템이기 때문에 리눅스를 종료하기에 앞서 접속된 사용자들에게 미리 종료 메시지를 전달해 주는 것이 바람직합니다. 만약 3분 후 리눅스 시스템을 종료한다는 메시지를 보내고자 하는 경우에는 다음과 같이 명령을 수행하면 됩니다.

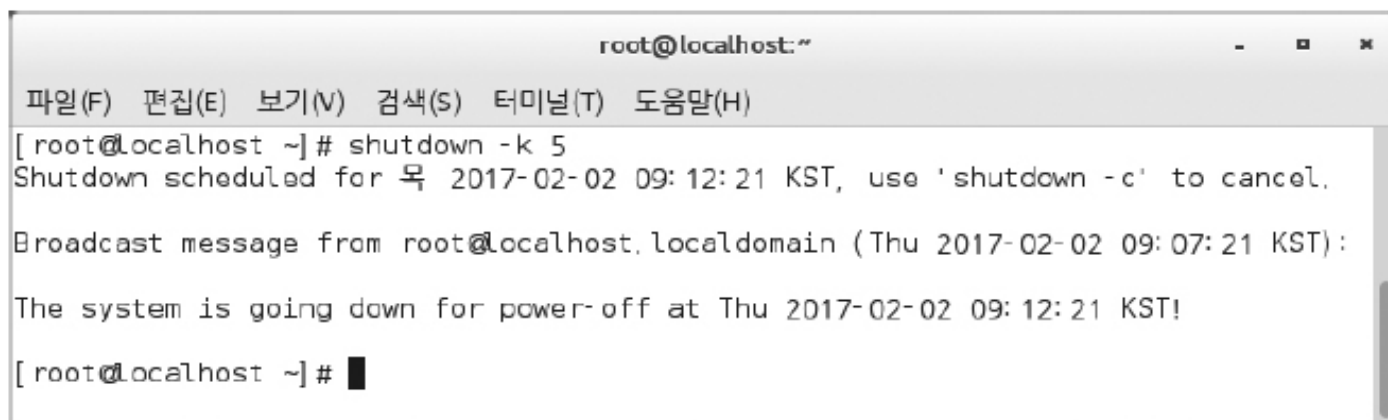
```
# shutdown -h +3 "It is 3 minutes before the shutdown."
```

- 실제로 시스템을 종료하지 않고 종료 메시지만 보내기

| 예제 6-28 |

리눅스 시스템을 실제로 종료하지는 않고 5분 후 시스템을 종료한다는 메시지만 사용자들에게 전달하려면 다음과 같이 명령을 수행하면 됩니다.

```
# shutdown -k 5
```



```
root@localhost:~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
[root@localhost ~]# shutdown -k 5  
Shutdown scheduled for 목 2017-02-02 09:12:21 KST, use 'shutdown -c' to cancel.  
Broadcast message from root@localhost.localdomain (Thu 2017-02-02 09:07:21 KST):  
The system is going down for power-off at Thu 2017-02-02 09:12:21 KST!  
[root@localhost ~]# █
```

[그림 6-52] 5분후 시스템 종료 안내

기타 명령어

shutdown 명령 외에 시스템을 종료하거나 재시작을 위해 사용할 수 있는 명령으로 런레벨 변경과 halt, poweroff, reboot 명령이 있습니다. halt, poweroff, reboot 명령은 /var/log/vtmp 파일에 시스템 종료와 관련된 기록을 남기고 시스템을 종료하거나 재시작합니다. 터미널 창에서 마운트 및 패키지 설치 등으로 인해 시스템을 재부팅할 경우가 발생하게 됩니다. 이와 같은 경우 리눅스 시스템 전체를 종료하지 않고 터미널 창에서 시스템을 재부팅할 때는 reboot 명령을 사용하면 됩니다.

```
# reboot
```

이상으로 관리자 계정접속과 사용자 관리계정 및 그룹관리, 소유자와 소유그룹 변경에 대해 살펴보았습니다. 다음 장에서는 리눅스 파일시스템과 디스크 관리에 대해서 다루겠습니다.